

University of Southampton Research Repository

Copyright © and Moral Rights for this thesis and, where applicable, any accompanying data are retained by the author and/or other copyright owners. A copy can be downloaded for personal non-commercial research or study, without prior permission or charge. This thesis and the accompanying data cannot be reproduced or quoted extensively from without first obtaining permission in writing from the copyright holder/s. The content of the thesis and accompanying research data (where applicable) must not be changed in any way or sold commercially in any format or medium without the formal permission of the copyright holder/s.

When referring to this thesis and any accompanying data, full bibliographic details must be given, e.g.

Thesis: Haider Al-Shareefy (2025) "AIC predictive architecting for designing and assuring safety-critical autonomous systems in open, complicated environments", University of Southampton, name of the University Faculty of Physical Science, School of Electronics & Computer Science, PhD Thesis, pagination.

Data: Haider Al-Shareefy (2025) Titles:

- A. Definition of the Multi-domain ODD Natural Surrounding Taxonomy
- B. ODD Uncertainty Grading for Safety Training of Autonomous Systems
- C. Extended Literature Review
- D. Safety Validation Report for AVOID Dataset using the CuneiForm method
- E. AIC Systems Approach Key Concepts Extended Definitions
- F. Detailed Summary of AIC Systems Approach Processes
- G. Applying Current SE Methods (HAZOP, STAMP, FRAM)
- H. Unsafe Train Tracks Problem Detailed Implementation Case Study
- I. Mid-air Collision Avoidance Case Study
- J. Extended work for the Unsafe Train Tracks Case Study
- K. Mid-air Collision Case Study Extended Work
- L. AMLAS and SACE Safety Case Construction
- M. AVOIDDS Case Study Summary

University of Southampton

Faculty of Engineering and Physical Sciences

School of Electronics and Computer Science

Towards an Advancement in Holistic Systems Engineering Method for Machine Learning based Safety Critical Systems

by

Haider M. Al-Shareefy

ORCID ID <https://orcid.org/0000-0002-8291-5620>

Thesis for the degree of Doctor of Philosophy

June 2025

University of Southampton

Abstract

Faculty of Engineering and Physical Sciences

School of Electronics and Computer Science

Doctor of Philosophy

Applied Systems Science and Thinking-based Predictive Architecting for Engineering Safe, Resilient, Complicated Systems

**Towards an Advancement in Holistic Systems Engineering Methods for Machine
Learning based Safety Critical Systems**

by

Haider M. Al-Shareefy

Ensuring the safety and trustworthiness of autonomous systems in open, complex environments requires not only addressing foreseeable hazards (“White Swans”) but also actively seeking and mitigating unforeseen, high-impact failures (“Black Swans”) that arise from experiencing unexpected scenarios in the operational problem space. If we model the operational domain correctly, we can predict those environmental Black Swan scenarios and prepare our ML-based systems with the proper training. This thesis develops and validates a Systems-Thinking and Science-based approach, rooted in reducing the architect’s epistemic uncertainty, by employing a holistic perspective on general systems. It constructs comprehensive safety cases that demonstrate the minimisation of unknown unknowns “as low as reasonably practical” at both the architect’s predictive-thinking level and the machine-learning model level.

The current safety and systems engineering approaches appear to be biased towards control thinking. Such bias limits an architect’s lateral thinking, which in turn leads to missing critical scenarios that may seem irrelevant or illogical. This predictive thinking attitude could cause an architect to miss considering rarely concerning situations, such as a moon resembling a traffic light for a Tesla car, or a person walking with a bicycle not being recognised as a person by an Uber autonomous car, potentially leading to catastrophic outcomes and failures. Hence, we consider the architect as part of the problem,

and enhancing their predictive thinking using lateral and vertical thinking helps reduce the uncertainty of such complicated situations.

We identify three core challenges in safety-critical Autonomous-Systems Engineering (ASE):

1. Architects' epistemic uncertainty may lead to oversimplified problem definitions and the emergence of risks in open, complicated environments.
2. Traditional safety processes rarely predict or validate behaviour under Black Swan scenarios.
3. A semantic gap exists between written requirements and pictorial datasets, hindering traceability and uncertainty quantification.

To overcome these challenges, we introduce the AIC (Appreciation–Influence–Control) systems approach and its associated Predictive Architect thinking pipeline, which systematically exposes long-tailed events in the operational environment, guides lateral-thinking perspective shifts to uncover novel hazards, and structures both system-level and ML requirements around predicted Black Swans. We further present “CuneiForm”, a methodology for closing the semantic gap by mapping written safety intents into structured pictorial training syllabi, enabling targeted Black-Swan-driven dataset generation and robustness testing. We also introduce a Black Swan-driven machine learning (ML) training pipeline to demonstrate objectively that, in addition to addressing easy and hard-to-predict hazards, the trained machine is also ready for Black Swan operations.

Through two applied case studies, securing a train track zone in a local residential area from adversarial drone intrusions and enabling the autonomous avoidance of mid-air collisions for a potential autonomous aircraft, we demonstrate that:

- AIC-driven processes uncover previously unrecognised failure modes and validate ML behaviour under Black Swan scenarios.
- Black-Swan-focused ML training and testing measurably enhance safety-case value compared to conventional White-Swan-only approaches.
- CuneiForm pictorial requirements effectively help architects to expose gaps in datasets and demonstrate holistically how safety requirements and the architect's intent are captured in the ML development pictorial datasets.

This work contributes a rigorously defined, repeatable ASE framework that equips architects, regulators, and customers with transparent, Black-Swan-validated assurance of autonomous system trustworthiness.

Table of Contents

Table of Contents	4
Table of Tables	16
Table of Figures	19
List of Accompanying Materials	24
Research Thesis: Declaration of Authorship.....	25
Acknowledgements.....	27
Dedication	28
Definitions and Abbreviations.....	30
Chapter 1 Introduction to the research	31
1.1 Research motivation.....	32
1.2 Summary of contribution	34
1.3 Mapping published papers to thesis Sections	36
1.3.1 Research paper 1: Comprehensive problem articulation.....	36
1.3.2 Research paper 2: Safety and Security-driven system and ML requirements derivation	37
1.3.3 Research paper 3: Black Swan-driven system and ML requirements derivation	37
1.3.4 Research paper 4: Uncertainty graded multi-environment ODD	37
1.3.5 Research paper 5: CuneiForm development for bridging the Semantic Gap 37	
1.4 Research formulation	38
1.4.1 Research methodology.....	38
1.4.1.1 Research problem domain exploration.....	38
1.4.1.2 Problem definition	38
1.4.1.3 Review of existing literature	39
1.4.1.4 Hypothesis generation	39
1.4.1.5 Validation through applied case studies	40
1.4.1.6 Consolidation with industry guidance	40

Table of Contents

1.4.1.7 Consolidation with general systems theory.....	41
1.4.1.8 Consolidation of findings through peer review.....	41
1.4.2 Research problem definition	41
1.4.2.1 ASE problem 1	42
1.4.2.2 ASE problem 2	42
1.4.2.3 ASE problem 3	43
1.4.2.4 Regulator's epistemic uncertainty due to the semantic gap	43
1.4.3 Research questions	44
1.4.3.1 Research question 1	44
1.4.3.2 Research question 2	45
1.4.3.3 Research question 3	45
1.5 Introduction to case studies	45
1.5.1 The unsafe train tracks problem case study	46
1.5.1.1 Problem statement	46
1.5.1.2 Eagle robot system overview.....	46
1.5.2 Aircraft Mid-Air Collision Avoidance for Autonomous Airliners.....	49
1.5.2.1 Background.....	49
1.5.2.2 Problem statement	49
1.5.2.3 Dataset overview	50
Chapter 2 Literature Review	52
2.1 Research Questions 1 & 2 Literature Review	52
2.1.1 General Systems Science Concepts.....	52
2.1.1.1 Smith's AIC General Systems Theory.....	54
2.1.1.2 Black Swan Events.....	55
2.1.1.3 Epistemic Uncertainty and Ignorance.....	56
2.1.2 Holistic Systems Engineering Processes for Automated CPS.....	59
2.1.2.1 INCOSE Systems Engineering Approach.....	59
2.1.2.2 MBSE Methods	60

Table of Contents

2.1.2.3 Requirements Elicitation and Capture Methods	62
2.1.3 Safety and Security Analysis Methods.....	62
2.1.3.1 Safety I: HAZOP Analysis	63
2.1.3.2 Safety I: Event-B	65
2.1.3.3 Safety II: FRAM.....	65
2.1.3.4 Safety II: STAMP (STPA)	66
2.1.3.5 Safety II: SHARCS	66
2.1.4 Academic Research on Holistic Systems Engineering Processes for Autonomous Systems	67
2.1.5 Industrial Holistic Systems Engineering Practice for Autonomous Systems	72
2.1.5.1 SACE	73
2.1.5.2 AMLAS	75
2.1.5.3 SASWG Framework.....	78
2.1.6 Chain-of-Thought (CoT) techniques for large NLP	79
2.1.7 Autonomous Systems Operational Domain Definition	80
2.2 Research Question 3 Literature review	83
2.2.1 The semantic gap between written requirements and pictorial datasets	84
Chapter 3 Examining Existing Safety Processes	87
3.1 Thought experiment: examination question	88
3.2 The novelty test of predicted hazards	89
3.3 Applying the HAZOP method	90
3.3.1 Architect predictive attention distribution of HAZOP types of failures....	91
3.3.2 Application of HAZOP discussion	93
3.3.2.1 Strengths of applying HAZOP for problem articulation	95
3.3.2.2 Weaknesses of applying HAZOP for problem articulation	95
3.4 Applying the STAMP/STPA method.....	97
3.4.1 Application of STAMP discussion	97
3.4.1.1 Strengths of the STAMP Framework.....	97
3.4.1.2 Weaknesses of the STAMP Framework	100

Table of Contents

3.5 Applying the FRAM method	102
3.5.1 Strengths of the FRAM Framework.....	102
3.5.2 Weaknesses of FRAM's application	103
3.6 Reflections from Applying Current Safety Approaches	105
3.6.1 Reflection 3.1: HAZOP performance	105
3.6.2 Reflection 3.2: STAMP performance	106
3.6.3 Reflection 3.3: FRAM performance	106
3.6.4 Reflection 3.4: Evaluating FRAM predictive thinking coverage using AIC107	
3.6.5 Reflection 3.5: Guidewords-based approaches limitation	107
3.7 Answering the exam question	111
Chapter 4 Introducing AIC Systems Approach Key Concepts	112
4.1 AIC Approach to Complicatedness Resolution.....	114
4.1.1 Systemness of a General Complex	117
4.1.2 The Predictive Architect.....	122
4.1.3 Compounded Uncertainty of Open Complicated Problems.....	124
4.1.4 General Lateral Thinking Process	127
4.1.5 Situation and Interaction	130
4.1.6 Source Complex	135
4.1.7 Sink Complex.....	135
4.1.8 Relationship	135
4.1.9 Supra-Complex, Supra-Source and Supra-Sink	137
4.1.10 Supra System Purpose, Primary Purpose, Goal and Action	138
4.1.11 Supra System Primary Purpose (PrimeP).....	140
4.1.12 Appreciation, Influence and Control Key Differentiation:.....	142
4.1.13 AIC-Core thought process	143
4.1.14 Goals and actions have an emergent impact property that induces change	
144	
4.1.15 The intrinsic hazard of appreciation.....	144
4.1.16 White Swans, Black Swans	145

Table of Contents

4.2 Architect's Predictive Thinking Process (ArcPT)	147
4.2.1 Predictive SECoT Structure	149
4.3 CuneiForm: Pictorial Epistemic Uncertainty Reduction	151
4.3.1 CuneiForm: a pictorial-type requirement	152
4.3.1.1 Machine Training Process Concepts	153
4.3.2 Pictorial Hazard	154
4.3.3 CuneiForm Abstraction Process	155
4.4 Predictive Lateral Thinking Technique: Disordered AIC Perspective Shift	155
4.4.1 Perspective Shifts as an Objective Predictive Lateral Thinking Process	157
Chapter 5 Summary of AIC Systems Approach Processes	158
5.1 Overall process description	160
5.2 Stage 1: Uncertainty Problem Articulation and Operational Environment Modelling	163
5.2.1 Identification of problematic Black Swan events	165
5.3 Stage 2: Architect Intent and Autonomous Solution Needs Definition	167
5.4 Stage 3A: HazTOPs and Ordered AIC-Driven Autonomous System Requirements Development	169
5.4.1 AIC-based 4-Whats-&-How refinements	171
5.4.1.1 4-Whats-&-How (4WnH) Process Overview	171
5.5 Stage 3B: Comprehensive Operational Environment Definition	173
5.6 Stage 4: Disordered AIC-Driven Black Swan Scenarios Prediction	173
5.7 Stage 5: CuneiForm-based Syllabus for Safety-Driven ML Development	175
5.8 Stage 6: Black Swan-driven ML Development and Testing pipeline	178
5.8.1 Designing and Validating Robustness during Black Swan operations experiment	179
5.8.1.1 Key Insights	181
5.8.2 Black Swan-driven training pipeline approach	185
5.9 Stage 7: SACE and AMLAS Safety Case Production	189
5.10 SECoT Validation Process	190

Table of Contents

5.10.1 Purpose of validation.....	191
5.10.2 Define the Scope of validation.....	191
5.10.3 Define SECoT output validation criteria.....	192
5.10.3.1 Validation Criteria	192
5.10.3.2 Validation Results Table	192
5.10.4 Define Summary and Next Steps	193
5.10.4.1 Summary of Findings	193
5.10.4.2 Recommended Actions	193
5.10.5 Appendices (Optional).....	193
5.10.6 SECoT Validation Review Process	193
Chapter 6 Unsafe Train Tracks Part A	195
6.1 Introduction	195
6.1.1 A Thought Experiment.....	196
6.2 Stage 1: Uncertainty Problem Articulation and Operational Environment Modelling	197
6.2.1 Predictive Thinking Pipeline 1: Appreciate the Complex of the Problem Complexity Field	198
6.2.2 Predictive Thinking Pipeline 2: Appreciate the Complex of the Problem Complexity Field	200
6.2.3 Predictive Thinking Pipeline 3: Predict the Emergence of AIC Complexity Field for Detailed Operational Scenario Articulation	203
6.2.4 Predictive Thinking Pipeline 4: Predict and Evaluate Problem Domain Factors and Assumptions.	205
6.3 Stage 2: Architect Intent and Autonomous Solution Needs Definition ...	209
6.4 Stage 3A: HazTOPs and Ordered AIC-Driven Autonomous System Requirements Development	211
6.4.1 Predictive Thinking Pipeline 1: Introducing Autonomous systems into Forward-Feed complexity	211
6.4.2 Predictive Thinking Pipeline 2: Designing the affecting Backward-Feed complexity field	213

Table of Contents

6.4.3	Predictive Thinking Pipeline 3: Hazards, Threats and Opportunities	
	Scenarios (HazTOPS) Analysis	218
6.4.3.1	Step 1) Scope the HazTOPS context domain	219
6.4.3.2	Step 2) Characterise the scoped interactions.	220
6.4.3.3	Step 3) Apply potential complications predictive guide words.	222
6.4.4	Predictive Thinking Pipeline 4: Elicitate ordered AIC System-Level	
	Requirements and training requirements	223
6.4.4.1	Step 1) Model the Complex of Interest Operating Scenario Context	
		224
6.4.4.2	Step 2) Model hazards mitigation ordered-AIC complexity field ..	225
6.4.4.3	Step 3) Ordered-AIC-based System or Safety Requirements Derivation	
		227
6.4.4.4	Step 4) Extended Concrete Safe Operating Concept and ML Safety- Training Concept	228
6.5	Stage 3B: Comprehensive Operational Environment Definition.....	231
6.6	Stage 4: Disordered AIC-Driven Black Swan Scenarios Prediction	234
6.6.1	Step 1) Define the interactions	234
6.6.2	Step 2) Define the AIC-Matrix.....	235
6.6.3	Step 3) Perform the Perspective Shift.....	236
6.6.4	Step 4) Predict Harder-to-foresee emergent scenarios (Black Swan scenario)	
	238	
6.6.5	Step 5) Define mitigating ML Development and Safety Requirements...	238
6.7	Reflections on applying SECoT_1 in Stage 1 process.....	240
6.7.1	Reflection 6.1: AIC exposed the nature of problems.....	240
6.7.2	Reflection 6.2: Reduction of epistemic uncertainty could be validated.	241
6.7.3	Reflection 6.3: AIC reveals a Heavy-Tail distribution of complexity.....	243
	6.7.3.1 Connection to Black Swan Theory:	244
	6.7.3.2 Assessing the risk of rarely concerning factors	245
6.7.4	Reflection 6.4: power-law curve fitting as a measure	246
6.7.5	Unsafe Train Tracks Thought Experiment	246

Table of Contents

6.7.6	Reflection 6.5: Application of Action Matrix and Lateral Thinking Process	248
6.8	Applying SECoT_3 in Stage 3A	248
6.8.1	Reflection 6.6: AIC brings lateral thinking to guidewords-approach	248
6.8.2	Reflection 6.7: Safety and Security Concepts in the Context of AIC	250
6.8.2.1	Hard Safety Hazard	251
6.8.2.2	Soft Safety Hazard	251
6.8.2.3	The inherent security threat of appreciation	252
6.9	Applying AIC Perspective Shift (Stage 4)	253
6.9.1	Reflection 6.8: AIC perspective shift helped to predict Black Swan pictorial scenarios	253
6.9.1.1	Evidence of AIC-perspective shift led to predicting missing Black Swans	254
6.9.1.2	Predicting Unknown-Unknowns through Perspective Shifts	255
Chapter 7	Unsafe Train Tracks Case Study B	258
7.1	Stage 5: CuneiForm-based Syllabus for Safety-Driven ML Epistemic Intelligence Development	258
7.1.1	Step A) Articulate the pictorial problem context:	258
7.1.2	Step B) Characterise the Training Classes for CuneiForms	259
7.1.3	Final CuneiForm	262
7.1.4	Develop the Training, Testing and Black Swan Validation Datasets	264
7.1.5	More CuneiForms	266
7.1.5.1	A CuneiForm-based non-typical Black Swan scenario	266
7.1.5.2	A CuneiForm-based typical operations scenarios	269
7.1.5.3	A CuneiForm-based out-of-context example	276
7.2	Stage 6: Black Swan-driven ML Development and Testing	277
7.2.1	Training syllabus configurations	279
7.2.2	Step 1: Design CuneiForms	280
7.2.2.1	In-context Black Swans.	280
7.2.2.2	In-context Typical Operations.	280

Table of Contents

7.2.2.3 Out-of-context CuneiForms (OOC dataset)	281
7.2.3 Step 2: Validate CuneiForm compliance to SOC.....	281
7.2.4 Step 3: Instantiate CuneiForms	281
7.2.4.1 White Swan datasets generation.....	281
7.2.4.2 Black Swan datasets generation.	282
7.2.4.3 Out-of-context dataset generation	282
7.2.5 Step 4: Validate Instantiated images compliance to CuneiForms	283
7.2.6 Step 5: Dataset Similarity Testing	284
7.2.7 Step 6: Develop the White Swan model.....	285
7.2.8 Step 7: Black Swan operations training and testing.....	285
7.2.8.1 Config 1: White Swan Model	286
7.2.8.2 Config 2: Black Swan testing 1	287
7.2.8.3 Config 3: Black Swan training 1	288
7.2.8.4 Config 4: Black Swan testing 2	289
7.2.8.5 Config 5: Black Swan training 2	290
7.2.8.6 Config 6: Testing on all types of data	290
7.2.9 Step 8: Dataset augmentation for adversarial attacks robustness (Config 7)	292
7.2.10 Step 9: Develop final model	293
7.2.11 Step 10: Develop AMLAS Safety Case	293
7.2.12 Dataset Holistic Consolidation	294
7.3 SECoT Validation Report for Unsafe Train Tracks case study.....	295
7.3.1 Purpose of the Report.....	295
7.3.2 Scope of Validation	296
7.3.3 SECoT Output Validation	296
7.3.3.1 Validation Criteria.....	296
7.3.4 Summary and Next Steps.....	298
7.3.4.1 Summary of Findings	298
7.3.4.2 Recommended Actions.....	298

Table of Contents

7.3.5 Appendices (optional)	298
7.3.6 SECoT Validation Review Process	299
7.4 Reflection from Applying CuneiForm-based assurance framework (Stage 5)	299
7.4.1 Reflection 7.1: We managed to bridge the semantic gap using CuneiForms 299	
7.4.2 Reflection 7.2: CuneiForm Pictorial Complexity Field limit	300
7.4.3 Reflection 7.3: Addressing the problem of sufficiency	301
7.4.4 Mid-air Collision problem Thought Experiment	302
7.4.4.1 Reflection 7.4: CuneiForm validation process helped us to discover gaps in existing dataset	302
7.4.4.1.1 Evaluating Cloud Types Coverage	302
7.4.4.1.2 Analysis of Black Swan Event Handling	303
7.4.4.1.3 Pictorial Distance Coverage	303
7.4.4.1.4 TOI Positioning and Horizon Attitude Analysis	303
7.5 Reflections on Black Swan-driven training pipeline	304
7.5.1 Reflection 7.5: Impact of Black Swan Inclusion in Training	304
7.5.2 Reflection 7.6: Limitations of typical operations and out-of-context training 304	
7.5.3 Reflection 7.7: Testing Strategy Implications	304
7.5.4 Reflection 7.8: Role of Stage 4 in Assurance	305
7.5.5 Reflection 7.9: Demonstrating lowering of risks during Black Swans ALARP 305	
7.6 Reflections with regard SECoT Validation Process	307
7.6.1 Reflection 7.10: The SECoT validation report was a useful tool	307
Chapter 8 Safety Case Development	307
8.1 Developing Partial SACE Safety Case	308
8.1.1 SACE Stage 1: Operating Context Assurance	308
8.1.2 SACE Stage 2: Autonomous System Hazardous Scenario Identification	310

Table of Contents

8.1.3	SACE Stage 3: Safe Operating Concept Assurance	311
8.1.4	SACE Stage 4: Safety Requirements Assurance	313
8.1.5	Stage 5: AS Design Assurance	314
8.1.6	Stage 6: Hazardous Failures Management	316
8.2	Developing Partial AMLAS Safety Case	317
8.2.1	AMLAS Stage 1: ML Safety Assurance Scoping.....	317
8.2.2	AMLAS Stage 2: ML Requirements Assurance.....	318
8.2.3	AMLAS Stage 3: Data Management Assurance	319
8.2.4	AMLAS Stage 4: Model Learning Assurance	321
8.3	AMLAS and SACE application reflection.....	323
8.3.1	AIC can enhance AMLAS and SACE	323
8.3.2	Potential Enhancement of AMLAS and SACE	325
8.4	Summary: AIC complementarity with SACE and AMLAS.....	326
8.4.1	Problem-Domain Versus Solution-Domain Starting Points	326
8.4.2	Gap in SACE/AMLAS: Detailed Scenario & Black-Swan Prediction	327
8.4.3	Rich Artefacts & Traceable Evidence	327
8.4.4	Holistic Coverage Across SACE & AMLAS.....	327
Chapter 9	Conclusion.....	328
9.1	Predictive Lateral, Vertical and T-Thinking	329
9.1.1	Fundamental Framework for AIC Balanced Predictive T-Thinking.....	331
9.1.2	Architect Predictive Systems Thinking	332
9.2	SECoT helped us trust our predictions under CUP	334
9.3	PhD research formulation reflections.....	335
9.3.1	Research Question 1	335
9.3.1.1	Solution Methods for RQ1.....	335
9.3.2	Research Question 2	337
9.3.2.1	Solution Methods for RQ2.....	337
9.3.3	Research Question 3	338

Table of Contents

9.3.3.1 Solution Methods for RQ3.....	338
9.4 Future Research Direction	339
9.4.1 Fine-Tuning on Black Swan-driven ML development pipeline	339
9.4.2 AIC-enhanced FRAM process.....	340
9.4.3 White Ghosts, Black Ghosts	340
9.5 In Conclusion	341
Glossary of Terms	343
Bibliography.....	344

Table of Tables

Table 2.1 Arbitrary Training Strategy to train a perception system on the complexity of a typical time period of a day on earth.....	57
Table 2.2 Comparison between Aleatoric and Epistemic.....	59
Table 2.3 HAZOP guidewords examples	64
Table 2.4 State-of-the-art Integrated ODD taxonomy for autonomous motor vehicle.....	81
Table 3.1 Architect's attentive mentions of types of failures we predicted using HAZOP analysis	91
Table 4.1 General SECoT example.....	150
Table 4.2 Semantic bridge between handwritten requirements and pictorial datasets ..	153
Table 5.1 Black Swan prediction with respect to Architect Concern Distribution	167
Table 5.2 4-Whats-&-How process with an example	172
Table 5.3 Experiment characterisation	179
Table 5.4 SECoT validation results table	192
Table 6.1 Actions Matrix example	200
Table 6.2 Problematic interactions classification.....	201
Table 6.3 Design problem selection process	202
Table 6.4 AIC extended scenario for n1 Interaction SECoT definition	204
Table 6.5A Sample of Predicted Factors Output.....	206
Table 6.6 Architect High-Level Solution Prescription	209
Table 6.7 Mapping AIC interactions of the Eagle Drone and adversarial drone behaviours in mitigating train derailment risks	212
Table 6.8 AIC Actions Matrix (Red: influence, green: appreciation, blue: control)	217
Table 6.9 AIC interactions table for Eagle Drone in 6.12 the complexity field	218

Table of Tables

Table 6.10 The table describes the AIC interaction dynamics between Eagle Drones and adversarial drones.....	221
Table 6.11 HazTOPS Analysis of "3 Drones Attack" Scenario with Risk and Surprise Assessment	222
Table 6.12 Adversarial Drone hiding behind fence HazTOPS.....	223
Table 6.13 AIC interactions table	226
Table 6.14 Safety requirements derivations to mitigate the concealed drone problem. .	227
Table 6.15 ML Safety Requirements Derivation	230
Table 6.16 ML dataset requirements derivation	230
Table 6.17 Operational Design Definition for Eagle Robot Deployment in Train Track Zone	232
Table 6.18 Deep AIC factorisation for Eagle Drone and adversarial drone shapes.....	235
Table 6.19 AIC-type perspective shift	237
Table 6.20 Harder-to-foresee Emergent Scenario (black swan scenario)	238
Table 6.21 Eagle Drone Safety-Training Requirements for Black Swan Scenarios	239
Table 6.22 Black Swans driven dataset requirements	239
Table 6.23 Unsafe Train Tracks problem space complexity (long-tailed distribution)	240
Table 9.2 Answering thought experiment tests.....	246
Table 6.25 lists of rare factors associated with the problem	249
Table 6.26 AIC-type perspective shift for unsafe training tracks case study.....	253
Table 7.1 CuneiForm Pictorial situation articulation.....	258
Table 7.2 Characteristic Training Classes definitions for a CuneiForm abstract image ..	259
Table 7.3 CuneiForm-H.36 artefact	262
Table 7.4 Set of Black Swan CuneiForms examples	266
Table 7.5 Safety Training dataset development requirements.....	270
Table 7.6 Typical operations CuneiForm characterisation	271
Table 7.7 Set of typical operations CuneiForms satisfying Table 7.28	273

Table of Tables

Table 7.8 Chosen training and testing syllabus with required performance	278
Table 7.9 Renaming experiments to training syllabus configuration	279
Table 7.10 Black Swan Scenarios Batch A and B CuneiForms.....	280
Table 7.11 Typical operations CuneiForms	280
Table 7.12 Black Swan-driven incremental ML development process to assure model performance under Black Swan operations	285
Table 7.13 Integrating model training activities in AMLAS safety case	293
Table 7.14 SECoT Validation acceptance criteria	297
Table 8.1 AIC systems processes to SACE Stage 1 design process	308
Table 8.2 Mapping AIC systems processes to SACE Stage 1 design process	310
Table 8.3 Mapping AIC systems processes to SACE Stage 1 design process	312
Table 8.4 Mapping AIC systems processes to SACE Stage 4 design process	313
Table 8.5 Mapping AIC systems processes to SACE Stage 5 design process	315
Table 8.6 Mapping AIC systems processes to SACE Stage 5 design process	316
Table 8.7 Mapping AIC systems processes to AMLAS Stage 1 design process	317
Table 8.8 Mapping AIC systems processes to AMLAS Stage 2 design process	318
Table 8.9 Mapping AIC systems processes to AMLAS Stage 3 design process	319
Table 8.10 Mapping AIC systems processes to AMLAS Stage 4 design process	321
Table 8.11 Mapping AIC systems approach to SACE and AMLAS	323

Table of Figures

Figure 1.1 food delivery robot incident (sourced from [2])	33
Figure 1.2 The Eagle Robot, an Eagle Drone and a docking station (Eagle nest) installed on a street lamp post.	47
Figure 1.3 Human operator controlling and operating the Eagle Robot system	48
Figure 1.4 The Eagle Robot in a railway application.	48
Figure 1.5 The Eagle Robot performing an autonomous security patrolling mission	48
Figure 1.6 The Eagle Robot autonomously responding to an accident on the rail tracks ...	49
Figure 1.7 example set of images in the AVOIDDS sample dataset.....	51
Figure 2.1 Reconstructed PASSI Model as described by [68].....	68
Figure 2.2 AI Holistic Systems Engineering Life-cycle process in a continuous loop fashion [69]	69
Figure 2.3 AI development life cycle as described and illustrated by [69]	70
Figure 2.4 Machine Learning Lifecycle model as depicted and described in [71]	72
Figure 2.5 Overview of AMLAS lifecycle process as illustrated by [18]	75
Figure 2.6 AMLAS process in generating ML Epistemic Training and testing data	76
Figure 2.7 Intelligent systems training process as suggested in AMLAS methodology.	77
Figure 2.8 Cuneiform language-based sale contract specification Image reused from [105]85	
Figure 3.1 A snapshot taken from DuckDuckGo web browser maps of a strip of train tracks in Holy Town, Scotland, UK.....	88
Figure 3.2 Problem domain model.....	90
Figure 3.3 Guide words architect predictive attention distribution exposing our predictive thinking bias and focus our attention on White Swan events only. Any deviations outside the guidelines are possible Black Swans. How can we predict them?	92

Table of Figures

Figure 3.4 HAZOP and Formal Methods predictive attention distribution of problem complexity	94
Figure 3.5 Functional cell of FRAM model taken from [16]	109
Figure 3.6 FRAM, STAMP and CHARCS fundamental perspective about the complexity of any problem domain.....	110
Figure 4.1 AIC Systems Approach for Architect, Machine Intelligence and Goal Hierarchy in Autonomous Cyber-Physical Systems (right). AIC model as defined by Smith (left).....	115
Figure 4.2 Types of architect approaches to complexity of in CUP. Compounded Epistemic Uncertainty Problem (CUP).....	125
Figure 4.3 INCOSE definition of emergence (left) [116] and the format of interaction (right)	131
Figure 4.5 The right is an example of an ancient Sumerian sale contract specification. On the left is an example CuneiForm that describes a Black Swan pictorial scenario.	152
Figure 4.5 Predictive Disordered AIC Perspective Shift in Original vs. Emergent Scenarios.	156
Figure 5.1 proposed classification of Safety and Systems Engineering.....	159
Figure 5.2 The overall AIC Systems Approach Framework mapped to AMLAS and SACE	161
Figure 5.3 Stage 1 process model.....	165
Figure 5.4 Stage 3A process model.....	170
Figure 5.5 4WnH thought process	172
Figure 5.6 AIC Hierarchical decomposition of interactions	172
Figure 5.7 Designing for Black Swan operations	175
Figure 5.8 CuneiForm Syllabus development and validation process.....	176
Figure 5.9 Similarity test between training syllabus 9.4 and 10	183
Figure 5.10 Exp.5 reveals that it is possible some OOD scenarios can be mitigated by the diversity of the training and validation set.....	184
Figure 5.11 Black Swan-driven safety-critical ML development pipeline	187
Figure 6.1 A snapshot taken from DuckDuckGo web browser maps of a strip of train tracks in Holy Town, Scotland, UK.....	196

Table of Figures

Figure 6.2 Problem Environment Depiction.....	199
Figure 6.3 Forward-feed partial AIC-SECoT.....	203
Figure 6.4 Modelling AIC scenario from interaction n1	203
Figure 6.5 Architect predictive attention distribution curve of the derived factors for unsafe train tracks problem domain	207
Figure 6.6 AIC general systems rule to model the engineered solution's complete reaction to the problem complexity.....	212
Figure 6.7 Introducing part of the solution into the mix of AIC problem complexity.	212
Figure 6.8 Eagle Robot operational environment.....	214
Figure 6.9 Unsafe Train Tracks Operational Domain context diagram	215
Figure 6.10 Abstract interaction for Eagle Drone denying adversarial drone access.....	215
Figure 6.11 Backward-feed complexity field	216
Figure 6.12 AIC complexity field of the interaction between Eagle Drone and Adversarial Drone	217
Figure 6.13 Complicatedness surrounding Eagle Drone	218
Figure 6.14 Hazards Complexity Field Scope: graphically scoping the hazards within the complexity field by placing hazard icons on target interaction.	219
Figure 6.15 Threats, Complexity Field Scope	219
Figure 6.16 Opportunities Complexity Feels Scope	220
Figure 6.17 Hazards associated with Eagle Drone preventing derailed train complexity field	221
Figure 6.18 Soft Hazard Complexity Field Model for local people's reaction to police decisions, which could lead to an unsafe environment for people and the system	223
Figure 6.19 Problem context definition for concealed adversarial drone	224
Figure 6.20 Refining the concealed adversarial drone problem context.....	225
Figure 6.21 Mitigating the Concealed Drone problem scenario	225
Figure 6.22 Updated abstract relationship.....	226

Table of Figures

Figure 6.23 Example complexity field. TZ stands for track zone.....	234
Figure 6.24 Example relationship of interest to perform the perspective shift	235
Figure 6.25 Extending the interactions with reactions among systems (binary relationships)	235
Figure 6.26 Shifted perspective from influence to control.....	237
Figure 6.27 Architect predictive attention distribution curve of the derived factors in AVOIDDS problem domain complexity field	242
Figure 6.28 Hard and Soft Hazards definition	252
Figure 6.29 Example Black Swan CuneiForm H.37 used to assure the final model performance during the Black Swan operational scenario	254
Figure 7.1 represents the Cuneiform (H.36) characterisation for the Black Swan Scenario Validation.	262
Figure 7.2 Example of a compliant image to CuneiForm in H.36 in Black Swan Validation set	265
Figure 7.3 Example of compliant images to CuneiForm in H.36-41	266
Figure 7.4 CuneiForm ML development strategy based on TOI 3D orientation.	270
Figure 7.5 H.54 Out-of-context CuneiForm of drones	276
Figure 7.6 CuneiForm H.54 Out-of-context depiction of drones.....	281
Figure 7.7 Typical operations of CuneiForms and instantiated images	282
Figure 7.8 Black Swan images generation	282
Figure 7.9 Examples of out-of-context dataset.....	283
Figure 7.10 Validating a Black Swan image to a Black Swan (batch B) CuneiForm	283
Figure 7.11 Config1 training and testing strategy	286
Figure 7.12 Config 2 similarity test result	288
Figure 7.13 Config4 training and testing similarity test result	290
Figure 7.14 Chosen final training and test strategy configuration 6	291
Figure 7.15 CuneiForm consolidation report.....	295

Table of Figures

Figure 9.1 illustrates the structure of the SECoT method.....	333
---	-----

List of Accompanying Materials

- Appendix A: [Definition of the Multi-domain ODD Natural Surrounding Taxonomy](#)
- Appendix B: [ODD Uncertainty Grading for Safety Training of Autonomous Systems](#)
- Appendix C: [Extended Literature Review](#)
- Appendix D: [Safety Validation Report for AVOID Dataset using the CuneiForm method](#)
- Appendix E: [AIC Systems Approach Key Concepts Extended Definitions](#)
- Appendix F: [Detailed Summary of AIC Systems Approach Processes](#)
- Appendix G: [Applying Current SE Methods \(HAZOP, STAMP, FRAM\)](#)
- Appendix H: [Unsafe Train Tracks Problem Detailed Implementation Case Study](#)
- Appendix I: [Mid-air Collision Avoidance Case Study](#)
- Appendix J: [Extended work for the Unsafe Train Tracks Case Study](#)
- Appendix K: [Mid-air Collision Case Study Extended Work](#)
- Appendix L: [AMLAS and SACE Safety Case Construction](#)
- Appendix M: [AVOIDDS Case Study Summary](#)

Research Thesis: Declaration of Authorship

Print name: Haider Al-Shareefy

Title of thesis: Towards an Advancement in Holistic Systems Engineering Method for Machine Learning based Safety Critical Systems

I declare that this thesis and the work presented in it are my own and has been generated by me as the result of my own original research.

I confirm that:

This work was done wholly or mainly while in candidature for a research degree at this University;

Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated;

Where I have consulted the published work of others, this is always clearly attributed;

Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work;

I have acknowledged all main sources of help;

Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself;

None Parts of this work have been published as:

- **Title:** An AIC-based approach for articulating unpredictable problems in open, complex environments: H. Al-Shareefy, M. Butler and S. Hoang, "An AIC-based approach for articulating unpredictable problems in open, complex environments," in 19th European Dependable Computing Conference, Leuven, Belgium, 2024.
- **Title:** AIC approach for intelligent systems requirements elicitation: H. Al-Shareefy, M. Butler and S. Hoang, "AIC Approach for Intelligent Systems Requirements Elicitation," in 7th International Conference on System Reliability and Safety (ICSRS), Bologna, 2023.
- **Title:** AIC-based unforeseen emergent scenarios (Black Swan events) discovery method for autonomous systems safety design: H. Al-Shareefy, M. Butler and T. Hoang, "AIC-based unforeseen emergent scenarios (Black Swan events) discovery method for

Research Thesis: Declaration of Authorship

autonomous systems safety design,” in 68th International Society for Systems Sciences Annual Conference, Washington, USA, 2024.

- **Title:** Comprehensive Multi-environment operational design domain (ODD) taxonomy for intelligent systems safety design: H. Al-Shareefy, M. Butler and T. Hoang, “Comprehensive Multi-environment operational design domain (ODD) taxonomy for intelligent systems safety design,” in 68th International Society for Systems Sciences Annual Conference, Washington, D.C, 2024.
- **Title:** Cuneiform method for assuring the safety of ML-based computer vision development datasets: H. Al-Shareefy, M. Butler and T. S. Hoang, “CuneiForm Method for Assuring the Safety of ML-Based Computer Vision Development Datasets,” in 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW), Reykjavik, Iceland, 2024.

A handwritten signature in black ink, consisting of several loops and a long horizontal stroke extending to the right.

Signature:

..... Date: 21/06/2025

Acknowledgements

Al-Shareefy is supported by a Thales EPSRC iCASE Award. Butler and Hoang are supported by the HD-Sec project, which was funded by the Digital Security by Design (DSbD) Programme delivered by UKRI to support the DSbD ecosystem.

Dedication

This thesis is lovingly dedicated to my wonderful parents, Sundus and Mahmood, whose unwavering support and sacrifices have been my bedrock; without their encouragement, I could never have embarked on or completed this journey.

To my brother, Ahmed, and his wonderful wife, Dr. Maha, thank you for your tireless help in the final week before submission, and to my younger brothers, Mustafa (Fofo) and Muntadher (Minato Sensei), whose wit and humour kept my spirits high even in the darkest moments. Heartfelt thanks also to my sister, whose boundless energy fuelled my determination during the most intense final days.

I am profoundly grateful to my supervisor, Professor Michael Butler, and to Dr. Son Hoang, who tolerated my most adventurous ideas and poetic “strings too hard” attitude; your patience (even when it meant headaches on Fridays before your long-earned weekends) made this work possible.

To my ISSS family, Gary Smith, Professor Michele Friend, Jen Makar, Jen Wilby, and Dr. William Smith, thank you for showing me what it truly means to think in systems. As for Dr William Smith, thank you for inventing AIC theory, which has become my guiding philosophy in systems, so as all future AIC architects. A special smile goes to Dennis, ever-ready to lead me on another systems pub crawl.

In loving memory of my mentor, Mr. Fadhil Al-Marsromy, your profound reflections on reality and systems continue to shape my thinking and fuel my resolve to seek and share truth.

Finally, to Dr. Junaid Akram, Dr. Charlie Faltas, and Andrijas Dusa, your steadfast encouragement and belief in me have made all the difference. your

This work stands upon the shoulders of you all.

Dedication

To my Al-Rabbani Guiding Father, Spirit and Mentor, I dedicate this humble contribution to the advancement of the human mind and soul

Definitions and Abbreviations

Appreciation	Definition
ADS	Automated Driving Systems
AIRR	Architect-Intended Real-World Representation
AMLAS	Assurance of Machine Learning in Autonomous Systems
ArcPM	Architect Predictive Mental Model
ArcPT	Architect Predictive Thinking Process
ArcUC	Architect Epistemic Uncertainty Curve for a given dataset
ASE	Autonomous-Systems Engineering
ATTOL	Autonomous Taxi, Take-Off and Landing
AutoSys	Autonomous System
AVOIDDS	Aircraft Vision-based Object Identification and Detection Dataset System
CPS	Cyber Physical System
CUP	Compounded Epistemic Uncertainty
DoS	Denial of Service
DSL	Domain-Specific Language
ESHA	Environmental Survey Hazard Analysis
FRAM	Functional Resonance Analysis Method
HAZOP	Hazards and Operability
HIMR	Horizon Indication Measurement Ruler
HMI	Human Machine Interface
HSE	Holistic Systems Engineering
InD	In-distribution for similarity between datasets
MBSE	Model-Based Systems Engineering
NHTSA	National Highway Traffic Safety Administration
ODD	Operational Domain Definition
OEDR	Object and Event Detection and Response
OOB	Out-of-Context dataset
OOD	Out-of-distribution
PDF	Probability Density Function
pHash	Perceptual Hash (Python library)
PHI	Pictorial Visible Horizon Attitude Indicator
SASWG	Safety of Autonomous Systems Working Group
SECoT	Systems Engineering Chain of Thought
SE-STPA	Security-Enhanced System Theoretic Process Analysis
SHARCS	Systematic Hierarchical Analysis of Requirements for Critical Systems
STAMP	System-Theoretic Accident Model and Processes
STPA	Systems-Theoretic Process Analysis
TOI	Target Objects of Interest for Detection
V&V	Verification and Validation
Vis&V	Visualise and Validate

Chapter 1 Introduction to the research

This PhD work aimed to establish the theory and merits of the following intelligent systems' trustworthiness principle:

Assuring the trustworthiness of autonomous systems entails establishing a comprehensive safety case that demonstrates confidence in reducing the architect and ML-component's epistemic uncertainty as low as reasonably practical.

However, why should systems safety engineering practice view an architect's intelligence and machine intelligence as a single whole rather than two separate parts?

We explored an answer to this question through systems science, systems thinking, and computer science to create a holistic architecture paradigm that demonstrates the reduction of epistemic uncertainty ALARP for safety-critical autonomous systems in an open and complicated operational environment.

Autonomy, in general, changes the dynamics of the relationship between humans and machines; thus, our definitions of systems and systems thinking need to be updated to accommodate the connotation of human architect and machine intelligence. In this PhD, we use the concept of intelligent machine, the intelligent component of an autonomous cyber-physical system, to mean non-deterministic supervised machine learning-based engineered systems that can be trained to make autonomous decisions in expected and unforeseen scenarios. Engineering unsupervised machine intelligence may benefit from our approach; however, we focused on the supervised type of intelligence in our research.

Humans recognise patterns; intelligent machines, through mathematical analysis, recognise patterns. The human intelligence system does not rely solely on mathematical logic to deduce or induce patterns; various faculties within the system, which we have yet to fully understand, are also involved in pattern recognition. In this research, we explored what remains not thoroughly unexplored in systems engineering practice:

- *How can we comprehensively understand the problem domain as-is and to-be to make reasonable predictions of emergent scenarios when an autonomous system is introduced into the natural world's complexity, thus improving requirements coverage and mitigating missing requirements risk?*
- *How can we demonstrate effectively and with minimal ambiguity that safety requirements are preserved during rare, high-impact scenarios (Black Swans)?*
- *How can we close the semantic gap between how humans define objects in requirements and what machines understand about objects?*

Upon close examination, we identified that the above issues are caused by the degree of epistemic uncertainty in both the architect's intelligence perspective of the problem and how this perspective impacted the resilience of their engineered intelligent machine. Epistemic uncertainty can be understood as the difference between what a predictive observer thinks the reality will be and what the reality becomes, given the completeness of initial base-knowledge about the concerning reality at the present, using some ontological classification. Epistemic uncertainty is reducible through an iterative process of belief-set update (in a Bayesian sense).

Through this work, we come to understand that architects indeed need to comprehend the new dynamic relationship between humans and intelligent machines to assess the adequacy of our holistic systems engineering practices when developing and evaluating the trustworthiness of artificial intelligence (AI), particularly in safety-critical applications. This PhD will explore the trustworthiness of such systems from a systems science and holistic systems engineering point of view.

Thales jointly funded this PhD as part of the iCASE award, providing valuable input to formulate the problem and research objectives. Thales experts provided valuable input to help develop the research problem definition from an industrial perspective.

1.1 Research motivation

The introduction Section highlights some background information that provides context for why ensuring ML is an important topic to pursue. To illustrate our motivation, we will use an example incident in the us concerning an autonomous food delivery robot. In a dynamic urban environment, autonomous delivery robots must operate reliably amidst many unforeseeable and non-standard events. A notable incident in September 2022 illustrated such a challenge when an autonomous food delivery robot, during its service route, encountered a crime scene perimeter established in response to a reported emergency at a local high school. The robot's standard operation protocols led it to navigate underneath the scene's delineating boundary, potentially compromising the integrity of a sensitive area and causing confusion among law enforcement personnel.

This situation was further amplified by the interaction with a bystander, a news reporter, who inadvertently endorsed the robot's action by capturing and disseminating the footage, thereby influencing public perception [1]. This interaction inadvertently led to the journalist assigning a secondary value to the robot's presence, viewing it as a newsworthy element within the crime scene rather than an intruding entity. Figure 1.1 depicts the food delivery robot on the street, accompanied by a police presence and a cordoned-off area, suggesting an unusual situation or emergency.



Figure 1.1 food delivery robot incident (sourced from [2])

This situation incident can be viewed as an observed complexity field involving an autonomous system behaving to manifest its primary purpose of delivering food while interacting with a human bystander who aimed to capture news for a local news agency. In this context, the robot and the cameraman formed an emergent, unintended interaction. The robot and the bystander's primary purposes have cooperated, prompting the human to adopt an emergent, unexpected auxiliary goal to support the robot's mission. Consequently, the news reporter appreciated the robot's action (acquiring an appreciation goal) by allowing the robot's behaviour to proceed through the crime scene (performing an appreciative action).

The earlier interaction demonstrated that the main challenge is the uncertainty and variability of the dynamic, open environments in which these systems operate, rendering it nearly impossible to predict their behaviour with reasonable certainty. This challenge is exacerbated by the relatively unpredictable nature of adaptive systems (such as ML-based). The primary issue in the earlier scenario arises when the robot (with the assistance of a human) is permitted to engage in unethical and unsafe behaviour. Since every part of the crime scene could potentially hold valuable evidence for solving the crime, the crime scene ought to be considered safety-critical. There is a need for an additional layer of intelligence beyond perception to think objectively about such scenarios and scrutinise the behaviour. The primary concern arises when these systems, through interaction with humans or autonomous decision-making, engage in actions that could be deemed unethical or unsafe, such as compromising the integrity of a safety-critical crime scene.

the above situation illustrates the challenges surrounding ML's safety analysis and assurance methods. They are considerably complex, especially regarding certification [3]. The literature showed us that there is a minimal contribution towards assuring safety at the training Stage of ML-based systems; in fact, it remains an open question [4]. ML algorithms are no better than the data sets which train them; thus, data sets directly influence the safety of ML systems [5]. Considering a brief common sense causal analysis, we can conclude that the quality of a training

Stage depends on the quality of the process, from problem definition to gathering datasets for training, which falls under the realm of systems engineering methods [6]. Thus, these questions arise:

How can current systems engineering practices deal with the non-deterministic nature of artificially learning machines with perception and the level of openness we must appreciate in their (artificially learning machines) operational context?

There are no universally agreed methods for verifying and validating ML in safety-critical systems, and existing holistic systems engineering standards focus on deterministic processes, where validation maps test cases to requirements based on the assumption that if a system passes a test, then testers can be certain that the system will always behave as demonstrated in the test [7]. However, ML components do not follow such an assumption; even if an ML component detects a person (for example) in a broad daylight scenario, there is no guarantee it will do the same in all exact scenarios. This situation raises concerns about the regulator's or evaluator's confidence in the adequacy of any ML-component validation processes that ensure requirements are met in real-world applications beyond the design and testing phases.

1.2 Summary of contribution

There are two views of this thesis: The summary view and the detailed view. The chapters present a concise summary, while the appendices provide a detailed technical view. The reader can review the appendices to gain a detailed understanding of the contribution.

AIC stands for Appreciation, Influence, and Control, a systems-theoretic approach to resolving the confusion of hard-to-predict systems in complex operational environments. Based on concepts of systems, uncertainty, and predictability, we underpinned an ontological system to analyse, design, and validate environments that are harder to predict, with a focus on discovering Black Swan scenarios. The main contributions of this thesis are as follows:

1. AIC Systems-Approach Framework (Chapter 4 and Appendix E, Section E.1)

- a. Development of the **Deep AIC-Matrix** and accompanying **Actions Matrix** for systematic resolution of predictive complicatedness. The Matrix utilises AIC colour coding schema to express the type of goal a certain action is intended to manifest. Thus, adding a deeper layer to understanding the hidden, unseen activities when observing the complexity of compounded uncertainty problems. [Section F.1]
- b. Formulation of an **Emergent Relationship Factorisation** process to uncover and quantify hidden dependencies within complex systems. [Section F.2]
- c. AIC modelling schema for open, complicated problem domains. [Section 4.1]

- d. A set of measurable definitions based on theoretical understanding of uncertainty and general systems theories and systems thinking approaches.[Section 4.1]

2. SECoT (Systems Engineering Chain-of-Thought) Methodologies

- a. **SECoT**: A predictive thinking chain of thought processes to resolve the predictability of open, complicated environments and engineering hard to predict systems. [see Sections 4.2, E.6]
- b. **SECoT_1**: A four-Stage predictive thinking pipeline for articulating compounded epistemic-uncertainty problems by (i) appreciating complexity fields, (ii) resolving patterns of complicatedness, (iii) forecasting complexity emergence for detailed scenario design, and (iv) evaluating domain factors and assumptions. [see Sections 5.2, F.3].
- c. **SECoT_2 (HazTOPS)**: A structured scenario-generation technique using hazards, threats, opportunities and affordances, guided by predictive “complicatedness” keywords. [Sections 5.3, F.4]
- d. **SECoT_3**: An ordered, HazTOPS-driven requirements-elicitation process that integrates forward- and backward-feed complexity analyses to derive system-level AIC requirements. [Sections 5.4, F.5]

3. Multi-Purpose Operational Design Domain (ODD) Definition

- a. A six-step method to define, grade, and tabulate ODD uncertainty for both land and air applications, enabling consistent characterization of operational contexts. [Sections 5.5, F.6]

4. Predictive Lateral-Thinking for “Black Swan” Events

- a. **General predictive lateral thinking** is based on the idea that it is acceptable to embrace contradictory interactions in a complicated environment and also expect rarely concerning event with high impact. [Sections 4.1.4, F.6]

This perspective differs from Vertical thinking, which only accepts immediate concerns and logical interactions. Vertical thinking is suitable for normally distributed complexity fields with known unknowns, but not effective in long-tailed distributional complexity fields dominated by Black Swan events.

- b. Introduction of the **Disordered AIC Perspective-Shift** technique to systematically explore non-linear complexity and predict low-probability, high-impact scenarios. A generalised, stepwise “4-Stage” process for Black-Swan scenario prediction and corresponding safety-requirement derivation. [Sections 4.4,5.6, F.7]

5. AIC-Based “4-Whats-&-How” Refinement

- a. A structured approach to refine high-level objectives into actionable tasks and system behaviours.[Section 5.4.1]

6. CuneiForm Framework for Context-Aware Dataset Development

- a. Design **ML Training Concept** based on Safety Concept derived from ordered and disordered AIC analysis.[Section 5.4, 5.6, F.5.4(d), F.8.5]
- b. Design of a **CuneiForm Training Syllabus**, complete with artefact characterisation and implementation steps, for generating pictorial problem contexts derived from safety-driven training concept. [Section 4.3, 5.7, E.7,F.10]
- c. A **CuneiForm Validation Process** (Vis&V) to verify multi-dimensional visual representations, including horizon attitudes, object dynamics, and positioning zones. [Section F.11]

7. Validation and Integration with Safety Assurance Processes

- a. Definition of SECoT output validation criteria and a review process to ensure methodological soundness.[Section 5.10]
- b. Interfacing the AIC Systems-Approach with established safety-assurance frameworks (SACE for autonomous systems, AMLAS for machine-learning components) to support partial safety-case construction.[Section 5.9, Chapter 8, Appendix L].

Together, these contributions provide a comprehensive, AIC-centered toolkit for predictive architecting, encompassing complexity resolution, uncertainty articulation, scenario prediction, requirements elicitation, dataset generation, and validation, which can enhance the safety and robustness of autonomous systems operating in open, complex environments.

1.3 Mapping published papers to thesis Sections

During the PhD, we successfully published papers in conferences and journals. For further detailed traceability between the papers and the thesis topic review Appendix M, Section M.5.2. In this Section, we will describe how each published paper maps to the concepts of our work:

1.3.1 Research paper 1: Comprehensive problem articulation

- **Title:** An AIC-based approach for articulating unpredictable problems in open, complex environments [8]
- **Authors:** Haider Al-Shareefy, Michael Butler, Son Hoang.
- **Conference:** EDCC 2024 - 19th European Dependable Computing Conference, Leuven, Belgium, 8-11 April 2024
- **Underpins the following thesis content:** Section 4.1, F.2, 4.2, E.6, 5.2, F.3, 5.4.1

1.3.2 Research paper 2: Safety and Security-driven system and ML requirements derivation

- **Title:** AIC approach for intelligent systems requirements elicitation [9]
- **Authors:** Haider Al-Shareefy, Michael Butler, Son Hoang.
- **Conference:** 2023 7th International Conference On System Reliability And Safety (ICSRS), Bologna, Italy, 22-24 November 2023
- **Underpins the following thesis content:** 5.4, F.5, 4.1, F.2, 5.3, F.4, 5.4.1

1.3.3 Research paper 3: Black Swan-driven system and ML requirements derivation

- **Title:** AIC-based unforeseen emergent scenarios (Black Swan events) discovery method for autonomous systems safety design [10]
- **Authors:** Haider Al-Shareefy, Michael Butler, Son Hoang.
- **Conference:** 68th International Society for Systems Sciences Annual Conference, Washington, D.C.
- **Underpins the following thesis content:** 4.1.4, F.6, 9.9, 4.4, 5.6, F.7, 5.4, 5.6, F.5.4(d), F.8.5

1.3.4 Research paper 4: Uncertainty graded multi-environment ODD

- **Title:** Comprehensive Multi-environment operational design domain (ODD) taxonomy for intelligent systems safety design [11].
- **Authors:** Haider Al-Shareefy, Michael Butler, Son Hoang.
- **Conference:** 68th International Society for Systems Sciences Annual Conference, Washington, D.C.
- **Underpins the following thesis content:** 5.5, F.6, 4.3, 5.7, E.7, F.10

1.3.5 Research paper 5: CuneiForm development for bridging the Semantic Gap

- **Title:** Cuneiform method for assuring the safety of ML-based computer vision development datasets [12].
- **Authors:** Haider Al-Shareefy, Michael Butler, Son Hoang.
- **Conference:** 2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW, Reykjavik, Iceland).
- **Underpins the following thesis content:** 5.4, 5.6, F.5.4(d), F.8.5,

1.4 Research formulation

This Section outlines the research methodology, identifying key problems and objectives of research. The methodology comprises seven phases: problem domain exploration, problem definition, literature review, hypothesis generation, theoretical validation, experimental validation, and peer review consolidation.

1.4.1 Research methodology

Our research methodology follows a structured scientific approach to ensure rigorous examination and validation of our hypotheses. The steps in our methods are designed to systematically address the research questions and ensure the reliability and relevance of our findings. The approach can be broken down into several key phases:

1.4.1.1 Research problem domain exploration

The initial phase of our research focused on thoroughly exploring the research problem domain. This phase was crucial for gaining a nuanced understanding of the context and boundaries related to the challenges surrounding the trustworthiness of machine learning (ML) algorithms, particularly in safety-critical systems. During this exploration, we examined various elements, including existing literature, current technological advancements, and the potential risks associated with deploying ML in environments where safety was paramount. We also identified key industrial guidance and ethical implications that influenced the trustworthiness of these systems.

By systematically analysing these factors, we aimed to uncover specific vulnerabilities, biases, and intellectually unresolved issues that impact systems engineering processes to manage ML behaviour assurance. This foundational understanding served as the basis for our subsequent steps in tackling the complexities of ensuring the safety and effectiveness of ML-based solutions in high-stakes scenarios.

1.4.1.2 Problem definition

Based on the insights gained during the exploration phase and discussions with Thales' research and development team, we constructed an initial understanding of potential gaps in the current literature that needed to be addressed. This involved articulating specific challenges related to the safety and trustworthiness of ML-based systems, particularly in safety-critical applications. Our problem definition is informed by observed gaps in current solutions and theoretical frameworks, which guide the formulation of targeted research questions. The problem definition

evolved across the lifecycle of this PhD until it was stabilised before the production of research papers.

1.4.1.3 Review of existing literature

We reviewed past work to understand how similar problems have been approached. This review encompasses academic literature and industry reports, providing a comprehensive perspective on both theoretical and practical advancements. The goal was to identify existing solutions, methodologies, and frameworks that have been proposed and to assess their effectiveness in addressing the identified problems. We also attempted to apply existing methods to solve the issues, such as HAZOP [13], STAMP [14], SHARCS [15], and FRAM [16] methods. Then, we evaluated their intellectual rigour to discover rare and impactful Black Swan scenarios. We also included relevant literature related to narrowing the gaps identified, for example:

- General systems theories and systems thinking: (appreciation, influence, control) AIC systems theory, soft systems methodology, Bogdanov tektology, INCOSE systems engineering, Klir's general systems problem solver [17].
- Epistemic and aleatoric uncertainty.
- LLM Chain-of-Thought techniques.
- Ancient pictorial objects' abstraction language (CuneiForm).
- Existing operational design domain definition processes (ODD).

As part of the exploration of the problem and its definition, we systematically applied existing techniques: HAZOP, STAMP and FRAM to solve our chosen case study problem. This hands-on experience enabled us to evaluate the effectiveness of each process in solving open, complex environmental problems with a safety-critical ML component as a solution. We then formulated a hypothesis about a possible extension to the current paradigm that could help architects avoid the risk of missing critical requirements when solving compounded epistemic uncertainty problems.

1.4.1.4 Hypothesis generation

Drawing from the insights gained in previous phases, we generated a hypothesis as a systems-theoretic and systematic process of a holistic systems approach. The approach considers the architect (predictive problem solver), the complexity of the problem and solution systems as a whole, then systematically distils the problem into system-level requirements. The general concept of our solution can be explained in three dimensions:

- Black Swan scenarios are the Achilles' heel of any safety case for a safety-critical autonomous solution in open, complicated scenarios. The prediction of such issues requires the integration of the following design activities:

- Lateral predictive thinking.
- Systematic or Vertical predictive thinking.
- Multifaceted systems approach.
- The architect's predictive thinking capability and epistemic uncertainty are part of any problem/solution which requires assurance through SECoTs.
- The reduction of epistemic risks associated with ML training datasets coverage of the operational domain needs to be demonstrated through the following approaches:
 - Requirements are systematically captured in context-aware datasets (using CuneiForms).
 - Successful handling of Black Swan dedicated testing datasets (unmitigated out-of-distribution testing datasets that are traced to Black Swan CuneiForms).

Our hypothesis aims to offer a comprehensive knowledge acquisition solution that enhances the trustworthiness of ML-based safety-critical systems by addressing identified gaps and leveraging existing knowledge.

1.4.1.5 Validation through applied case studies

One dimension of hypothesis validation is carried out through experimental case studies. These case studies are designed to test the hypothesis in real-world problems or simulated environments, providing empirical evidence of its effectiveness. The validation criterion of our approach entails an answer to the following questions:

1. How effective is the predictive thinking process in supporting the architect to discover harder-to-predict, rare, impactful scenarios (Black Swans)?
2. Can the predictive thought process enable the architect to discover an unmitigated Black Swan dataset?

And we have validated against those criteria.

1.4.1.6 Consolidation with industry guidance

This is another form of validation for the final approach. As part of this research, we also attempted to produce a partial safety case that wraps up the output artefacts in our case studies. We mapped the Stages approach and artefacts to AMLAS [18] and SACE [19], a state-of-the-art autonomous systems safety case pattern. By doing so, we indirectly validated our approach's suitability to meet established industrial guidance in assuring the safety of autonomous systems. We do not claim full compliance, but what we have produced within the time frame of this PhD complies with AMLAS and SACE standards. We also propose improvements and potential extensions of the processes.

1.4.1.7 Consolidation with general systems theory

To further ensure the scientific soundness of our proposed systems engineering approach, we derived its principles from established systems science and thinking frameworks. Specifically, we derived our systems approach from AIC systems theory and the reduction of epistemic uncertainty, applying the SHARCS approach, defining measurable definitions for complexity and complicatedness, along with other systems concepts. Chapter 4 provides the scientific basis for the processes proposed in Chapter 5. The theoretical validation of our approach would be in demonstrating its effectiveness in adapting general systems theory. This involves validating that our proposed approach adheres to established principles of systems theory and that it logically addresses the complexities and dynamics of safety-critical ML systems. For example, we define a formalism of AIC interaction based on the general definition of emergence set out by the INCOSE handbook [20] This step ensures that our hypothesis is grounded in established theoretical frameworks.

1.4.1.8 Consolidation of findings through peer review

Finally, we consolidate our findings by engaging with experts in the field through peer-reviewed conferences. This step involved presenting our research at relevant conferences and submitting papers to academic journals for review. Peer review provides critical feedback, helps refine our approach, and ensures that our findings contribute meaningfully to the field.

1.4.2 Research problem definition

Initially, we set out to examine potential concerns to understand the scope of the problem domain. The concerns were mainly related to the following preconceptions:

1. There is no clear approach to developing a comprehensive training process for machine learning models by designing training data. To our understanding, no universally adopted or standardised process exists for designing and ensuring training data for machine learning models.
2. There is no clear methodology for reducing an architect's uncertainty about the dataset's trustworthiness, arising from the semantic gap between desired emergent intelligent functionality and pictorial data sets.

Throughout the PhD research, the problem was revisited several times, and we redefined the issues in terms of risks that may be encountered during the design of autonomous systems or challenges that are uniquely difficult for autonomous-systems engineering (ASE) processes:

1.4.2.1 ASE problem 1

The risk of solving the wrong or incomplete problem due to the architect's epistemic uncertainty and biased perception of the problem complexity dynamics

If the design process underestimates the complexity of the problem domain, it may lead to oversimplified solutions that overlook critical interdependencies, emergent risks, and unforeseen scenarios. This oversight can compromise the system's reliability, safety, and effectiveness in real-world applications. The tendency to underestimate perceived complexity is closely linked to the architect's predictive thinking process and their level of situational awareness regarding the problem. This awareness is essential for making well-informed engineering decisions and addressing the situation's complexity in a manner that makes the resulting design decisions sufficiently trustworthy.

Such insights are crucial for designing systems engineering methods to achieve reliable and trustworthy performance in complex, real-world environments. As such we recognise the following challenge:

ASE challenge 1: Understanding and predicting “emergence” in a compounded epistemic uncertainty problem involving a hard-to-predict system operating in a hard-to-predict operational environment is challenging for architects.

1.4.2.2 ASE problem 2

The risk of missing unforeseeable Black Swan scenarios

There is a risk that during the requirements engineering process, unforeseen, rare, and impactful events, often inaccurately referred to as “edge cases” [21], may be missed due to their unpredictable nature. These scenarios are particularly challenging to anticipate in the context of autonomous systems operating in complex and evolving environments. Failure to account for such events during system design and requirements engineering poses the risk of system failure in critical situations or unexpected behaviour, compromising safety. This risk is amplified by the complexity and unpredictability of autonomous systems' intelligent components, which can lead to potential vulnerabilities and critical failures.

Given the above articulation, we define a third challenge that is required to be overcome by our intended systems approach:

ASE challenge 2: Establishing confidence that a trained emergent intelligent behaviour will consistently maintain its designed safety qualities during Black Swan scenarios is hard to demonstrate.

1.4.2.3 ASE problem 3

The risk of ineffective management of operational domain epistemic uncertainty in the ML training process due to inherent architect biases

This epistemic coverage of a dataset reveals something about the dataset strategy, which impacts the forged intelligence performance in both foreseeable and unforeseeable scenarios. Due to the constraints of available resources and the inherent variability of operational environments, acquiring a large, high-quality, and uniformly distributed dataset can prove challenging and costly. Consequently, extensive datasets may be susceptible to hidden biases. It is crucial to comprehend the epistemic uncertainty associated with a given dataset to identify the circumstances in which it may fail to support a reliable intelligent model. To do so, the architect will face two main risks:

1. The risk of hidden epistemic uncertainty related to the operational domain in the dataset for a given classification system (the hidden biases in datasets that capture the real world).
2. The semantic gap between requirements and large datasets hinders the risk of inability to trace requirements down to the dataset level.

1.4.2.4 Regulator's epistemic uncertainty due to the semantic gap

Suppose the architect has engineered a large dataset of 213,000 images. The architect must demonstrate, to a regulator, that their dataset faithfully captures the required behaviour in given scenarios. How can the architect show that written requirements are truly captured in those 213,000 images? Mainly by including them in a limited safety case. How can the quality assurance team, a regulator, or a court (in case of a catastrophic failure) check all 213,000 images and validate that requirement are truly captured?

There is a significant risk that machine learning datasets for developing ML-based computer vision systems may need to capture or represent the necessary safety requirements fully. These datasets are crucial in shaping the behaviour and decision-making capabilities of the AI system. If key safety or are not included or properly represented during the dataset development, the resulting system may behave unpredictably in real-world scenarios. This can lead to vulnerabilities, including susceptibility to adversarial attacks, biased decision-making, or unsafe operations, compromising the system's reliability, ethical standards, and user safety.

The main challenge of this risk lies in the following premise:

It is difficult to close the semantic gap [22] [23] between the written description of a requirement (for example, safety) and the infinite possible ways they can appear in a pictorial form.

Given the above articulation, we realise that:

ASE challenge 3: it is very hard to predict, trace, and risk-manage the size of infinite acceptable input scenarios associated with an open, complicated environment interacting with an autonomous system and vice versa.

What do we mean by “***infinite acceptable inputs***”?

For example, consider a perception system run over a neural network, which is tasked with detecting a flying drone; there are infinitely many ways that a flying drone may visually appear as input. Each possible way is classed as an acceptable input. Therefore, systems engineers must specify infinite possible inputs in a traceable manner that cannot be precisely measured.

1.4.3 Research questions

Given the above research problems, we then developed specific questions to tackle the most challenging aspect of each.

1.4.3.1 Research question 1

Assumption 1: comprehensive and objective knowledge acquisition processes can minimise encountering unknown problem complexity.

Given ASE problem 1:

Research question 1: How do we comprehensively appreciate problem and operational complexity domains to reduce the architect’s epistemic uncertainty as low as reasonably practical and predict problem-level Black Swan events?

- **Sub-question:** how can we demonstrate that epistemic uncertainty is effectively reduced at the engineered dataset level?

This also includes unsafe scenario discovery that can handle a variety of types of systems, engineered, social and environmental.

Note: Reducing epistemic uncertainty is qualified by increasing traceable and justified new knowledge (Black Swan events) relative to an initial lack of knowledge about those events. Lack of knowledge means the absence of explicit mentioning and processing to an implementable and influential decision.

1.4.3.2 Research question 2

Given ASE problem 2:

Research question 2: How can we ensure that safety requirements and architect intent will be preserved during Black Swan situations (unforeseeable high-impact scenarios)?

- **Sub-question:** How can we validate designed intelligent behaviour safety properties during Black Swan scenarios?

1.4.3.3 Research question 3

Given the ASE problem 3 and ASE challenge 3: it is very hard to predict, trace, and risk-manage the share size of infinite acceptable input scenarios associated with an open, complicated environment interacting with an autonomous system and vice versa.

Research question 3: How can we trace and validate developed pictorial datasets to written requirements (closing the semantic gap)?

1.5 Introduction to case studies

Note: The case studies are performed in the following:

Unsafe train tracks (Eagle Robot) summary of implementation covered in chapters 6 and 7. While a detailed implementation is covered in Appendix H.

AVOIDDS case study summary covered in Appendix M, while the detailed implementation is covered in Appendix I.

The AMLAS and SACE safety cases implementation summary is covered in chapter 8, and the detailed implementation is covered in Appendix L.

A case study-based research approach enables in-depth, multifaceted exploration of complex issues in a real-life setting [24]. It is a valuable validation and problem-discovery tool for theory-based studies, demonstrating the value of an idea in practice. This Section Introduces a case study approach focusing on the Eagle Robot system and AVOIDDS (aircraft vision-based object identification and detection dataset system) as solutions to safety and security challenges in railway and aviation contexts. The Eagle Robot study addresses threats like trespassing and drone incursions in the UK railway network, emphasising operational challenges, Black Swan scenarios, and the development of machine learning datasets for adversarial drone detection and response. It explores CuneiForm image representations to enhance model training and system adaptability in high-risk zones.

The AVOIDDS study concentrates on refining vision-based collision avoidance for autonomous aircraft. Applying the Appreciation, Influence, and Control (AIC) systems approach and

incorporating CuneiForm images aims to enhance dataset complexity and resilience, ensuring robust performance in dynamic airspace. Both case studies seek to advance safety and reliability in autonomous system training through innovative methodologies and comprehensive datasets.

1.5.1 The unsafe train tracks problem case study

The Eagle Robot case study is based partially on the real-life problem the UK Network Rail company faces. The company oversees the integrity of over 15000 km of rail tracks and 23 train operating companies [25]. Such a complex task generates many complexities across technical, social, economic, and political dimensions. The network has already conducted successful trials on deploying the beyond-visual-line-of-sight project Falcon, a drone-in-box system designed to provide a first response to human intrusion [26]. In a hypothetical scenario, Network Rail would like to enhance the ML perception system's capability to detect adversarial drone intrusion into the train track zone.

1.5.1.1 Problem statement

Trespassing and illegal drone flights around railways are escalating concerns for Network Rail, affecting the UK's railway system's safety, security, and operational stability. These issues disrupt the railway infrastructure and operations, causing delays, damage, and heightened safety risks. With the widespread availability of drones, unauthorised drone flights in sensitive railway areas, especially in high-traffic zones, require immediate intervention. This case study explores a potential solution using an autonomous, intelligent drone system to detect and respond to unauthorized drones within the railway vicinity.

1.5.1.2 Eagle robot system overview

In collaboration with the hypothetical engineering firm, **Xspace Engineering**, **Network Rail** has conceptualised the *Eagle Robot system*, a fully autonomous drone and docking station setup designed for remote monitoring and proactive intervention. The Eagle Robot system includes:

- **Eagle Robot:** a drone (Eagle Drone) housed in a docking station (Eagle Nest) positioned at a secure location along the railway, programmed to conduct patrol missions and respond autonomously to identified intrusions.
- **Remote control capabilities:** while the system can operate autonomously, a remote human operator can oversee and manage drone activity as needed, ensuring effective responses to complex situations.
- **Targeted deployment:** For the pilot phase, the system will focus on Holy Town train station in Scotland, which has been identified as a hotspot for trespassing incidents, including unauthorised drone activity.

Chapter 1

The Eagle Robot system aims to intercept drones through an intelligent detection framework. With a deployable net mechanism, it leverages machine learning (ML) for image recognition and target capture. This approach allows real-time response to drone intrusions, and the system can adapt to emerging threats on the railway.

Figures 1.2 to 1.6 overview the Eagle Robot system and its various applications. Figure 1.2 illustrates the Eagle Robot setup, showcasing the drone and docking station installed on a streetlamp post. It highlights the drone's automated operation as it is released to address specific scenarios. In Figure 1.3, a human operator is depicted controlling and managing the system, emphasising the importance of human oversight and autonomous operations.

Figure 1.4 demonstrates the Eagle Robot's application in a railway setting, where the system is strategically positioned along the tracks to enhance safety and monitoring. Figure 1.5 captures the drone conducting a night-time security patrolling mission over railway tracks, underscoring its capabilities in surveillance during low-visibility conditions. Finally, Figure 1.6 illustrates the Eagle Robot autonomously responding to a train crash accident, showcasing its ability to relay live streaming data back to the ground station for timely response and intervention. Together, these figures highlight the versatility and operational scope of the Eagle Robot drone system in enhancing safety and efficiency in various environments.

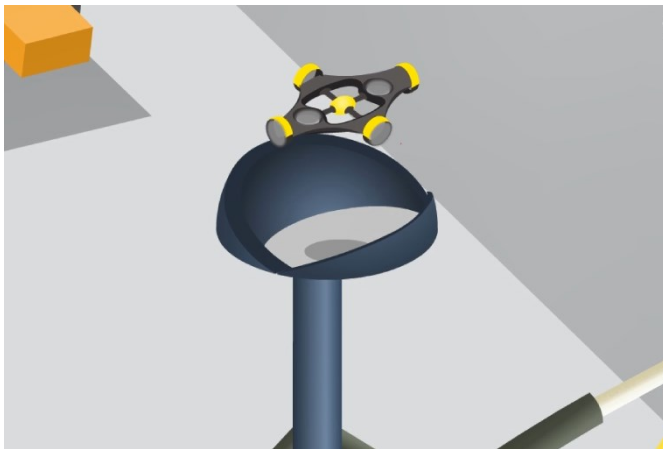


Figure 1.2 The Eagle Robot, an Eagle Drone and a docking station (Eagle nest) installed on a street lamp post.

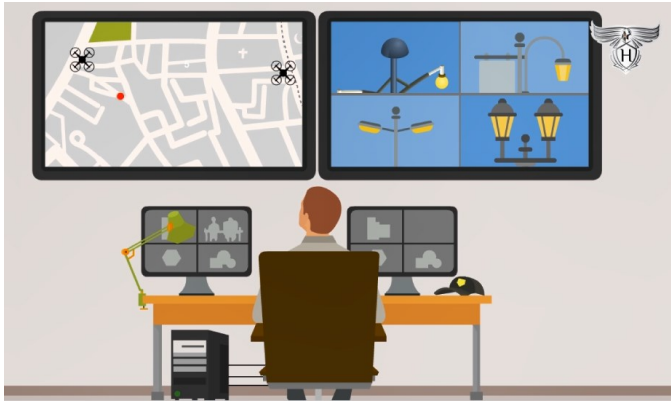


Figure 1.3 Human operator controlling and operating the Eagle Robot system

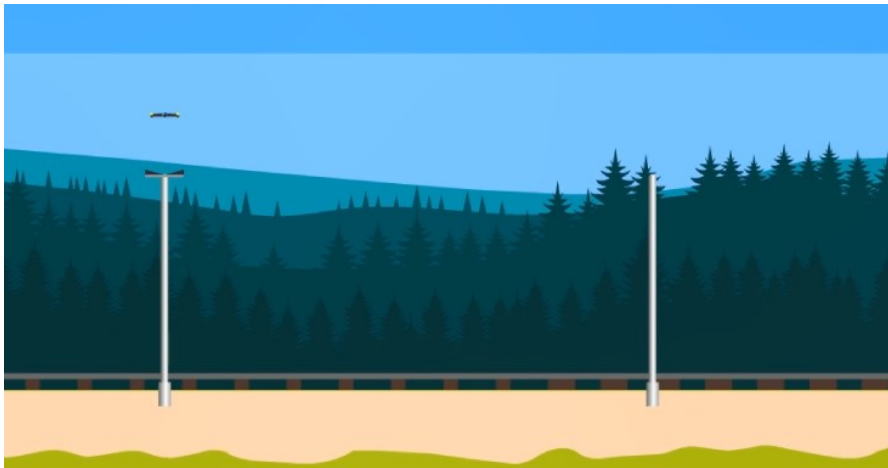


Figure 1.4 The Eagle Robot in a railway application.



Figure 1.5 The Eagle Robot performing an autonomous security patrolling mission

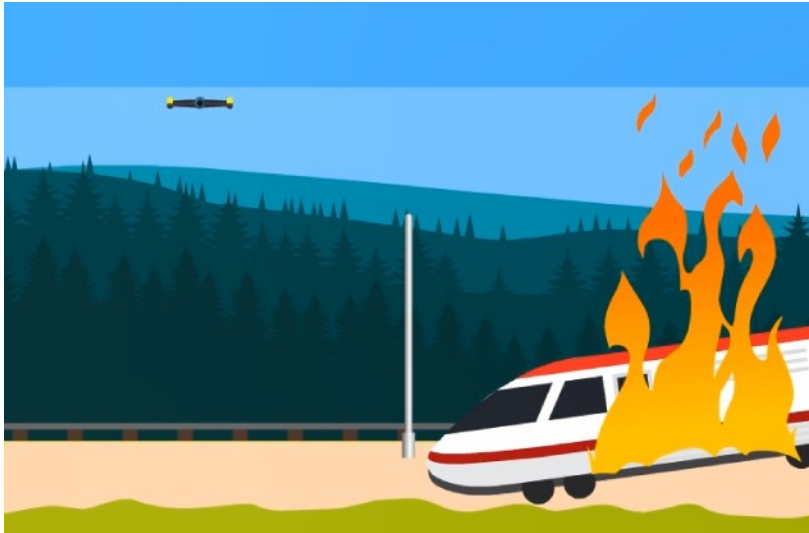


Figure 1.6 The Eagle Robot autonomously responding to an accident on the rail tracks

1.5.2 Aircraft Mid-Air Collision Avoidance for Autonomous Airliners

1.5.2.1 Background

The aviation industry is rapidly developing, and the dream of creating an autonomous airliner has already accelerated, as exemplified by the Airbus Autonomous Taxi, Take-Off, and Landing (ATTOL) project [27]. If such a dream is to become reality, at some point, autonomous airliners will need to upgrade their mid-air collision avoidance perception systems to be autonomous or at least serve as a tool to assist remotely pilots. Aircraft collision avoidance systems are crucial for maintaining safe airspace, primarily by detecting and tracking nearby intruder aircraft to provide timely advisory alerts. In manned aviation, these systems rely on a mix of transponders and onboard radar, often supplemented by the pilot's visual cues. However, additional sensor solutions are essential for autonomous aircraft to ensure robust performance, both as redundancy to existing systems and as replacements for human visual inputs.

In a hypothetical scenario, Airbus approached Xspace Engineering to validate the trustworthiness of the AVOID dataset [28] and produce a dataset validation report and a partial safety case for a potential autonomous mid-air detection of aircraft. In other words, they are interested in assessing the quality of the AVOID dataset to determine whether it can be trusted for training safe perception systems. For this purpose, they require a report to identify any gaps and define the types of safety requirements included in the datasets. The client is satisfied with conducting a quality check on the small sample dataset to ensure it is representative of the entire dataset.

1.5.2.2 Problem statement

Autonomous aircraft need reliable vision-based systems to detect and avoid collisions with other aircraft, especially in uncontrolled airspace. Vision-based sensors, such as intelligent cameras,

have been identified as promising tools to fulfil this need. However, the effectiveness of these systems hinges on extensive, diverse datasets that enable robust image processing and detection algorithms capable of performing reliably under various conditions. Specifically, datasets need to enable trained models to handle Black Swan scenarios demonstrably.

1.5.2.3 Dataset overview

AVOIDDS (Aircraft Vision-Based Object Identification and Detection Dataset System) [28] is a comprehensive dataset designed to advance aircraft detection via vision-based systems. This dataset includes over 72,000 labelled images of intruder aircraft captured across Multiple conditions:

- **Lighting:** varying illumination levels, from daylight to low-light scenarios.
- **Weather conditions:** different weather effects enhance the dataset's utility for training models in real-world scenarios.
- **Relative geometries:** images showcasing aircraft at various distances, angles, and orientations relative to the camera.
- **Geographic locations:** diversity in backgrounds and settings to ensure wide applicability and adaptability of detection algorithms.

The dataset supports the training and benchmarking of models by simulating diverse environmental factors and enhancing generalisation for real-world performance.

We will be trying to answer the following question:

Is AVOIDDS complete and safe as a baseline for perception-based mid-air collision avoidance tasks?

We will aim to evaluate the completeness of the AVOIDDS dataset by assessing the quality of the small sample folder. We assume that the small sample is statistically sound representation of the whole dataset thus any quality issues we discover on the small dataset, is true for the whole dataset.

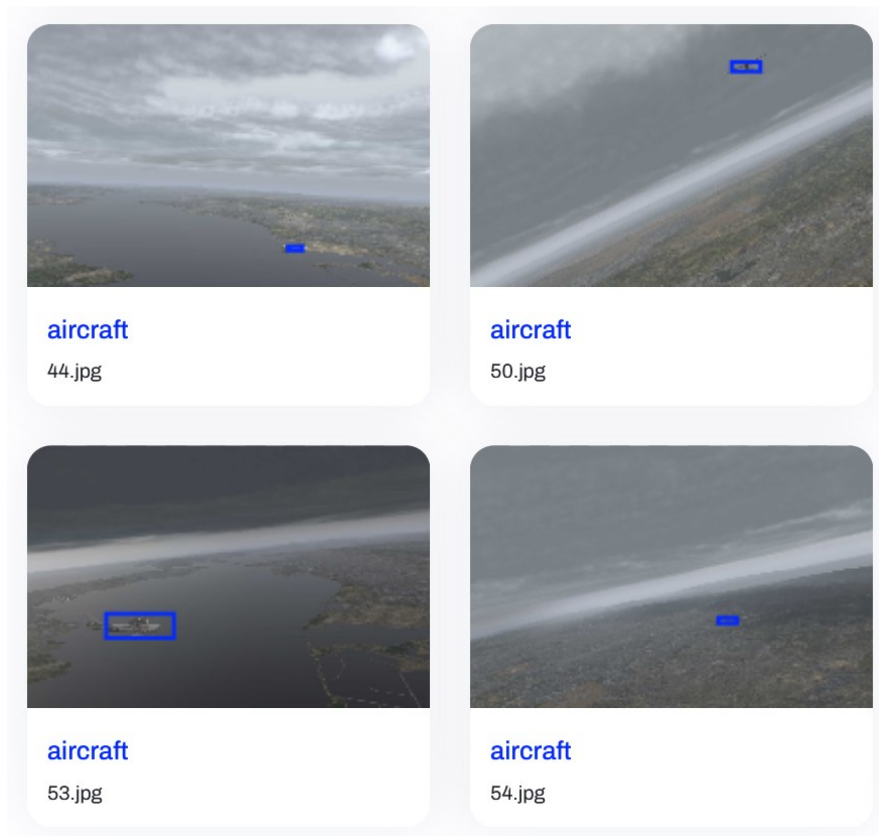


Figure 1.7 example set of images in the AVOIDDS sample dataset

Chapter 2 Literature Review

This Chapter will examine the existing literature concerning systems engineering methods for design knowledge acquisition. Additionally, we will investigate how the body of knowledge addresses the discovery of black swan scenarios within complicated problems involving traditional and autonomous systems. This review aims to understand the methodologies and frameworks for navigating problem complexities and identifying unforeseeable events. This chapter summarises a more comprehensive literature review we included in this PhD which can be found in Appendix C.

2.1 Research Questions 1 & 2 Literature Review

The main concerning topics in research questions 1 and 2 are:

- How does the state-of-the-art handle the problem of accounting for all possible interactions in an open, complicated environment?
- How does the state-of-the-art handle the problem of assuring desired intelligent behaviours in Black Swan events?

With the above concept in mind, we sought relevant literature in systems science and systems engineering that represents the current schools of thought providing solutions to these problems.

2.1.1 General Systems Science Concepts

The interdisciplinary frameworks of systems science and systems thinking provide useful conceptual and methodological tools for addressing complicated problems, including autonomous, cyber-physical, human, and environmental systems [29]. In this thesis, we are mainly concerned with Compounded Epistemic Uncertainty Problems, which are the types of problems where the architect (as problem solver) epistemic uncertainty compounds as more factors and players are added in the problem space. Hence, we saw systems science and systems thinking approaches as a directly relevant basis for any approach to solve such problems.

To refresh the reader's systems literacy, we start with a fundamental concept in systems approaches, the theory of the General Systems Problem Solver (GSPS) [17]. We use this concept to organise our thought process regarding the AIC systems approach as a whole. Klir's definition of GSPS compares it to an expert system assistant designed to help solve complex problems systematically. It focuses on understanding "systems" as a whole (systemhood expertise) instead of just individual components (thinghood expertise). Alexander Bogdanov's Tektology [30],

introduced the concept of “organisation experience” and regards the genesis of systems movement. One core principle which we used in our approach was the concept of “complex” as a general descriptor of any organisation in the natural universe. A complex is not merely a collection of parts, but an organized whole whose coordinated interactions create properties greater than, or less than, the mere sum of their elements.

System boundaries and hierarchical levels, Bogdanov notes, are observer-dependent. A system, subsystem, or element only exists in relation to the way an observer chooses to "carve up" interactions over several frequencies and scales [31] [30]. We adopted this perspective to underpin our assumption about what constitutes a system and what does not. We understood that to have a measurable definition of a system, we cannot exclude the perception of the observer, in this case, the architect. This idea prompted us to delve deeper into how the architect constructs their perception of a complex observation, which led us to appreciate how systems thinking and an awareness of general systems patterns help inform objective architectural engineering judgement regarding predictions of the unknown unknowns. The answer was: enhance mental models to reduce epistemic uncertainty. This means that, whatever complex is observed, the judgment of whether it is a system or not is based on whether the architect's mental model effectively reduces their epistemic uncertainty about the present, past, and future of the observed complex behaviours.

Hence, Bogdanov continues his articulation of the nature of systems as a perspective-based experience that is actively influenced by organisational means; thus, it is not passively received [32]. This also means, as he proclaims, that any act of observation is an organisational decision that influences what is seen, how parts are aggregated, and what dynamics are considered significant. Such a perspective is a profoundly valuable clue to understanding where we should look if we want to reduce errors and missing requirements when dealing with open, complex organisations, such as those that include autonomous systems and an open environment. Bogdanov tell us to look at the source where a complex is judged to be a system or not, and that is the observer's predictive thinking, and for us is the architect and their machine.

Critical systems heuristics by Werner Ulrich [33], and methodologies such as Peter Checkland's Soft Systems Methodology (SSM) [34] also provide a precursor to our approach and inform how we can deal with the reduction of architect epistemic uncertainty. SSM gave us the idea of using icons to represent and model complexity, and Werner inspired us to consider questioning-based enquiry to stimulate coverage about a problem. However, his heuristics are driven solely by the managerial purposes of human organisations. We need a questioning process that is generic enough to predict the future dynamics of any complex system, especially those that include robots, people, the natural environment, and unknown unknowns.

2.1.1.1 Smith's AIC General Systems Theory

Our proposed systems approach is based on applying systems scientific theory known as AIC. AIC stands for Appreciation, Influence and Control and was invented by a systems scientist, William Smith [35]. Originally, it was intended to be a framework for effective leadership and organisations. However, the latest version was generalised as a systems theory defining the cybernetic relationships between humans and their natural environment. The theory states that humans have three types of relationships with any whole they exist in:

- Control – our relationships to the part of the whole we can control.
- Influence – our fluid relationships to those things outside of our control but that we can influence.
- Appreciation – our relationship to all those things that affect our purpose but which we cannot influence or control.

Smith describes those relationships as the forms of powers available to people that they can use to manifest any given purpose. He also defined that purpose itself can be classified into those three types of AIC. The first inception of AIC comes from the need to understand systems, in general, to understand all types of possible factors that need to be considered, which affect what a practitioner is trying to do. From that, Smith defined wholeness as:

“Everything that affects or is affected by what I am trying to do”

From that, Smith defined the System as:

“a set of ordered relationships between a purpose and everything that affects or is affected by that purpose”

When considering Purpose, the practitioner must consider “context”, so a system based on purpose has to understand both purpose and context. From that Smith defined context as a set of three types of relationships to any given purpose:

1. That can be controlled.
2. That can't be controlled but can be influenced.
3. One that can neither be controlled nor influenced but can only be appreciated.

Purpose also can be classified into three types:

1. Goal – a result that can be directly controlled
2. Values – a result that can't be directly controlled, but can be influenced
3. Ideals – a high level mission which neither be controlled nor influenced but can only appreciated.

In this PhD, we will use AIC theory as our Systems Theory basis to develop a Systems Thinking tool that helps Systems Designers organise their mental representation of an observed phenomenon (problem) at any Stage of the systems design process.

2.1.1.2 Black Swan Events

In his seminal work *The Black Swan*, Nassim Nicholas Taleb [36] introduces the concept of Black Swan events, rare, unpredictable occurrences that have disproportionately large impacts and fall outside the realm of regular expectations. These events, often associated with heavy-tailed observed statistical distributions, challenge traditional risk assessment and prediction models, particularly in complex systems. To frame the nature of different statistical environments, Taleb distinguishes between Mediocristan and Extremistan. In Mediocristan, variables tend to follow thin-tailed (e.g., Gaussian) distributions where deviations from the mean are modest and predictable, such as human height or daily temperature.

In contrast, Extremistan is characterised by fat-tailed distributions, where a single extreme event can dominate the total impact. Examples include financial markets, internet traffic, and the popularity of books or videos. For instance, while no single person's height drastically alters the average in Mediocristan, one extremely successful tech start-up can skew the entire market capitalisation distribution in Extremistan. Taleb argues that most modern social, technological, and economic systems reside in Extremistan, rendering conventional modelling tools inadequate. As such, designing for safety and resilience in these domains requires a paradigm shift, one that emphasises robustness and antifragility rather than precise prediction.

But why should we care about Taleb's work?

A recent paper produced by NASA [37] on mitigating Black Swan events in space missions identifies several root causes, undiscovered contributory factors in complex designs, unpredictable operational environments, insufficient tools and processes (e.g., limitations of FMEA), incomplete failure-mode identification, and inadequate safety inspections, that not only imperil mission success but also mirror the challenges faced in developing safe autonomous systems. In open, dynamic contexts, hidden interactions and rare scenarios can elude conventional analyses. Furthermore, the lack of a clear methodology to verify that safety-critical requirements, such as mitigating Black Swan scenarios, are embedded in intelligence development datasets further compounds epistemic uncertainty. By advocating a robust risk-management framework, one underpinned by strategic decision-making and resilient assumptions throughout a program's lifecycle, NASA underscores the need for a holistic, systems-level approach to enhance predictive thinking and proactively address compounded uncertainties. These insights directly inform and validate the objectives of this PhD research.

2.1.1.3 Epistemic Uncertainty and Ignorance

Our approach is concerned with finding a comprehensive methodology to solve Compound Epistemic Uncertainty Problems (CUPs) (problems that involve autonomous systems, people, traditionally engineered systems, open complicated environments, etc.). One of the initial efforts we had to make was to understand uncertainty in general and the nature of the problem. This Section laid out the foundation for articulating the distinction between lateral and Vertical thinking approaches in Section 4.1. Hence the relevance.

In their work, Moeller et al. [38] present a decision-theoretic framework for defining safety applicable across diverse systems and domains. They conceptualise safety as the proactive effort to reduce or minimise both risk and epistemic uncertainty linked to undesirable outcomes, particularly those severe enough to be deemed harmful.

Lack of knowledge, also referred to as ignorance, highlights a system or decision-maker's limitations in fully understanding or capturing a domain's complexities [39]. This concept becomes critical in fields like machine learning and probabilistic modelling, where knowledge gaps can significantly impact the reliability and predictability of models. Ignorance is closely tied to epistemic uncertainty, which stems from incomplete or imperfect knowledge about the world [39], as opposed to aleatoric uncertainty, which arises from inherent randomness or stochasticity in data [40].

Our research aimed to understand the nature of failure modes in machine learning epistemic training strategies due to architectural misconceptions or an incomplete understanding of real-world complexity. We suspected a direct dependency between the architect's epistemic uncertainty (lack of knowledge or ignorance) and the machine's aleatoric uncertainty (uncertainties related to accurate predictions), due to a lack of coverage in the datasets (which is a form of epistemic uncertainty on the machine's side). The central objective of any ML training strategy is to mitigate the residual risk associated with ignorance, or more specifically, a lack of knowledge about the operational domain in which the system will operate. The literature conceptualises "ignorance" as a reducible epistemic uncertainty related to a trained agent's knowledge deficiencies [39]. We also consider the architect as an agent. We utilise the concept of ignorance to gain a deeper understanding of the machine learning training strategy.

When we mention an ML Epistemic Training strategy, we mean the following:

1. The architect must comprehend the real-world complexity field as completely and accurately as practicably possible. Thus, the problem and solution domains must be

modelled, and the architect must choose scenarios to include in training the ML component.

2. Architect's choice of how to represent the real-world complexity (basis of assumptions) in terms of:

- Types of categories (training classes). For example, subdividing the types of pictorial distance of a target object for detection.
- The granularity of the categorical training classes system (number of training modules (scenarios)). For example, how granular the class system is to define how far an object is from the perceptive model.

3. Architects make engineering judgements about two main aspects of training:

- Nature of the examples' diversity. For example, out-of-context examples or context-aware examples.
- The architect's attentive mentions (number of examples) of each category captured in the dataset. For example, 100 times of a far object, 80 times of a closer object.

Both factors directly impact the assurance of the trained model during Black Swan operations.

Also, from an assurance perspective (by assurance we mean mapping desired functions to actual subsystem engineering implementation), substantiable rigour in the training strategy requirements helps traceable validation. It draws attention to certain layers of detail that could lead to improved quality. Thus, it enhances the fidelity of safety cases (as a true representation of the engineered systems) for safety-critical applications.

To this end, we investigated the literature on epistemic uncertainty, specifically the concepts of “lack of knowledge” or “ignorance”. To help us make sense of the concepts, we will use the following arbitrary example perception training strategy for a typical time period of a day in the real-world:

Table 2.1 Arbitrary Training Strategy to train a perception system on the complexity of a typical time period of a day on earth

Training class	Class examples count strategy
Night	168

Morning	104
Afternoon	64
Early Morning (Dawn)	40
Mid-Morning	24
Midday (Noon)	16
Late Afternoon	8
Evening (Dusk)	8
Total	432

Table 2.1 demonstrates the architect's knowledge and interpretation of the real world, reflected in the trained model's understanding of it. A reliable and robust representation of sources of uncertainty is crucial in machine learning, particularly within safety-critical domains such as medicine. This aspect should be regarded as a fundamental characteristic of any machine learning methodology [41] [42]. Each training class represents epistemic uncertainty for the architect, which is translated into a source of epistemic uncertainty for the trainee machine. Therefore, any ignorance in the architect's epistemic uncertainty will lead to ignorance in the training strategy, which leads to ignorance in the trainee machine. Hence, we needed to investigate further the concept of uncertainty to understand how it can be reflected in the safety development of the training process for ML components.

The following definitions are taken from the following reference [43], and recontextualised in the case of developing training datasets:

Aleatoric Uncertainty:

More information **cannot reduce** Aleatoric uncertainty. It arises from randomness or inherent variability in the system being studied. For example, Weather conditions during "Night" in the real world may inherently vary, with no fixed pattern, making it impossible to predict with certainty whether it will rain, snow, or remain clear on any given night. No matter how many images you collect for "Night", this randomness remains.

Epistemic Uncertainty

Epistemic uncertainty refers to uncertainty that **can be reduced** by collecting more data or improving the model. It arises due to a lack of knowledge about the system, such as incomplete data or incorrect model assumptions. For example, the "Evening" training class has only 8

examples. This small sample size introduces epistemic uncertainty because it may lack sufficient data to capture real-world conditions fully. Collecting more images for "Evening" could reduce this uncertainty. Categories with fewer instances, such as "Late Afternoon" (8 training examples) and "Evening" (8 training examples), signal higher epistemic uncertainty being introduced by the training architect. Increasing the number of images for these categories could reduce this uncertainty for the machine, leading to a more robust training regime. The machine's epistemic uncertainty is entirely dependent on the architect's epistemic uncertainty about the real-world complexity and their chosen assumptions. Table 2.2 summarises the difference between the two types of uncertainties:

Table 2.2 Comparison between Aleatoric and Epistemic

Aspect	Aleatoric Uncertainty	Epistemic Uncertainty
Cause	Inherent randomness or similarity in data points	Lack of type of examples or features.
Can it be reduced?	No, collecting more data won't help. More data, more noise.	Yes, more data or more granular features can reduce it. More data, less ignorance.
Example	"Morning" and "Afternoon" images overlap due to similar lighting.	A smaller number of images for "Evening" creates knowledge gaps.

Lack of knowledge / Epistemic Ignorance is the main cause of epistemic uncertainty [44]. The literature presents various methods for quantifying the following. This is an important topic as it relates to the nature of failure in datasets.

2.1.2 Holistic Systems Engineering Processes for Automated CPS

The literature on methods to engineer and ensure the safety of automated systems' compliance with requirements is vast [45]. Various life cycle models have been developed and applied to automated engineering systems. The top-down approach in Systems Engineering processes has evolved since its inception in Bell Labs [46]. The Software Engineering discipline introduced several various methods, such as Waterfall [47] Spiral [48] Vee and its widely adopted variation "Vee Model XT" [49].

Furthermore, there are a great number of standards that directly deal with the application of SE in the context of traditional engineering projects; a list can be obtained from the Systems Engineering Body of Knowledge (SEBok) [9]. However, all the practices recalled share a common theme: They are methods that assume a high level of determinism in engineered systems; thus, they may not be adequate with non-deterministic systems such as ML.

2.1.2.1 INCOSE Systems Engineering Approach

The International Council for Systems Engineering (INCOSE) established a generic industry-independent Holistic Systems Engineering practice. The SE model produced in their Handbook

[50] is based on interpreting the Systems and Software Engineering – System life cycle processes ISO15288:2015 standards. The handbook attempts to contextualise the standards to be more Holistic Systems Engineering specific by specifying some high-level elaboration for methods and definitions specified in the standards. One can consider the INCOSE handbook as general guidance on what to do from a project management perspective. It does not tell you how to solve the problem, but encourages using some systems thinking techniques like soft systems. The latest version of the handbook does not mention anything about Black Swans and how to handle their prediction.

2.1.2.2 MBSE Methods

As cyber-physical systems grew in complexity, text and schematic-based representing Systems Engineering activities added negatively to the overall complexity of projects, especially concerning the communication of design artefacts with various stakeholders [51]. Consequently, a move towards more graphical-based intuitive ontologies such as UML emerged to provide an alternative paradigm for software specification, design, and V&V documentation. With the advent of UML in software engineering communities, Systems Engineers saw that UML is software-oriented and thus not necessarily generic enough to cover SE data outside the code specification, hence the emergence of SysML [52]. Overall, MBSE methods represent knowledge graphically instead of deriving new knowledge.

Systems Diagramming

The concept of diagramming for design conceptualisation is extremely ubiquitous among all kind of engineering design disciplines. How people think relates directly to how they use and view diagrams [53]. Some people prefer sequentially linear flows of steps showing clear causality, and others like to see more complicated presentations that care more about the big picture. Diagramming allows for effective communication among people, especially when it comes to critical systems for engineering safety because a simple miscommunication can lead to serious disasters. Therefore, diagramming in engineering has become an assistive means of reducing human error when appreciating a piece of communication. Mental Models, thus, are used across various disciplines to get the intended point across and enhance the alignment of Mental Models among communicators [54].

The Diagramming Cycle includes three important phases: Creativity, Connectivity and Communication [55]. This cycle can be applied in different situations where modellers need to make sense of the faced complexity. Below is a list of diagramming techniques as proposed by Hudson:

- To enable deeper understanding, diagrams can be developed during the creativity of the diagramming cycle; Context Diagrams are developed to define the high-level

representation of system boundaries that directly influence the system of interest [56]. Soft Systems Methodology by Checkland introduced the concept of Rich Pictures, which expands on context diagrams to include all possible interactions in the problem domain that may directly or indirectly influence the system of interest [34]. One of the main philosophies in implementing rich pictures is not to seek the problem-solution paradigm but to seek the situational appreciation paradigm.

- To study system connectivity, we can use relationship diagrams, influence diagrams, or system maps. These help examine the complexity of a system's connections. Causal Diagrams provide similar functionality as influence diagrams with the added notion of positive and negative feedback loops.

Diagramming techniques indirectly affect the trustworthiness of systems design. They can be viewed as evidence that the design team considered a variety of techniques in analysing the problem or design solution.

Formal Methods

Formal methods refer to techniques that utilise mathematical constructs to specify properties precisely. Thus, they offer a comprehensive process to describe, identify, develop and verify systems systematically. A method is considered formal when it has a sound mathematical formulation, which is usually dictated by the chosen formal language. This format allows developers to utilise tools that automatically verify desired system behaviours concerning a set of assumptions. Consequently, it provides a means to prove that the system is correct by construction without the need to run it [57]. Formal Specification languages provide the medium through which formal modelling can be realised. The main components of formal languages constitute Syntax (set of symbols) and semantics (set of data classes, states, events, transitions and state machines).

SysML

Systems Modelling Language (SysML) is a multipurpose modelling ontology based on graphical representations of concepts and relations [52]. SysML notation includes concepts similar to UML, with the difference being the purpose of the diagramming paradigm. UML is intended to model software programs, while SysML is intended to model systems in general, where software may be part.

MBSE ontologies and methods are practical techniques with a great history of applicability. However, they rely solely on the thinking strategy of the modeller. They can present and simulate system models and encourage the modeller to think about the problem differently (from different views). Alas, they all share the same Vertical thinking approach to problem solving. We learned

from our research that to solve Compounded Uncertainty Problems, we need both Vertical and lateral thinking. The latter requires a different approach to modelling.

2.1.2.3 Requirements Elicitation and Capture Methods

At the heart of any Systems Engineering Process is the management of information. Although the term Requirements Engineering is commonly understood to be top left prior for system design, it is broader in practice. Requirements, in general, are a list of required things to be made available for someone or something so that the end receiver can perform a particular activity. From such a broad view of requirements, the reader can appreciate that “requirements” are not only a “customer” thing; they are for everyone at every level of SE practice involved in the project. Some levels require more rigour than others, for example, capturing the full intention of the customer, or some may be captured on the fly, like when a domain expert meets with the system engineer in the office kitchen area and requests a particular input [58].

Requirement elicitation employs various tools and techniques, each with unique strengths, limitations, and suitable contexts. These methods cater to diverse stakeholder groups and project sizes, facilitating practical requirements gathering in software development. Appendix C includes a summary of techniques taken from [59].

However, most techniques suffer from one significant risk: bias and a lack of accurate knowledge. The techniques can be viewed as different attempts to mitigate the risk. We believe that more needs to be done for ML-based safety-critical systems.

2.1.3 Safety and Security Analysis Methods

There are numerous methods and techniques with regard the safety assurance of automated [60] [61]. Each technique has its strengths and weaknesses. Leveson [14] argues that traditional assumptions made by the current Safety paradigm are not completely fit for purpose in the face of the exponential pace of technological development. FMEA (Failure Modes and Effects Analysis) and FTA (Fault Tree Analysis), for example, given the increased complexity of socio-technical states of affairs, are no longer adequate to account for hazards beyond physical parts' failure. The argument becomes even more compelling when we consider fully Autonomous machines. The author examined current Safety assumptions and made complete arguments for those assumptions to be replaced with new ones. Leveson explained the difference between legacy safety culture and her proposed shift. Now we will be using Levelson's comparison to highlight the legacy Safety I culture, and current Safety II culture, with the intent to highlight a potential proposal for Safety III related to safety in designing Complex Adaptive Systems through the lens of AIC and the confused predictive architect.

Analysing Leveson's arguments, we can deduce a systemic pattern emerging about the legacy safety analysis methods: the absence of the consideration of appreciative aspects related to real-world systems dynamics, especially at the system context level. This also includes how the engineered system may affect other systems. In the real world, complicated systems contain concrete (tangible) and appreciative (social, intangible) elements. In other words, legacy approaches in safety emerged to respond to hard technical, quantifiable hazards, which directly affect the engineered analogue subparts. Still, they fell short in considering the soft context in which the system of interest operates, which includes humans and the unintended consequences.

One of the most essential categories determining the quality of safety and security tools (and systems engineering processes) is their ability to identify unknown hazards. Kritzinger [60] emphasised that the main problem is not the identified hazards but rather the “unidentified hazards”. Once you identify a hazard, you can plan its mitigation; unknown hazards are the surprises that no design team wishes to encounter. Based on the above articulation, we categorise the prevailing safety schools of thought into two distinct classifications based on the work done by Ham [62]:

- **Safety I:** Where the failure problem is assumed to be like a domino puzzle. HAZOP [13], and Formal Methods (such Event-B) are iconic approaches for such a paradigm.
- **Safety II:** Where the failure problem is assumed to be a more nuanced, multifaceted problem, where no single factor describes its causality. STAMP/STPA based approaches are examples of this paradigm (SE-STPA, SHARCS [15]). Also, FRAM [16] is the leading approach in this category of safety engineering.

It is essential to recognise that systems and safety engineering approaches are not in competition regarding superiority. Each technique has advantages and disadvantages depending on the type of problem and project requirements. Below is a brief description of some of the Safety Analysis methods and techniques:

2.1.3.1 Safety I: HAZOP Analysis

Hazard and Operability (HAZOP) study [13] systematically evaluates a system operation. The process starts with the expectation that the design process has generated enough context information about the system, such as:

1. Defined design intentions.
2. Detailed design.
3. Operating procedures.
4. Material specification.
5. Legacy hazard studies.

Chapter 2

Before starting a HAZOP study, the following preconditions need to be satisfied:

1. Design intentions are provided
2. The design must be firmly based lined
3. System detailed components diagrams are provided

The purpose of HAZOP study is to:

- Systematically uncover residual operational hazards, including those with potential environmental ramifications, that were not detected during the preliminary design phase
- Diagnose critical operability and quality issues stemming from system reliability and performance, and assess their implications for unplanned downtime, equipment damage, product nonconformity (e.g., loss, spoilage), and associated turnaround and disposal costs.

The success of the HAZOP study is directly influenced by:

1. The level of accuracy and detail of the information available.
2. The level of expertise and experience of the team.
3. The way the study is conducted.
4. How systematic, creative, and imaginative the examination can be.

Generally speaking, HAZOP studies focus on hazards predominately related to Hard Systems. Hazards are then captured by investigating how system design may deviate from design intentions. The process is entirely based on team members' perception of the system, its context and the level of expertise involved. It assumes that the design process has produced a well-founded design artefact and has evaluated the problem thoroughly. This fact poses a question:

How do we know that the team's perception of the context in which the system is operating is properly captured and understood?

HAZOP is often described as guideword-driven. One of the techniques involved in the process is applying a list of recommended guidewords to consider as a brainstorming stimulation. The list represents a definition of completeness. We consider those guidewords as predictive keywords to enable the architect to resolve the confusion of completeness. The following is a list of guidewords and their meanings [63]:

Table 2.3 HAZOP guidewords examples

Guideword	Meaning	Example
No	None of the design intent is achieved.	A water pump fails to start, resulting in no water flow through the system.
More	Quantitative increase in a parameter.	Excessive pressure builds up in a pipeline due to a malfunctioning valve.

Less	Quantitative decrease in a parameter.	A chemical dosing pump delivers less reagent than required.
-------------	---------------------------------------	---

2.1.3.2 Safety I: Event-B

Event-B enables rigorous mathematical modelling and verification of systems to ensure correctness and reliability [64]. A state-based formal method rooted in set theory and refinement supports incremental system development through abstraction and proof-based verification [65]. Its application in safety-critical systems is well-documented, particularly for identifying and mitigating hazards early in the design phase [15]. For instance, Event-B's refinement mechanism allows engineers to decompose high-level safety requirements into verifiable system properties, ensuring traceability between safety goals and implementation. While Event-B excels in precision, complicated open problems where the precision of interaction is unlikely render the approach less effective. We view Event-B as an enabling ontology to examine the consistency of AIC models across the design Stages. However, some form of adaptation may be required to achieve a synergy of practice.

2.1.3.3 Safety II: FRAM

The Functional Resonance Analysis Method (FRAM) [16] is a safety and performance analysis tool designed to understand how complicated socio-technical systems behave in dynamic and unpredictable environments. Rather than focusing solely on failures or breakdowns, FRAM aims to explore how variability in everyday operations might resonate and lead to both positive and negative outcomes. This method is beneficial for examining how small changes in individual system components can interact to produce unpredictable or emergent behaviours in the entire system.

In FRAM, the goal is to identify potential failure points and understand how regular variability in human actions, technology, and organisational processes can lead to safe and unsafe outcomes. By representing the system as an interconnected function, we can assess how these functions vary and how their interactions might cause functional resonance, where small variabilities in different functions combine in ways that lead to either desired or undesired results.

In Chapter 3, we applied the process to examine its advantages and disadvantages as a systems engineering approach. The process appears to handle the discover of hard-to-predict interactions with complexity better than STAMP and HAZOP due to the flexibility of functional resonance. However, since it is what we label as a control-theory approach to complexity, it misses more subtle, nuanced interactions where control is not manifested.

2.1.3.4 Safety II: STAMP (STPA)

Leveson [14] proposed a paradigm for safety based on Systems Theory known as the Systems-Theoretic Accident Model and Processes (STAMP) model and the Systems-Theoretic Process Analysis (STPA). STAMP assumes that variability accidents arise from inadequate control of complicated systems rather than simply failures of individual components or human errors.

The traditional event-chain models, which focus on failure events and causality, often fall short when dealing with complicated systems that involve non-linear interactions, software-driven processes, and human-automation interfaces. STAMP, in contrast, recognises that accidents can occur not only due to component failures but also due to flawed system design, inadequate control actions, and poor communication among system elements. STAMP is built on three key concepts:

1. **Constraints** – Accidents happen when system components fail to enforce necessary safety constraints.
2. **Control structures** – Safety is an emergent property of a system that depends on the interactions and feedback mechanisms within control structures.
3. **Process models** – Decision-makers operate based on models of the system they control. Inadequate or inaccurate models can lead to unsafe decisions.

Within STAMP, STPA (System-Theoretic Process Analysis) is a technique that applies the principles of STAMP to identify unsafe interactions, determine potential causes of accidents, and recommend safety improvements. STPA can prevent human life loss and address security, privacy, and system performance.

2.1.3.5 Safety II: SHARCS

SHARCS stands for Systematic Hierarchical Analysis of Requirements for Critical Systems [66]. It is a system requirements elicitation technique that combines STPA and formal methods to specify the control architecture of critical safety systems. The technique exploits STPA and Event-B formal language features to ensure rigorous requirements derivation from high-level system purposes to low-level components and to discover hazards systematically and unambiguously. SHARCS can be viewed as a generalised and improved method of SE-STPA [67] in the sense that SE-STPA is a pilot version of the SHARCS paradigm in the context of safety and security analysis. Both methods are iterative. However, the main difference is that SHARCS takes a relatively incremental, abstract-driven approach to managing modelling informational complexity. At the same time, SE-STPA attempts to produce a comprehensive complicated model in every iteration.

The main drawback of the latter approach (SE-STPA) lies in the fact that the holistic modelling experience can get complicated faster, thus challenging the explainability of design items (which in turn impacts the effectiveness of traceability of the output design artefacts, in other words, how easy to trace a design decision back to purpose), significantly when systems increase in size and complexity. However, SHARCS is still in the development process and requires further maturity, especially in terms of clarity on what system the developer should ask themselves to use to instantiate the intended process paradigm in every step correctly.

2.1.4 Academic Research on Holistic Systems Engineering Processes for Autonomous Systems

This Section will investigate existing literature regarding the current system lifecycles for Autonomous Systems. It will also examine the various safety and security analysis techniques used in the trustworthiness models. Starting from generic Holistic Systems Engineering approaches, we discuss specific strategies for intelligent CPS. It will investigate how various models ensure safety requirements encoded in training and test datasets for ML-based systems.

Cossentino [68] defined a lifecycle process for designing and developing intelligent multi-agent societies using UML from requirements to code. The methodology addresses the fidelity issue between models and smart systems. The process includes five distinctive activities: System Requirements, Agent Society, Agent Implementation, Code, and Deployment. The process is intended to be an iterative process.

The flow diagram in Figure 2.3 represents a lifecycle methodology for designing and developing intelligent multi-agent societies, as defined in Cossentino's PASSI model. This approach utilises Unified Modelling Language (UML) to ensure traceability and consistency from initial requirements analysis to code development and deployment. The methodology emphasises iterative refinement to adapt to system changes and improve outputs. The process does not deviate enough to manage confidence in assuring emergent behaviours for a safety case. The only unique aspect of the process is the concept of the “Agent Society mode and Society test”. The Agent Society Model outlines the ontology description, which sets a shared vocabulary for agent communication, essential for multi-agent interactions. It also includes Role Descriptions that clarify each agent's responsibilities and Protocol Descriptions that define the interaction rules and communication protocols governing agents. The Society Test evaluates the interactions and performance of the entire multi-agent system as a cohesive unit. However, the author does not delve deeper into a description of how we design such rules and how we validate their expected deviation from these rules. Agents are expected to deviate from control rules at some point.

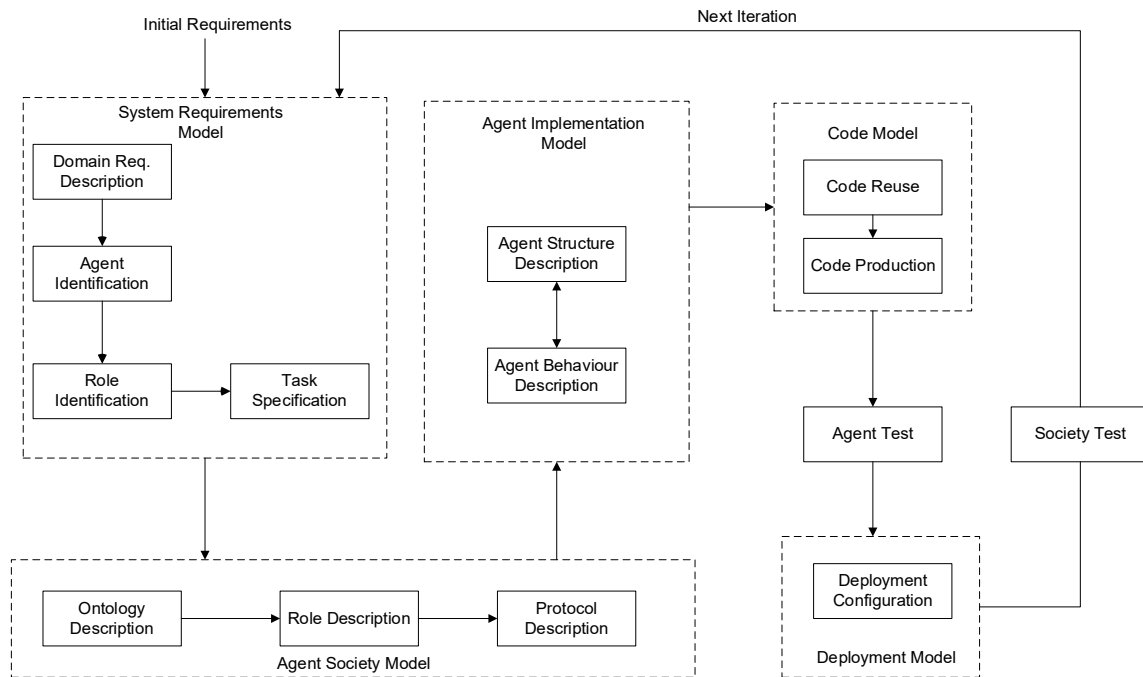


Figure 2.1 Reconstructed PASSI Model as described by [68]

Fischer et al. [69] summarise an AI Holistic Systems Engineering life cycle process. The model represents a continuous development cycle even after deployment due to the evolutionary nature of AI. The authors in [69] highlight several challenges and issues in the AI Holistic Systems Engineering lifecycle. The following is a list of relevant challenges that will influence our methodology:

1. The issue of Dataset Shifting in ML Epistemic Training is related to how much a real-life operational environment has shifted from the day we trained an ML. For example, a computer vision model trained to detect underage individuals at checkout in 2000, how much should a development team need to be concerned with applying the system in 2020 for a different store?
2. Data Augmentation techniques (augmenting existing data with various graphical manipulations and distortions) introduce biases caused by their assumptions about the task at hand. The side effects of data augmentation have hardly been investigated yet.
3. Less Research has paid less attention to the problem of monitoring the quality of the data supplied to AI.

The AI system engineering model presented in [69] encompasses a comprehensive set of topics that guide the development process. These topics include understanding the problem, specifying requirements, selecting the appropriate AI model, acquiring data, conditioning the data for analysis, and finally deploying the AI system.

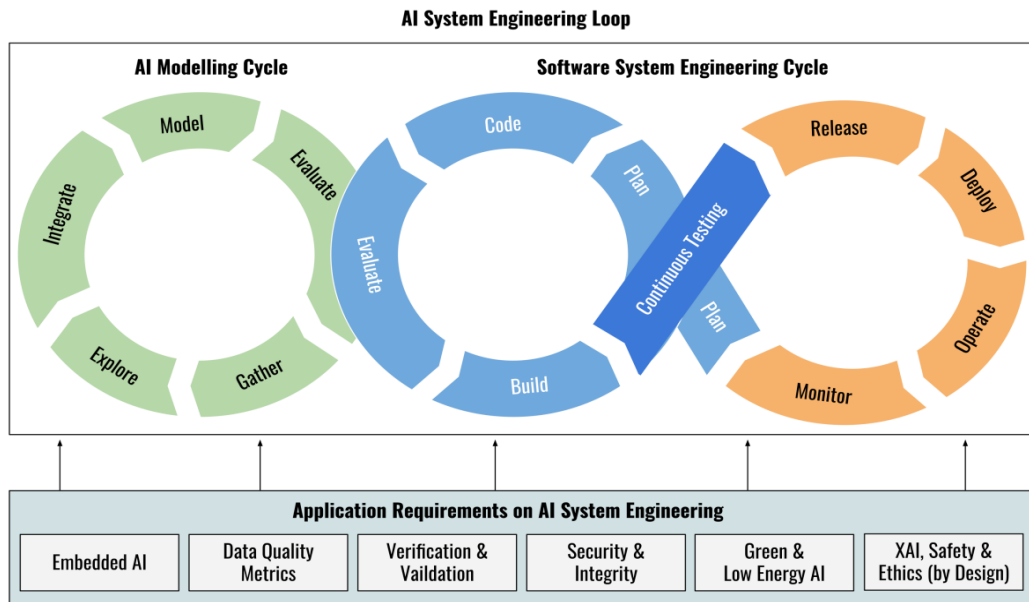


Figure 2.2 AI Holistic Systems Engineering Life-cycle process in a continuous loop fashion [69]

The AI Systems Engineering Loop in Figure 2.4 represents a lifecycle process for developing and deploying AI systems, integrating both AI-specific and traditional software engineering activities. The AI Modelling Cycle emphasises exploring, gathering data, modelling, evaluating, and integrating AI models into larger systems, focusing on creating and refining AI capabilities. The Software System Engineering Cycle complements this by focusing on traditional software engineering practices, including planning, building, coding, testing, releasing, deploying, operating, and monitoring the system. At the core of this process is Continuous Testing, which ensures the reliability and robustness of the system through iterative validation.

The lifecycle aligns with various application requirements for AI systems, including ensuring embedded AI integration, maintaining data quality, performing rigorous verification and validation, ensuring security and integrity, promoting green and low-energy AI, and adhering to explainable AI (XAI) principles with safety and ethical considerations "by design". Together, these interconnected cycles provide a holistic framework for developing reliable, efficient, and ethically responsible AI systems while continuously improving their performance.

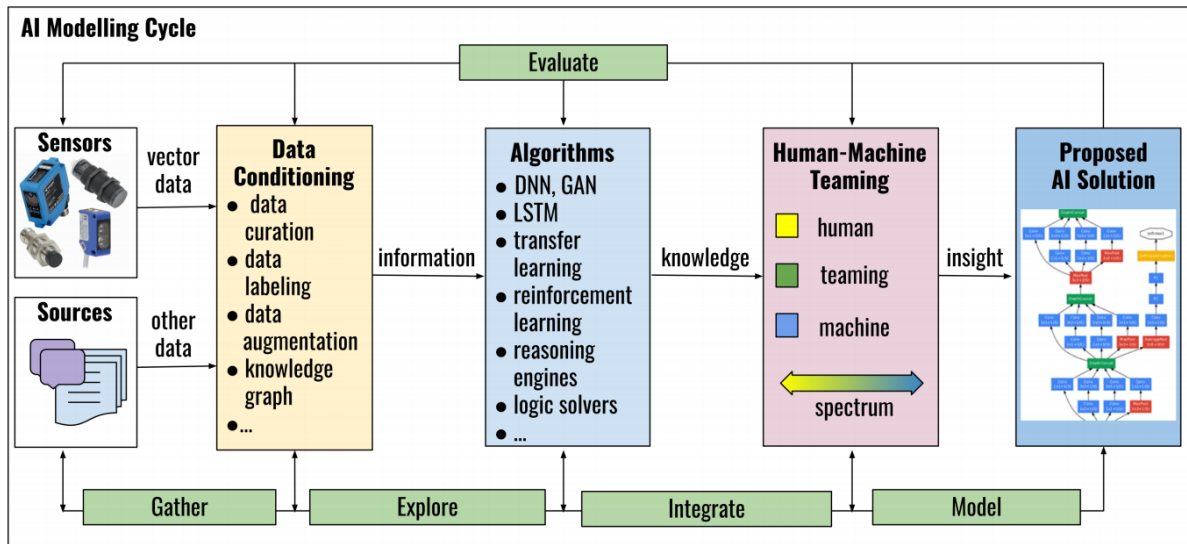


Figure 2.3 AI development life cycle as described and illustrated by [69]

Figure 2.3 illustrates that an AI system engineering process starts with a Stage related to gathering data to train the AI model. Then, exploration of AI techniques such as DNN (Deep Neural Network) and reinforcement learning. Once the AI is designed and trained, integration within the intended application is the third Stage. The process does not stop at the deployment Stage; the solution performance within a human team is dynamically assessed for insights to improve the process. Throughout the development life cycle, the model is consistently evaluated against several quality criteria and set performance indicators [69]. It can be noted that the AI SE process is continuously iterated throughout its application life-cycle; in other words, unlike the traditional CPS SE process, which usually ends with the deployment of the system, the evolutionary nature of autonomous systems performance demands that the iterative process carries on further for as long as the system in operation, hence the infinity sign representation in Figure 2.2.

Matthaci et al. [70] proposed a methodology to systemise and combine localization-driven and perception-driven systems architectures into a structured top-down functional system architecture for autonomous on-road vehicles. The authors emphasised that handling complexity in autonomous vehicles is a great challenge, especially when the SE approach attempts to combine multiple architectures for a scalable system. The architecture includes the following functional requirements: operating, mission accomplishment, map data, localisation, environmental perception, cooperation, safety, and self-perception.

Ashmore et al [71] summarise the state-of-the-art assurance processes for machine learning utilisation in safety-critical systems. ML development typically integrates into an umbrella system's development lifecycle. Thus, ML requirements are usually derived from higher-level system requirements. The main difference between the ML and traditional system lifecycle is the nature of acquiring data to synthesise a model. Traditionally, system requirements are translated deterministically. However, in the case of ML, the requirements are embedded indirectly within

the data sets, and error optimisation is used to train and test ML components. Tracing such requirements from the system to the ML level becomes a real challenge as the justification of traceability tends to depend on human interpretation of how datasets exactly implement the requirements. Machine Learning workflow encompasses the first three Stages of the ML Lifecycle.

The authors [71] describe the data-gathering Stage as the core of the Lifecycle. Datasets are subdivided into training and verification. Data gathering or collection is related to the process of which datasets are gathered and processed [72]. The gathering process includes missing data augmentation, pre-processing data collected from multiple sources, complexity management [73], and data labelling. Data analysis is the activity whereby data augmentation, pre-processing, complexity and labelling needs and methods are evaluated and selected [74]. The training activity of ML aims to produce a model that minimises the error calculated by the chosen loss function. When performing the training, ML engineers usually have to make engineering judgements on:

1. **Dataset strategy:** What data should be included and how much data should be used for training, validation, and verification?
2. How much data is needed for each training epoch, known as minibatch methods [39]?
3. Hyperparameter selection includes decisions regarding model hyperparameters, such as learning rate and size of training epochs.
4. Transfer learning of successful models to related contexts, i.e., using the model parameters and hyperparameters as a starting point of training another model in some related context, for example, teaching the model to recognise UK road signs, the model parameters and hyperparameters can be used as a starting point for road signs in France.

The survey [71] defines the following activities for the data management Stage:

1. **Data collection:** data collection may follow a controlled or uncontrolled process.
2. **Pre-processing:** includes actions such as ensuring images are the same size or manual addition of labels to images.
3. **Augmentation:** This involves adding synthesised new data to compensate for missing data, perhaps due to ethical reasons or costs [40]. It also involves instilling certain properties, such as robustness, by including adversarial examples [75].
4. **Analysis:** for example, data gathered exhibit a set of assurance properties in data bias analysis.

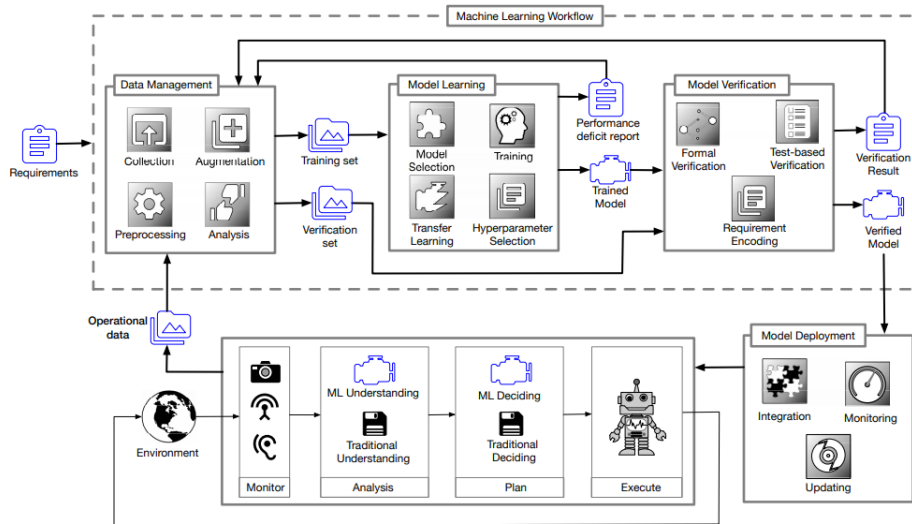


Figure 2.4 Machine Learning Lifecycle model as depicted and described in [71]

Figure 2.6 represents the ML lifecycles consisting of 4 main Stages:

1. **Data management:** Focuses on obtaining the data for training and verification of the model. It includes data collection, data reprocessing, data labelling [76] and data augmentation [77].
2. **Model Learning:** Activity of synthesising patterns from gathered data using a chosen ML algorithm and hyperparameters [78].
3. **Model Verification:** This proves the ML model complies with requirements.
4. **Model Deployment:** Integrating the trained model with its application environment [71].

Some research was conducted to address the types of possible errors that could occur during dataset curation, as well as the potential failures that could arise from it datasets. The authors in [65] identify several common causes of poor data quality, including errors during the data gathering process, inadequate management practices, and errors in data usage. Additionally, datasets may become less relevant over time due to shifts in context.

2.1.5 Industrial Holistic Systems Engineering Practice for Autonomous Systems

The purpose of this Section is to review the current state of the art in safety assurance of autonomous systems as applied in industry, including potentially relevant Holistic Systems Engineering practices. We review the common safety standards that are generic for various industries and examine how applicable they are to autonomous systems, specifically in the notion of safety assurance of the training process of AI. The aim is to demonstrate to the reader the extent to which the current industrial state of the art in assurance deals with ML.

Functional Safety is a concept originated by ISO 26262, which relates to functional failures; that is, in the event of failure, the system still retains some functionalities [79]. However, ISO 26262 does not cover safety hazards that do not result in system failure; hence, ISO 21448 was

introduced to address the concept of Safety of the Intended Use (SOTIF) and account for the Safety of Autonomous Vehicles [80]. The goal of ISO21448 is to reduce potential unknowns and unsafe conditions. However, this approach is rather broad and does not entirely cover how we can demonstrate that we have accounted for all false positives and false negatives edge cases. The UL 4600 [81] standard is a specialist guideline designed to assure the safety of autonomous vehicles. The standard is intended to accommodate new safety practices that may not be conventional to address autonomy. The main topics covered by UL4600 are safety case construction, risk analysis, safety design process, testing, tool qualification, autonomy validation, data integrity, human-machine interface analysis, system lifecycle, metrics and conformance assessment [81].

The National Highway Traffic Safety Administration (NHTSA) introduced non-regularity guidelines for Automated Driving Systems (ADS) development focusing on safety assurance [82]. The guidelines' structure is based on defining 12 areas of Safety Assurance that automotive companies must consider to certify their Autonomous Systems.

2.1.5.1 SACE

The Safety Assurance of Autonomous Systems in Complex Environments (SACE) [19] is a structured framework designed to provide systematic safety assurance for autonomous systems (AS) operating in complex and dynamic environments. The methodology, developed under the Assuring Autonomy International Programme (AAIP) at the University of York, aims to bridge the gap between traditional system safety engineering approaches and the emerging challenges posed by autonomous decision-making systems.

The SACE process is structured into eight key Stages, each addressing different aspects of safety assurance, from operational context definition to verification of system performance in real-world settings. Each Stage of SACE contributes to progressively refining and verifying safety arguments, ensuring that safety justifications are backed by strong evidence before deployment.

The first Stage, Operating Context Assurance, establishes the fundamental operational constraints for an AS by defining autonomous capabilities, validating the Operational Domain Model (ODM), and identifying operating scenarios. This Stage relies on inputs such as AS concept definition and design constraints to produce a validated ODM, documented autonomous capabilities, and an operational assurance argument forming the safety case's basis.

The second Stage, AS Hazardous Scenarios Identification, focuses on recognising potentially hazardous scenarios that could impact the system's safety. Using inputs like the ODM and predefined operating cases, this Stage produces a list of identified hazards, decision analysis

reports, and risk validation assessments, ensuring a systematic understanding of risks that the AS might encounter.

Following hazard identification, the Safe Operating Concept (SOC) Assurance Stage establishes a structured safety framework for AS operations. This step defines and validates safe operating behaviours using assumptions on safety constraints, risk assessment, and hazard mitigation strategies. The outputs of this Stage include a formalised SOC definition, justification reports, and an assurance argument, ensuring the AS operates within a controlled and predictable environment.

The AS Safety Requirements Assurance Stage translates hazard assessments and SOC constraints into safety requirements across different levels of system development. By validating and justifying tiered safety requirements, this Stage ensures traceability and compliance through structured safety documentation, including justification reports and safety requirement arguments.

Once safety requirements are established, the AS Design Assurance Stage ensures that the system design aligns with safety expectations. It integrates engineering artefacts, design processes, and architectural constraints to justify and validate design choices, ensuring the system meets all predefined safety requirements before implementation. The results include structured design validation, compliance reports, and assurance arguments.

The Hazardous Failures Management Stage examines how the AS might fail despite operating under its SOC. This Stage ensures that potential failures are anticipated and addressed by identifying failure scenarios, defining risk mitigation strategies, and conducting system failure analysis. The outputs include safety analysis reports, failure mitigation plans, and hazard management arguments.

In cases where an AS operates outside its predefined ODM, the Out of Context Operation Assurance Stage ensures adaptive safety mechanisms are in place. This involves assessing unexpected environmental changes, defining operational boundaries, and implementing fallback strategies to maintain safety even in unforeseen conditions. The outputs of this Stage include risk assessment reports, transition analysis, and contingency plans.

Finally, AS Verification Assurance is the culmination of the SACE process, ensuring that all safety requirements, operational constraints, and mitigation strategies are effectively implemented. Through structured testing, validation, and verification strategies, this Stage provides documented verification results, safety compliance logs, and assurance arguments, demonstrating that the AS is safe and reliable for real-world deployment.

Overall, the SACE framework provides a structured, traceable, and iterative process that integrates context definition, risk assessment, safety planning, failure management, and system validation. We use SACE to construct a partial safety case and consolidate our process in Chapter 8.

2.1.5.2 AMLAS

The Assuring Autonomy International Program (AAIP) proposed a method for ensuring machine learning for use in autonomous systems [18]. The AMLAS (Assurance of Machine Learning for Automated Systems) process is a structured, iterative framework that facilitates integrating machine learning (ML) components into safety-critical systems. It systematically develops an explicit safety case for ML components within a given system context, ensuring their reliability and compliance with safety requirements. The process integrates assurance activities alongside ML component development, maintaining adaptability through iterative feedback mechanisms.

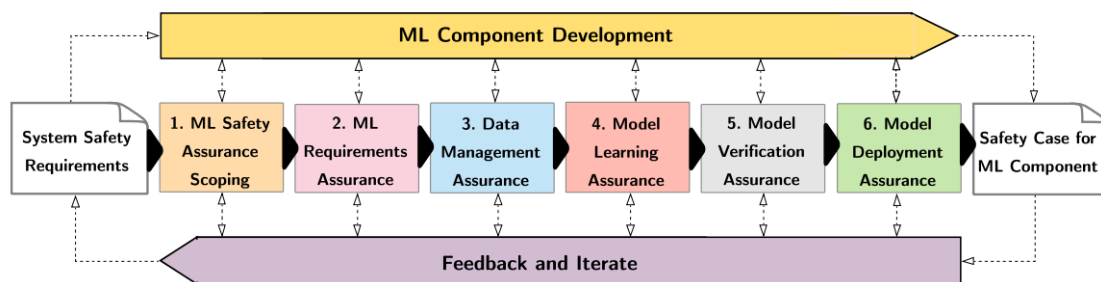


Figure 2.5 Overview of AMLAS lifecycle process as illustrated by [18]

Figure 2.7 represents the AMLAS process, encompassing six Stages linked to the overarching "Feedback and Iterate" thread. These Stages address various facets of ML assurance:

1. **ML Safety Assurance Scoping** defines the ML component's boundaries and safety assurance objectives.
2. **ML Requirements Assurance** develops and validates the safety requirements specific to the ML system.
3. **Data Management Assurance** ensures that data used in training and validation meets the necessary quality and safety standards.
4. **Model Learning Assurance** governs the creation and evaluation of the ML model to ensure safety objectives are achieved.
5. **Model Verification Assurance** validates the model's behaviour against its safety requirements.
6. **Model Deployment Assurance** oversees the integration and testing of the ML model in its operational environment.

Each Stage involves detailed activities and artefacts that contribute to the overall safety case. These include defining objectives, specifying inputs and outputs, and developing structured safety arguments supported by evidence. A key feature of AMLAS is its iterative nature, which allows for the reassessment and refinement of outputs based on the interdependencies between Stages. For instance, model verification results may necessitate revisiting data management or even redefining safety requirements.

The process relies on a structured definition of a safety case as “a compelling, comprehensible, and valid argument, supported by evidence, that a system is safe for a given application and environment”. AMLAS supports the instantiation of reusable safety case patterns, enabling systematic assurance tailored to specific ML applications while addressing common challenges and misunderstandings in ML development.

AMLAS Data Management Process:

The process comprises four main activities: define data requirements, generate ML data, validate ML data, and then instantiate ML data argument patterns. The flow of AMLAS takes system-level safety requirements and then translates them into safety requirements allocated to ML, which in turn are translated into ML safety requirements. ML safety requirements we then define data requirements (see Figure).

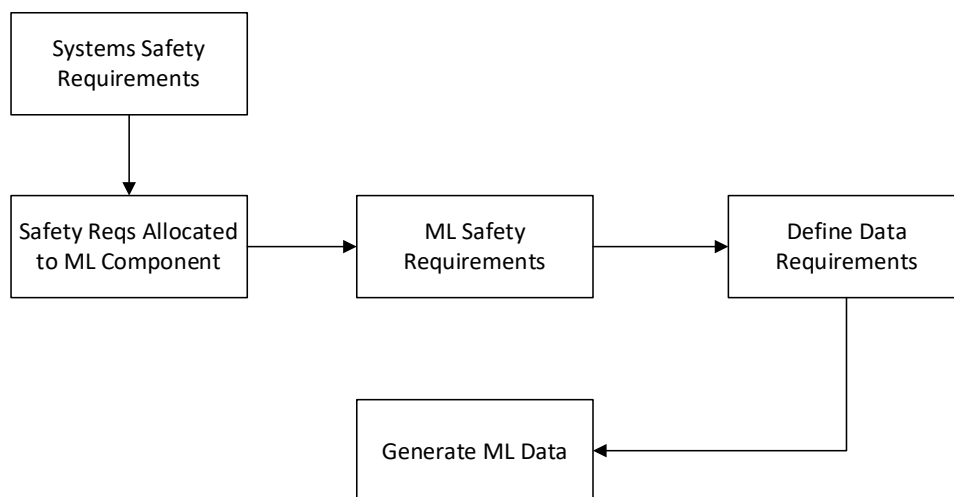


Figure 2.6 AMLAS process in generating ML Epistemic Training and testing data

As for validation of ML Data, AAIP sets out a validation acceptance criterion for what good datasets should be verified to be: complete, relevant, and balanced. Figure 7 is taken from the examples given in the AMLAS document for each process step. Below is an example extracted from the AMLAS handbook to illustrate a suggested approach to establishing a training process for intelligent systems.

The process (Figures 2.8 and 2.9) begins with establishing the system's safety requirements, which dictate that the system must be able to safely stop at zebra crossings when a pedestrian is

detected crossing the road. Specific safety constraints are assigned to the machine learning (ML) component to meet this requirement. For instance, when the ego vehicle is within 50 meters of the crossing, the object detection component is tasked with accurately identifying pedestrians who are on or near the crossing, ensuring their positions are correctly determined. The ML safety requirements further specify that the ML component must be capable of recognising individuals regardless of their pose in relation to the camera, thereby enhancing detection reliability. Finally, to support effective training and validation of the ML model, data requirements are outlined, mandating that all pedestrian positions be recorded based on the extremity closest to the roadway during data labelling.

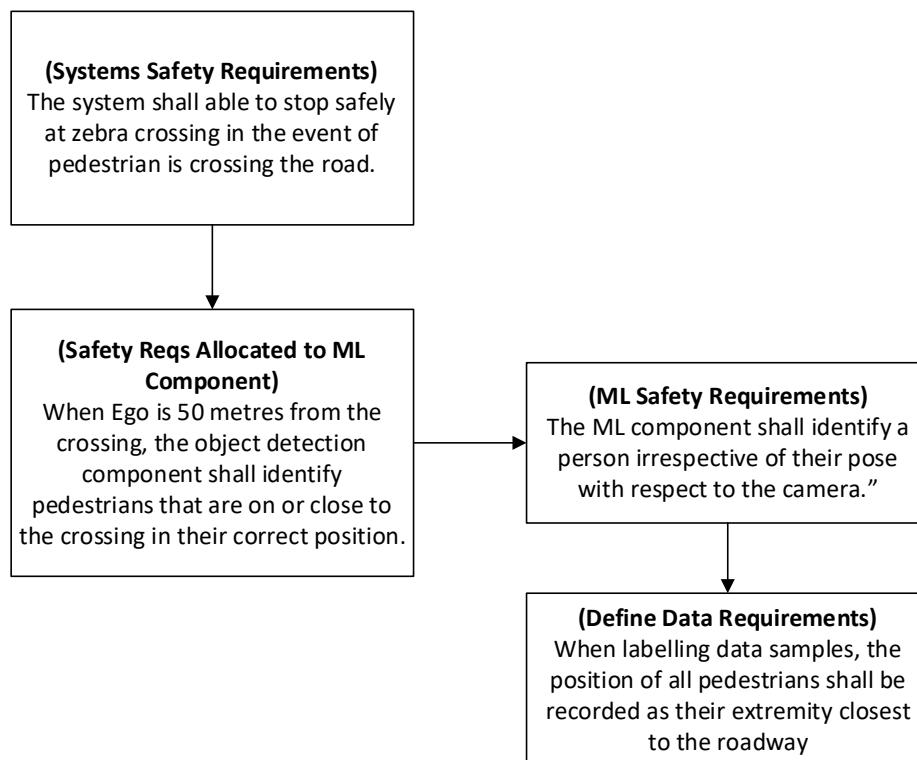


Figure 2.7 Intelligent systems training process as suggested in AMLAS methodology.

Figure 2.9 involves the following steps:

- **Step 1: Systems Safety Requirements:** The system shall be able to safely stop at a zebra crossing when a pedestrian is crossing the road.
- **Step 2: Safety Requirements Allocated to the ML Component:** When the ego vehicle is 50 meters from the crossing, the object detection component shall identify pedestrians who are on or near the crossing and determine their correct positions.
- **Step 3: ML Safety Requirements:** The ML component shall identify a person regardless of their pose relative to the camera.
- **Step 4: Define Data Requirements:** When labelling data samples, the position of all pedestrians shall be recorded as the extremity closest to the roadway.

In Summary:

- **AMLAS** is specialised in providing a structured methodology for assuring the safety of ML components in autonomous systems. It integrates safety assurance throughout the ML lifecycle.
- **SACE** provides a broader safety assurance framework for autonomous systems in complex environments, covering hazard identification, operational safety, and system-level verification.

2.1.5.3 SASWG Framework

The Safety of Autonomous Systems Working Group (SASWG) proposed a framework for assuring the safety of autonomous systems [83]. The guidance subdivided the problem of guaranteeing autonomy into three interrelated frameworks: computation, autonomy architecture, and platform. The Vertical step concerns implementation at software and Vertical hardware levels. The architecture step framework concerns how computation is integrated into the system. The platform step framework mainly focuses on platform requirements, the highest conceptual level.

The framework is illustrated through multiple projections, which provide different views. The output of following the guidelines is to allow practitioners to elicit design objectives for every Stage. The Vertical-step framework emphasizes rigorous dataset management throughout the machine-learning lifecycle. It mandates clear objectives for data generation, whether collected through observational studies, controlled trials, or synthetically created, alongside a documented process that accounts for environmental variables such as weather and system architecture. All acquisition software must be verified, and any externally sourced data must undergo authenticity checks, validation against “data poisoning” attacks, and screening for inadvertent labelling or translation errors. Quantification of sensor noise and other uncertainties is required, as is explicit management of ambiguous labels (for example, distinguishing pedestrians from cyclists). The framework further prescribes careful configuration management, formal control of any changes to data-collection methods, and special provisions for online-learning scenarios, including pre-processing activities like augmentation and ensuring that training sets faithfully represent real-world operational contexts.

While comprehensive, the SASWG safety guidance omits several critical facets of dataset quality assurance. It does not prescribe methodologies for ensuring demographic and scenario diversity to guard against biased model behaviour, nor does it address the need for statistical validation of sample size sufficiency or power analysis to guarantee representativeness. It does not provide any guidance with regards handling the prediction of operational problem space Black Swan scenarios for example.

2.1.6 Chain-of-Thought (CoT) techniques for large NLP

A fundamental aspect of our systems approach involves enhancing the predictive thinking capability of architects by applying systems theories and thinking frameworks. This allows for objective predictions regarding any complexity observed in the present (as-is) or anticipated in the future (to-be). To achieve this, we established a logical framework that guides the predictive reasoning process architects must engage in, grounded in a general system pattern. To this end, we adopted the concept of Chain-of-Thought (CoT) from the field of prompt engineering, as we found parallels between the architect's thought process and that of a large language model.

The concept of "Chain-of-thought" in artificial intelligence and natural language processing refers to a method where a model explicates its reasoning process step by step, akin to how a human might solve a complicated problem. This approach, particularly prominent in large language models, enables the model to break down complicated tasks into smaller, more manageable steps, making the reasoning process more transparent and interpretable [84].

Generating a Chain-of-Thought that consists of a series of intermediate reasoning steps improves the ability of large language models to perform complicated reasoning. This ability emerges naturally in sufficiently large language models through a simple method called Chain-of-Thought prompting. The study in [85] delves into how these prompting techniques enhance the ability of large language models to perform complicated reasoning tasks.

We noticed that sense-making in the chain of thought can be adapted to employ general systems theories, re-adapting the latter to employ systems thinking to enhance Architect Predictive Cognition.

We view systems engineering processes as methods to structure information for better systematic knowledge extraction. From such a standpoint, we notice a shared pattern between the function of SECoT with LLM and a system engineering process with an architect or system engineer. We realise that architect predictive intelligence is analogous to a super-intelligent LLM. This means we consider Architect Predictive Cognition as an intelligent black box device that can be trained and influenced by thinking operations such as CoT. Therefore, we combined both concepts and explored the idea of designing SECoT based on system thinking theories to be used as systems engineering methods. Here, we noticed the possibility of being able to control and verify the type of sense-making and thus control engineering Perspective shift and find a useful application in enhancing Architect Predictive Cognition.

We believe it is possible to assist intelligent machine architects in improving their predictive capability through a system thinking approach that employs a variety of systems perspectives across the design process. Also, by changing perspective (or shifting perspective), Using a

perspective-shift-based technique, architects' predictive ability may be improved. This approach involves experiencing Perspective shifts throughout the different design phases, which means changing the way one thinks and looking at the information from different viewpoints and angles.

2.1.7 Autonomous Systems Operational Domain Definition

The Operational Environment Definition process plays a crucial role in the architecture of trustworthy AS, particularly those utilising machine learning components [86]. It aids in defining a wide range of environmental conditions under which the system must function reliably [87]. These conditions inform the type and variety of data required to train and test intelligent components such as machine learning. Understanding the operational environment facilitates the engineering of comprehensive datasets that capture safety requirements by addressing all critical Operational Domains. This process also guides the creation of robust, trustworthy, intelligent systems that can effectively operate in real-world conditions, thereby enhancing their practical applicability and reliability.

When we reviewed the literature, to the best of our knowledge, we realised that no generic ODD covers a variety of IS. The ODD literature predominately tackles autonomous road vehicles and does not address various IS operational environments (air, space, sea or subsea). This paper is part of a series of papers aiming to develop a trustworthy system engineering method to assure the trustworthiness of intelligent systems [9] [88]. We used an aerial drone system as a case study to validate our method, and part of it was developing a perception system. here, we needed an appropriate ODD for aerial drone development that covers all possible environments in which a drone may operate. The ODD literature did not give us a satisfying answer. Therefore, we realised the usefulness of a comprehensive general ODD that provides baseline features that cover a variety of IS applications (space, land, air, and sea). In this work, we will focus on Land and Air applications, and in the future, we will incorporate sea-level, subsea and space.

The literature suggests a hierarchical taxonomy to help model and understand ODDs [86]. Standards such as the BSI PAS 1883 [87] and ISO 34503 (International Organisation for Standardisation, 2021) seek to enhance the taxonomy requirements by specifying environmental elements' measurability and definition formats. Other significant contributions include the SAE AVSC lexicon [89] and ASAM OpenXOntology [90], which aim to standardise and extend the taxonomy for autonomous vehicle (AV) systems.

The proposed ODD taxonomy by NHTSA [91] and other related standards are primarily tailored for self-driving cars. The focus is on scenarios and conditions relevant to road vehicles, including roadway types, traffic conditions, and specific environmental conditions that impact vehicular traffic. This specialisation means the taxonomy does not comprehensively address the broader

Chapter 2

range of autonomous systems. For instance, autonomous drones, underwater vehicles, or robotic systems operating in industrial environments have different operational needs and constraints that are not covered in this taxonomy.

The taxonomy lacks detailed consideration of complicated landscape shapes and terrain variations. While it mentions roadway geometry and surfaces, it does not extend to more intricate environmental structures that might be encountered by autonomous systems operating off-road or in non-urban settings. There are other domain-specific ODD taxonomies; for example, Ugé [92] specifies a taxonomy for above-sea-level marine applications. Another ODD related to the mining industry [93] extends the existing taxonomies with mining operational domain aspects. In summary, the taxonomy in Exhibit 1 integrates all taxonomies reviewed in this literature.

Table 2.4 presents a partial view of the comprehensive Operational Domain Definition (ODD) taxonomy tailored for autonomous motor vehicles which we gathered from the references above. The full table contains 5 levels and can be found in [94]. The full version of this taxonomy is organised into 5 levels, each detailing the essential attributes and categories that define the operational environment of autonomous road vehicles. The hierarchical structure ensures a thorough understanding and classification of all relevant factors influencing the performance and safety of autonomous vehicles.

Table 2.4 State-of-the-art Integrated ODD taxonomy for autonomous motor vehicle

Level 1: Class	Level 2 Attribute
Physical Infrastructure	Roadway Types
	Roadway Surfaces
	Road Surface Conditions
	Roadway edges
	Roadway Geometry
	Lanes specification
Operational constraints	Speed limit
	Traffic conditions
Objects	Traffic control devices
	Signage

	Roadway users
	Non-roadway user obstacles/objects
Connectivity	Vehicles
	Traffic density info
	Crowdsourced data
	V2I
	Remote fleet management system
	Communication obstructions
	Infrastructure sensors and communication
Weather-related conditions	Probability forecast
	Wind
	Precipitation types
	Sun angle
	Haze
	Sky condition
	Temperature
	Weather-induced roadway conditions
	Particulate matter
	Illumination (luminous flux)
Zones	Geo-fencing
	Traffic management zones
	School/construction zones
	Regions/states
	Interference zones
	Dynamic zones
	Non-static roadside objects

Additional categories	Additional requirements
------------------------------	-------------------------

2.2 Research Question 3 Literature review

This Section is related to AS problem 3: Bridging the semantic gap and AI training processes. Missing data in datasets represents a safety issue in training AI, not because AI needs more information to learn accurately alone, but because of how the human designer approaches data completeness. Therefore, a safety-driven quality criterion would consider the integrity of the gathering process so that any bias was not intentionally or unintentionally introduced. As for the Duplication of Data in datasets, the impact should remain the same accuracy of ML [95].

The outlying value problem is viewed from the perspective of “interesting sensor events”, whereby sensors detect patterns from an environment that may lead to misconceptions about the intelligent system [96]. It is also commonly known as event detection. This is typically related to sudden changes in the underlying data records, which manifest as either contextual or collective anomalies over relative timestamps [96]. The relevance of anomalous event detection to this research will mainly be related to understanding how such anomalies may arise during the training of the intelligent system. Thus, the following open question arises:

Open Question

How can we capture and interpret anomalous events during the training of intelligent CPS and use this information to inform the design process?

Open Question

How does the method of discovering anomalous events relate to the discovery of edge cases?

A great deal of literature was found on the topic of ML Training data gathering and management [4] [97] [98] [99] [100] [101] [102] however, it has been found that to the best of our knowledge, there is no sufficient work done on systematic process for constructing context-aware dataset that address the issue of guaranteeing safety requirements embedded in datasets. Certainly, from a systems science and systems engineering point of view. This gap in literature was confirmed by Kuwajima’s [4] claim that the literature demonstrated a lack of attention on the Training of AI-based CPS throughout academic research:

“One of the challenges with the lack of design specification is the establishment of a training process for machine learning models by designing training data and models...however, [a source] does not show what types of data must be added”. Also, the latter confirms:

“Engineers cannot design the training; however, they can design the training data... Yet, to the best of our knowledge, there is no standard or widely accepted process of designing training data for machine learning models”.

2.2.1 The semantic gap between written requirements and pictorial datasets

In the realm of computer vision, defining correct behaviour precisely is a complicated task. For instance, to recognise a drone using computer vision, a definition of a desired capability, such as “Eagle Drone Training datasets shall enable a robust and generalised computer vision model capable of identifying various adversarial drones, especially unusual or stealthy ones”, can be interpreted in multiple ways. This is because there could be many unspoken or unrevealed behaviours that should be correctly recognised to identify an “adversarial” drone accurately. This challenge becomes more apparent when the technology fails; it is difficult to determine the exact cause of the failure. Therefore, ensuring the safety of computer vision technology requires identifying all possible behaviours in which a drone should be recognised. The CuneiForm method, with its unique ability to construct abstract images that encapsulate these behaviours, offers a practical solution to this complicated problem.

The issue is related to the semantic gap between requirements over pictorial datasets and images that are supposed to satisfy them. The semantic gap is an established systems engineering challenge in assuring the safety of computer vision systems [22] [23]. Therefore, closing the semantic gap between a human-written requirement for an image and the pictorial representation in that image is a non-trivial task, mainly when the requirement involves subjective classifications, such as distinguishing an “adversarial drone” from any other type of drone. Unlike quantifiable attributes like voltage, which can be precisely measured with instruments like a voltmeter, the characteristics that define an object in a pictorial sense are more abstract and hard to define. To address this, we would need to create a robust set of criteria that collectively define what constitutes an object in a pictorial sense.

We are aware of literature concerning the metadata qualities of datasets, for example, the concept of datasheets for datasets [103] and a Domain-Specific Language (DSL) [104] to describe ML datasets. There is no clear guidance on systematically defining required pictorial objects' behaviours in a set of images. This is due to the multiplicity of behaviours and environments that must be determined. Therefore, any method that describes an object in an image cannot be

Chapter 2

complete without describing its environment. The main challenge is how we systematically derive potential pictorial Target Objects of Interest (TOIs) behaviours (for object detection using computer vision) and the possible environments in a way that can clearly be translated in a pictorial sense.

The term Cuneiform is related to the earliest system of writing, which the Sumerians of ancient Mesopotamia developed. Cuneiform writing started as a system of pictographs, where the scribes would draw pictures representing the words they wished to convey. The system has served humanity by communicating various specifications (social, technical, and business) for hundreds of years; for example, Figure 1 represents a cuneiform specification for a sale contract. When we observe Cuneiform at a high level, we note the following characteristics:

A collection of globally agreed-upon general icons that represent natural objects. Based on this idea, we envisage our approach, “CuneiForm”, as a technical pictorial characterisation of objects with an archive of universally accepted abstract symbols. Furthermore, cuneiform tablets are dissected into compartments that house symbolic ideas. From this idea, we divide a generic CuneiForm image into 9 compartments to define the location of TOIs in an image. Also, Cuneiform writing captures the dynamic behaviour of objects and quantities. From this idea, we include information about objects' dynamic behaviour and pictorial characteristics.



Figure 2.8 Cuneiform language-based sale contract specification Image reused from [105]

Engineering practice uses pictorial symbols to link human-written requirements and engineered systems. Icons are commonly used in technical specifications across various engineering disciplines, including electronic engineering. This practice involves using abstract icons to represent different types of electronic components. For instance, a simple abstract image of a capacitor can represent various concrete images of capacitors.

The closest work we found to our work is the work done by Tanaka et.al. [106] which introduces the Bounding Box Specification Language (BBSL), an approach for creating precise, formal specifications in Autonomous Driving Systems (ADS). BBSL utilises bounding boxes as a two-dimensional pictorial interval to represent various objects in a driving environment. The

Chapter 2

approach focuses on specifying relative positions among objects but does not include the variety of visual complexities that affect the reliability of the perception system. Our approach attempts to provide a richer ontology and systematic method to specify

Chapter 3 Examining Existing Safety Processes

This Chapter examines the current state-of-the-art systems engineering approaches for safety, namely HAZOP, STAMP, and FRAM. Appendix G showcases our application of these methods to solve the unsafe train tracks problem, so we will not repeat the work in this Chapter. Instead, we will discuss our experience with these methods to set a reference for comparison with our approach. The reader is encouraged to review Appendix G for reference.

Kritzinger [60] emphasised that the main problem in safety engineering is not the identified hazards but rather the “unidentified hazards.” This makes a suitable criterion for comparing the ability of safety approaches to identify novel hazards in an open and complex environment. After all, in Section 4.1, we explained the nature of such problems, which are often dominated by novel hazards and Black Swans. Ensuring autonomous systems perform as expected in Black Swan operations is critical for ensuring the safety of those systems. Therefore, we chose a novel hazard and assessed legacy approaches (HAZOP, STAMP, FRAM) to bring us closer to discovering the novel hazard. We expect legacy approaches' control-theory paradigm renders them insufficient to predict novel and emergent hazards associated with the complexity of compounded uncertainty problems.

Traditional systems engineering approaches to date can be classified as a pure control-theory-based approach to complexity. Meaning, the architect assumes that any complexity field can be viewed like an electronic board where we have specifiable components with clear, unchanging boundaries. While such simplification proved to be useful for traditional deterministic cps problems, it may not be effective worldview of open, complicated environments and autonomous systems. We experience this attitude towards complexity first-hand in this Chapter as we examine their application to solve our proposed problem.

The main problem we are trying to address in our approach can be described in two dimensions:

- Addressing the problem of predicting harder-to-predict, novel, unforeseen emergent scenarios related to unexpected emergent behaviour of autonomous systems in harder-to-predict complicated operational environments.
- Ascertaining that emergent behaviours fulfil requirements (especially safety ones) are preserved when autonomous systems interact with their environment in any possible emergent scenario that may or may not have been thought of during design and training time.

Both dimensions are attempted to be addressed in our approach. Our solution for handling complicated emergence problems is to enhance the architect's predictive cognition through the

Chapter 3

use of systemic (through the application of general systems intuition) and systematic thinking approaches. The second concern is that we propose justification-based traceability of how artefacts are developed, combined with demonstrating the thinking and intuition employed when artefacts are developed through the application of SECoT methods. Every Stage in our approach prompts the architect to tackle the initial problem from a different point of view. Imagine a problematic situation, such as an unknown alien object landing in front of the architect, who has never encountered such a level of complexity.

Since the object has never been seen before, the architect's predictive thinking ability experiences a state of confusion. Our approach aims to provide the architect with generic systems-thinking methods that can guide and streamline their application regardless of how unknown and alien a problematic situation is. We view the power of adapting general systems theories as making systems engineering approaches problem-agnostic. Attempting to predict the future complexity and complicatedness of an unknown complicated problem is an example of a confusing complex.

The following is the problem brief:

An anonymous drone intruded into a bounded train track zone while the train passed by. Police officers were called to the scene and tried to capture the drone but were unsuccessful.

3.1 Thought experiment: examination question

Before we start the process, we would like to prompt your predictive thinking about the scenario. We have a particular reason for doing so, which will be revealed later. Let's take you back to the case study we described in Section 1.5.1. Imagine a situation like in the image below:



Figure 3.1 A snapshot taken from DuckDuckGo web browser maps of a strip of train tracks in Holy Town, Scotland, UK

Answer and record your answer for the following questions:

1. Is the train tracks problem domain completely defenceless against adversarial drones, with no effective defence mechanism already in place in the existing structure of the train tracks zone?
2. Is there a concerning risk that train tracks (as they are) pose to the surrounding vegetation, which impacts the safety of the train track zone?
3. Why should we discuss the interaction between train tracks and vegetation instead of addressing the adversarial drone? If the primary problem is the adversarial drone, why are we concerned about train tracks and vegetation?

We will describe the methods and then demonstrate their application in selected examples.

In this Chapter, we will apply three different methods to 2 aspects of the processes to compare the results of our approach to them. We do not intend to apply the processes comprehensively, but rather only the main concepts of each process. The idea is that we want to compare how each process handles two systems engineering problems:

- Articulating the problem domain to discover problems that were not considered initially.
- Discover Black Swan events and associated hazards. Especially related to computer vision anomalies.

3.2 The novelty test of predicted hazards

The main assumption made on for this type of acceptance test is:

To solve the compounded uncertainty problem effectively, the systems engineering approach must enable a novice architect to predict novel failure modes (Black Swans).

A very important criterion to test legacy approaches is whether they could enable us to think of novel hazards that are hard for a skilled safety engineer to develop if they used the approach. We judge the novelty of a predicted hazard based on the following qualifications:

- Imagine you are an expert in deterministic control systems and a safety engineering, who assumes any complexity to be reducible like an electric circuit board. Is the hazard something we have never heard of or considered logically possible? Would you consider the discovered hazards as something you would have never thought of, even in the context of the problem? Would you usually consider it? Is it rarely concerning for you?
- Another way to think of it is: if you say it out loud to an expert, would they think of you as stating the obvious? Would you be called “Captain Obvious”?

An example of a non-novel hazard would be;

- An adversarial drone that enters the train track earlier than expected.

If the approach made you notice the obvious thing for example an adversarial drone entering the airspace, nothing new about it. This is not something surprising given the main problem is that. Another set of obvious failures or easy to spot without the help of the approach:

- Adversarial drone not operational: leads to falling from the sky.
- Adversarial drone operational but hit the electrical power lines or infrastructure.

An example of a novel hazard would be;

- A flying balloon with an image of a drone on it leads the Eagle Drone to think it is an actual adversarial drone. Imagine many balloons being blown onto the train track.
- A laser with drone drone-shaped stencil spoofing the eagle drone to think there is a drone on the wall flying straight into it, perhaps the train itself.
- A new gloss paint on the train that startled an adversarial drone from recognising it as a train.
- Another example of a novel hazard: a Tesla Model perception system confuses a moon with a traffic light.

3.3 Applying the HAZOP method

This process will attempt to apply HAZOP methods to solve two aspects. The first is articulating the problem, and the other is articulating the hazards associated with introducing the eagle drone to counter adversarial drone behaviour. For the detailed application, see Section G.1. We needed a complete model of the problem as HAZOP require a complete model of the system prior to conducting a HAZOP study. So we defined the problem (being the system of interest) as follows:

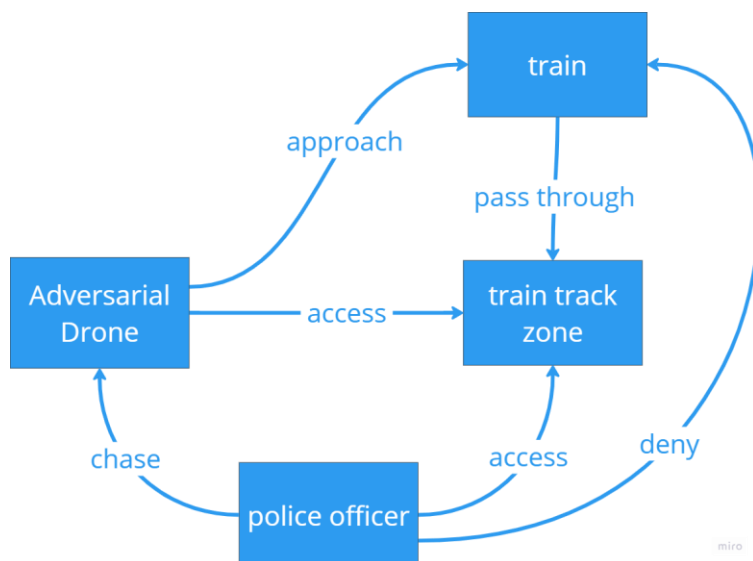


Figure 3.2 Problem domain model

3.3.1 Architect predictive attention distribution of HAZOP types of failures

In this Section, we add an extra analysis step to examine how we approached the problem, using the architect predictive attention distribution of the “guide words” we used to trigger our predictive thinking in identifying potential failures. Below is a distribution Table of the number of failures predicted per guide word, which accounts for the architect's attentive mentions of those guide words being recalled in our analysis:

Table 3.1 Architect's attentive mentions of types of failures we predicted using HAZOP analysis

Type of failure	Number of mentions
No	6
Faster	3
More	2
Other than	2
Less	1
As well as	1
Early	1
Reverse	1
Before	1
Late	1
Part of	0
Where else	0
Late	0
After	0
Slower	0

The following is a histogram of our predictive attention profile (that exposes what guidewords we paid more attention to predict than others) as guided by the HAZOP process.

The histogram (Figure 3.3) exposes a bias in our predictive thinking, as we focused. we applied the skewness statistical test, and it revealed 2.1 (using Excel function SKEW()), which indicates right-skewed. what makes this curve different is that we have 0 mentions for a number of guide words. this suggests that we are making a tacit assumption that such types of problematic situations are irrelevant based on our knowledge. This tendency is what gives rise to the normal-driven attention distribution over observed complexity, as induced by the HAZOP process.

Figure 3.3 confirms that when we rely on limited guidewords, the architect or practitioner will only consider those types of deviations. Such a predictive thinking strategy can be efficient for

traditional deterministic systems, like a regular controller. However, for open, complex problems or non-deterministic systems, Black Swans lurk in deviations that are outside the limits of guidewords, which the architect will not consider due to the constraining effect of conforming to guidewords only.

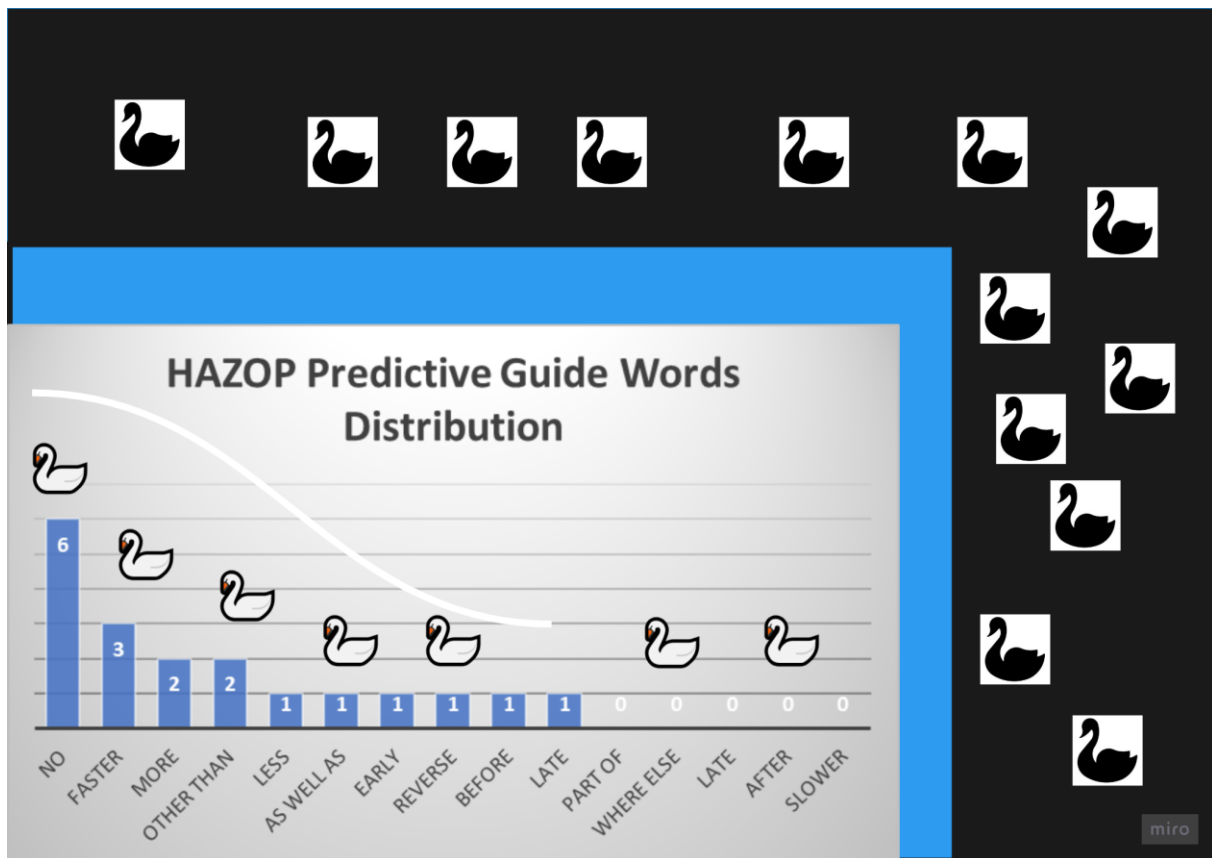


Figure 3.3 Guide words architect predictive attention distribution exposing our predictive thinking bias and focus our attention on White Swan events only. Any deviations outside the guidelines are possible Black Swans. How can we predict them?

HAZOP does not mandate the coverage of all possible deviations, guide words only, or what the architect deems to be appropriate. This can lead to complete dependence on the architect's experience and bias towards a particular subset of deviations. We couldn't think of possible late, after, or elsewhere deviations. Furthermore, we were not compelled to do so by the process; if the process had demanded to cover all possible deviations, we might be able to start thinking of the unthinkable. However, the fundamental premise of HAZOP is that the complexity of the problem can be reduced to a normally distributed set of events where a subset of deviations is sufficient to be concerned with, and what is deemed to be rare can be ignored.

Technically speaking, and looking at the histogram distribution, if a regulator asks us why we didn't consider other deviations, we can respond by saying:

- Those deviations are negligible for a given interaction.

- This is our interpretation of ALARP, and in our experience, given the body of knowledge that supports our claims, the chosen deviations are the most critical ones.

Such a mindset may be permissible for a traditional deterministic view of complexity (like a domino effect view of failures), but in an open, complicated environment where Black Swans and strange emergent behaviours dominate the complexity. Such an attitude towards safety can lead to unwanted surprises.

3.3.2 Application of HAZOP discussion

While HAZOP is a valuable tool for systematically identifying risks and designing safeguards in systems with predictable behaviour, its application for problem articulation in highly uncertain or autonomous systems environments is limited. It excels at mapping potential issues within a controlled operational Framework but does not capture the depth needed for problem definition in complicated uncertainty scenarios. The analysis didn't help us identify Black Swan events, such as how trackside vegetation may be a factor in the problem domain.

In general, our experience with HAZOP made us realise that the practice fundamentally chooses to reduce the problem complexity into a set of interactions that can either exactly deviate (in a hard-system sense) to an undesirable state, or it does not. Meaning, by virtue of applying guidewords, only direct undesirable states (which can only be a subset of concerning scenarios) are those which will be dealt with. Any potential scenarios outside the subset are irrelevant. For example, if a metal pipe in a hydraulic system is our focus of HAZOP study, then the question becomes as simple as:

What can prevent the pipe operability or lead to a hazardous situation?

By default, this question misses situations in which pipe operability's success may harm other operational environment elements outside the study's concern. This mindset works reasonably well in highly controlled problems with a high level of determinism and predictability. In other words, if HAZOP helps us predict an event, it is 100% certain that such an event directly impacts the safety of the component and given a set of conditions to be true, it will 100% manifest. Therefore, we can imagine a HAZOP view of complexity would be something like the following diagram:

In an ideal world, assuming that the complexity of any problem is like a domino puzzle with a single or multiple exact and predictable causes of a failure would be the dream of any architect who likes to live longer due to peace of mind. However, Compounded Uncertainty problems, like the one we aim to tackle, bet to differ. Figure 3.4 depicts our understanding (through the practice) of what type of problem solver we would have become. We were looking for the exact

deviation word that tells us how a deviation in an interaction may manifest. We understand the reader may be tempted to think, why don't we just adapt HAZOP and force the designer to think of all possible failure types? This is not as straightforward as you may think. Sure, let's adapt HAZOP (in fact, we generalised HAZOP into HazTOPs in our approach), however, a significant rewording and changing of what guide words need to mean to cover for variety of interactions that are AIC based so much so that the new approach may not look the same as HAZOP. Hence, we proposed HAZTOPs.

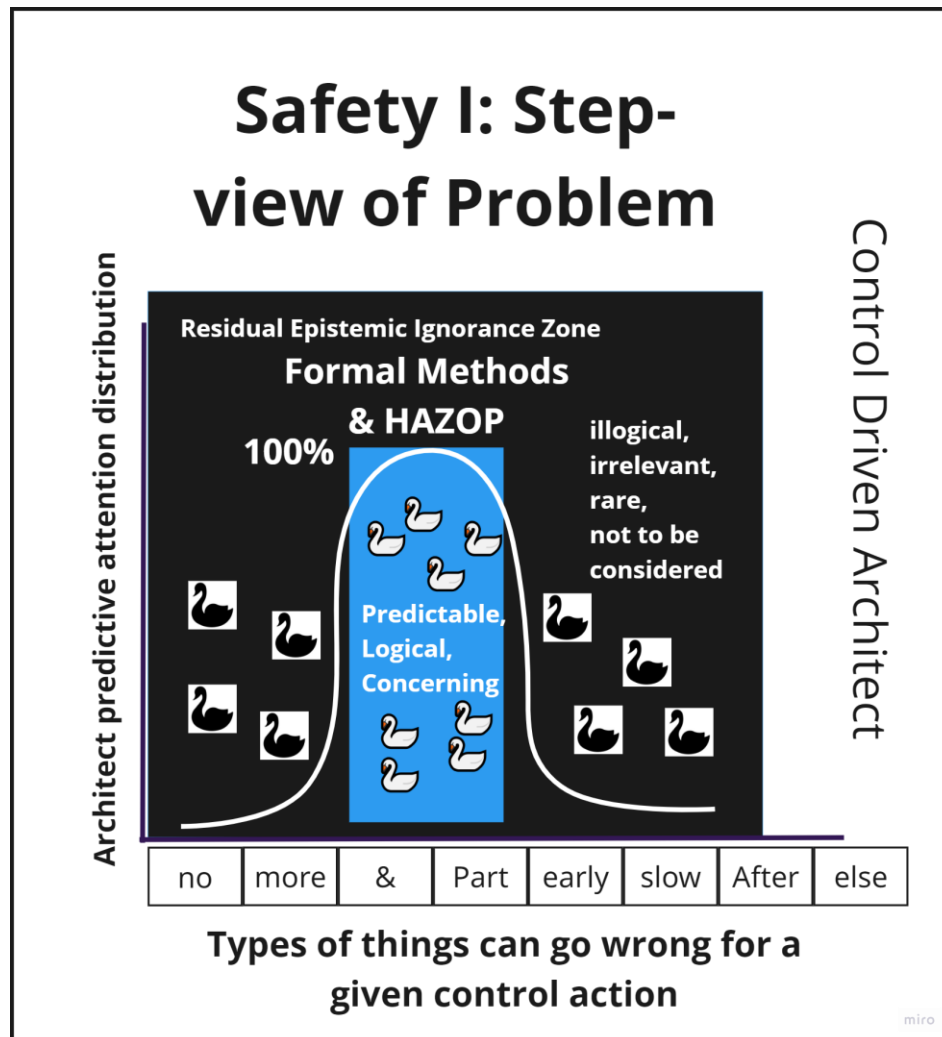


Figure 3.4 HAZOP and Formal Methods predictive attention distribution of problem complexity

The main takeaway from Figure 3.4, is that the HAZOP, as its standard practice, assumes that the model of the system or the problem is complete and certain. For example, in the model we made for the problem domain (Figure 3.2) we defined an interaction between a police officer and a train, considering it as “deny”. HAZOP assumes such interaction is 100% certain. However, in reality, we are not sure whether the police will have the ability to deny train access or not. A police officer is an agent and not an engine in a car. In general, we describe such a view of complexity as a step-

view of the problem, meaning the architect expects the complexity of the problem can be reduced to a subset of specific deterministic interactions.

3.3.2.1 Strengths of applying HAZOP for problem articulation

1. **Realisation of Ignorance:** Guide words acted like a general rule of deviation in complexity that is most often undesirable by the designer. We noted that those words played a direct role in our predictive capability, so in any complexity interaction, we could determine whether such interaction would deviate into undesirable (failure or hazardous) complications in the future. The abstract nature of guidewords raised an inquisitive sense about how the abstract rule may manifest in the context of an interaction. This created a sense of ignorance, which prompted us to start predictive thinking to bridge the gap and make connections we did not make prior to the application of HAZOP
2. **Systematic identification of known issues:** HAZOP's structured nature ensures that potential hazards are identified methodically through predefined guidewords. For instance, in the problem domain involving a train, track, drone, and police officers, HAZOP allowed for a comprehensive examination of potential failure points such as timing mismatches, miscommunication, or adversarial drone behaviours.
3. **Holistic examination of interactions:** HAZOP forces an investigation of each interaction (e.g., train-track zone, drone-train, etc.), promoting a holistic approach to understanding possible system deviations. This holistic analysis helps capture the system's complexity and highlights safeguards and potential corrective actions.

3.3.2.2 Weaknesses of applying HAZOP for problem articulation

1. **Ineffective¹ cognitive remedy to a sense of realised ignorance:** guidewords were effective in helping us realise a difference between the abstract rule of the guide word and the context of interactions in which we could predict certain undesirable deviations. However, it was not effective enough to make cognitive connections to be able to answer the exam questions. This is because of the tacit premise that there is only a small, tight, direct impact between any two elements, and denying such effects is worth our concern. That is the fundamental hard-systems perspective of complexity. There is no direct

1 By the term “ineffective”, we mean it does not help us to consider that all factors, regardless of their relevance and likelihood, are equally concerning and thus require prediction and mitigation, or justifying why we should not bother about them. But they need to be discovered. such general perspective is necessary to avoid missing Black Swans.

impact between vegetation and train operations that warrants our concern, hence we did not even consider a particular deviation to predict what could go wrong if a train passes through and moves the tips of grass stems. Such an event is irrelevant to our problem; there are more important problems to be concerned about, like an adversarial drone crashing into a passing train.

2. **Hazard novelty test criterion:** none of the hazards predicted using HAZOP in the context of the problem domain passed the novelty criterion. The reason is that HAZOP focuses our attention on choosing guidewords that we can easily think of plausible scenarios that we can comprehend logically.

For example, “adversarial drone – train track zone” we picked the guide word “no” because it is logical to think that the train track zone offers no deterrence to adversarial drone access. We could not think of something that would be surprising to us. We did not choose “late/early” because it does not make sense how can train track zone can be late or early. Another example, “adversarial drone – train track zone”, we chose the keyword “more” because it is evident that more drones have a worse impact than a single drone. We did not pick “late/early” at this time, but if we did, we would have thought about the drone entering late or early, concerning the passing train. Being late would constitute no hazard, early would allow it to be detectable, and our response team may have more time to respond. As you can see, we were driven by what was convenient and what we thought was logically likely.

3. **Limited adaptability to unusual (Black Swan) events:** One primary limitation is HAZOP’s guidewords, such as “no”, “more”, and “less”, are best suited for predictable deviations within a known design or operational Framework.

These guidewords fall short in identifying anomalies or unprecedented risks, such as “Black Swan” events, which are highly unpredictable and fall outside typical operational parameters. They may also fail to reveal unexpected vulnerabilities in autonomous and adversarial scenarios, where unknown variables may emerge.

4. **Designed for solution-oriented analysis:** HAZOP is traditionally intended for solution analysis rather than initial problem articulation. It was initially developed to optimise the safety and functionality of systems with known operational goals and specifications.

As a result, adapting HAZOP for unstructured problem definition requires substantial modification and often leads to a superficial articulation. This limitation is evident when defining problems in complicated systems with interdependent and unpredictable behaviours.

5. **Inadequate for compound uncertainty and autonomous systems:** in cases involving autonomous systems or adaptive adversarial elements (e.g., a drone entering the training zone with potential hostile intentions), HAZOP struggles to anticipate all possible states or interactions due to the inherent unpredictability. Autonomous behaviours introduce dynamic and unplanned actions that are difficult to capture using a static guideword-driven analysis.

Autonomous systems in uncertain environments also compound the difficulty in defining behaviours, as they often act in ways that evolve with context, adapting to environmental cues that may not be predictable in a traditional HAZOP Framework.

3.4 Applying the STAMP/STPA method

In this Section, we will attempt to apply the STAMP methodology to the same aspects as HAZOP and see whether methods help us to arrive to some Black Swan events. Black Swan events are discovered from unexpected or unfamiliar interactions within the analysed complexity field. For complete application detail, see Section G.2

3.4.1 Application of STAMP discussion

The STAMP (systems theoretic accident model and processes) Framework provides a structured approach to understanding and managing system safety through the lens of control structures. This method has several strengths that influence practitioners' attention and their problem-solving capabilities, particularly when addressing safety concerns in complicated systems like adversarial drone detection sensors and surveillance systems.

3.4.1.1 Strengths of the STAMP Framework

1. **Focus on hierarchical control structures:** One of the primary strengths of the STAMP method is its emphasis on control structures, which directs practitioners to examine how system components interact and exert control over one another. The hierarchical view of any complexity allows for some margins in uncertainty, because one abstraction (higher order of control) does not necessarily translate into exact behaviours in a lower order of the hierarchy, the same as an analogue or mechanical instrument.

Leveson's main intention in introducing a control-based cybernetics was to objectivise the problem and remove responsibility on a single component (like operator error). In her articulation of STAMP as systems theory-based, Leveson says that a failure event is a more holistic problem than a simple domino effect. In other words, she was trying to instil in our sense of reasoning that

if something goes wrong, any cause we discover is only partly to blame. And that failure is an emergent property bigger than the sum of causal factors. We need to manage human deficiency in judgement by designing between general control processes rather than firing them at will for making a mistake, because if we blame the operator, we will be biased away from realising potential flaws of the general control processes.

The nature of her chosen guide words to predict deviations of control actions, such as “What can go wrong if issued?” opens up a discussion in our mind to predict multiple possible answers. This is a looser approach than HAZOP simplistic yes/no guidewords (such as “no or not”). For example, when we analysed the interaction “1. Drone detection sensors and surveillance system detects adversarial drone”, then asked ourselves “What can go wrong if issued?”, we were prompted to consider a scenario where issuing detection can lead to something going wrong. So we predicted “false positives”. HAZOP does not provide guidelines on the correct delivery of control actions (see Table 2.3 HAZOP guidewords). They all assume a negative denial of service connotation. In effect, Leveson tried to encourage the consideration of side effects when things do go as planned. And that is the usefulness of applying a systems approach, which promotes the consideration of side effects.

Another example, where articulating the consideration of side effects went well, was limited by the control connotation, which is related to thinking about direct causality.

18. Railway traffic control system channels traffic in the track zone

Now, what could go wrong if the traffic control system effectively assists the train traffic in passing through the track zone as intended? That is the ultimate success scenario for the entire train network, including the eagle drone. It is akin to asking what could go wrong when the control command accurately directs the train through the track zone. Because we are biased towards the connotation of “control”, from a direct control perspective, nothing comes to mind. This represents a success scenario. AIC (appreciation, influence and control) aspect sees control as only 33% of the problem. 66% perception of a behaviour is based on “no-control”. This means we will need to think of what factors are indirectly influenced by the correct passage of control commands and other factors that must be appreciated such a successful case scenario. Well in this case, if we have an adversarial drone involved, then an adversarial drone would be appreciative of such success. It banks on it. So what can go wrong of control command succeeds? it can allow an adversarial attack scheme to be correct in its prediction of when the train will pass.

So what?

So, if we consider AIC, we realise that adversarial attacks' worst nightmare is the randomness of the train schedule and time delays. Hence, unexpected delays in different parts of the train network are part of the natural defence of the train network against planned adversarial drones. and the more reliable and predictable the network is, makes more vulnerable to intelligent trespass and adversarial attacks that use this to efficiently and last-minute attacks.

Now let me ask you this: When you read the thought experiment, did you expect that the success scenario of “safe and reliable train passage through track zone” is counter-productive vulnerability in the face of an intelligent adversarial drone? Did you expect an unreliable train network could make attacks harder (more likelihood of failure to achieve goals) and easier for users to catch off guard? How about such a perspective shift! We could predict by expanding our attention from control only to AIC. What would Network Rail's attitude become towards the problem of delays when they are told of such a complicated opportunity associated with an unreliable train network and the safety hazard of a reliable, predictable train network? Hence, we have the saying of “be careful what you wish for”. This is testimony for the fundamental difference between reliability and safety, and how context-dependent it is.

In comparison to HAZOP, the STAMP perspective fosters a more effective understanding of safety-critical processes by practitioners familiar with the concept of “control”. Yet it still retains the concept of hard dependency between elements for the practitioner to be concerned about such interaction type. The general notion of “The factor is not my concern if it does not make the train stop or topple”. It broadens the idea to go beyond the system of interest and enables teams to identify potential failures and unsafe actions related to control dynamics. By analysing how various subsystems communicate and manage each other's behaviours, practitioners can design more robust safety protocols and minimise the risks of accidents.

2. **Systematic problem identification:** STAMP systematically identifies and articulates problems within complicated systems. By considering the control actions and feedback loops in detail, practitioners can uncover underlying issues that may not be apparent through traditional failure analysis methods.

This method encourages a thorough examination of interactions and dependencies, enabling practitioners to pre-emptively address potential safety concerns before they escalate into real-world incidents.

3. **Application of general systems:** a unique feature STAMP attempted to bring to safety engineering practice is the application of general systems theory to model safety as a dynamic control problem rather than a static component failure issue.

By Framing accidents as results of flawed control processes, STAMP shifts focus from isolated events to system-level interactions and emergent behaviours. This approach integrates socio-technical systems (human, organisational, and technical components) into a unified model, enabling analysis of how mismatches in communication, feedback, or constraints across subsystems lead to hazards.

3.4.1.2 Weaknesses of the STAMP Framework

Despite its strengths, the STAMP Framework has limitations that can impede effective problem articulation, particularly in complex, dynamic environments.

1. **Ineffective² Cognitive remedy to a sense of realised ignorance:** Although the STAMP deviation guide words helped us predict more deviations, we did not consider them at the beginning of the analysis. Our focus on control action, which prompts considering hard compounding among elements, blinded us to considering softer factors.
2. **Hazard novelty test criterion:** None of the hazards predicted using STAMP in the problem domain passed the novelty criterion. STAMP focuses our attention on choosing guide words that we can easily imagine plausible scenarios for logically comprehending. If we thought a guide word made no sense or gave a positive outcome (not a hazard) we would ignore it as not applicable for safety. We are not compelled to think of a scenario or a situation in which the guideword could be meaningful.

For example, “18. Railway traffic control system channels traffic in track zone” control action. We couldn’t think of a potential hazard associated with “what can go wrong if issued?” We couldn’t think of anything new about it. If channelling the traffic through the track zone is issued, then there would be no hazard. Again, we couldn’t think of something out-of-the-ordinary, logical or plausible scenario.

2 By the term “ineffective”, we mean it does not help to consider that all factors, regardless of their relevance and likelihood, are equally concerning and thus require prediction and mitigation, or justifying why we should not bother about them. But they need to be discovered.

3. **Bias towards controllable factors:** The STAMP method often biases practitioners' thought processes toward directly controllable elements, making it challenging to envision scenarios beyond familiar boundaries.

This was apparent during the application of the step 'identifying system-level hazards' where we were tasked with predicting system-level hazards. This step heavily relied on our prior knowledge of potential hazards, focusing our attention on what we can observe in the problem (the system boundary definition). For instance, while practitioners may thoroughly analyse the detection and response systems for adversarial drones, they may overlook external factors or unexpected interactions that cannot be easily managed, such as environmental changes or unanticipated behaviours of objects in the patrol zone, like a fallen leaf or random litter carried by wind (such instances can be influential on the reliability of ml-based perception systems).

4. **Challenges with non-linear interactions:** the framework assumes that complexity can be simplified into manageable control structure problems. Deviations terms include the following:
 - No provision of control command, provision of control command, too early or too late or out-of-order control command, stopped too soon or applied too long control command

As you can see from the predictive guideword, focus on interaction where provision of control or influence is prioritised. However, there are interactions where no influence or control (appreciative) can be experienced. For example, the weather can influence the perception system, or we could assume it can control, but what about the interaction between perception and the weather? That is an appreciative interaction, not control. Such interaction would make no sense in the deviation analysis. Hence, they might be missed by a control-based predictive thought process.

Another example, a flying bird may send a specific control signal, as its presence is recognised to influence the algorithm's behaviour, not control it. The machine's perception of a bird cannot influence or control the bird's behaviour; it can only appreciate its presence. This disconnect between perception and control highlights a limitation in the STAMP model, as it fails to account for interactions outside the conventional control Framework, including those involving influence or appreciation (the absence of influence or control).

5. **Difficulty in predicting Black Swan events:** the STAMP approach may struggle to address Black Swan events, rare and unpredictable occurrences that have a significant impact.

When we applied the method, we realised that it leads us to think of the usual or expected failures; an expert in the field may easily pick up on them. However, the focus on control structures can lead to a neglect of low-probability, high-impact scenarios that are difficult to predict but could dramatically affect system safety. For example, the thought process of control structures may not anticipate the air turbulence created by the gap between a passing train and the track zone fence. It is hard to imagine how an expert in the field would think of a “train controller” to control the air between the train and the fence, resulting in underestimating potential risks.

6. **Vertical thinking bias and limit of lateral thinking:** given the inherent practitioner's sequential thinking bias, the implication of considering the concept of “control” can further limit the problem solver's ability to consider alternative scenarios or actions outside the established control paradigms.

Practitioners may become entrenched in a linear thought process that overlooks systems' complicated, interdependent nature, ultimately hindering innovative problem-solving and risk assessment. For example, if the practitioner starts the thought process with a physical controller system, it is hard to imagine some interaction outside the connotation of “control”.

In summary, the STAMP Framework offers valuable insights into safety management through its focus on control structures and systematic problem identification. However, its limitations, particularly regarding the consideration of non-linear interactions, Black Swan events, and the bias toward controllable factors, necessitate a broader approach to safety analysis. Practitioners must remain vigilant in exploring the complexities of their systems beyond the STAMP paradigm, ensuring they are prepared for unexpected challenges that lie outside the realm of direct control.

3.5 Applying the FRAM method

Overall, FRAM has proven to be a powerful method for handling system complexities and surfacing indirect and emergent risks. Its emphasis on functional resonance reveals layered interdependencies and allows for a deeper analysis of socio-technical systems compared to traditional methods.

3.5.1 Strengths of the FRAM Framework

Compared to STAMP and HAZOP, FRAM facilitates a more nuanced representation of system behaviour, enabling us to predict and analyse 92 variations across 30 scenarios. We were able to discover a greater variety of interactions compared to the other two. However. Most of the

discovered interactions wouldn't be considered as extremely rare and unexpected (not Black Swan events).

A particularly valuable feature of FRAM is its capacity to expose hidden or complicated interactions within scenarios. For instance, scenarios such as inadequate fencing and misaligned surveillance orientation illustrate how combined small failures can amplify error rates, contributing to false-positive or false-negative security alerts. This nuanced articulation includes scenarios where unexpected environmental conditions may hinder the effectiveness of a response team during an adversarial attack or where an adversarial drone's target zone deviates unexpectedly, confounding typical protective assumptions. Such emergent scenarios underline the strength of FRAM's resonance modelling, where even indirect relationships contribute to outcomes, expanding beyond the direct cause-effect associations emphasised in Frameworks like STAMP and HAZOP.

3.5.2 Weaknesses of FRAM's application

However, applying FRAM to our analysis has revealed several challenges:

1. **Hazard novelty test criterion:** none of the hazards predicted using FRAM in the problem domain passed the novelty criterion. FRAM focuses on assessing the suitability of variability (deviations) that we can easily imagine as plausible scenarios for logical comprehension. If we thought a guide word made no sense or gave a positive outcome (not a hazard) we would ignore it as not applicable for safety. The process does not prompt us to consider deviations which may not make logical sense.

For example, <train track zone facilitating safe passage for trains> output [safe transport of people] into the input factor of <passenger safe arrival to destination >, we couldn't think of novel scenarios to cover "too late" or "too early" variability, what aspect about "safe transportation of people" that if late or early will lead to loss of life? Late arrival at the destination is not necessarily a novel hazard nor an actual life-threatening hazard. For "object" guided deviation, we could assume that an object on the train tracks could lead to an unsafe travel experience. However, this is not a novel hazard but a common one. As for "direction", we could imagine the train performing a sudden reverse unplanned direction. And for the "distance" guide word, we cannot articulate a meaningful hazard in this case.

2. **Relationship interpretation difficulties:** one notable challenge in our case was interpreting relationships between functions such as track zone surveillance and adversarial drone activity.

FRAM traditionally presumes positively FRAMed operational relationships (e.g., inputs, preconditions), but defining such a relationship between surveillance and an adversarial drone is somewhat confusing since “monitoring” does not strictly serve as an input or control factor for the drone itself. This suggests a gap in FRAM’s relational categorisation when applied to scenarios without influence or control, yet not a background function.

3. **Ambiguities in defining control:** another ambiguity arose in determining whether the relationship between an adversarial drone and train collision should be classified as a control or input.

While traditionally defined control relationships in FRAM involve directly controlling behaviour, the operational reality is more complicated, where adversarial interaction influences collision risk without a governance intent. Adopting aic’s definition of control as governance in our method provided more generality. However, FRAM’s limitations in refining the boundaries of control relationships for adversarial interactions could benefit from more flexible definitions that accommodate hostile or competitive interactions.

4. **Selective feature utilisation and Black Swan events:** FRAM does not enforce exhaustive exploration of all CRIPTO (control, resource, input, precondition, time, output) features, potentially narrowing the scope for unconventional or "Black Swan" scenarios that are outside the limited type.

Some plausible interactions fall outside typical modelling logic; for instance, defining adversarial drone behaviour as input to environmental conditions initially appears nonsensical, but could inspire considerations for indirect drone influence on perceived environmental conditions information, leading to impacting drone performance. Adjusting FRAM to accommodate unusual scenarios, where interactions are counterintuitive, may improve its sensitivity to rare events and enhance robustness against Black Swan events.

5. **Functional assumptions and the problem of legacy bias:** FRAM assumes that system complexity can be reduced to specific “functional problems”. Each of which is solvable within its own scope. However, the guide questions aimed at model completeness, such as checking for unrepresented functions, are unlikely to inspire the discovery of new knowledge when all known functions are modelled.

Further, a complete and accurate FRAM model relies heavily on comprehensive domain knowledge, and practitioners with extensive experience may unintentionally impose "legacy bias". Restricting the imagination required to anticipate Black Swan events. Traditional assumptions may discount minor factors, like the impact of small insects near security drones,

and overlook potential failure points. Hence, FRAM's methodology could be enriched by an approach that deliberately challenges common knowledge, making it less dependent on existing expertise and better equipped to identify outlier risks. We solve this issue with the "perspective shift" technique to force practitioners to think laterally beyond their common wisdom.

3.6 Reflections from Applying Current Safety Approaches

To answer the thought experiment, the systems engineering approach is required to make us appreciate the following factors:

- Visual uncertainties in the train tracks (rarely concerning actions) may cause an indirect impact on the reliability of adversarial drone perception. The adversarial drone has no influence or control over the random visual actions that can occur, which may impact the adversarial drone's behaviour. Therefore, the train track zone is not entirely defenceless against drones. Random events that may not have been part of adversarial drone training may cause the drones to fail.
- For the adversarial drone to successfully perform its missions, its perception system must be reliable during rarely concerning and impactful scenarios. For example, the motion of air between

Using HAZOP, STAMP and FRAM we were not able to arrive at such conclusions.

3.6.1 Reflection 3.1: HAZOP performance

HAZOP is useful for designing deterministic, clear systems, but not for confusing, complicated systems to solve open complicated problems.

Applying the HAZOP method for problem articulation reveals some strengths and limitations. While HAZOP excels in systematically identifying known hazards and promoting a holistic analysis of system interactions, its effectiveness diminishes in highly uncertain and autonomous environments. The predefined guidewords used in HAZOP are not effectively helpful for recognising unpredictable risks (unknown unsafe interactions), such as "black swan" events. The method's design as a solution-oriented tool often renders it less suitable for initial complicated problem definition. It requires some advanced articulation and distillation into a more concrete solution architecture to be useful. Additionally, the dynamic behaviours of autonomous systems introduce complexities that HAZOP struggled to capture, highlighting the need for alternative approaches when addressing these intricacies. For example, we would not have been able to consider the motion of vegetation as an influential factor if we had continued the analysis.

3.6.2 Reflection 3.2: STAMP performance

STAMP performed better than HAZOP but remained constrained by a control mindset. Useful for White Swan failures but did not help us to predict Black Swan failures.

The STAMP (Systems Theoretic Accident Model and Processes) framework offers significant advantages over HAZOP in addressing safety concerns in complex problems, primarily through its focus on control structures and systematic hierarchical problem identification. However, its limitations are evident, including a bias towards controllable factors, challenges in accommodating non-linear interactions (within the sphere of appreciation or influence), and difficulties in predicting rare "black swan" events that could lead to missing requirements for system safety (if we compare it to AIC approach that takes into consideration two more natures beside control). This bias can hinder practitioners from considering unexpected influences or alternative scenarios outside conventional control paradigms, potentially resulting in a relatively incomplete risk assessment compared to AIC. Overall, while STAMP offers a valuable perspective on safety management (in comparison to HAZOP and, dare we say, FMEA too), its shortcomings underscore the need for a more comprehensive approach to identifying and articulating problems in dynamic environments.

3.6.3 Reflection 3.3: FRAM performance

FRAM performed better than the rest but remained constrained by a control. A lack of appreciative actions modelling, and a strict functional mindset. These may lead to missing critical Black Swan events in complicated environments.

The application of the FRAM method demonstrates its effectiveness in revealing more nuanced interdependencies and emergent risks within socio-technical systems, surpassing traditional approaches such as STAMP and HAZOP in analysing varied scenarios. While FRAM successfully uncovers some hidden interactions, challenges arise in interpreting relationships, particularly in cases where direct control or influence is lacking, such as the connection between surveillance cameras and adversarial drone behaviour. How can the function of "surveillance" directly control the movements of adversarial drones? Additionally, ambiguities in defining control relationships and the selective use of FRAM features may limit its ability to address unconventional or "black swan" scenarios, suggesting a need for more flexible definitions and adjustments to enhance its analytical robustness. FRAM, just like HAZOP and STAMP, is fundamentally based on the premise that interactions in complicated systems can be reduced to a set of control loops. AIC, on the other hand, involves two additional distinct interactions: influence and appreciation.

3.6.4 Reflection 3.4: Evaluating FRAM predictive thinking coverage using AIC

In Section G.4, Appendix G, we attempted to evaluate how the FRAM ontology can handle modelling the fundamental AIC framework to examine the completeness of predictive thinking involved in the FRAM modelling technique. It was an interesting piece of work because it helped us identify a list of potential types of situations that may be missed by those who intend to use FRAM.

We learned that, fundamentally, FRAM analyses may miss seven key interaction types, functions that merely “appreciate” other functions without controlling it, indirect influence constraints, unintended side-effects, hidden feedback loops, temporal ordering issues, non-functional environmental factors, and interactions among background functions. Thinking in terms of Appreciation, Influence, and Control (AIC) helps to enhance FRAM to catch them by prompting the architect to (1) explicitly declare which components strictly observe impactful aspects (2) distinguish soft “influence” links from hard “control” links to prevent unrealistic bypasses, (3) label potential side-effects as influences that may surface under certain conditions, (4) trace any cycle where a function’s output comes back as its own (or another’s) input, (5) enforce needed delays or activation sequences when an influence depends on prior control actions, (6) recognise which factors that cannot be modelled as functions, and (7) considering the epistemic risks associated with background functions, how one background factor (e.g., sunlight) influences another (e.g., rain) so that models account for combined effects rather than treating them in isolation.

3.6.5 Reflection 3.5: Guidewords-based approaches limitation

Guidewords may constrain an architect’s lateral thinking: traditional approaches are suitable for known concerns (guidewords), but struggle with emerging, rarely concerning events that fall outside the scope of what guidewords can inspire.

Limited guideword-based analyses inherently channel engineering judgment toward identifying the smallest, most directly relevant set of concerns (those issues that map neatly onto pre-defined semantic categories) to avoid overengineering. In structured domains with well-understood “known unknowns” such as passenger aviation, this minimalist mindset achieves high efficiency and safety by concentrating effort on the most probable failure modes. However, when the operational landscape is unstructured (populated by nondeterministic robots, human actors, open environments, and true “unknown unknowns”), the same drive for minimal coverage overlooks phenomena that lack obvious semantic ties to existing guidewords. By optimising only for the directly controlled, statistically frequent risks, practitioners inadvertently blind themselves to Black Swan behaviours, rare, emergent interactions or environmental factors that

fall outside the traditional guideword taxonomy but can nonetheless inflict catastrophic harm on system viability.

Hence, in our process, we abstained from employing such an approach during Stage 1, which pertains to problem articulation. Our analysis was limited to a guide-word-based evaluation exclusively during the HazTOPS step of Stage 3A. Furthermore, within the HazTOPS process, we prompt the architect to consider and envision all possible scenarios for each guide word. The omission of any guide word suggests the potential for overlooking a Black Swan event. Furthermore, to address any unintended residual ignorance resulting from restricting the architect's imagination to guide words, we implemented Stage 4, which involves a shift in perspective regarding the models designed in the preceding Stage 3A from an AIC perspective.

FRAM handbook highlights the main difference between the practice and STAMP based on the way of accident causality:

“Whereas the focus of accident analysis methods such as HFACS, TRIPOD, the Bowtie, and STAMP is on why something has gone wrong, the focus of the FRAM is on how something is done.”

Thus, it assumes the complexity of an organisation as a process of functional interactions that lead to an emergent outcome. For example, the complexity of drinking tea can be modelled as a set of functions involved in the process, then highlighting which functions control what and what resources are required for a function, what inputs, where an output goes, what preconditions needed to be available for a function to be enabled, and temporal constraints related to time or sequence.

Both STAMP and FRAM assume failures and successes as emergent properties where causality can be attributed to multiple factors. FRAM constrains the type of interactions among functions to 6 fixed aspects of concern we call them CRIPTO (which constitute the definition of completeness in terms of boundary definition of function):

- Control
- Resource
- Input
- Precondition
- Time
- Output

Then it instructed us to consider different types of variability per coupled functions scenarios between two functions. However, they both assume that a system boundary is whatever the practitioner considers their concern. That is a constructive and subjective view of boundary

definition, which leads to favouritism, totally dependent on hoping the practitioner's experience is enough to ensure that whatever they think is the genuine concern, is truly the only concern that needs to be considered. This means, the practitioner can pick and choose what to focus and what to consider as irrelevant.

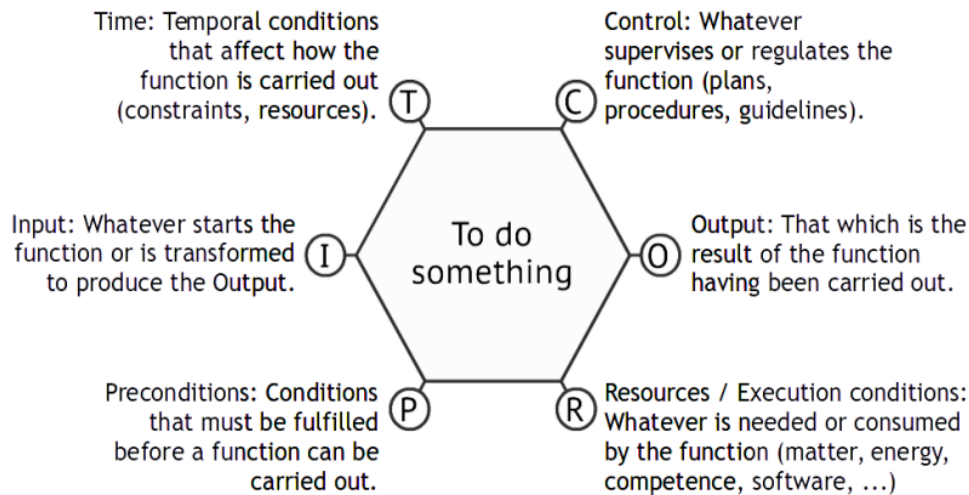


Figure 3.5 Functional cell of FRAM model taken from [16]

Suppose we apply the view of subjective choice of boundary in an open, complex environment, dominated by Black Swan scenarios (such as a moon appearing to be a traffic light). Would the traditional systems architect think of a moon looking like a traffic light? Or would they deem it as an extremely rare concern and irrelevant consideration?

Compared to the step-function-like HAZOP approach to problem complexity, FRAM and STAMP failure events and successes are considered a more holistic and hierarchical property that the sum of causal interactions cannot precisely explain. In FRAM, emergence is captured through cycles and functional resonance. In STAMP, emergence is captured through an abstract control structure assuming some theoretical controller. Based on this, we believe that the fundamental view of STAMP and FRAM about the complexity of a problem can be described in Figure 3.6, where the architect assumes that there is a subset of things worth my concern, which have direct control interactions, whose impact and therefore concerns profile is normally distributed. Anything else outside the bubble of the bell curve is not my concern. The main limitation we found in FRAM, which we believe prevented us from considering vegetation-related factors, was:

- How do we think of moving grass tips and vegetation branches as a function that impacts any of the elements described in Figure 3.2?

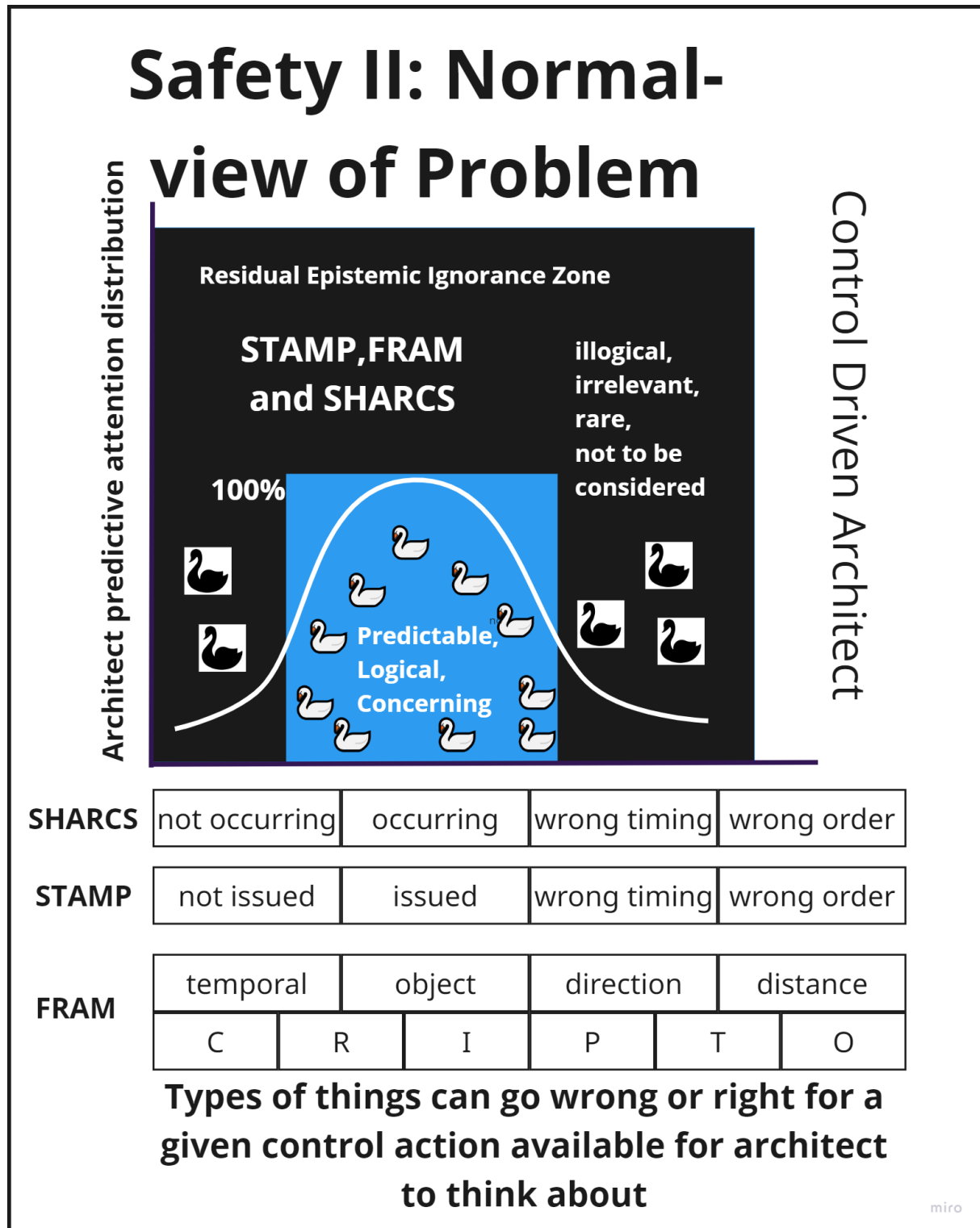


Figure 3.6 FRAM, STAMP and CHARCS fundamental perspective about the complexity of any problem domain

If we think about tangible control, such a factor is somewhat remotely in control of anything in the problem domain in the traditional systems sense. Such factor is rarely concerning and we have sufficient belief that traditional systems architects, using traditional systems approaches, may struggle with emergent rarely concerning failure modes outside what guide words can inspire.

3.7 Answering the exam question

In Section 3.1, we set out an exam question and challenged ourselves to answer the three questions. In the end, the methods could not truly give us a direct answer to them. FRAM was the most likely candidate to be able to pick up on nuanced interactions that could potentially answer the questions, yet the strict adherence to functional interpretation of complexity, with control being the dominant lookout interaction, kept our imagination rather rigid in figuring out those direct control actions and relationships among components.

FRAM, however, was less rigid than STAMP. It allowed for consideration of resonances where some interaction in complexity could lead to a change in another interaction, yet the model could not consider appreciative interactions (interactions where no influence of control is accepted). Both STAMP and HAZOP were religiously focused on the concept of control, which hindered our imagination from considering non-control interactions.

All in all, our questions remain unanswered. Hence, in the following few Chapters, we will describe a holistic systems approach capable of embracing HAZOP, STAMP, and FRAM and extending them further by introducing concepts such as influence and appreciation besides control.

Chapter 4 Introducing AIC Systems Approach Key Concepts

The authors of SACE guidance make a noteworthy observation about safety assurance approach for autonomous systems [19]:

“Demonstrating that the intent of the safety requirements is captured requires more than simply stating that a relationship exists between safety requirements at different tiers. Some explanation and justification for the sufficiency of that relationship must be provided. Concepts such as “Rich Traceability” could help in this regard.

Ensuring the intent of the safety requirements is maintained throughout decomposition may often be more challenging for AS than it is for traditional systems due to the sometimes large “semantic gaps” that can exist. The nature of these gaps and the challenges of addressing them for AS is discussed in more detail in.”

To demonstrate “Rich Traceability” [107] behind our approach, we look to a theoretical rationale to underpin our recommended design approach. Rich traceability focuses on the logical structure of requirements and the provision of how they are satisfied and qualified (through validation). An essential aspect is the provision of justification and rationale for why claims of satisfaction and qualification truly meet the requirements of a design. Since we are starting from the complexity of the problem domain, the architect will need to explain why they predict certain scenarios that are not being directly observed in the problem domain. So, how can they convince a client (and by extension a regulator) who may not have thought of some of those scenarios? This is where we need a first-principles-based theoretical background to help the architect understand why we chose such an approach and what it means objectively.

To help the architect understand deeply where our design decisions come from, we need to substantiate the rationale behind our proposed practice. To do so, we needed to understand systems from first principles and work out an approach based on our understanding.

For example, we needed to study uncertainty and its types in order to make sense of “Black Swans” and architect epistemic uncertainty, predictive thinking processes and how they relate to each other. We needed to theorise the definition of compounded uncertainty and the role of the architect's predictive chain of thought in reducing it. As for closing the semantic bridge recalled above, we needed to explain what CuneiForm is and how its

Chapter 4

pictorial abstraction method helps to transition meaning from regular safety requirements to the pictorial dataset; Thus demonstrating the intent of a safety requirement is truly persevered by the training and testing images. Hence, in this chapter, we lay out the substantiating theory behind our processes and thus enabling richer traceability to principles of systems, complexity and emergence.

This chapter introduces foundational concepts of the AIC Systems Approach, which integrates Appreciation, Influence, and Control (AIC) GST as a framework for understanding, predicting, and designing autonomous systems solutions for Compounded Epistemic Uncertainty Problems. The chapter defines key concepts, such as the distinction between complex and system, highlighting how AIC aims to transform Stochastic Confusion complexity into system complexity. Situations and scenarios are detailed using a structured notation to map relationships and interactions between entities, emphasising the significance of emergent complexity fields (a field is a domain which has a complex of subcomplexes).

The chapter also explores Supra-Systems, goals, and actions, distinguishing between primary and auxiliary purposes and actions (A-goals, I-goals, C-goals). Ordered and disordered AIC timing for harmonic and non-harmonic interactions is also explained. We also introduce CuneiForm, a pictorial requirements language that maps textual requirements to pictorial representations. It emphasises traceability and dataset validation. It systematically defines Machine Training syllabus through guidelines for characterising Target Objects of Interest (TOIs) and their environmental training classes, aiming to minimise epistemic uncertainty in computer vision training datasets.

The Multi-Operational Design Domain (Multi-ODD) framework is proposed for systematic environmental categorisation (to support the development of training concepts and training classes). It is supported by an uncertainty Grading System tailored for safety-critical applications.

Firstly!

What do we mean by an autonomous system?

By Autonomous systems, we mean:

- An engineered complex of interacting easy-to-predict and hard-to-predict engineered cyber-physical systems,
- Which employ non-deterministic ML-assisted and deterministic subsystems,
- Capable of making emergent autonomous decisions and executing emergent actions
- in response to foreseeable and unforeseeable emergent complexity.

Autonomous Systems are expected to manage an unbounded number of possible variations of desired inputs and accurately perform intelligent functions in all possible variations, whether trained (foreseeable) or untrained (unforeseeable) for them. Examples include a self-driving car or an autonomous drone.

4.1 AIC Approach to Complicatedness Resolution

In Section E.1.1 (Appendix E)³, we articulated the Uber fatal accident and realised that relying on a control-only type of thinking leads us to miss potential failures related to appreciation. Control-only thinking resembles the tunnel vision of reality, which caused the Uber architects, the unfortunate victim, and subsequently the machine (as a result of the Uber architects' omission of key scenarios in training) to overlook the criticality of thorough appreciation of the potentially complicated properties of the environment over which they have no influence or control being.

In Section E.1.9, we also learned that classifying types of systems based on the nature of the observed statistical distribution of events helps us understand why a control-only mindset is ineffective in dealing with an open and complicated problem space. That is because it can lead to bias, which may result in missing critical events and behaviours (Black Swans). This means we need a new approach that is more holistic and inclusive, allowing us to capture as many Black Swans as possible. Here, we found AIC (Appreciation, Influence, and Control) to be a very useful approach for handling complexity in CUP.

Dr William Smith, pioneer of AIC systems theory, defines the following [108]:

“AIC is a model that describes the relationship between purpose and power of a system as a part of some larger whole, and the purpose and power of the whole that impact the said system. The model is named after the three fundamental and universal relationships involved in the design of any purposeful system: the relationship to the whole (appreciation), the relationship between the parts of the whole system (influence) and the relationship of the individual part to itself (control). Just as the mixing of three primary colours can give millions of different colours, so too can the mixing of (A), (I) and (C) give millions of shadings of power relationships.”

³ Further elaboration on the theoretical grounding behind the concepts in this chapter can be reviewed in Appendix E, Section E.1, particularly regarding concepts related to the underlying general systems perspective. This chapter aims to provide the minimal systems-theoretical underpinning behind our processes.

AIC places people at the centre of their relationship with the environment. We place the predictive architect or observer at the same centre, looking at a complexity field. We re-used the AIC model definition and adapted it for a system engineering approach as follows:

AIC stands for Appreciation, Influence, and Control, a predictive mental model used to approximate an ideal system from any natural complex system. The perfect system is a predictable complex to which a problem-solving architect, a predictive observer, is sure of its purpose and how its AIC interactions affect its operational environment, and is affected by the environment's AIC interactions with it in all possible scenarios.

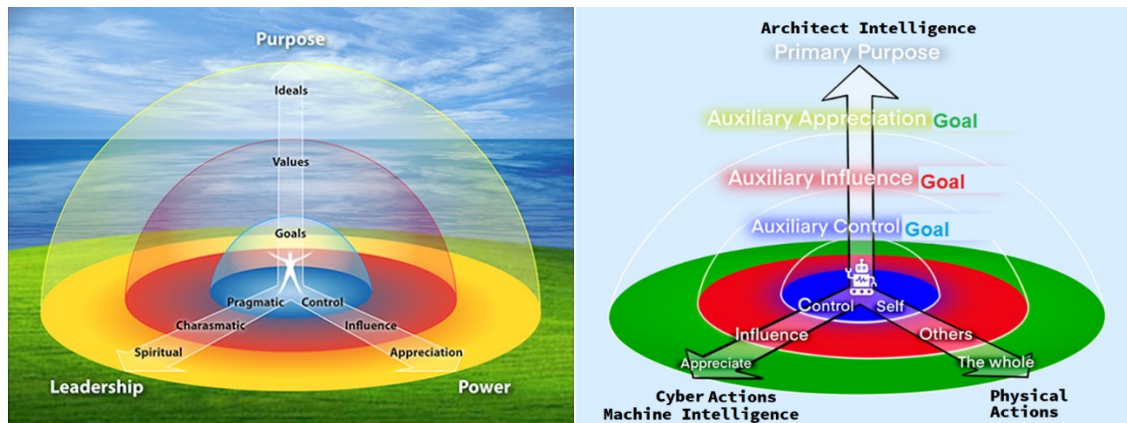


Figure 4.1 AIC Systems Approach for Architect, Machine Intelligence and Goal Hierarchy in Autonomous Cyber-Physical Systems (right). AIC model as defined by Smith (left).

The depiction in Figure 4.2 highlights the AIC Systems Approach (right-hand figure), which integrates machine intelligence, cyber-physical actions, and architectural intelligence to address CUPs, thereby designing trustworthy Autonomous Systems. The visualisation captures three interconnected dimensions:

1. The Cyber actions and Machine Intelligence.
2. The Physical platform and actions.
3. The Architect Intelligence and predictive thinking approach.

Furthermore, it also encompasses the following operational domains:

- Self-Control and immediate others within sphere of controls (easier-to-manipulate), to;
- Influence other complexes within the sphere of influence (harder to manipulate directly).

- Appreciating the sphere of the whole (environmental behaviours that cannot be influenced or controlled by the ideal system capability), only appreciated (cannot be manipulated).

Figure 4.2 Horizontal Plane: represents the complexity field of the design and operational domain. This plane represents the actions and operational components of the system:

- **Cyber Actions:** Actions executed through software and algorithms (e.g., decision-making, optimisation).
- **Physical Actions:** Within-immediate-physical-reach actions involving hardware and interaction with the physical environment (e.g., braking, patrolling).
- **Machine Intelligence:** The decision-making and operational logic that enables the system to manifest the architect's will. It also includes the knowledge system (training dataset), which enables the model to exhibit intelligence.

Traditional Approach Limitation: Conventional system design approaches (such as HAZOP, STAMP and FMEA) are fundamentally based on viewing the complexity of problems to be reducible to two categorical events:

- Failure Events that are of obvious concern.
- Rare or less impactful failure events that can be neglected.

Such an attitude aligns with a tacit assumption that the observed statistical distribution of failure events can be reduced to be normally distributed complexity. They mainly operate within the sphere of control, assessing the system's ability to meet functional requirements. However, this approach alone is insufficient for addressing the complexities of AI in dynamic environments. A more comprehensive architecting strategy is necessary, one that goes beyond traditional control-sphere-based system engineering Processes typically used for conventional systems.

The primary distinguishing challenge between engineering AI and non-AI systems is predicting unconventional 'Black Swan' events that no one would have anticipated. For example, an autonomous car may suddenly change lanes unexpectedly because it has learned a pattern that the driver always used to change lanes at a particular time and place. Another example is when a Tesla car perceived the moon as a traffic light [109]. Architects require innovative techniques to anticipate unexpected, high-impact, and unforeseen events. Relying solely on conventional processes, such as HAZOP and STAMP, and treating the challenges of Autonomous Systems as traditional problems can weaken confidence in safety cases and evidence claims.

Figure 4.2 Vertical Plane: represents the knowledge of the operational complexity field and the predictive architecting domain. This dimension represents the Architect's Intelligence (as a function of epistemic uncertainty reduction), which brings predictive, non-biased reasoning to

the system's design and operation. The following are some of the factors involved in this dimension:

- The architect's knowledge system is part of the design problem.
- Architect intelligence involves understanding the system's environment's past, present, and future complexities.
- The key constraint that impacts the whole is the architect's epistemic uncertainty.

Key Insight: To trust the machine, we must trust the safety case, and to trust the safety case, we need to trust the architect's knowledge system to be clear of any epistemic uncertainty as much as practically possible. Architect intelligence ensures that the machine acts as expected in all foreseeable (defined) scenarios and has good insight into how it will behave in unforeseeable scenarios, comprehensively addressing uncertainties.

Color-Coded AIC Goals:

The color-coding schema is inspired by AIC systems theory. The depiction uses the RGB colour scheme to represent three types of goals or relationships:

- Blue (Control): Direct, active governance over a situation (e.g., patrolling an area).
- Red (Influence): Indirect control to influence or achieve a desired outcome (e.g., inhibiting adversarial behaviour).
- Green (Appreciation): Understanding situational awareness without controlling or influencing (e.g., adapting to weather conditions).

These goals form a hierarchical relationship (more details in Section E.4):

1. Appreciation (green) enables control (blue),
2. Control and Appreciation enable Influence (red),
3. All of these goals collectively support the manifestation of the system's Primary Purpose.

With the above in mind, the following is a set of concepts that will be used in the approach:

4.1.1 Systemness of a General Complex

We start our articulation for a general definition of a system (which we need to base the definition of Compounded Uncertainty Problem) by defining a general fundamental term:

General Complex: a set of coexisting elements and events (regardless of nature) that may or may not be interacting. The definition also includes all possible relationships or interactions that could emerge among those elements.

We reuse Bogdanov's definition [30] of the complex as an entity, a combination of elements. We distinguish complexes by saying that not any complex of things is a system. However, the general systems scholars tend to define a system as everything is a system. George Mobus introduces the concept of Systemness [110]:

“Systemness: Bounded networks of relations among parts constitute a holistic unit.

Systems interact with other systems, forming yet larger systems. The universe is composed of systems of systems.”

Now, the definition remains too abstract to be practical from an objective engineering perspective, where we need a quantifiable notion to assess as precisely as possible whether we are right or wrong about something. In our view of systems engineering, the problem boils down to the following general test:

Have I produced a predictable and explainable set of interacting situations (a complex), regardless of any possible context this complex may find itself in an operational environment, such that my intended purpose is guaranteed for as long as the complex exists?

In more detail,;

- Have I managed to predict all possible contexts of the operational environment?
- Have I made a predictable and explainable system of responses to that context?
- Have I guaranteed that the primary purpose of the engineered complex shall be preserved in all possible operational contexts?

All in all, we see ourselves repeating one primary concept, “predictability” of a set of situations or events. Where there is predictability involved, there is associated “probability”. Where there is probability, there is “uncertainty”. Luckily, all concepts “Predictability”, “uncertainty” and “statistical distribution of an observation” are measurable. Therefore, for us to base our systems approach on a quantifiable definition of Systemness, that we can re-use and validate our creations and models, it will have to be associated to:

- Predictability
- Uncertainty
- Statistical distribution.

From that, as part of our foundational theory, we will make an assumption about the Systemness of a complex and say:

Systemness of a complex is the presence of predictable rules that govern the randomness of the observable statistical distribution in a complex. We also understand

Systemness as the likelihood that a complex verifiably adheres to constraining rules assumed by a predictive architect.

An example of a complex that has the capacity to be studied and predicted would be an ideal gas piston. Although the ideal gas's microscopic evolution is random and chaotic, the emergent statistical rules (Maxwell–Boltzmann distribution, ideal-gas law, etc.) are robust. Known physics fixes the set of possible interactions (elastic collisions, boundary reflections), and there is no hidden “intelligence” reprogramming the particles on the fly. Therefore, an architect's hypothesised “constraint” (e.g., “every experiment at volume V and temperature T yields pressure p obeying $pV=Nk_BT$ ”) is correct to within diminishing error margins.

Some predictability exists in the set of events that will have an impact, as well as in the predictability of the observed statistical distribution of those events occurring in a complex of interacting situations that affect the belief state of a predictive observer's uncertainty (whether an architect or a machine). Such that:

- **Complicatedness** *is a function of the predictability of a complex relative to prior knowledge of a predictive observer about the complex. We define Complicatedness as being directly related to the epistemic uncertainty of the predictive observer (in case of the architect, it would relate to the architect's knowledge and the process of knowledge acquisition, and for the machine by the training corpus and the training process for the ML model regarding the problem space) on the observer's predictive attention management, which entails determining:*
 - *What types of events should be paid predictive attention to.*
 - *Which can be guided (influenced) by the type of ontology used to describe any observation. For example, AIC interactions or HAZOP guidewords.*
 - *What appropriate distribution of predictive attention should be. T*
 - *Which can be classified into: normal distribution or abnormal distribution (e.g. long-tailed or uniformed distribution). For. For example, in HAZOP, the architect is prompted to focus only on what they perceive as immediately necessary. Attention distribution can be biased.*

In other words, Complicatedness costs predictive attention resources.

Chapter 4

Complicatedness affects the predictive observer's predictive attention management, which is heavily dependent on the quality of the mental model used to process information perceived about the statistical distribution of an observation. Mental models can be characterised by their comprehensiveness (depth and layers of hierarchy) and the efficiency of their ontological system in describing observations, allowing for accurate predictions. By efficiency, we mean how much predictive processing a predictive observer should undertake to resolve the confusion of complicatedness using an ontological system and attention distribution strategy.

After all, ontology also requires memorisation and processing to classify things in any observation, which also consumes predictive thinking resources to determine what, and how to allocate attention. Biased predictive attention distribution tends to prevail over time and increase in scale of complexity, as it allows for more concentrated attention on fewer types of things (i.e., fewer moving things to worry about), thereby increasing confidence in the predictability of what is being observed within a some sphere of concern. Ultimately, predictive attention serves a primary purpose, which is to create a sense of resilient confidence that the organisational experience for the predictive observer will never become confusing at some point in the future.

Complicatedness resolution of a complexity depends on the breadth of knowledge possessed. It can be analysed by monitoring the distribution of attention paid by the predictive observer, focusing on which types of events are being focused on and which types are not. At the same time, ontology resolves complicatedness in the form of epistemic uncertainty reductions, aleatoric uncertainty, a property of the observable statistical distribution of an observation, and challenges ontology's vulnerability. Hence, over time, ontologies (for example, HAZOP, STAMP, FRAM) tend to be challenged in handling the prediction of unknown unknowns at a more complex scale efficiently.

Therefore, while epistemic uncertainty is reducible, complicatedness may be efficiently reducible (the rate of epistemic uncertainty reduction with respect to aleatoric uncertainty increase), depending on the nature of the approach's predictive thinking model that guides the predictive observer's predictive attention management of the observation.

There are three types of impacts complicatedness can have on the predictive observer's predictive attention management:

- **A Clear complicatedness:** where the architect's epistemic uncertainty about a particular complexity field, past, present and future events and situations is relatively low. You can think of it as “complicated clarity”.

Such a state of organisational experience between the architect and the observable statistical distribution of an observed phenomenon (let it be events in an engineered system or an environmental complex) characterises the Systemness of the observed complex as clear and predictable. Such a state defines the observed complex as **a system** relative to the observing architect. The architect is clear that their predictive attention distribution is sufficiently adequate. In terms of AIC, such a system can be said as “controlled”. This means the architect can narrow down the focus of their predictive attention to fewer factors than initially needed. An **ideal system** is a complex whose complexity is entirely predictable, for example, a light switch.

- **A Confusing complicatedness:** where the architect's epistemic uncertainty is relatively high about a particular complexity field, past, present and future events and states. You can think of it as “complicated confusion”.

In contrast to systems, when the architect is unsure about the adequacy of their mental model construction to make sense of the statistical distribution of observations, it renders the observation as stochastic confusion. This does not constitute a system for the observer architect. Nonetheless, AIC provides a solution to deal with such a situation, and that is to “appreciate” which means to accept that all possible events and things have equal distribution of attention.

Stochastic confusion complex with respect to a predictive observer is not a system.

- **A fuzzy complicatedness:** a mixed spectrum of clarity and confusion. It sits in between the two opposing attitudes. The architect may give a long-tailed attention distribution over things involved in the complexity, which means paying attention to some, but keeping an open mind (lateral thinking mind) to any other rarely concerning factors involved in the observation.

Now, we have some level of understanding about what complicatedness is, which is a shared property between the statistical distribution of the observation and the attention distribution of the observer. But the statistical distribution of an observation exists outside the predictive observer world. And that is complexity.

- **Complexity** is a function of the nature of the randomness of the observable statistical distribution of event occurrences among the elements of a complex. We define Complexity to be related to the aleatoric uncertainty associated with the observation. It can be of two possible natures:

- Normally distributed.
- Abnormally distributed (long-tailed or uniformed)

Chapter 4

Complexity cannot be reduced by collecting more data. However, the more information gathered, the clearer the nature of complexity becomes for a predictive observer. The scale of information gathered about a complex may or may not be manageable by reducing its complicatedness for a predictive observer.

We realised the definition above after observing the output of our AIC analysis in Stage 1 (see Section 6.2.4) Additionally, after conducting experiments in Section 7.2, we trained an ML model and tested its confidence in a set of images that capture the observation of systems interacting. We noted that the ML model lost considerable confidence in Black Swan datasets (a testing dataset which was visually out-of-distribution with its prior training and validation). We trained a model with nearly 85,000 images of drones⁴, and when we tested it on a set of out-of-distribution images (which can be thought of as informational systems), the model failed drastically. But for us as humans, those hard images for the ML are easy. This situation inspired us to realise that:

- Black Swan images had complexity. This complexity is somewhat related to the histogram distribution of a set of pixels that capture certain physical events. They appear meaningful to us, and we can easily predict what is in them.
- Those Black Swan images appear to be **confusing** for the ML, **harder to predict**, but they are **clear** for us, **easy to predict**.

Since the images are clear and predictable, we can see the presence of a recognisable pattern of a system. For a sufficiently trained model, Black Swans were unpredictable; clearly, there is no system. So, the definition of “system” or “no system” was purely based on whether an observed statistical distribution was predictable or unpredictable with respect to a predictive observer's prior experiences and intelligence (inference methods).

4.1.2 The Predictive Architect

The predictive architect is also a complex part of the problem complexity.

We define the architect as a predictive problem-solving agent who relies on systems approaches to construct predictive mental models that aim to make sense of and predict complexity fields with the intention of either appreciating, influencing, controlling, or all of these. Our definition of the architect draws inspiration from Klier's definition of GSPS [17] and Bogdanov's understanding of the relationship between the observer and the organisational experience that occurs in complicated systems (see Section 2.1 literature review). In this, we care about how we can objectively identify if a problem space complicatedness solving approach (like HAZOP, STAMP

⁴ See Section 6.9, H.10 for the experiments we conducted.

etc) is appropriate to the nature of the problem being approached (for example controlled space engineered system like a traditional car vs an open complicated environment with autonomous cars operating in).

The predictive architect relies on strategic predictive attention management to make predictions about observed complexity. That entails two activities:

- What types of things to distribute predictive attention over.
- What type of general predictive attention nature to use for a given observed complexity.

In other words, should the architect approach the complexity with normally distributed attention (which means assuming there is a finite and limited number of events worth paying attention to and others can be negligible) or abnormally distributed attention (assuming all things deserve equal attention or at least there is a set of things that can be used a priori for attention, but keep an open lateral thinking mind to welcome rarely concerning things too).

For the definition of systemness, we can understand four different possible organisational experiences between the architect and the observed problem space:

- **Normal predictive attention distribution paid → Normally-distributed statistical complexity:** In this case, the architect is correctly adopting a normally distribution attention to a normally distributed statistical observation because the complexity of the real world is indeed normally distributed. Such interplay can be described as being in harmony and is effective.

Normal complexity means there can only be a finite type of events happening and nothing outside that, like a traditional car. Event-B is a normal attention distribution approach, which means it helps the architect to discover those specific events and thus help with coverage. Hence, Event-B modelling can be used to resolve the complexity of designing a traditional car.

- **Abnormal predictive attention distribution paid → Abnormally-distributed statistical complexity:** In this case, it can also be said that the organisational experience between the architect's approach to resolving the complicatedness of a long-tailed statistical distribution of problem space is in harmony or efficient.

This is also the right approach to the right type of problem space. For example, applying AIC to solve a complicated problem like the train tracks zone security (chapter 6). In other words:

Normal thinking is sufficient for normal problems, but Abnormal problems require abnormal thinking.

Now, you can probably guess what a less efficient attention management approach for a problem space type might be. That is:

- **Normal predictive attention distribution paid → Abnormally-distributed statistical complexity:** Here, the architect who has had successes in the past with normal complexity through normal attention may lead to a fundamental misjudgements, increasing the likelihood of failure due to missing types of information and requirements. For example, applying HAZOP guide words to predict failure in open, complicated environments.

However, the other type of misalignment; **Abnormal predictive attention distribution paid → Normally-distributed statistical complexity**, may not necessarily lead to engineering misjudgements, but it may lead to over-engineering. This is entirely dependent on the level of rigor is required, for example, developing a new airliner, although aircraft are normal problems, meaning they are deterministic (yes with large scale), applying AIC can be justifiable and useful to speed up the discovered of unknown unknowns.

To better describe the idea in this Section, below is a diagram illustrating how various attention distribution strategies handle the complexities of problem space in Figure 4.2. The figure demonstrates the two types of predictive attention distribution that problem-solving approaches inspire in the architect's attention. Our analysis of HAZOP and STAMP revealed that they focus our attention solely on control aspects, which means they consider direct interactions, such as one complex directly impacting another. AIC (Appreciation, Influence and Control) prompts the architect to think of the problem like a fractal. It is an abnormal distribution of attention to an abnormally distributed complexity. Whereby, some interactions cause direct changes among interacting complexes, while others are more subtle, occurring over time and with complexity delays, and are not observed immediately. Additionally, some interactions cannot be influenced or controlled but must be accounted for nonetheless.

4.1.3 Compounded Uncertainty of Open Complicated Problems

“The complexity of an object is in the eyes of the observer” Klir [17]

SACE (Safe Assurance of Autonomous Systems in Complex Environments), guidance suggests the following observation about the relationships between autonomous systems and an open operational environment:

“The different possible scenarios and environmental states that relate to the decision can then be identified by considering the real-world state and the belief state of the AS at the point at which the decision is made, along with each of the possible options. The real-world state represents the actual state of the operating environment, whilst the

belief state represents the understanding that the AS has of the state of the operating environment”

We can summarise the observation as a general expression:

The open real-world states influence the AS open-ended belief state about them, and vice versa.

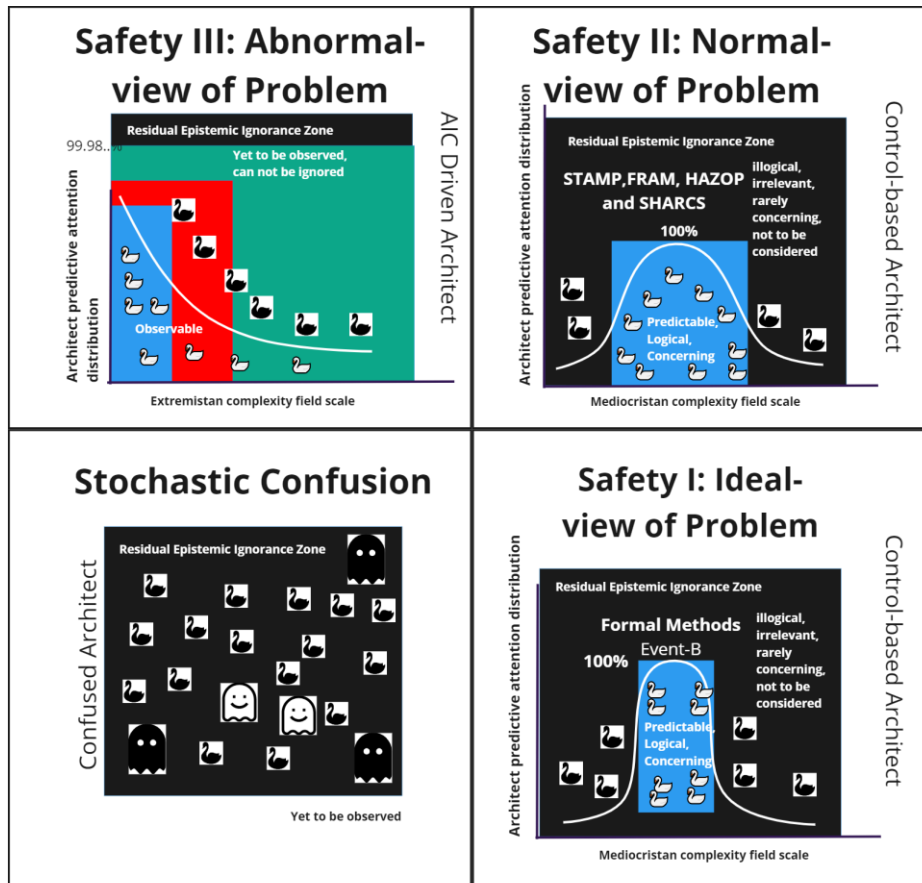


Figure 4.2 Types of architect approaches to complexity of in CUP. Compounded Epistemic Uncertainty Problem (CUP)

Why did we say “open-ended”? This is because when we teach an ML model to identify a person (for example), we don’t specifically tell it what a person exactly is (in a similar fashion as we would specify an acceptable voltage value for a deterministic controller). We give it an open-ended characterisation, open to infinite possible changes of what a person is. The pictorial definition of a person depends entirely on the context within which the ML model operates. The challenging part here for the architect is predicting all possible contexts that the problem domain may evolve into, so that they can be accounted for in the training process.

In other words, we provide many possibilities of what a person “could be”, but ultimately, we expect the trained ML model to decide what constitutes a person when it observes one, hence it is always a probability of what it has learned in general through our datasets. Thus, “openness” can be understood as the boundlessness of possibilities, some of which are “meaningful” or “intended through design”, others are “meaningless” or “unintended as a by-product of intended design”. By “meaningful”, we mean we train a model to understand something significant to us (solving a particular problem) and expect it to follow through on its own.

So, how do we understand the parts of the problem articulated by SACE?

Well, it seems there are a number of key concepts involved which we need to make sense of and work out a general solution for them as a whole:

- Openness of the operational environment and open-endedness of machine understanding of the operational environment.
- Real-world states.
- Machine’s belief states.
- How does uncertainty behave in a problem involving an open, complicated environment and an open-ended belief and decision maker (other than a human)?

We further add another component to the above articulation, which is essential to the general systems approach solution:

- The architect’s (as the problem solver) approach to managing their predictive attention and how epistemic uncertainty can be reduced about the open environment and the autonomous system behaviour in response.

We look at the components of the problem from the standpoint of “Architect vs Problem” , so we understand the situation as:

- Architect predictive capability problem (ArcPC).

In the above articulation, we recognise three main problems to manage:

- The operating environment is open and complicated with a multitude of real-world state types (dominated by hidden Black Swan real-world states).
- We expect the engineered AS to behave or execute its designed belief states as expected in those black swans.
- The architect's belief system that predicts and defines those states.

In this context, the **architect's belief system** refers to the internal mental model, predictive assumptions, and reasoning framework the architect uses to understand, anticipate, and design for both the real-world operational environment and the behaviour of the autonomous system (AS). It encompasses the architect’s current knowledge, expectations, and uncertainties about how the AS will perceive and act in open, unpredictable scenarios, particularly in relation to unknown or Black Swan events. So, from an architect's belief-system development perspective, the more categories of parts and the more parts in each category or type are involved in the open

environment and the behaviour of their engineered system, the faster the architect's epistemic uncertainty about the robustness of their assumptions compounds. Hence, we use the term “Compounded Uncertainty Problem” or CUP in short.

A **Compounded Epistemic Uncertainty Problem** is a Stochastic Confusion (non-system) involving agents, deterministic systems and an open, complicated environment, where any changes in any part of the problem lead to unpredictable whole for the predictive observer. The epistemic uncertainty of the predictive observer is understood to compound nonlinearly as the complexity of the problem domain changes unpredictably.

There is a set of properties in such problems which can be sources of risks:

1. Any prior beliefs about such problems by the observer can be volatile. There is a very high likelihood that prior beliefs will not prevail over time.
2. Confusing, purposeless, uniformly or non-normally distributed complexity of events and behaviours.

4.1.4 General Lateral Thinking Process

Lateral thinking, popularised by De Bono [111], involves solving problems not in a structured step-by-step manner. It deliberately diverges from vertical thinking by embracing creativity and non-linear problem-solving approaches. Unlike vertical thinking, which follows a linear, stepwise, and methodical process to develop existing patterns within strict boundaries of relevance and certainty, lateral thinking seeks to restructure these patterns through multidirectional exploration. While vertical thinking prioritises depth (in a decision tree-like manner) of knowledge and systematic progression, rewarding precision and logic, lateral thinking thrives on uncertainty, welcoming novel emergence and unexpected connections. We view lateral thinking as a key to solving CUPs. It encourages breadth over depth, often drawing from irrelevant or unconventional information to spark the discovery of new scenarios. For further elaboration on the impact of lateral thinking on safety in open, complicated environments, see Section E.1.6.

However, our approach to systematising lateral thinking (for repeatable use) will be based on our understanding of uncertainty (see Section 2.1.1.5 for the literature). We view that the fundamental difference between Vertical thinking, and lateral thinking lies in the architect's view of the world, and there are two ways we recognise as distinct views of how the observed statistical distribution of events in a complexity:

- **Predictive attention should be equally distributed over all factors in a Complexity:** All possible categories of events (known or yet to be known) are equally important and

deserve equal attention (failure events, impactful events). This means illogical interactions and rarely concerning events in a complicated environment **are equally** important for consideration with logical and perceived important events.

- **Predictive attention should not be equally distributed over all factors in a Complexity:** not all possible categories of events are equally important; only a subset is worth attention (failure events, impactful events). This means illogical interactions and rarely concerning events in a complicated environment **are not equally** important for consideration with logical and perceived important events.

We understand the architect's cognition as a black box, where both conflicting views of the world can sometimes lead to confusion about which approach to start with or preserve. In the vertical thinking school of thought, as seen in HAZOP and software engineering, the general guidance revolves around adhering to the principle of finding the most resilient subset of assumptions about the operational domain that are necessary to engineer a desired subset of events.

Our view on the insufficiency of systematic and vertical thinking to solve CUP is also shared by Johnson [112] in his research into a theory of engineering Complex Adaptive SoS (CASoS). The primary differing view for us is that we consider CASoS from the perspective of a predictive problem solver and examine what tangibly happens to their predictive capability, which is the primary concern of our predictive systems' thinking approach. We also have a more distinct definition of “complexity” and “complicatedness” (see Section 4.1.1).

From this stance, we understand Jackson's concept of “Critical Systems Approach” [113] to be an artistic, objective approach to combining lateral and vertical thinking processes to establish well-reasoned confidence about a solution for CUP.

We view lateral thinking as a form of predictive thinking where the problem solver spreads their attention laterally to other possible interactions that are initially deemed irrelevant or illogical (using vertical thinking). There is an element of uniform distribution and the pursuit of indifference between how logical a subset of events is and how illogical another subset of events is. Hence, to understand this further, we need to understand two foundational principles of uncertainty that we need to understand are the principles of indifference and maximum entropy:

The **principle of maximum entropy** tells us that when we wish to assign probabilities to describe a system but don't know much about it, we should choose the observed statistical distribution with the **highest entropy**, in other words, the one that spreads the probabilities as widely and evenly as possible, given the constraints we know [114]. This means that, if the architect must estimate the real-world distribution of the time-period-of-a-day complexity, in an ideal world, the architect should assume the highest entropy possible. It doesn't directly tell us to assume full-on uniform distribution; this is where **the principle of indifference** comes in. It says that when you

have no specific knowledge about a system, you should assume all outcomes are equally likely [115].

Well, how is the principle of indifference going to help us?

It guides us in managing our attention-distribution strategy when facing complex situations.

The architect starts being confused about the diversity and ideal number of training examples to include, nor has an idea of what a sufficient distribution of the dataset is allowable (considering that projects are limited in resources and the availability of datasets, the skewed nature of the real-world operational domain, or the costs to generate dataset may limit the architect ability to guarantee a balanced distribution). The principle of indifference tells us that:

It is better to consider a training syllabus that uniformly distributes the data examples across all training classes than a training syllabus that results in a biased distribution. However, prior to this, the architect's predictive attention must be equally distributed to all possible interactions and events within a complexity field. Here, vertical-thinking approaches may not be practical because they constrain the architect's attention to only a finite number of event types.

Based on these demands and considering that architect epistemic uncertainty impacts directly the quality of ML development datasets in representing the real world, It is best if the systems approach encourages a uniformly distributed view of the real world complexity, in which all possible interactions within the complexity field of the problem are equally valid and there is no superior subset of them worthy of more attention than others. AIC Matrix captures all possible interactions in a problem, and our approach guides the architect to make engineering judgement about each.

From the above, to systemise a general lateral thinking process, the following principles need to be applied by the predictive thinking:

1. To reduce the risk of a normally distributed attention in CUP, we need N^2 matrices that map complexity elements to each other and define all possible interactions. This is the basis for ArcMatrix and Action Matrix tools in Section F.1.
2. Prompt the architect to broaden their view of how general rules of complexity may emerge or be designed concerning less about strict logical tractability.
3. Consideration of analogies between unrelated events or scenarios.

Based on the three principles of lateral thinking, and a thought experiment we conducted in Section E.4 we proposed a general predictive lateral thinking process in Section E.5. The process entails three predictive steps:

1. **Broaden Your Perspective:** Deliberately set aside whatever assumptions and views you made about the system in its widest context, treat even “irrelevant” elements (e.g., a stationary lamp post shadow) as potential influencers, and consider both direct and indirect interactions. In our process, we set the base perspective during Stage 3A and then we simulate a state of relative chaos in the design complexity field by shifting the AIC perspective in Stage 4.
2. **Draw Cross-Domain Analogies:** Look beyond your immediate attention distribution for inspiration, compare your problem’s components to analogous elements in unrelated domains (e.g., how a cloud may look like a tree, or a moon lay look like a traffic light (Tesla incident)) and even entertain humorous or counterfactual scenarios to spark unconventional insights.
3. **Formulate & Iterate Predictions:** Turn each insight or analogy into a concrete hypothesis about possible system behaviours (e.g., how shadows might fool sensors) and continuously loop these hypotheses back into your knowledge base, refining, re-associating, and expanding your understanding to capture ever-emerging “Black Swan” scenarios.

By following those three steps, the architect would have applied the principle of indifference, in the sense that their attention distribution is indifferent to any type of event or factor in a complexity field. We used this process to objectively explain how interactions in our case study (see Sections 6.2.2 and H.4.2.1), though appearing unrelated (such as the movement of trees and cars driving across a bridge), could be understood through the lateral thinking process, allowing us to identify the interaction that we accepted as relevant. We contend that would be unlikely to even think of such interaction if we had approached the problem from a vertical thinking perspective.

4.1.5 Situation and Interaction

A situation describes a complex in a specific dynamic or static state. For example:

- "Unsafe train track zones" (state: unsafe; complex: train track zones)
- "Secured passage of trains" (state: secured; complex: passage of trains)

Generally, a situation combines an adjective (describing some state) with a noun (the complex), sometimes including an adverb to express a predicted quality. We use a particular notation to capture the situation:

{adjective+noun}

Example: A situation involving a food delivery robot could include multiple states:

- A robot safely (predicted quality) passes through (situation 1) heavy traffic roads (situation 2), busy sidewalks (situation 3), in-between fast-walking pedestrians (situation 4), and a cordoned-off crime scene (situation 5).

Each of these 5 situations has a complicated description in a particular state.

- **Situation 1:** {safely passing robot}
- **Situation 2:** {heavy traffic roads}

An interaction, on the other hand, is a sequence of two or more interrelated situations. For instance, the food delivery robot navigating through the above situations forms a scenario. The emergence of a scenario can be expressed in the following format:

[{source situation}]_[AIC-action]_{sink situation}

Written in the following syntax: **[{adjective+noun}]_[verbal phrase]_{adjective+noun}**

For example: **[{accelerating robot}]_[monitors]_{heavy traffic roads}**

The structure of the general interaction format comes from adopting INCOSE definition of emergence:

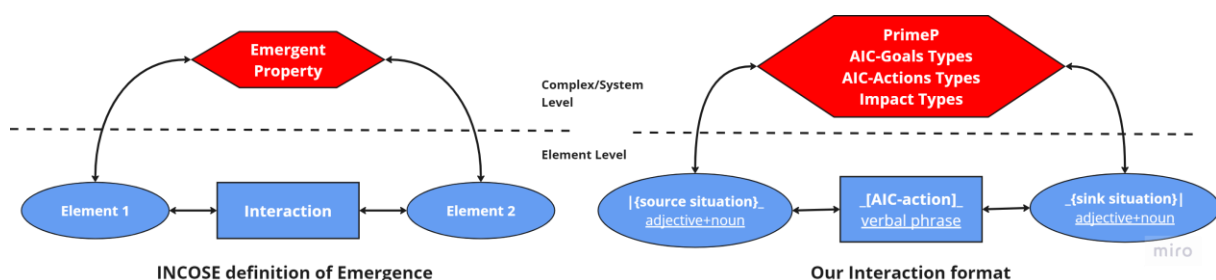


Figure 4.3 INCOSE definition of emergence (left) [116] and the format of interaction (right)

Figure 4.3 demonstrates how the interplay among elements can give rise to emergent properties in any complex. Our definition of Systemness of a complex, as stated in Section 4.2, states that Systemness is the predictability of the observed statistical distribution of events in a complex of interacting situations, which impacts the prior beliefs of a predictive observer.

So, assuming the predictive observer is an architect, then:

- **Interacting situations:** We have two situations, with the least number of situations between which an interaction can be observed. The source situation, sink situation, and AIC action are the observable interactions.
- **Impacts the belief states of the architect:** There are four essential factors possessed by the interactions and invisible to the architect, yet they influence the architect's predictive capability to predict how one interaction will impact others and the whole, and vice versa. Those four invisible factors (emergent) are:
 - Prime P.
 - Goal type (AIC).
 - Action type (AIC).
 - Impact type (supportive, obstructive, neutral).

These are the four factors that the architect needs to predict or make assumptions about.

- There are several quantifiable properties of an interaction:
 - The architect's attentive mentions of its being recalled in a predictive thought process. This measures the predictive observer's level of concern about the interaction and how complicated the architect's prior knowledge about the.
 - The probability it will deliver the PrimeP given a scenario.
 - The observed statistical distribution of the events (like change of state) occurring in each situation when the action is enacted or enabled.
 - The predictability of the interaction to maintain its initial emergent properties (the four factors) in the event of interacting with others in a complexity field.

This figure highlights the basic principles of emergence as defined in INCOSE handbook, which we use to base the structure of the interaction definition on. First, individual elements alone cannot display emergence at a higher level within a complex. Second, the presence of two or more elements is essential for emergence. Third, emergence takes place at a level beyond the individual elements. The interaction formula explicitly defines the action between the sink and source. As for the emergent property, it is inherent in the following emergent properties of any interaction (think of them as the invisible components of an interaction):

- The Primary Purpose.
- The AIC goal types.
- Action type.
- and Impact type.

Emergent properties are context-sensitive; they may change depending on context. They are architect perspective setters, such that if they were to change or shift, the architect would be

dealing with an entirely different scenario, even if the components of an interaction remain the same. For example:

Scenario 1: Medical Package Delivery in a humanitarian application

- **Components:** GPS, obstacle avoidance sensors (LiDAR), communication module, flight controller.
- **Interactions:**
 - GPS provides route data to the flight controller.
 - LiDAR detects buildings and other drones, feeding data to the flight controller.
 - Communication module relays real-time updates to a central logistics system.
- **Emergent Properties:**
 - **Primary Purpose:** Safe medical transportation.
 - **AIC Goals:**
 - **Control:** Follow predefined routes.
 - **Appreciation:** Dynamic obstacle rerouting, Frequent check-ins for route optimisation.
 - **Influence:** save the life of a patient.
 - **Action Type:** Influence.
 - **Impact Type:** supportive.

Emergent Behaviour: The drone is a lifesaver.

Scenario 2: Military Surveillance (from an opponent's perspective)

Same components, different context:

- **Interactions:**
 - GPS provides route data to the flight controller.
 - LiDAR detects buildings and other drones, feeding data to the flight controller.
 - Communication module relays real-time updates to a central logistics system.
- **Emergent Properties:**
 - **Primary Purpose:** Stealthy intelligence gathering.
 - **AIC Goals:**
 - **Control:** Follow predefined routes.
 - **Appreciation:** Dynamic obstacle rerouting, Frequent check-ins for route optimisation.
 - **Influence:** save the life of a wounded soldier behind enemy lines.
 - **Action Type:** Influence.

- **Impact Type:** Obstructive.

Emergent Behaviour: From an opponent's point of view, the perspective shifts from defence intent to adversarial intent. The drone operates stealthily, avoiding detection while collecting critical intelligence.

Key Insight: Despite identical hardware/software components, the **context** (mission type) redefines:

1. **Primary Purpose:** Delivery versus Surveillance.
2. **AIC Goals:** Humanitarian reasons versus Stealth.
3. **Action Type:** Influence and Influence.
4. **Impact Type:** Supportive versus Obstructive.

Differences in mission type will have architectural Implications such as:

- Adjusting interaction parameters (e.g., LiDAR sensitivity for terrain masking vs. obstacle detection).
- Redesigning communication protocols (high-architect's attentive mentions updates vs. low-probability-of-intercept LPI bursts).
- Reprioritising flight controller logic (route efficiency vs. radar avoidance).

Emergent properties are not inherent to components but arise from their **interactions in specific contexts**. Changing the context (e.g., mission type) reconfigures these interactions, leading to new system behaviours. Architects must model these interactions dynamically to ensure adaptability across contexts. Suppose the architect wishes to maintain the humanitarian emergent behaviour for the opponent not to consider such a drone an adversary. In that case, they will need to include extra support systems to ensure that there is no miscommunication with opponents.

Designing and assurance of emergent properties for an interaction entails the architect's attempt to ensure that those emergent properties remain constant in all possible contexts as intended, by accounting for as many scenarios as possible. Architect design goals would involve the following activities:

- Define a desired emergent property for a given interaction.
- Define the construct of the interaction that delivers the emergent property.
- Ensure the defined emergent properties are resilient to change in any possible complexity change.

The observable or visible components of an interaction are:

Chapter 4

- The source complex.
- The sink complex.
- the source behaviour or action.
- The sink behaviour or reaction.

An interaction's emergent complexity field can be thoroughly explained when visible and invisible components are made explicit. The architect can decompose an interaction into its visible and invisible factors relative to a given scenario or context:

{**Source:** robot, **Source situation:** safely passing, **Sink:** traffic roads, **Sink situation:** heavy, **Goal:** to manage, **Goal type:** Appreciation, **Action:** monitors, **Action type:** appreciative, **Effect:** neutral}

Note that the scenario's connotation has induced the "Goal" factor. The scenario definition did not explicitly mention the goal, which is why factorising any situation is essential to revealing what AIC goal type is associated with an action. Deep articulation of interaction is critical when we apply perspective shift to discover Black Swan scenarios.

4.1.6 Source Complex

In each AIC interaction, a source complex acts upon a receiving sink complex with some interaction. A source is an initiating element that appreciates, influences or controls another complex. For example, the car's sensor system is a source complex, collecting real-time data (like traffic density) and influencing the car's speed optimisation strategy (sink complex). Note that we said "influence" and not control because the sensor reading and speed optimisation are informational.

4.1.7 Sink Complex

The source complex appreciates, influences, or controls the receiving end sink of an AIC interaction. For example, the car's braking system is a sink complex when it receives signals from the braking control system to slow down due to detected obstacles.

4.1.8 Relationship

An interaction occurs when a source complex takes a one-way action (behaviour) with a goal of appreciation, influence, or control to impact a receiving sink situation. There are 4 possible types of goals behind any action:

- Utilisation of the sink situation,

Chapter 4

- Inhibition of the sink situation,
- Management of the sink situation (neutralising its impact), or
- Initialisation or triggering some aspect of the sink situation.

Example: An autonomous car (source) senses (action) changes in weather conditions (sink) to manage their impact on its operation.

Intra-actions are expressed using this format:

[AIC type-action]

Written in the following syntax:

[verbal phrase]

For example, `{{flying police robot}}_{{learns humans' visual profiles}}_{{distressed people}}`.

Relationship:

A relationship is a two-way interaction between a source complex and a sink complex, forming a feedback loop. In a scenario, both act on and react to each other.

Example: In the context of an autonomous car:

- The car (source) influences the human driver by guiding attention.
- The human driver (sink) reacts by driving the car to its destination.

The formal structure of the AIC relationship:

`{{Supra-Source situation}}_{{primary purpose}}_{{sink or Supra-Sink situation}}` whereby;

`{{source situation}}_{{AIC-action}}_{{sink situation}}`, while, `{{sink situation}}_{{AIC-reaction}}_{{source situation}}`

The term of the verbal formula is written in the following syntax:

`{{adjective+noun}}_{{verbal phrase}}_{{adjective+noun}}`

For example,

`{{alerted Police Force}}_{{manage safely}}_{{active Accidents}}` whereby;

{flying police robot}_[learns humans' visual profiles]_{distressed people}], while
{distressed people}_[stand underneath]_{flying police robot}]

The above example is an appreciation relationship between flying police robots and distressed people, contextualised within the police force Primary Purpose. The emergence of an appreciation relationship is attributed to the combined emergence of an appreciative interaction and a reacting influencing interaction.

4.1.9 Supra-Complex, Supra-Source and Supra-Sink

Supra-System means a larger-than-observed complex of concern whole or System of interacting purposeful Systems [117]. Similarly:

- A **Supra-Complex** is a larger-than-observed complex of concern, a collection of complexes where a complex of interest (of a problem-solving architect) is part of.
- A **Supra-Source** is a larger source complex that contains smaller source complexes.
- A **Supra-Sink** is a larger sink complex encompassing smaller ones.

Examples:

- If a policing drone is a source in an interaction, the Police Force is the Supra-Source that includes the drone.
- If a train is a sink complex, the Train Network is the Supra-Sink that encompasses the train.

Naming Convention

- **Supra-Complexes:** Written in all capital letters (e.g., {Train Network}).
- **Constituent Complexes:** Written in lowercase (e.g., {train}, {policing drone}).

For example:

- **Source complex:** {adversarial drone}.
- **Supra-Source:** {Adversarial Attack}.
- **Sink complex:** {train}.
- **Supra-Sink:** {Train Network}.

4.1.10 Supra System Purpose, Primary Purpose, Goal and Action

The purpose of a system is the high-level outcome or service it aims to achieve towards some larger whole. It is supported by goals, which are specific objectives, and actions, which are the steps taken to fulfil those goals.

We structure system's purpose into goals in the following way:

- **Goals:** Specific objectives or intent derived from the purpose that an action can not directly observe, but only inferred. Goals can be classified as auxiliary objectives that assist in the manifestation of purposes, yet are invisible to direct observation. Purpose and goals are formed within the architect's intent in design time. While a machine performs actions. Goals categorised as:
 - **Control-type:** Direct objective to regulate, govern or maintain some situation. These are mainly within-immediate-reach situations, but they can be informational situations.
 - **Influence-type:** indirect objective to extract a value or provide a service. Think of a Harder-to-reach immediately expected outcome or service.
 - **Appreciation-type:** objectives involving understanding or situational awareness of other situations that impact the system of interest, but the system of interest can influence control. Mainly informational situations, but it is possible to appreciate within-immediate-reach situations.
- **Actions:** Observable, verifiable steps or tasks performed to achieve goals. Actions are written as direct verbs without the term “to”. So instead of “to deny”, which can be a goal or purpose, action is “deny”.

Examples:

- **Autonomous Car**
 - **Purpose:** To provide safe, efficient, and reliable transportation. This is a systemic service of the transportation supra-system as a whole.
 - **Goals:**
 1. To regulate safe speed among cars (control-type). This is a within-immediate-reach situation since it involves a within-immediate-reach component state.
 2. To minimise travel time (influence-type). This is a Harder-to-reach service since travel time is a Harder-to-reach informational complex.

3. To monitor the separation distance among cars (appreciation-type). This is a Harder-to-reach informational situation since separation distance is measurable information.
- **Actions:**
 1. Brake and accelerate smoothly for safety (control action: directly stop motion).
 2. Choose optimal routes based on traffic (appreciative action; understand which is the optimal root out of many options).
 3. Optimise fuel consumption (influence action, indirectly changing fuel consumption rate).
 4. Switch to electric mode (control action, toggling between fuel or electric driving).
 5. Reduce car emissions (influence action, indirectly bringing a global carbon level down).
- **Autonomous Security Drone**
 - **Purpose:** Share the same purpose as the autonomous car but performs different goals.
 - **Goals with train tracks zone of operations:**
 1. To perceive unauthorised intrusions (appreciation type).
 2. To enhance high-value asset security (influence type).
 3. To command a security officer's reaction (control type).
 - **Actions:**
 1. Patrol zones to prevent intrusions (control action).
 2. Adjust sensors for better detection (appreciation action).
 3. Transmit live feeds to a central system and demand immediate reaction (control action).
 4. Deny unauthorised intrusion (influence action).

Actions and goals can be interchangeable (meaning the architect can choose); as long as the classification of “goal” or “action” is used with the “to verbs”, then it will be clear for a problem-solving architect which one is the intent and which one is the action.

Primary and Secondary Purposes:

A system may have multiple purposes (multiple reasons it exists or serving multiple purposes). The architect may choose primary or secondary purposes. The second purpose is related to affordances that an engineered system can be used for.

Example: Autonomous Agricultural Robot (Tractor)

- **Primary Purpose:** To provide efficient tending crops (e.g., planting, watering, harvesting).
 - **Goals:** To navigate fields, to apply water/fertiliser, and to harvest crops.
- **Secondary Purposes:**
 - **Transporting equipment**
 - **Goal:** to secure tools.
 - **Action:** Lock and stabilise tools during transit.
 - **Transporting individuals**
 - **Goal:** to safely carry passengers.
 - **Action:** Stabilise cargo or passenger zones.
 - **Towing heavy equipment or vehicles**
 - **Goal:** to safely tow.
 - **Action:** Engage towing mechanisms and adjust driving parameters.

A system's purpose defines its existence; goals are desired situations that make it actionable, and actions fulfil those goals to realise its purpose. Secondary purposes broaden a system's utility while supporting its primary function.

4.1.11 Supra System Primary Purpose (PrimeP)

A property of supra-systems. It can be defined as the entire supra-system's primary, unchanging intent or extracted value or service. All systems within the supra-system ultimately impact this PrimeP. The following are some key properties of PrimeP:

- **Consistency of PrimeP:**
 - While a system's specific goals may change depending on context or conditions, the PrimeP may remain constant regardless of the scenario.
 - All systems' actions and goals should manifest the PrimeP in all scenarios. Actions may differ in form and nature, given the context of the experienced scenario, so long as the PrimeP is preserved. In other words, the architect may not care much if the robot's actions deviate from the designed or specified action, as long as the intended PrimeP is persevered (in this case, the architect's intent).
 - So! While traditional engineers assess whether the system indeed and precisely performs specified actions, in ML-driven behaviours, the architect does not mind deviations in behaviours, as long as all experienced behaviours give confidence that the architect's intent constrains the machine.

This is the influence-mindset of the AIC architect, which differs from that of the Control-driven architect. The idea is that the influence-driven architect cares less about controlling emergent behaviours and more about ensuring that their intent truly constrains the emergence, allowing the influenced machine to have an artificial sense of freedom in its actions. This means the architect will only consider a test to be a failure if the intent is not manifested over a series of test observations, even if the actions emerging from the machine deviate by an allowable boundary from specified actions.

For instance, consider a scenario where an AS car perception system is designed to identify a human, yet it mistakenly identifies an image of a human featured in an advertisement on the bus. A control-driven architect would regard this as a failure in behaviour, as an image of a human does not constitute an actual human being. Conversely, an influence-driven architect (an AIC thinker) would accept this as an acceptable deviation, maintaining that their intent is persevered in this case, which is to identify a human, and the system indeed identified a representation of a human. This identification could lead to a series of diminished performance outcomes, such as inhibiting acceleration in the presence of a human. In this instance, reducing speed is considered safer than continuing to accelerate, ultimately prioritising safety. Architect intent is the PrimeP of machines' emergent behaviours to be constrained by.

- **Example (Train Network):**

- PrimeP: “Transport people and goods safely”.
- A fence surrounding train tracks in the network, though just one component, shares this PrimeP.
- Goal for the fence: prevent intrusion to maintain safety, aligning with the network’s PrimeP.
- If the fence prevents an intruder from entering, such action (though passive) indeed manifests the PrimeP. If the fence prevents some gusts of wind blowing debris onto the train tracks (although this is not a designed action), such emergent affordance also manifests the PrimeP.

- **Influence on Decision-Making:**

- The PrimeP guides how the architect predicts system responses in complicated scenarios.
- Example (Self-Driving Car) in the context of avoiding traffic: If the PrimeP is “transport passengers efficiently”, the architect may choose to innate the ML component to choose the riskier route to avoid delays.
- If the PrimeP includes “safely”, the architect may innate the car to prioritise a safer but longer route.

- **Design and Validation:**

- The architect's Epistemic Uncertainty may be reduced when PrimeP is assured or made certain of.
- The PrimeP must be defined clearly at design time and agreed upon among stakeholders.
- Architects must anticipate possible situations and ensure the system's goals/actions align with the PrimeP.
- Setting goals and defining actions suffice to validate the PrimeP as the system's conceptual property captures the architect's intent.
- **Choosing the Right PrimeP:**
 - The chosen PrimeP influences the prediction of all emergent behaviours. Anticipating or defining different PrimePs for the same system will affect the types of emergent goals and actions to be predicted. For example, if we choose PrimeP for an autonomous car to win races, we will predict different goals and actions than if we define PrimeP for a car to transport heavy goods.
 - If "safely" is included, the system's behaviour shifts towards safer outcomes.
 - A comprehensive analysis is needed to define a PrimeP that leads to desired, predictable responses.
- **Predictability of system's possible behaviours:**
 - Knowing the PrimeP of a system, makes the system behaviour predictable (in all possible scenarios) based on the following assumptions:
 - The architect is assured that the PrimeP had been innated into the system design or emergence.
 - The architect is forever certain of the consistency of PrimeP.
 - The observed system is forever enslaved by its PrimeP.

Though "forever" seems like an unattainable quality, as far as "making an assumption" is concerned, it is viable to think so. The architect may soften the certainty to be constrained in some conditions; however, such engineering decisions are down to the circumstances faced as a whole.

4.1.12 Appreciation, Influence and Control Key Differentiation:

- **Appreciation (A):** Minimal governance; to appreciate means more influenced than influencing.
- **Control (C):** Direct and strong governance with higher certainty.
- **Influence (I):** Indirect governance with moderate or lower certainty of causing immediate change, mediated through intermediaries.

The primary distinction between Appreciation, Influence, and Control in the AIC approach and traditional control theory or systems engineering lies in its probabilistic nature.

Control in Traditional Systems Engineering:

- Defined with the expectation of absolute certainty.
- For example, in control theory or frameworks like STAMP, control is assumed to be precise and deterministic, often described as “control loops”.
- Example: A drone pilot controls a drone" implies full and unquestionable governance.

Control in AIC:

- Defined with the expectation of being “**very likely**” or “**almost certain**”, acknowledging uncertainty. It is a softer form of control as per control theory. We view it as a “governance” rather than actual hard control, with a pinch of uncertainty and doubt in it.
- Example: "The drone pilot is almost completely in control of the drone, but not quite".
- This subtle shift encourages architects to remain aware of uncertainties rather than assume absolute certainty in control. Such a perspective is tacitly encouraged by control-based approaches like HAZOP, STAMP and FRAM.

Core Insight: By adopting a probabilistic view, the AIC approach enables architects to embrace uncertainty and prepares them to address potential deviations in expected behaviours or events in Compound Uncertainty Problems. This mindset shift, viewing control as near-certain but not absolute, is foundational in solving complicated, multi-faceted problems by predicting more unforeseeable challenges.

4.1.13 AIC-Core thought process

We adopt the following fundamental thought process as the minimum comprehensive chain of thought that enables better-informed predictions about a phenomenon. We employ this thought process in situations where an engineering judgment is required. For example, in Sections 6.4.2 and M.4.2, we utilised AIC-Core to aid in predicting backwards-feed interactions in a complex setting. The following are the general thought steps that the architect can use to inquire about any observed problem to reach an AIC-informed engineering solution judgment:

1. **Influence:** What complex does the problem try to influence?
2. **Control:** What controllable complex does the problem use to exert its influence?
3. **Appreciation:** What environmental complex should the problem appreciate to enable the intended control to be implemented, and thus correctly manifest the desired influence?

It is essential to recognise that AIC-Core is an adaptable thought process, whereby the architect can tailor the wording and context of each aspect to suit the specific problem at hand. The order matters, meaning there is potential for change in the architect's prediction depending on where the architect starts from. The architect anchors the chain of thought on A, I, or C (meaning starting to enquire with either aspect). In Section E.2 we introduce ordered and disordered AIC thought processes.

4.1.14 Goals and actions have an emergent impact property that induces change

AIC systems theory defines a system as “the set of ordered relationships that affect and are affected by purpose”. The definition includes the requisite effectiveness to be experienced by the system due to its purpose. The effect of a purpose or goal can be understood as its Impact upon other purposes or goals, and actions are then evaluated for Effectiveness to deliver the purpose and intended impact. Impact and effectiveness explain the resultant effect of a system's purposes/goals and behaviours on other systems in the context of some relationship.

AIC goals can have the following types of impacts on other AIC goals in any relationship: supportive, obstructive, or neutral. For example, the Eagle Drone has an obstructive control purpose over the Adversarial Drone, a supportive appreciation purpose over the train track zone fence, and a neutral appreciation purpose over weather conditions. At the same time, the effectiveness of actions can be classified into effective, ineffective, or absent. For example, the Police's security patrolling capability is ineffective in denying adversarial drone obstructive control over the train tracks zone open space.

4.1.15 The intrinsic hazard of appreciation

All appreciative interactions are the complexity field's weakest links for maintaining the emergence of a primary purpose and AIC actions' impact types.

The hazard is related to the potential safety and security vulnerability associated with appreciation. When system A is in an appreciative interaction with some situation D, whoever controls or influences D, unavoidably influences or controls system A. This means that for any appreciative relationship, the appreciative system also implicitly appreciates other situations that can influence or control its appreciated situation. For example, an autonomous car (system A) appreciates visible traffic signs (situation D); however, whoever or whatever influences or controls the clarity and visibility of a traffic sign unavoidably influences or controls any passing autonomous vehicle.

This is an important concept, especially in cybersecurity, as whenever an appreciative interaction is defined, a cybersecurity or safety threat is inherently associated with it. If appreciative

interactions are exploited, the system is open to whatever exploiting system's goal or PrimeP. AIC schemas enable the modeller to naturally consider potential soft and hard hazards associated with appreciative interactions in the complexity field. For detailed elaboration of soft hazards and hard hazards in the context of AIC see Section E.3.

4.1.16 White Swans, Black Swans

In popular literature, the concept of Black Swans as described by Taleb [118] refers to events that meet the following criteria:

- **Outliers:** Things that we wouldn't have expected through our prior experience. They are very rare to occur in the real world.
- **Extremely Impactful:** We reasonably believe that they will contribute to the success or failure of our desired purpose.
- **Explainable after they occur:** When they do occur, we have a convincing rationale explaining why they would have been inevitable, but our experience lets us down in realising that earlier.

To redescribe Black Swan events from a predictive system thinking perspective, we understand such events to satisfy the following criteria:

- **Epistemic Outliers:** are predictions that reside outside the pool of knowledge acquired at some Stage of intellectual experience. For example, new predictions were made in predictive thinking pipeline 2 (Section 6.2.2) of the process that were not made during the intellectual experience of predictive thinking pipeline 1 (Section 6.2.1). Or predictions made about new refined factors that were not recalled in the initial problem brief.
 - **Architect Concern Outlier:** We also include the meaning of being "rarely concerning or rarely considered as problematic by the architect". Thus, they may be outliers in the architectural sphere of normal concern (note that 'normal' refers to a normal-distribution-based perception of some complexity field). For example, an architect may have never considered the shimmer of light on rain dew on leaves a concern or problematic factor in image recognition before.
- **Extremely Impactful:** We have sufficient rationale that they will contribute to the success or failure of our desired purpose. For example, an audit trail that demonstrates a set of images impacts the reliability of a trained model.
- **Explainable after they occur:** When we do predict them, we can unambiguously trace them back to some prior knowledge acquired. For example, a prediction based on the application of a general systems rule in a SECoT.

In the context of datasets, Black Swan pictorial scenarios are:

- **Pictorial outliers:** They are visually out-of-distribution relative to some existing training and validation datasets. They are also pictorial scenarios that the architect deems as rarely concerning. For example, a scenario where a moon resembles a traffic light is rarely concerning for an architect.
- **Extremely Impactful:** Suppose we have an existing machine learning (ML) model trained on training and validation datasets. It performs as expected on the validation and in-distribution testing datasets. However, if the model is tested on a Black Swan dataset or an image, it fails significantly.
- **Explainable after they occur:** We have a SECoT that explains why they are Black Swans and a reliable dissimilarity measure that explains why they are out-of-distribution.

With the above definition in mind, we devised a systematic process for training models for Black Swan operations; see Section 7.2 for how we practically did it (as inspired by the experiment in H.10). Taleb also describe a White Swan event, which is an event that satisfies the following criteria:

- **Relatively likely and predictable:** We know they will occur if we don't account for mitigations and corrections due to prior experience of such events. For example, GPS-jamming-induced patrol gap GNSS jamming/spoofing is a known threat in many drone operations.
- **Extremely impactful and measurable:** We can quantify their impact, and it is highly significant. For example, an Impact window: ~8–12 minutes of blind patrol per event. Adversarial drone plants/extracts illicit payload, disables track signalling, leading to a 4-hour train stoppage, passenger evacuation, and investigation. Such failure may cost approximately £ 200,000 in direct downtime and approximately £ 50,000 in reputational and regulatory fines.
- **Explainable how they would play out:** We have sufficient knowledge to estimate their likelihood and the conditions that make them likely to occur. For example, We know from field tests that a 3 W jammer within 200 m of the flight corridor will degrade GNSS to < 2 m accuracy within 30 s.

Most failure and hazard analysis techniques, such as HAZOP and STAMP, help mitigate White-Swan errors. In the context of predictive thinking, White-Swan scenarios are easy-to-predict scenarios, such as a trained and purposed adversarial drone crashing into a train track zone infrastructure. Such risk is easier to predict because we have a long history of moving things crashing into infrastructure. However, what would be very rare is for an adversarial drone to crash into a nearby pond of water because the infrastructure reflects on it.

In the context of datasets, White Swan pictorial scenarios satisfy the following criteria:

- **Relatively likely and predictable:** Typical operations are expected within the operations domain, such as green trees, train tracks, and regular DJI drones.
- **Extremely impactful and countable:** The trained model fails unexpectedly if we don't account for them. For example, a model fails in typical operations scenarios.
- **Explainable how they would play out:** We have sufficient knowledge to estimate their likelihood of occurring and the conditions that make them likely to occur. For example, a memory was overloaded due to a cyber-attack.

To see an example of how we accounted for White Swan pictorial scenarios, see Section 6.9.

4.2 Architect's Predictive Thinking Process (ArcPT) ⁵

***Predictive Thinking** is a process that reduces the complicatedness of a complex, by anticipating unforeseen scenarios before they actually happen, and demonstrates reduction of epistemic uncertainty, justifiably ALARP.*

In the context of uncertainty quantification, reducing epistemic uncertainty, our lack of knowledge about an engineering problem space, enables us to anticipate how aleatoric uncertainty, irreducible variability inherent in the problem space, will manifest, and thus to select the appropriate control interventions (e.g., engineering constraints or physical or informational boundaries) that guide real-world complexity toward desired behaviours. The reduction of epistemic uncertainty is “predictive thinking ArcPT”, which entails a meta-cognitive strategy in which one deliberately configures both the distribution of the systems thinker's predictive attention (for example, focusing on typical versus long-tail events) and the ontology or level of abstraction used to describe complex problem space. Effectively managing these choices, attention distribution and ontology granularity constitutes a higher-order decision process that an architect or system designer must govern in order to steer both their mental model's uncertainty profile and the measured (examined or designed) complex problem space toward predetermined objectives.

The output of ArcPT is a cognitive component of the Architect's Predictive Mental Model (ArcPM):

***A Predictive Mental Model** is a set of general rules and types about how a general complex behaves, which justifies the prediction of unforeseen behaviours before they actually occur, used to update initial priors.*

⁵ For a more detailed elaboration, refer to Appendix E, Section E.1.3. Section E.1.3 relates the reduction of epistemic risks in engineered datasets to the predictive thinking process.

Predictive Mental Model also includes a structured representation, governed by the abstract rule set and categories of aspects, that enables an architect to forecast types of information in a complexity field.

Predictive Mental Models are responsible for the architect's engineering judgment and decisions in the face of aleatoric uncertainty. While aleatoric uncertainty cannot be reduced (as more things are involved), epistemic uncertainty can be reduced through an enhanced Predictive Thinking Process.

Predictive Systems Thinking is the topic that provides the tools and techniques to enhance an architect's predictive thinking by minimising sources of epistemic ignorance. Epistemic ignorance is an inherent deficiency associated with the architect's limited ability to account for all possible sources of randomness in an open operational environment complexity field when perceived by the architect. Therefore;

The architect's epistemic perception is inherently deficient

Based on the above principle, we understand that the purpose of the systems approach and systematic design processes is to reduce the architect's perception deficiency. Since epistemic uncertainty is reducible by acquiring more knowledge, the architect needs a process of acquiring knowledge and making epistemic inferences.

Epistemic inference is an inference from a set of beliefs to new or similar beliefs about an observation that either updates or validates the original set [119]. Shannon defines information as a message from some source to a receiver, and that information can be characterised as the removal of uncertainty for the receiver [29]. However, the receiver who lacks (thus experiencing an epistemic need) the knowledge carried by the information needs to make sense of the message first. Suppose the source of information is an observation of some complexity, and the message processor is the observing architect, who has some epistemic uncertainty about the observed complexity. Based on their existing belief system, the architect's epistemic inference minimises their uncertainty⁶. Through epistemic uncertainty reduction processes, the architect makes a series of inferences, which can be thought of as predictions to be added to the architect's knowledge.

⁶ The core concepts are rooted in epistemology and the formal belief revision theory. We contextualise them in predictive architecting context.

4.2.1 Predictive SECoT Structure

We define a particular structure of fully elaborated SECoT, which comprises the following elements captured in a table:

- **SECoT Title:** Define a particular descriptive title.
- **SECoT Input:** Define the input information needed to implement the CoT. A description of some phenomenon (an interaction or a scenario).
- **SECoT Process:** Describe the steps of the thinking process that prompt the systems thinker to consider the intuition described in general systems basis, in the format of a question and a guiding prompt. The SECoT process can be organised and structured into the following types of activities:
 - **A Predictive Thinking Pipeline:** A conceptually interrelated series of predictive thinking steps. A Predictive Thinking Pipeline title represents the primary prediction goal, expected to be asserted by the interrelated constituent thinking steps.
 - **A Thinking Step:** is a thought-provocative prompt that influences the architect's predictive thinking processes (specifically architect's predictive attention distribution approach through altering or refining the meaning of the internal ontology used) to make an informed prediction based on some general system perspective. It intends to focus (influence) the architect's attention to synthesising a potentially complicated interaction from some observed context by applying general rules about systems. It contains the following thinking activities:
 - **Step's input:** an input of information that requires processing.
 - **Step's process,** which includes the following activities:
 - **General Systems Rules:** A set of general rules about any system dynamics. In other words, it is a general rule that makes any complexity behave like a desirable system. The output of the thinking process must preserve those rules. They help define what acceptable output looks like and doesn't. In this Section, various types of axioms can be used to guide the thought process of the designer:
 - Hard systems axioms, such as those related to physical systems dynamics.
 - Soft systems axioms, such as those related to soft concepts like ethical or human value consideration.
 - **Predictive Question:** Prediction-provoking Question

- **Guiding Prompt:** Prediction-Guiding Prompt. Include a notion of completion, in other words, how a step is qualified as complete.
- **Step completion criteria:** describe how the step is judged to be completed.
 - **Step's Output Inference:** The output inferred realisation or assertion.
 - **A Predictive Thinking Pipeline prediction:** Set of all inferences.
- **SECoT Output Prediction:** The series of predictions made across the steps from all thinking pipelines that define new anticipated situations or scenarios.

The above construct is captured in the format of a table. For example, Table 4.1:

Table 4.1 General SECoT example

SECoT Title	AIC perspective shift SECoT.
SECoT Input	Security drones inhibit human intrusion into train track zone.
SECoT Process	Predictive Thinking Pipeline 1: Perspective Shift by Altering Interaction's AIC Type. Thinking step 1: identify AIC interaction of concern. Thinking step 2: Alternate the goal's AIC type. Thinking step 3: describe a scenario that captures the change in complicatedness. Predictive Thinking Pipeline 2: Perspective Shift by reversing effect direction. Predictive Thinking Pipeline 3: Perspective Shift by alternating effect type. Predictive Thinking Pipeline 4: Perspective Shift by alternating multiple perspective shift types.
SECoT Output Prediction	Four different unforeseeable scenarios for every given system phenomenon.

Table 4.1 presents an example of a SECoT (Systems Engineering Chain of Thought) titled "AIC Perspective Shift SECoT", which illustrates its purpose, input, process, and output. The primary purpose of this SECoT is to assist in predicting unexpected and unforeseeable emergent scenarios resulting from anticipated systems interactions. The input focuses on security drones inhibiting human intrusion into a train track zone. The process is structured into a predictive thinking pipeline with multiple steps designed to shift perspectives by altering AIC interaction types, reversing effect directions, or alternating effect types. Each step prompts the architect to think about interactions in new ways, ultimately leading to SECoT's output prediction: four different unforeseeable scenarios for every system phenomenon.

4.3 CuneiForm: Pictorial Epistemic Uncertainty Reduction

CuneiForm-based Systematic Context-Aware ML Training and Testing Syllabus

CuneiForm is a controlled, ontology-driven visual characterisation language that defines a set of abstract pictorial primitives and processes, enabling the unambiguous, traceable mapping of high-level textual requirements into the infinite variety of real images. For a more detailed description, the reader may refer to Sections E.7 and E.8 for a more detailed articulation of the CuneiForm process.

It is possible to transform a physical object into an informational entity by defining a set of measurable or verifiable knowledge base or features describing it. Informational features may include height, width, length, distance, location in the 3D plane, time, curves, aesthetic attributes, etc, which can constitute a knowledge system. We view pictorial representations of physical objects as knowledge systems representing objects. The actual challenge here lies in identifying rigorous and concrete processes that can be systematically validated for correctness, linking a general pictorial representation of a physical object to the infinite possible manifestations of that physical object captured.

Suppose we want to define what a human is in a visual sense, precisely (as precise as measuring exactly 2m). How will we precisely define an abstract human object that can be verified in all possible ways to capture an actual human visually? This issue is a root cause challenge behind specifying verifiable or validateable requirements that allow the architect to make a trustworthy judgment that the requirement is truly captured in a pictorial dataset.

Given that training pictorial datasets for safety-critical applications can typically be huge and costly, writing a requirement that defines each image individually would be impractical. Hence, we need an intermediate semantic step that bridges descriptive scenarios (or requirements) and pictorial situations. CuneiForm (inspired by the Cuneiform writing system) is a tool that acts as the semantic bridge between written descriptions and pictorial representation in pictorial situations by generating abstract pictorial images, which are a set of actual photos. Refer to Figure 4.4 for an example of a CuneiForm image taken from Eagle Robot case study.

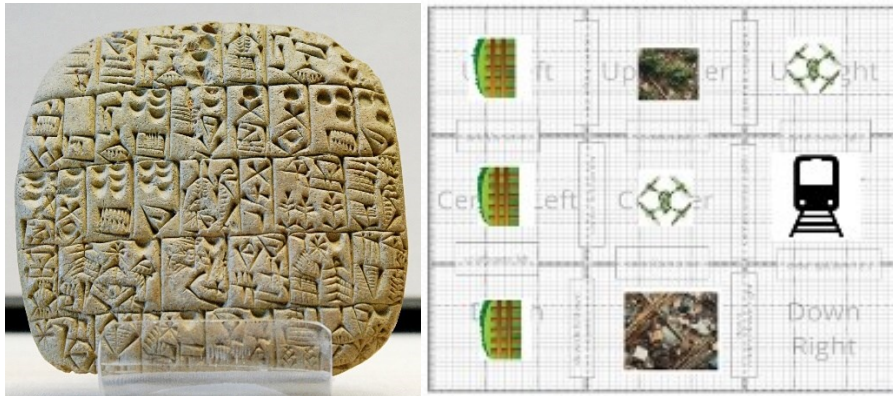


Figure 4.4 The right is an example of an ancient Sumerian sale contract specification. On the left is an example CuneiForm that describes a Black Swan pictorial scenario.

4.3.1 CuneiForm: a pictorial-type requirement

Traditionally, systems engineering employs several types of requirements, each serving specific purposes in the reduction of the architect's epistemic uncertainty during the design and development process:

- **Handwritten Requirements:** These are textual descriptions of desired system behaviours, often qualitative and open to interpretation. While accessible and flexible, handwritten requirements may lack the precision necessary for Vertical systems.
- **Formal Specifications:** These employ mathematical or logical constructs to define system requirements precisely, ensuring clarity and unambiguous interpretation, particularly for automated verification.
- **Graphical Representations:** Models such as UML (Unified Modelling Language) and SysML (Systems Modelling Language) provide visual representations of system architectures, processes, and behaviours, aiding communication and alignment among stakeholders.

Just as formal mathematical specification bridges the gap of concise specificity between human language regular requirements and potential software code, CuneiForm is proposed as a type of requirement that bridges the gap between traditional written requirements and computer vision development datasets. This approach provides a means to transform textual descriptions of system behaviours into pictorial representations that align with perception-based systems' training and validation needs, particularly for safety-critical applications. CuneiForm enables more comprehensive demonstration coverage of expected system behaviours in various operational scenarios by serving as an intermediary between abstract written specifications and concrete datasets.

Table 4.2 Semantic bridge between handwritten requirements and pictorial datasets

Safety Training Requirement (Training Concept)	CuneiForm Abstract Images (Training Syllabus)	Training dataset images (Training Examples)
Eagle Drone Training datasets shall enable a robust and generalised computer vision model to identify various adversarial drones, especially unusual or stealthy ones.	CuneiForm Requirement	

Table 4.2 illustrates the semantic bridge between traditional handwritten requirements and the development of pictorial datasets using CuneiForm representations. Each CuneiForm is a training class specifying, in general, the training examples needed to train the ML. The table shows how a safety training requirement, such as ensuring robust and generalised computer vision models for identifying various adversarial drones, is transformed into CuneiForm abstract images.

These abstract images encapsulate key elements of the scenario, such as drone types, their configurations, and contextual variations, which are essential for capturing the complexity of the requirement. The CuneiForm representations then guide the creation of training dataset images, which include diverse visual scenarios, such as drones in different environmental contexts like railway tracks or rural areas. This process demonstrates the role of CuneiForm as an intermediary, ensuring the alignment of abstract requirements with the comprehensive datasets necessary for training effective perception systems.

4.3.1.1 Machine Training Process Concepts

There are some practical concepts we use to describe the process of training machines:

Machine Training Syllabus: A set of training, validation and testing classes (CuneiForms) that map to a training concept requirement. They are a syllabus required for the machine to be trained on in order to satisfy the training concept.

Machine Training Class: A single CuneiForm is considered a class or a lesson for the machine that needs to be trained on and pass the tests.

Machine Training Topics: Machine Training Classes are subdivided into **training topics**, which explain the real world to a model. We have **eight** training topics that need to be covered in Training Classes (TOI means Target Object of Interest):

- 1) **TOI's aesthetic complexity:** train the model to recognise how complicated the TOI shape is.
- 2) **TOI's motion complexity:** train the model to recognise how complicated the TOI's modes of motion in the real world.
- 3) **TOI's 3D spatial orientation complexity:** train the model to recognise how complicated a TOI's 3D orientation can be.
- 4) **TOI's spatial positioning complexity:** train the model to recognise how complicated a TOI's spatial positioning can be.
- 5) **TOI's distance complexity:** train the model to recognise how TOI's distance can complicate its detectability to the machine.
- 6) **Background objects complexity:** train the model to recognise its TOI among a complexity field of background objects.
- 7) **Background objects motion complexity:** train the model to recognise its TOI among a complexity field of background objects in motion.
- 8) **Pictorial Horizon:** Horizon may alter the pictorial complicatedness for an intelligent machine (ML model). It may introduce hidden biases towards a specific configuration of things in an image. In one part of an image, there can be a clear space for any object, but in another part, there can be a complicated pattern of objects.

4.3.2 Pictorial Hazard

While examining data sets in the AVOID case study (Appendix I), we learned that operational hazards could emerge due to missing crucial information about the operational environment in pictorial coverage (ignorance in pictorial datasets). We understand the pictorial hazard as “the absence of valuable information in image-based datasets that may lead to physical hazard”.

While physical hazards (such as power supply loss or mechanical lever breakage) can lead to direct harm,

Pictorial Hazards can be considered epistemic hazards associated with the relative loss or missing of valuable information that could have prevented the machine's harm.

Information may be about the operational domain that the AI model should have been aware of. One of the leading causes of Pictorial Hazard is the architect's epistemic ignorance about the problem at hand. Hence, we propose the CuneiForm as a method to systematically mitigate pictorial hazards, thereby further reducing the architect's epistemic ignorance of the operational

domain. At the same time, recording such a process as evidence for an assurance case to support claims related to reducing uncertainty.

Note that we used the term “relative” in front of “loss of valuable information” because it also includes a relative gain in information that may lead to an imbalance in distribution or a harmful skew in ML behaviours to serve some intended or unintended, desirable or undesirable purpose. A loss of information is directly related to a loss of epistemic certainty, which is technically known as “epistemic ignorance”.

4.3.3 CuneiForm Abstraction Process

The primary purpose of the CuneiForm process is to design and validate datasets systematically, provide tangible evidence and argumentation on dataset coverage of requirements (especially safety), and verify that the dataset accurately captures the requirements. In this Section, we define the general aspects of the CuneiForm process. Then, in Section 4.3, 5.7, we will explain the CuneiForm characterisation process by applying it directly to the case study (Section 7.1, and Appendix I.9). The CuneiForm-based characterisation focuses on carefully designing the pictorial situations a computer vision system must handle. It ensures that pictorial ML Training situations (captured in sets of instantiating images that share a common abstract parent image) can be traced to written requirements. The core aspect of designing a CuneiForm characterisation involves:

- Systematically constructing abstract images embodying objects and behaviours outlined in a requirement, such as the movements and optical appearance of adversarial drones in a set of images.
- Then, we instantiate the abstract images with a set of actual images that meet the abstract image characterisation.

4.4 Predictive Lateral Thinking Technique: Disordered AIC Perspective Shift⁷

Black swan Scenarios are a sequence of at least one event or situation that the architect deems remotely unexpected but concerning. For example, a falling dry leaf from a tree in front of an autonomous car causes the car to change lanes abruptly. When the architect models the AIC schema of a complexity field and rigorously justifies it, they make a tacit premise with regard to

⁷ For a detailed description of the process, see Section F.7.

expectation; that is, the AIC model complexity of the problem and solution is what we expect, hope and desire to happen.

Since the approach utilises a general mental model that describes the intricacies of any complexity field, by definition, the output of designing the AIC model is a declaration of an expected or predicted chain of events that is unfolding or will unfold in the future (after deploying the solution, for example). Therefore, its a claim of “foreseeability of events”. In other words, architects who produce AIC models are tacitly asserting that the model is what they expect in the complexity field of the problem as-is or to-be. Any alteration to the mental model, then by definition, means the following:

- An undesirable situation of the complexity field.
- Unforeseeable situations of affairs among the systems.
- Least expected interactions to occur since the original model is the most likely description of what is happening or will happen.

Given that Black Swan events are unexpected events in terms of occurrence or concern, altering or negating the AIC logic should guide us to think of what we initially did not think of, thus helping us to predict “unexpected, rare events”. We do so by “shifting the initial perspective”, and by doing so, we can predict new variations of potential scenarios. Those scenarios would qualify to be “Black Swan events”. For example consider the Eagle Drone case study:

In this example, Figure 4.5, the architect predicted the original complexity field to have the Eagle Robot be in supportive control interaction [+pilot] with the Eagle Drone. In a post-perspective shift scenario, the architect assumes that over time and for some complicated reasons, the possible complexity field changes such that the control interaction is no longer possible, and only a possible influence remains. At the same time, it was initially expected that the Eagle Drone would appreciate the Eagle Robot commands; in that futuristic scenario, the Eagle Drone controls the entire Eagle Robot decision-making process.

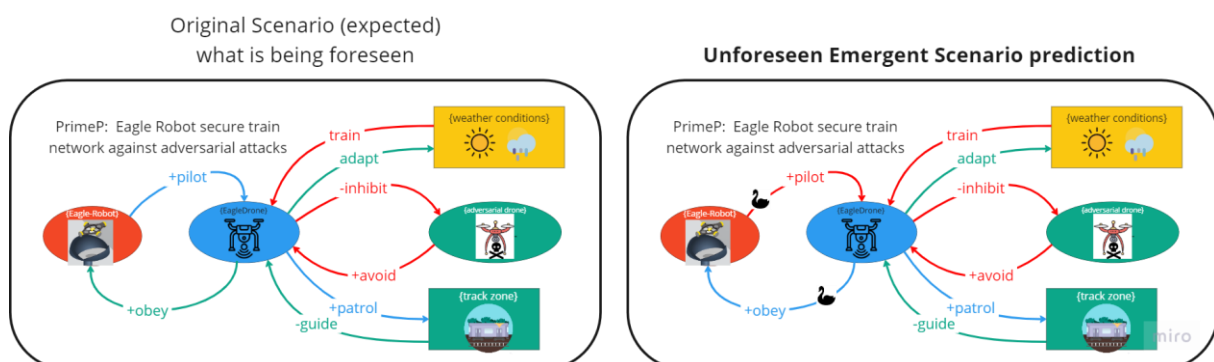


Figure 4.5 Predictive Disordered AIC Perspective Shift in Original vs. Emergent Scenarios.

4.4.1 Perspective Shifts as an Objective Predictive Lateral Thinking Process

Perspective-shifting within the AIC framework is a prime example of lateral thinking. Lateral thinking involves challenging conventional assumptions and reframing problems to explore novel solutions or scenarios that are not immediately apparent. In this context, applying an AIC perspective shift enables the designer to break away from linear or expected patterns of interaction and systematically and objectively explore unforeseeable or hard-to-imagine scenarios. This iterative process disrupts any build-up of bias during Stages of conventional thinking in a design process by encouraging the designer to consider alternative roles and interactions within a complexity field, such as shifting a supportive or appreciative interaction into a control interaction.

This deliberate act of imagining unconventional shifts aligns with lateral thinking's emphasis on generating new possibilities by questioning the limits of the current mental model. For instance, in the autonomous vehicle example, the process of envisioning how a pedestrian might control a path by interacting with objects like a shopping cart exemplifies how lateral thinking uncovers hidden complexities. These unexpected scenarios, emerging from the interplay of actors and environmental elements, highlight the potential for unknown unknowns to manifest. Thus, perspective-shifting enhances problem-solving creativity and systematically broadens the scope of potential risks and opportunities within the design framework, providing a robust approach to addressing complex, dynamic systems.

Chapter 5 Summary of AIC Systems Approach

Processes

In this Section, we will summarise our contribution processes with limited examples to give a general appreciation of what the design approach entails. Then, we will perform each case study in Chapters 6, 7, and 8 (for a detailed application, see Appendices H, I, and L). This Chapter is supported by Appendix F where Full SECoTs are defined. The theoretical basis for the processes is outlined in Chapter 4.

The idea behind our design approach is to imagine that we are reorganising or engineering the entire problem domain from the first principles of systems, with the autonomous system being introduced into the mix. We assume that the problem domain is a Stochastic Confusion⁸ (which is not a system), and that the complexity field's fundamental observed statistical distribution of events is assumed to be a long-tailed profile whereby rare but impactful events (Black Swans) can not be ignored (dominating such an environment)⁹. We argue that traditional systems engineering approaches (HAZOP, FMEA, STPA..etc) may fundamentally assume general complexity as a normally distributed problem whereby rare events can, in theory, be ignored.

Approaching a long-tailed open operational environment with a mindset accustomed or naturalised to the idea that some rare events can be irrelevant and ignored would be an architect's fundamental error, even before thinking to solve the problem. We encourage the reader to read chapter 4 (and appendix E) to appreciate the foundational theoretical aspects of our approach.

For long-tailed problems, the architect's epistemic uncertainty (ignorance)¹⁰ compounds as more factors are involved in the operational domain. Hence, Black Swans are the Achilles' heel for architectural intelligence about the problem itself, impacting the usefulness of solutions in such issues. Therefore, while STAMP views the problem domain complexity as a cybernetic control problem (see Figure 5.1), we shift the focus of problem-solving to enhancing the architect's predictive systems thinking to predict Black Swans by approaching the problem as a long-tailed probability-distributed complexity.

⁸ See Section 4.1.1

⁹ See Figure 4.2 for more details on the different types of approaches to the problem domain.

¹⁰ See Sections 2.1.1.5, 4.1.1, 4.1.3, E.1, for more details on uncertainty.

The solution to such problems would be a first-principled approach, whereby the architect aims to consider all possible interactions, assuming that there are no irrelevant or negligible factors. A Black Swan can lurk under the pool of the least relevant information.

Modelling an open, complicated environment with non-deterministic agent interactions as simple control loops would be an oversimplification that may result in unwelcome surprises for the architect, such as unexpected data shifts. The environment is adaptable and constantly changing, so we opted for an open fractal-like problem-solving approach that accepts control, no-control (appreciation), and indirect control (influence) as part of the final solution.

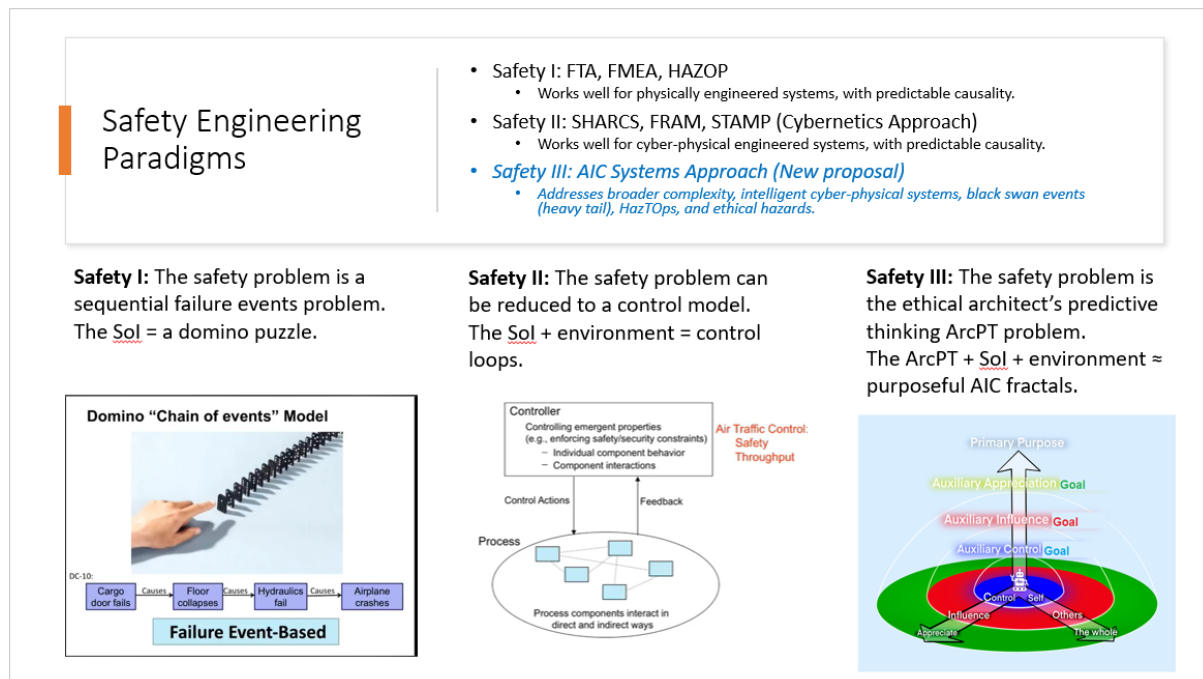


Figure 5.1 proposed classification of Safety and Systems Engineering¹¹

In our approach, we view the systematic processes as methods to constantly update the architect's belief system in a Bayesian sense. Whereby, the architect's belief system updated systematically, thus demonstrating an epistemic uncertainty reduction, as follows:

1. The start of the Architect belief system setting is at Stage 1, which is problem articulation, in which the architect focuses purely on the problem and refrains from thinking about the solution. For the architect to make informed predictions, they need some general complexity rules to assist them with predicting how a particular scenario may be developed in the problem. Some of those factors would be Black Swans or factors that the architect would rarely consider to be impactful if they were attempting to provide deterministic engineered systems.
2. The output of Stage 1 is translated into a set of factors and assumptions, which are then processed in Stage 2, where the architect and stakeholders make appropriate solution-

¹¹ This picture is an extension to slides presented by Leveson; see the video here.

level decisions. A solution is well articulated and driven by a deep appreciation of the problem.

3. The architect then starts to incorporate the solution into the complexity of the problem domain systematically to derive system-level requirements, which is performed in Stage 3. Stage 3 also includes a description of the operational environment envelope (the characteristics of the real-world composition) in Stage 3 b. We provide a comprehensive ontology to describe multi-type environments in Appendix A. The architect would employ Hazards, Threats and Opportunities analysis at this Stage.
4. At the end of Stage 3A the architect would have built a comprehensive mental model of what will happen when the solution is released to the wilderness of the open operational environment. However, Black Swans defy any sense the architect may assume to be common. To help the architect's predictive thinking distil their worldviews about the problem, we prompt the architect to shift their perspective on those complicated interactions. Stage 4 is predominantly designed to help the architect imagine scenarios where the complexity does not unfold as expected. Ultimately, this would predict “Data Shift” sources, where the problem domain may change.
 - The outputs of Stage 3a and 4 are the Safe Operating Concept and the Training Concept.
5. The training concept is translated into a safety context-aware training and testing dataset using CuneiFrom requirements. The output of Stage 5 would be a systematically constructed dataset which includes the following aspects:
 - Typical operations scenarios.
 - Black Swan scenarios.
 - Out-of-context (relevant) data for extra robustness.
6. Stage 6 would then be concerning ML development and testing.

We needed a set of processes to support the approach's overall objective. This chapter will detail these processes, providing limited examples to illustrate their application. We utilised the AMLAS and SACE frameworks to represent our approach, highlighting its adaptability to state-of-the-art guidance and demonstrating the synergy between our approach and the “how” required for the “what” needed in those processes.

5.1 Overall process description

The AIC Systems Approach is a structured framework for managing architects’ epistemic uncertainty in compound uncertainty problems. It involves predicting emergent risks in the development of safety-critical autonomous solutions. Each Stage follows a predictive and analytical systems thinking process, systematically uncovering problem complexities, defining

architectural intent, designing autonomous system interactions, and developing safety-driven machine learning models. Figure 5.2 below presents the tasks and objectives of each Stage in a formal format, accompanied by examples that illustrate their application.

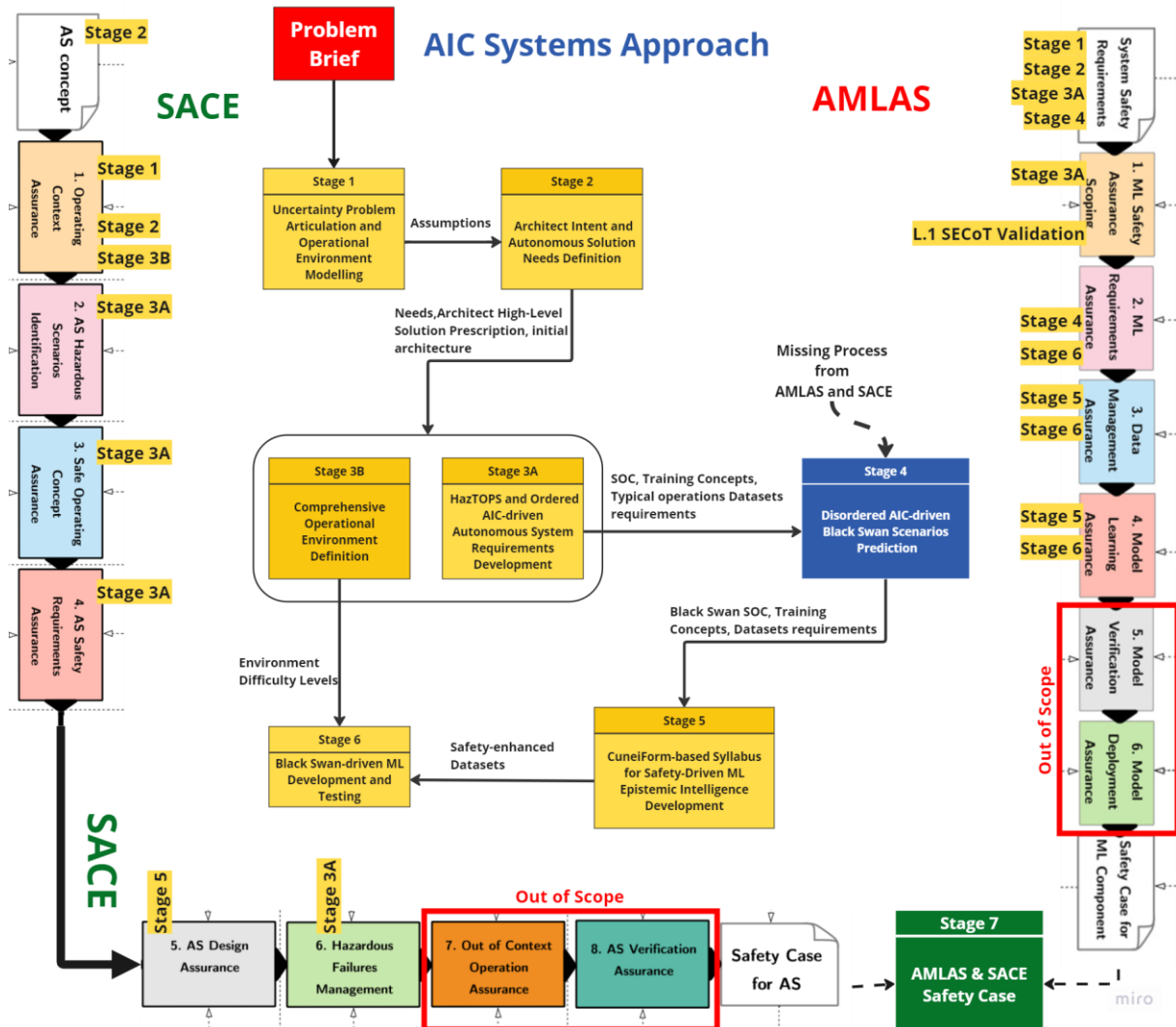


Figure 5.2 The overall AIC Systems Approach Framework mapped to AMLAS and SACE

The primary objective of this methodology is to identify and capture **Black Swan** events and scenarios through predictive thinking, the **CuneiForm** development process, and a **Black Swan-driven machine learning training pipeline**.

The overall process begins with articulating the problem to reveal latent Black Swan factors. These factors subsequently assist in recognising Black Swan pictorial scenarios for the purpose of training and testing the robustness of the machine learning model, whilst providing a substantiated argument (within the safety case) for enhanced safety in light of unforeseen scenarios. Following this, we delineate solution policies or design decisions concerning the appropriate responses and interventions for each factor. This activity falls under Stage 2, wherein the regulator, customer, and the architect collaboratively brainstorm ideas and make determinations regarding which problematic factors will be included and which will be excluded

(with a compelling justification of why so). Although in Stage 1, within the predictive thinking pipeline 2, step 2.3 (Section H.4.2.3), the architect exercises engineering judgment to assess what should be considered and what may be deferred, the architect subsequently consults with the customer and the regulator to establish a mutually agreed-upon problem space model through a comprehensive set of needs.

Stage 3 entails the formulation of the autonomous solution concept. In this phase, the architect undergoes a series of predictive thinking processes to integrate the solution into the AIC complexity field of the problem space, thereby addressing the newly emergent dynamics of the solution space model. A significant aspect of this process involves conducting assessments of Hazards, Threats, and Opportunities to further anticipate potential Black Swan events. The output of Stage 3A is regarded as the AIC-ordered view of the solution, alongside the definition of the operational design domain. Nonetheless, there remains a possibility that bias may influence the architect's perception of the operational domain. To mitigate this epistemic risk, we have introduced Stage 4, which presents an AIC-disordered view of the solution space.

In Stage 4, we prompt the architect to reconsider AIC perceptions; for instance, if the architect proposes a control interaction between the Eagle Drone (a security drone) and an adversarial drone, during this Stage, the architect is encouraged to shift their perspective and contemplate a scenario where the Eagle Drone finds itself in a situation devoid of influence or control over the adversarial drone, merely observing. Consequently, the architect must envision circumstances in which the adversarial drone exerts total control over the behaviour of the security drone. Disordering the AIC interactions in the problem space model simulates a scenario where the assumed model of reality is completely out of order and thus confusing. This approach clearly demonstrates to the regulator that the architect has indeed reduced their epistemic uncertainty when the complexity of the real world turns out to be completely different from what had been assumed in the design. In complicated open environments, shocking unforeseen surprises are the common experience of any complicated systems architect. Stage 4 aims to increase the confidence of the architect, customer, and stakeholder that chaos has been considered comprehensively.

The outcomes of Stages three and four yield a comprehensive set of system-level requirements, subsystem-level requirements, a safe operating concept, and a machine learning (ML) training concept. The ML training concept, which interprets the safety requirements, is subsequently remodelled as CuneiForm requirements through Stage five, which develops the ML training syllabus. The output of Stage five consists of a collection of CuneiForm requirements that are prepared for instantiation in Stage six. Stage six involves the Black Swan-driven training pipeline, which encompasses the instantiation of abstract CuneiForms into concrete representations,

followed by training and validation to ensure that the final ML model is both Black Swan-ready and adheres to safety standards.

Stage seven consolidates all information and artefacts produced in Stages one through six to instantiate the SACE and AMLAS safety cases (see Chapter 8 and Appendix L). Figure 5.2 illustrates the correlation of the AIC approach with the SACE and AMLAS safety cases and design processes. It is imperative to note that Stage four is not explicitly and definitively delineated in SACE and AMLAS, which constitutes our proposed addition to the processes.

5.2 Stage 1: Uncertainty Problem Articulation and Operational Environment Modelling¹²

This Stage focuses on systematically identifying and articulating the uncertainty associated with the complexity of the problem domain. The primary objective is to define the operational environment (without considering a solution) by recognising unsafe behaviours, modelling the complexity field, and refining the architect's understanding of the context of any solution. The following is a brief description of the main activities performed using the predictive thinking pipeline:

- **Identifying Unsafe and Confusing Behaviours:** The architect begins by listing behaviours that contradict the expected order of the environment, leading to either confusion or direct risks.

These are considered the sources of stochastic confusion that the architect experiences regarding the problem. For example, an adversarial drone hovering near train tracks without immediate hostile action might not be overtly unsafe, yet it normalises unauthorised drone presence, introducing potential long-term security risks. The confusion in such a scenario stems from the fact that the architect is uncertain about the factors that are directly, indirectly and not involved in such an environment.

- **Visualising Unsafe Behaviours:** The identified problematic scenarios are then translated into visual representations, ensuring all stakeholders can holistically appreciate the complexity.

¹² For full SECoT description see Section F.3 (Appendix F)

By generating descriptive images, engineers can better anticipate potential nuanced relationships in the problem, such as wind causing trees to oscillate in a manner that may impact the reliability of a security drone's computer vision system.

- **Defining the Complexity Field and Supra-Complexes:** The architect categorises various subsystems and actors within the problem domain, determining their roles, interactions, and governing constraints.

In this activity, the architect predicts the primary purpose of the holistic systems involved, which then helps the objective prediction of the involved problem complexes that explain their chosen behaviours and behaviours, leading to the problem domain's confusing status. For example, the train network, security monitoring system, and law enforcement units can collectively be viewed as a supra-system that maintains safe railway operations. With such information in mind, the architect starts to model the complexity field of the problem domain using the AIC schema.

- **Defining long-tail factors (Black Swans) and assumptions:** The architect categorises various subsystems and actors within the problem domain, determining the boundaries of their roles in the problem. In this activity, the architect faces the initial batch of potential Black Swans to which they can present to stakeholders and make the appropriate engineering decisions.

This step prioritises factors by relevance and influence over our concern, according to how we view the problem, listing them from most to least frequent (most concern to least concern). Thus exposing our epistemic ignorance and bias. This Stage lays the foundation for risk mitigation by articulating the uncertainty landscape and creating structured models that will inform system design and autonomous response planning. To understand the overall activities, see the following process model:

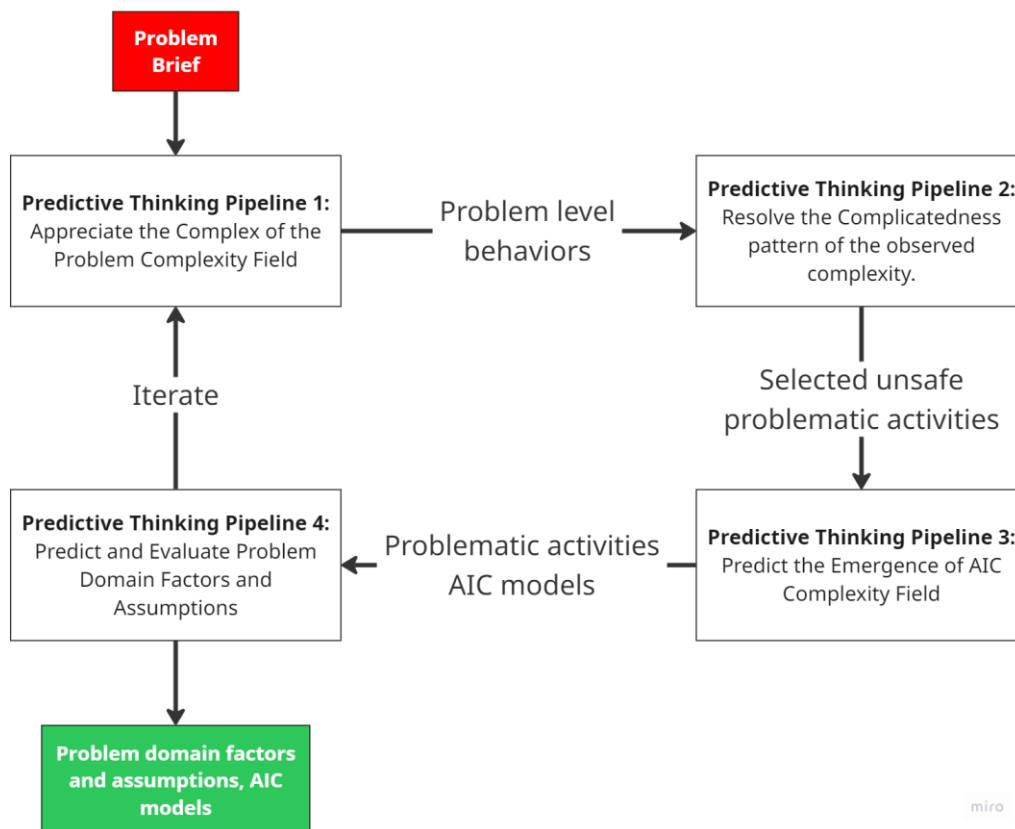


Figure 5.3 Stage 1 process model

Section 6.2 apply to this Stage. For the complete application, see Sections H.4 and I.4.

5.2.1 Identification of problematic Black Swan events¹³

The final step in this process is identifying Black Swan factors. As we gain a clearer picture of the factors contributing to the problem's complexity, we can make an informed judgment and identify those that we deem rare events within this complexity, particularly in the operational environment. The thought process in this activity involves identifying factors and explaining why we believe they are rare and have a significant impact. We can also allow ourselves to consider different situations involving the same factors, or if our predictive thought processes prompt us to imagine other scenarios.

Since we are challenging our epistemic certainty through predictive thinking, we could have a more definable characterisation of what factors may be considered Black Swans and what factors are not. Bear in mind that the output factors are our statement of predictions; we envision an unseen picture of the problem space complexity (which is not apparent at the initial perception

¹³ See Section H.4.4.1 for an implemented example

of the problem during the problem brief). So, since this is a process totally dependent on our knowledge, the challenge here:

What makes a factor a Black Swan or not qualitatively?

Quantitatively, we could examine the number of mentions of each factor during the predictive thinking pipeline. This indicates how much we are concerned about each predicted factor or what our mental model's perception and thinking are biased around. Therefore, whatever the definitive criteria may be for Black Swan, our perception of it would be a factor in it.

A guiding thought: You might find it helpful if you involve humour here. For example, think along the lines of “I tell you what, it would be funny if ...” Then think of something strange but plausible and impactful, or even not impactful. Note we included both situations: impactful and non-impactful, because in a long-tailed complexity field, non-impactful and rarely concerning (by the architect) events are also appreciated as a possibility for concern.

Therefore, to identify Black Swans in the predicted factors, the following criteria can be used as a guideline:

- Predicted factors or interactions. For example, the visual complexity of plants leaves.
- Current architect concern level (the ratios recalled in the analysis. For example, 1%.
- **Rarely concerning test:** the architect is prompted to assess whether a factor or an interaction rarely concerns them in their experience. For example, the architect had never encountered a harmful situation where the shape of a leaf was a primary concern.
- **Impact on ML perception reliability:** Would such a situation impact the reliability of the solution ML model?

Table 5.1 captures our attempt to predict those sources of Black Swan scenarios. We opted to colour the rows that answer yes to rarely concerning fields and impact on the ML model in green to indicate that these are relative Black Swans for us. These factors are then considered Black Swan events, and appropriate requirements, implementation, and testing are used to validate the emergence of safety in such scenarios. The finale table demonstrates that we have considered all factors, concerning and not concerning, which assures comprehensiveness in design.

Table 5.1 Black Swan prediction with respect to Architect Concern Distribution¹⁴

Predicted Factor	Current Concern Level	Rarely concerning	Impactful on ML model detection reliability
roaming_adversarial_drone	20.00%	no	yes
moving_train	17.85%	no	yes
train_tracks_structure	3.08%	no	yes
any_trained_computer_vision_agent_perception_capability	2.46%	no	yes
train_tracks_zone_fence_structure	2.46%	no	yes
train_derailment	2.15%	no	yes
fallen_vegetation_over_train_tracks_structure	2.15%	yes	yes 
powered_powerlines_cables_structure	1.85%	yes	yes 
visual_appearance_vegetation_complexity_growth	1.85%	yes	yes 

5.3 Stage 2: Architect Intent and Autonomous Solution Needs

Definition

Following the problem articulation, this Stage defines the architect's intent and the necessary capabilities to address the identified factors. The goal is to determine the autonomous system's objectives by aligning its actions with the operational problems in Stage 1. We envisage the architect as a complexity doctor who prescribes solutions for problematic situations, which are then translated into potential requirements. The primary purpose here is to get the architect communicating with stakeholders to determine a share vision for the appropriate solution.

¹⁴ Example snippet, for full table see Table H13B, Appendix H

The architect prescribes solutions to selected problems, capturing their high-level intents and assessing stakeholders' needs as supposed to be, given the problem defined. Such as deploying an Eagle Drone System to patrol train tracks and neutralise unauthorised drone threats. The main activities for a solution prescription are:

The process includes the following activities to be done for every interaction and assumption:

Input: The following artefacts:

- Interaction definition.
- AIC model schema.
- Definition of factors.
- Definition of assumptions.

Process: the following thinking activities are to be done:

1. **Stage 1 SECoT Validation:** Validate SECoT artefacts from Stage 1, using SECoT validation process (see Section 5.10 for SECoT validation process, Sections 5.10, L,2,7.3, Appendix L for example implementations to the case study)
2. **Plausibility test:** Is the assumption plausible or not?
3. **Why?:** Define your justification for the reason behind the choice of plausibility value.
4. **Architect Intent (mission):** How does the architect intend to manage the situation? This may involve involving other stakeholders to manage this situation and not necessarily the system of interest
5. **Autonomous Solution needs:** What would be the objective of the autonomous systems in managing the situation?
6. **Autonomous solution constraints:** What constraints should the autonomous solution be bounded within.
7. **Other support systems needed:** If the architect decides to involve other systems or stakeholders, what goals should they fulfil?
8. **Support Systems Constraints:** What constraints should the support systems be bounded within?

Output: The process's output defines the overall solution strategy as a set of prescriptions, including the architect's Intent (mission), the needs and constraints of the autonomous systems, and the support systems.

This Stage establishes a structured contract among the architect, stakeholders, and system developers, ensuring that all design decisions align with the architect's intent and mission. Trade studies can also be adopted to determine the appropriate overall architecture of the system. We will not conduct a trade study as part of this PhD.

Sections 6.3 apply to this Stage partially. For the complete application, see Sections H.5 and I.5.

5.4 Stage 3A: HazTOPs and Ordered AIC-Driven Autonomous System Requirements Development¹⁵

Every contractual solution prescription, along with the overall Sol architecture, is then individually analysed in the context of the associated problem-level AIC models. From that, the architect predicts the autonomous system's AIC behaviour in response to the problem scenarios. The Hazards, Threats, and Opportunities (HazTOPs) Analysis is systematically applied to elicit safety requirements. The following are the main activities performed using predictive thinking pipelines:

1. **Introducing Autonomous Systems into Forward-Feed Complexity:** The architect introduces autonomous capabilities into the problem domain and models their effects. For example, deploying a security drone into the train track surveillance zone introduces desirable (deterrence, detection) and undesirable (false alarms, interference) emergent behaviours.
2. **Designing Backward-Feed Complexity Field:** The backward-feed complexity analysis examines how the operational environment reacts to the presence of autonomous systems and vice versa. For instance, local civilians might perceive the security drone as intrusive, leading to new safety concerns, such as civilian drones interfering with security patrols.
3. **HazTOPs¹⁶:** With full complexity fields designed for solution and problem, we start generating a variety of scenarios that may pose safety and security issues. We also consider leveraging potential affordances from the environment or the system to help reduce the complexity of the system solution. For example, local birds nesting on trees can act as a natural warning system against a drone hiding among the trees represents a potential affordance or opportunity to utilise.
4. **Developing System-Level Safety Requirements:** Using the AIC interaction framework, the architect derives formal safety requirements to mitigate identified risks. For example, the Eagle Drone must avoid collision with power lines while ensuring it can track and deter unauthorised drones within the train track zone.

¹⁵ For full SECoT description, see Section F.5 (Appendix F)

¹⁶ For full SECoT description, see Section F.4 (Appendix F)

5. **Refinement of system-level requirements:** The process also include an AIC-based refinement process which adopts SHARCS mythology in a hierarchical based refinement technique.

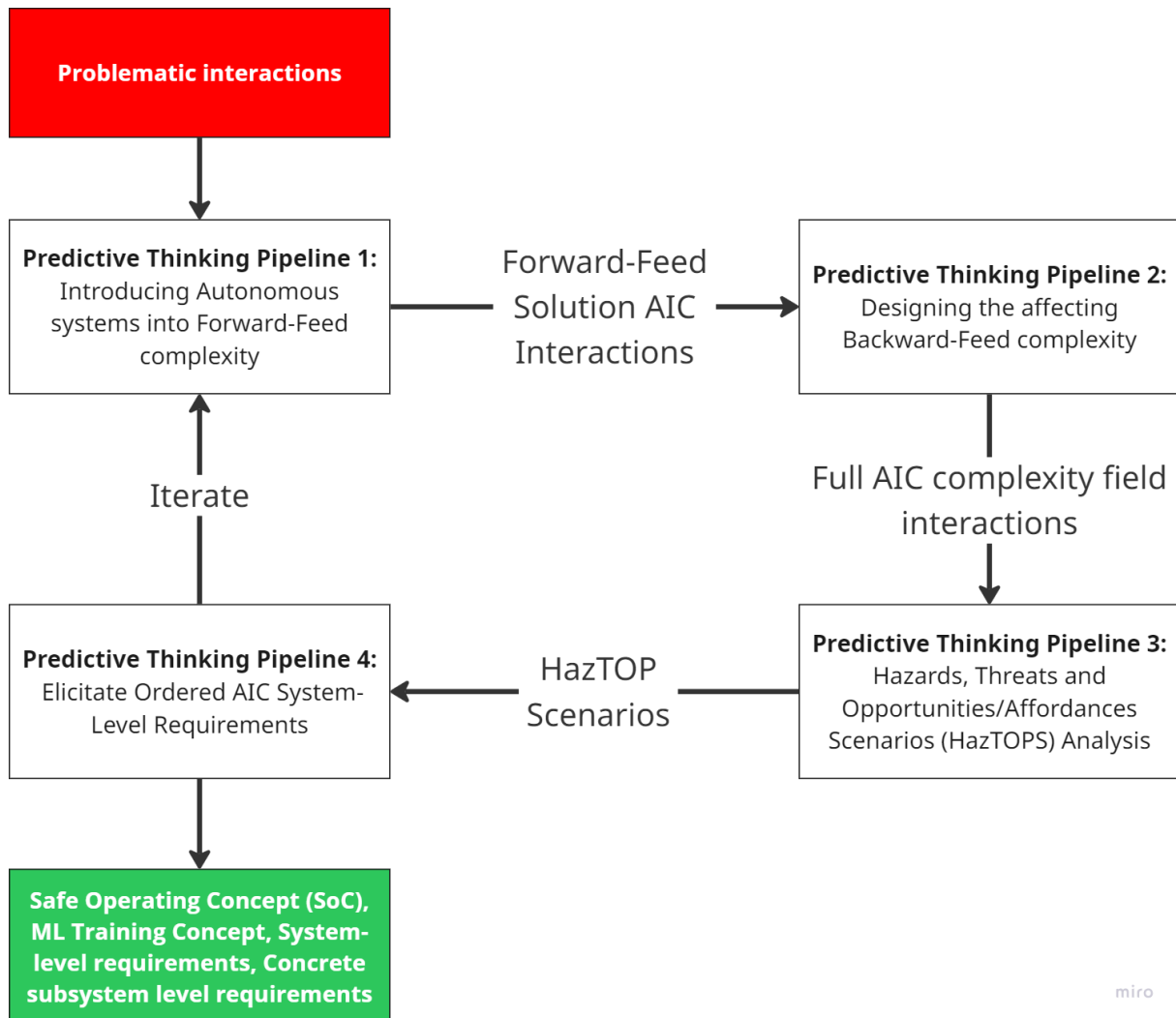


Figure 5.4 Stage 3A process model

Note, we said “ordered AIC”¹⁷ In this Stage, the architect would define the complex's natural behaviour. For example, the security drone aims to control the airspace of the train track zone. This would be a natural, ordered behaviour. A disordered behaviour (relative to what we defined as ordered) would be that the train track zone controls the drone's behaviour. However, there is no hard rule for the architect to consider some interactions as Black Swans. We encourage the architect to keep a vigilant eye on Black Swan at all process Stages. Therefore, it is possible that during this process, the architect may discover Black Swans. This Stage transforms system

¹⁷ See Section E.2 for a detailed definition of AIC timing.

uncertainties into systems and safety requirements, ensuring the autonomous system operates as intended within its mission constraints.

Sections 6.4 and 7.4 apply to this Stage partially. For the complete application, see Sections H.6 and I.6.

5.4.1 AIC-based 4-Whats-&-How refinements

The **4-Whats and How concrete refinement strategy** is designed to systematically elicit concrete sub-system-level requirements by analysing influence relationships within a complex field in a hierarchical manner. The process reuse **SHARCS** [15] hierarchical abstraction technique. This process focuses on breaking down abstract interactions into actionable Appreciation, Influence, and Control (AIC) relationships, enabling the identification of operational requirements that ensure system robustness and adaptability. By iteratively exploring "What" needs to be influenced, "how" to be controlled, and "what" to appreciate, the process ensures a comprehensive and traceable foundation for requirement elicitation.

The main objective is to model the complexity field driving emergent behaviours and define detailed system-level requirements by decomposing influence relationships into their constituent elements of control and appreciation. This process allows for precisely identifying operational parameters, interactions, and supporting mechanisms critical to achieving the system's Primary Purpose (PrimeP). Although the process aims to decompose influence interactions, it can also be used to decompose any control or appreciation interactions. The process is generic and can be adapted according to the need.

5.4.1.1 4-Whats-&-How (4WnH) Process Overview

The **4-Whats and How Analysis** (Figure 5.11) involves iteratively answering the following key questions for every influence relationship:

1. **What is the source situation?** Identify the initiating source complex or component responsible for the interaction.
2. **What is the interaction?** Define the action being exerted by the source on the sink (e.g., regulation, adjustment, or modulation).
3. **What is the targeted sink situation to influence?** Specify the target system, action, or behaviour that needs to be influenced.
4. **How can it be controlled?** Determine the specific control actions required to manage or direct the interaction effectively.
5. **What should be appreciated?** Identify the appreciation mechanisms (e.g., sensors, feedback systems) necessary for situational awareness and adaptive responses.

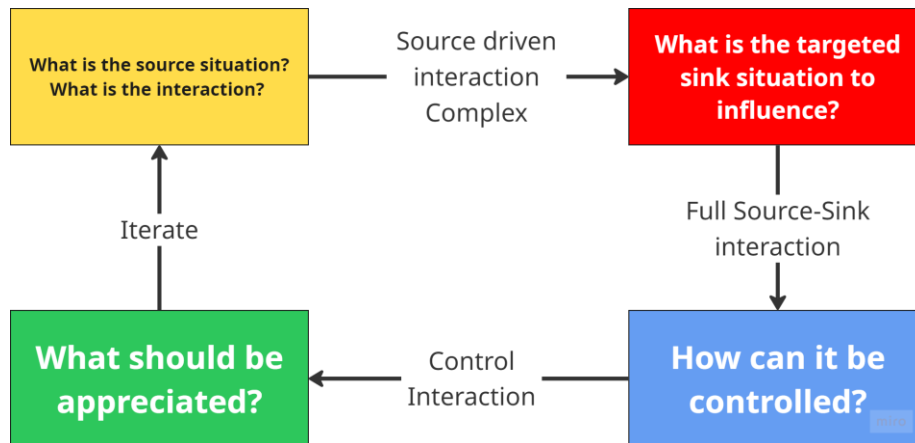


Figure 5.5 4WnH thought process

Repeat the process 5 times for every discovered control complex. Capture the process using the following table:

Table 5.2 4-Whats-&-How process with an example

Source situation	interaction	Target situation (What to influence)	Sink (What (control))	How to control it?	What to appreciate? (appreciation)
Eagle_drone	+flying	low_patrolling _altitude	+Regulate adaptive_ Flight_controller	+Visually scanning visible_terrain, GPS	

The process also includes an AIC hierarchical schema modelling to assist with visualising the whole fractal of subsystem components. The schema provides an interactive and layered understanding of the system's behaviour, highlighting interdependencies between influence, control, and appreciation.

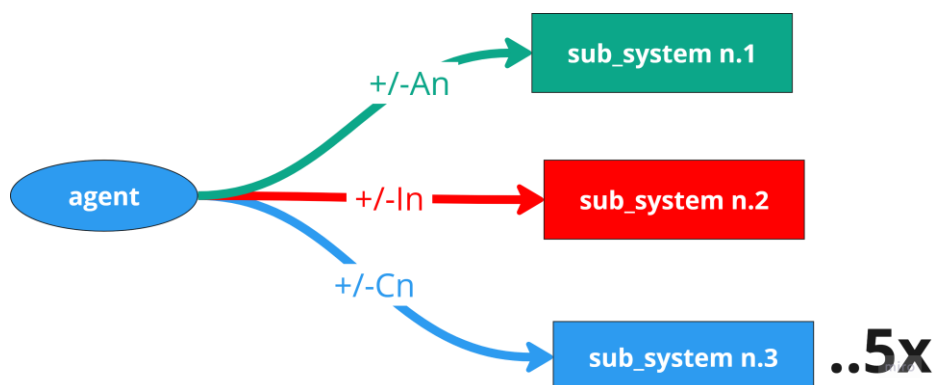


Figure 5.6 AIC Hierarchical decomposition of interactions

5.5 Stage 3B: Comprehensive Operational Environment Definition¹⁸

This Stage refines the operational design domain (ODD) by specifying environmental constraints, dynamic interactions, and potential Black Swan scenarios. The architect defines:

- ODD Uncertainty Grade¹⁹ (e.g. Grade A uncertainty).
- Lighting Conditions (e.g. Sunny).
- Weather Conditions (e.g. Precipitation, Wind, Visibility).
- Time of the Year: Season-specific environmental characteristics (e.g., spring-related environmental changes).
- Landscape type variety definition (e.g., Urban Landscape).
- Geographical region-specific natural phenomena (e.g. Flood)
- Time of the Day (e.g. Noon).
- Perceived Horizon Attitude (e.g. Level Horizon: Roll: -1 to +1, Pitch: 0 to +1).
- Sun sphere positioning (e.g. Mid-Morning/Mid-Afternoon (Medium Angle)).
- Moon sphere positioning (e.g. Waxing Crescent).
- Specialised zones features (e.g. Train Tracks Zone).

This Stage ensures that the autonomous system is ready for realistic deployment scenarios by systematically assessing the operational complexity. The number of factors involved will be application-dependent. Sections 6.5 and M.5 apply this Stage partially. For the complete application, see Sections H.7 and I.7.

5.6 Stage 4: Disordered AIC-Driven Black Swan Scenarios Prediction²⁰

Stage 3A defines what the architect deems to be a typical operation. This Stage challenges the architect's perspective by asking what if the complexity becomes abnormal. We employ a lateral thinking technique we call AIC-perspective shift²¹. This Stage predicts and validates rare, high-impact failure scenarios that Stage 3A might overlook. In this Stage, potential causes for data shift

¹⁸ For a detailed description of the process, see Section F.6.

¹⁹ See Section E.8.3 and Appendix B.

²⁰ See Section F.7 for full SECoT.

²¹ See Sections 4.4 and F.7.

may be predicted. The following activities explain the overall purpose of performing the predictive thinking pipelines:

1. **Stage 3A SECoT Validation:** Validate SECoT artefacts from Stage 3A, using SECoT validation process (see Section 5.10 for SECoT validation process, Sections 5.10, L,2, 7.3, Appendix L for example implementations to the case study)
2. **Defining Disordered AIC Interactions:** The architect shifts perspective to selected AIC complexity fields from Stage 3a, exploring unintended emergent behaviours. For example, what if the security drone misidentifies a bird flock as an adversarial drone swarm? Such an event could lead to unnecessary countermeasures and wasted resources.
3. **Performing Black Swan Scenario Prediction:** The architect examines unknowns by using a predictive thinking process of perspective-shifting system interactions (from Stage 3A) in unpredictable ways to predict Black Swan scenarios.
4. **Defining Safe operating and training concepts:** The architect then defines the appropriate safety and training requirements.
5. **Producing a Black Swan Training Subset strategy:** The identified Black Swan scenarios inform the development of additional safety mechanisms, such as a Black Swan Training Subset dataset, to assess the ML model's performance in Black Swan scenarios.

This Stage ensures that unforeseen risks (during Stages 1, 2, and 3) are anticipated and accounted for, reinforcing the system's robustness to operate under uncertain and dynamic conditions. **It attempts to simulate the state of relative chaos for the AIC-designed complexity, where the entire problem domain contradicts the architect's initial assumptions (output Stage 3A).**

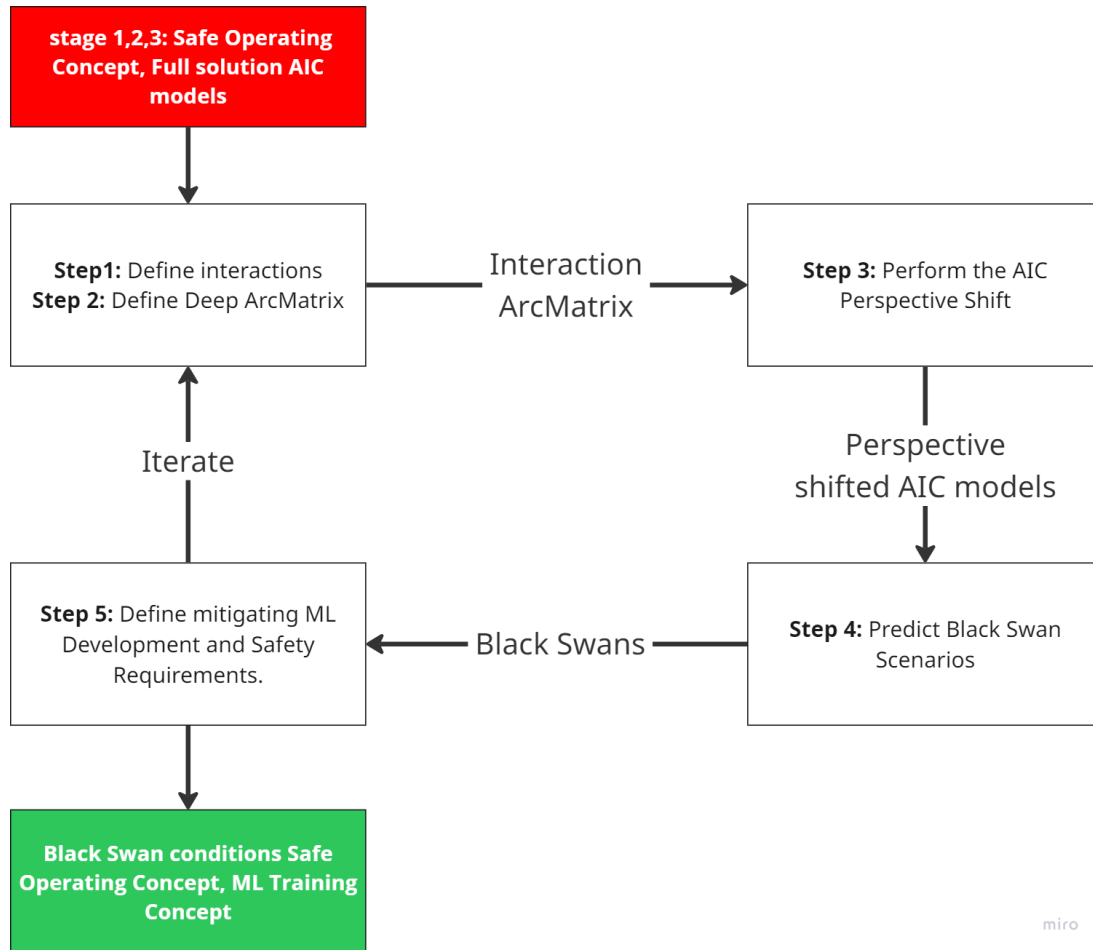


Figure 5.7 Designing for Black Swan operations

Sections 6.6 and 7.6 apply this Stage partially. For the complete application, see Sections H.8 and I.8.

5.7 Stage 5: CuneiForm-based Syllabus for Safety-Driven ML Development

“Epistemic intelligence” in the context of assuring the reduction of uncertainty²² through ML training process refers to the stored knowledge in a dataset that leads to a reduction in the epistemic uncertainty of an architect, regulator, and agent about the real-world complexity characteristics. The knowledge stored in the dataset, which represents the real world for an agent

22 Most safety standards (such as AMLAS, SACE and UL4600) expect the measurable demonstration of uncertainty reduction to be the result of any process. Here, we are being explicit that the CuneiForm production process was developed with the requirement in mind. We use Section 4.3, where we link the CuneiForms production to mitigate epistemic pictorial ignorance risk.

(as per the architect's interpretation of the real world), is then converted into an emergent capability, through training, to generalise well over unforeseen scenarios.

The CuneiForms captures the architect's epistemic uncertainty associated with the real-world complexity, before turning it into pictorial examples for training a model. Practically speaking, CuneiForms are about representing physical environments consisting of static and dynamic entities, aiming to specify image sets for ML training. An example of a CuneiForm and its instantiations is shown in Figure 5.8 and 4.2. In other words, the production of CuneiForms leads to the production of datasets which demonstrate a systematic reduction of epistemic uncertainty about the operational domain for both the model and the architect.

The ultimate objective of this Stage is to construct a safety-driven context-aware dataset systematically using the CuneiForm framework²³. In this Stage, we aim to clearly show how safety requirements were directly captured by thousands of images using abstract control pictorial language, which we call CuneiForm. Those CuneiForms are then described and explained in the final safety case, where a regulator may validate using statistical methods to check whether a subset of images truly captures the pictorial characterisation of the associated CuneiForm.

The following is a model of the CuneiForm generation process:

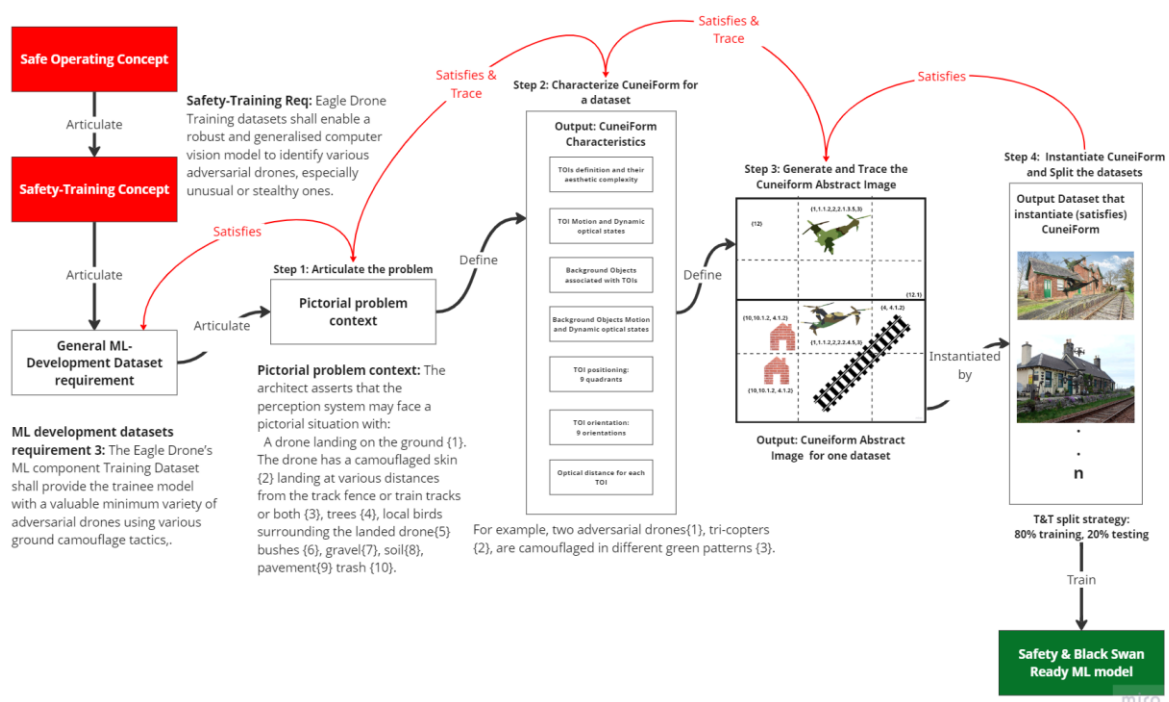


Figure 5.8 CuneiForm Syllabus development and validation process

²³ See Sections E.7, E.8 for the theoretical description of CuneiForm development and validation processes.

The main activities involved in this process:

Takes a general ML Development Dataset requirements as input, then:

1. Articulate pictorial problem context.
2. Characterise the CuneiForm dimensions.
 - a. Target Objects of Interest (TOI)s definition and their aesthetic complexity.
 - b. TOI Motion and Dynamic optical situations.
 - c. Background Objects associated with TOIs.
 - d. Background Objects Motion and Dynamic optical situations.
 - e. TOI's positioning: 9 quadrants.
 - f. TOI's 3D orientation: 9 orientations.
 - g. Optical distance for each TOI.
3. Generate the CuneiForm image.
4. Instantiate the CuneiForm with conformant in-context images.

The output from Stages 3 and 4 represents sources of epistemic uncertainties that the architect faces in the given problem domain. They also signify the machine's epistemic uncertainty about the real world it will enter. Therefore, the architect must describe the real world, including all sources of unexpected scenarios that the trainee machine will need to navigate and fulfil the architect's intent. Thus, we need to reduce these sources of uncertainty for the machine to be as low as reasonably practicable (ALARP). We achieve this through the systematic construction of epistemic uncertainty mitigation CuneiForms. Therefore, it is more accurate to describe this process as ML Epistemic Uncertainty Reduction Training, as it aims to enhance the robustness of the machine solely by adding more knowledge to the dataset about the real world rather than employing clever augmentations of the dataset.

Dataset augmentation techniques (such as manipulating the image colours, etc) will be carried out later; we consider such augmentations as "Aleatoric Uncertainty Reduction" since we are manipulating the aleatoric uncertainty of the pixels themselves (randomness) rather than the actual scenarios themselves. This Stage focuses on developing machine learning (ML) training datasets that align with the defined system and safety requirements. The CuneiForm process ensures:

- Training data is structured and valuable (e.g., drone detection images are curated based on real-world operational constraints and systematically to concrete requirements).
- Black Swan scenarios are embedded into dataset validation (e.g., including images of stealth drones that adversaries may use to bypass detection algorithms).

The final output datasets (Training, Validation, and Testing) would be composed of the following types of scenarios:

- In-context typical operations (e.g. regular drone flying around train tracks).
- In-context Black Swan Scenarios (e.g. a flock of birds appears to be a swarm of adversarial drones).
- out-of-context relevant data (e.g. online-sourced random images that includes a drone in it).

When we say “in/out-of-context” we mean in the context of a CuneiForm. A Black Swan image or dataset has the following properties:

- It directly instantiates a Black Swan CuneiForm.
- It is out-of-distribution (dissimilar) to the training and validation dataset combined²⁴.

With Black Swan-driven context awareness, the idea here is to predict challenging pictorial scenarios related to Black Swans in Stage 4, then generate CuneiForm and data. This Stage embeds safety into ML development, ensuring that AI-based decision-making remains robust and aligned with operational requirements.

5.8 Stage 6: Black Swan-driven ML Development and Testing pipeline

The final Stage integrates the ML models into real-world operations, ensuring their safety and reliability through rigorous testing.

The architect may not initially include Black Swans in the Training syllabus; instead, Black Swans serve as an assurance policy for Black Swan validation.

After the architect develops typical operations and trains the model, if it passes both typical operations and out-of-context testing examples, the architect tests it on the Black Swan

24 If a Black Swan image appears to be in distribution with the training and validation, then it is fair to expect that such image should not be a challenge for the model to pass and that the diversity of the training truly mitigate the Data Shift in the Black Swan. By definition it will not be classed as Black Swan, but we could call them Grey Swan meaning we don’t know until we see how the model will react to it.

scenarios. If it fails on those Black Swans, the architect instructs the ML development team to further increase the training and validation size and diversity with the Black Swan examples.

The goal is that the architect expects the size and variety of examples in the training and validation sets are sufficient to pass the Black Swans without generating additional data (we assume that generating more in-context data is expensive and time-consuming, with no clear stopping point). Thus, we use the Black Swan as the final pass to streamline the process and maintain training validation efficiency, while also using the Black Swan to guide additional training when testing fails

For example, suppose we predict 10 Black Swan scenarios. After the model (trained on typical operations and out-of-context data) passes the test on in-context typical operations and out-of-context data, the architect may test the model on the Black Swans prior to including them in the training. Suppose the model passes 3 out of 10 scenarios; that means the diversity and the size of the training and validation sets are insufficient to cover those Black Swans. Then, instead of generating more in-context Black Swan training data, we only generate data for 7 Black Swans where the model had failed. By doing so, we reduce the effort of generating more data while demonstrating the ability to handle Black Swan scenarios.

5.8.1 Designing and Validating Robustness during Black Swan operations experiment

To comprehend the interplay between Black Swan training and the model assurance process, we investigated the impact of Black Swans on model reliability. Furthermore, we needed to validate that the outputs of Stages 4 and 3A genuinely inspire the prediction of Black Swan scenarios, necessitating a clear understanding of what we mean by Black Swan in a visual representation sense.

We investigated the role of context-aware dataset construction, specifically, the inclusion of “Black Swan” (non-typical, out-of-distribution, OOD) scenarios, in shaping the robustness and trustworthiness of object detection models for safety-critical applications (e.g., autonomous drone detection near train tracks). Building on established findings that high in-distribution (InD) accuracy often gives a false sense of security in trusting trained models performance metrics , we sought to determine whether explicitly training on synthetically generated in-context Black Swan images improves downstream performance on rare, high-impact test cases and thereby strengthens assurance arguments. Table 5.2 provides a concise summary of the experiment (for a more detailed version, review Section H.10, and H.11, Appendix H):

Table 5.3 Experiment characterisation

Aspect	Summary
--------	---------

Hypothesis

Hyp1: Models trained and validated solely on typical or out-of-context images will show high performance on similar data but fail on OOD Black Swan scenarios. Conversely, inclusion of in-context Black Swan examples in training should yield measurable performance gains on Black Swan test sets.

Hyp2: Testing on a dedicated Black Swan test set is a crucial metric for revealing unmitigated OOD vulnerabilities and supports rigorous safety assurance.

**Chosen Model
and Metrics**

Architecture: Roboflow 3.0 Object Detection (Fast) with COCO pre-trained weights; no image augmentation.

Primary metric: mAP@50 (IoU threshold 0.5).

Visual-shift validation: Perceptual hashing (pHash) + Jaccard IoU on unique pHashes to confirm OOD status between splits.

Datasets

Out-of-context images: Five external drone datasets to simulate generic backgrounds.

Typical operations: Synthetic in-context scenes (CuneiForms H.42–50).

Black Swan scenarios: Two batches of synthetic in-context OOD events (CuneiForms H.36–41, H.51–53).

**Experiment
Groups (Nine
Configurations).**

Group 1: Out-of-context only (Exp. 1, 4, 5).

Group 2: Typical operations + out-of-context (Exp. 6, 7, 10).

Group 3: Black Swan + out-of-context (Exp. 2, 3, 8.1, 8.2).

Group 4: Comprehensive mixes (Exp. 9.1–9.5).

Each experiment varied the proportion of image types in training/validation and tested on separate typical and Black Swan test sets.

**Key Outcome
Patterns**

- Models trained exclusively on out-of-context data achieved high mAP@50 on similar test images (e.g. 97% in Exp. 1) but plummeted (to 24% in Exp. 4) on OOD Black Swan tests. Thus demonstrating a false sense of security over the actual reliability of the model.

- Inclusion of even a small fraction of Black Swan images in training (e.g. 30% in Exp. 3) boosted Black Swan test performance significantly (up to 67%).
- Merely enlarging out-of-context datasets (e.g. Exp. 10 with ~85 k images) failed to close the OOD gap (35% on Black Swan).
- Data augmentation (Exp. 9.3) did not compensate for missing Black Swan exposure.
- Certain out-of-distribution (OOD) shifts (e.g., Experiment 5's typical operations experiment) were "mitigated" by training diversity, suggesting that OOD status alone does not immediately mean potential failure. This implies that it is feasible to diversify the image pool and conduct a comprehensive assessment of the dataset's designed variety in terms of performance over the OOD test set. If it performs as anticipated, this does not negate the OOD set's classification as a non-Black Swan; rather, it indicates that the training process has indeed matured the model intelligence to manage such OOD scenarios effectively, thus reflecting the overall adequacy of pictorial variety, and the corresponding variety of the CuneiForm Syllabus.

5.8.1.1 Key Insights

For full detailed description of the experiments see Section H.10 and H.11. If we were to compare the training syllabus between Black Swan-experienced and inexperienced models, 9.4 and 10, we would realise clearly that size does not matter in terms of trust in performance over Black Swan scenarios. Figure 5.7 presents a Venn diagram comparing the similarity of training and validation across experiments for two datasets. We use this to validate the similarity between, for example, a Black Swan test set and a training-plus-validation dataset. This figure can be used as part of the arguments surrounding the validation of a given Black Swan dataset, satisfying an essential criterion of being a Black Swan, which is an out-of-distribution with the training in a pictorial sense.

Despite the training syllabus in Exp. 9.4, yielding a smaller set, it still enabled the training of a model that we can trust more in terms of its ability to handle Black Swan operations compared to the training syllabus that focused only on out-of-context training with some in-context images.

Models trained exclusively on out-of-context images may achieve impressive in-distribution accuracy, yet they can collapse under truly novel conditions. This failure, often masked by a high

in-distribution score, creates a false sense of security, underscoring the crucial need to weave rare, high-impact “Black Swan” scenarios into the development pipeline from the outset. When even a small proportion of in-context Black Swan images is deliberately included in training, the resulting performance gains on out-of-distribution tests are striking. For example, in Exp. 3 (Section H.11.6(i))., we included only 30% (2390 images) Black Swan images out of 8000 total training dataset the remaining 70% where out of context images), the trained model achieved 67% mAP50 over Black Swan batch A test set. in Experiment 10 (Section H.11.5(i)), which we included no Black Swans in the training, but rammed up the training set size to 85677 with 4% (3270 images) of in-context typical operations images (nearly 1000 more images in comparison to Exp.3 Black Swans), the same model architecture perfumed significantly lower at only 35% mAP50 on the exact same Black Swans Batch A test set. These targeted inclusions not only bolster empirical robustness but also serve as concrete evidence in support of safety and assurance claims for reliability during Black Swan data shift.

Verifying whether a given test set truly represents an out-of-distribution challenge can be conducted systematically through perceptual-hash similarity metrics. A zero-per cent overlap in unique perceptual hashes between training and testing datasets signals a bona fide Black Swan scenario, and the model’s performance drop on such tests confirms an unmitigated vulnerability. To systematise resilience against rare events, we advocate an iterative, Black Swan–driven development cycle: first, predict candidate Black Swans and specify them as structured CuneiForms, then assemble a non-Black-Swan reference dataset, followed by synthetic Black Swan images. After confirming their out-of-distribution status via similarity tests, a “White Swan” model is trained and challenged. Failures lead to reintegration of the problematic scenarios into training; successes indicate that the shift has been effectively mitigated.

Venn Diagram of Unique pHash Signatures
 Generated at: 2025-05-06 09:47:27
 Unique pHash: exp9_4_train+valid: 17360, exp10_train+valid: 45917
 Intersection: 9949
 shared visual distribution likelihood (unique pHash datasets IoU): 18.66%
 How much of A in B, B in A: exp9_4_train+valid-> exp10_train+valid: 57.31%
 exp10_train+valid-> exp9_4_train+valid: 21.67%

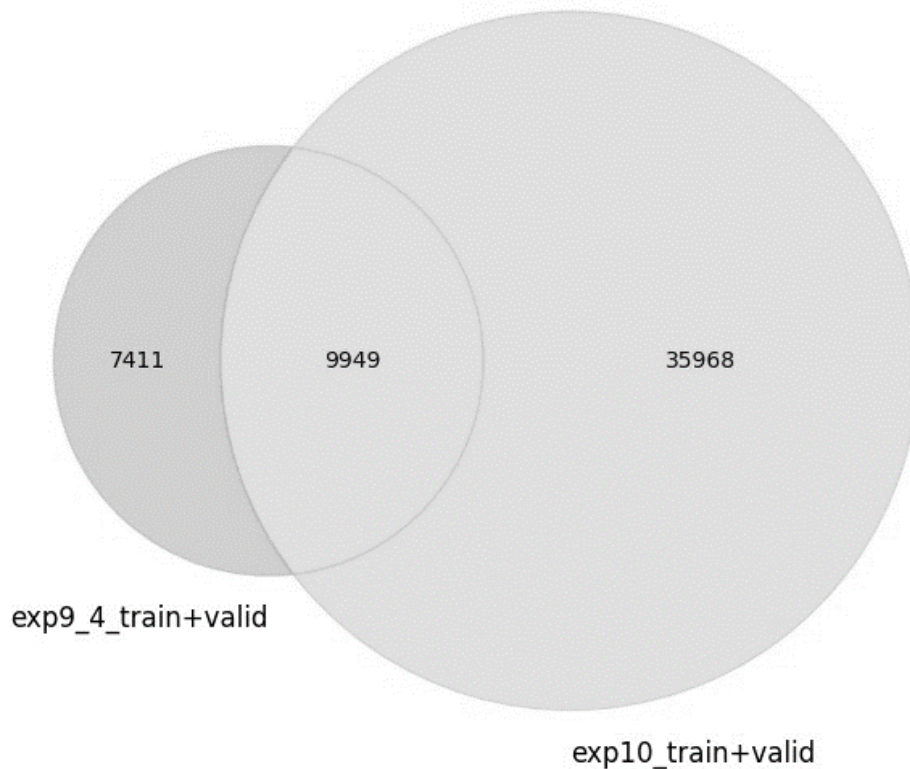


Figure 5.9 Similarity test between training syllabus 9.4 and 10

Our findings also demonstrate that sheer dataset volume cannot substitute for contextual relevance. A smaller (Exp.3 training of 8000 images / with 2390 in-context Black Swans, performing 67% mAP50 over Black Swan Batch A test), context-aware training mix outperformed a much larger (Exp.10 training of 85677 images / with 0 in-context Black Swans, performing 35% mAP50 over Black Swan Batch A test), context-unaware corpus on rare-event tests, illustrating that awareness of operational scenarios, particularly Black Swan events, is far more influential than raw data quantity alone.

Moreover, not all out-of-distribution shifts provoke equal difficulty. Some OOD test sets are “mitigated” by pre-existing pictorial diversity in the training data, For example, See Figure 5.10, in Exp.5 (Section H.11.4(iv)), we trained a model with the following mix: **Total:** 9924(69%), In-context images = 0, Out-of-context images = 9924 (100%), and tested it on **Total:** 3061, of which In-context typical operations images = 3061 (100%) and 0 OOC. The IoU between the test set and the training+validation = 0%. This means the in-context test set is out-of-distribution with the training. The results were surprising, the model was able to perform well on such OOD scenario. This means that the pictorial diversity provided by the training and validation test set were sufficient to handle such OOD.

This means, the distribution of the training has implicitly included the distribution in the test, but those images did not physically exist in the training set. In some sense, one can understand such phenomenon like there is a hidden ghost of distribution within the folds of the training and validation diversity of information to which the performance of a model can be trusted. So, if an out-of-distribution datasets can be mitigated by a variety of images in training and validation, then surely, we can assume trust on hidden ghost images which are in-distribution with the training, but not part of the training. We called those type of images as Friendly “White Ghost patterns”. We articulate some elaboration on this phenomenon because it has a direct impact on how we assure safety over unforeseen scenarios and how we specify requirements. In other words, understanding this ghost patterns objectively, allow us to make more daring claims which we can validate (using our Black Swan-driven training pipeline) for a safety case.

Exp.5 trained model still yielded reliable performance despite being unseen. Others remain “unmitigated” Black Ghost patterns (like in Exp.10) which we could refer to them as Black Ghost patterns, where the model fails. For more detailed articulation on this observation, see Section E.9. Distinguishing between them hinges on empirical performance under pHash-verified shifts.

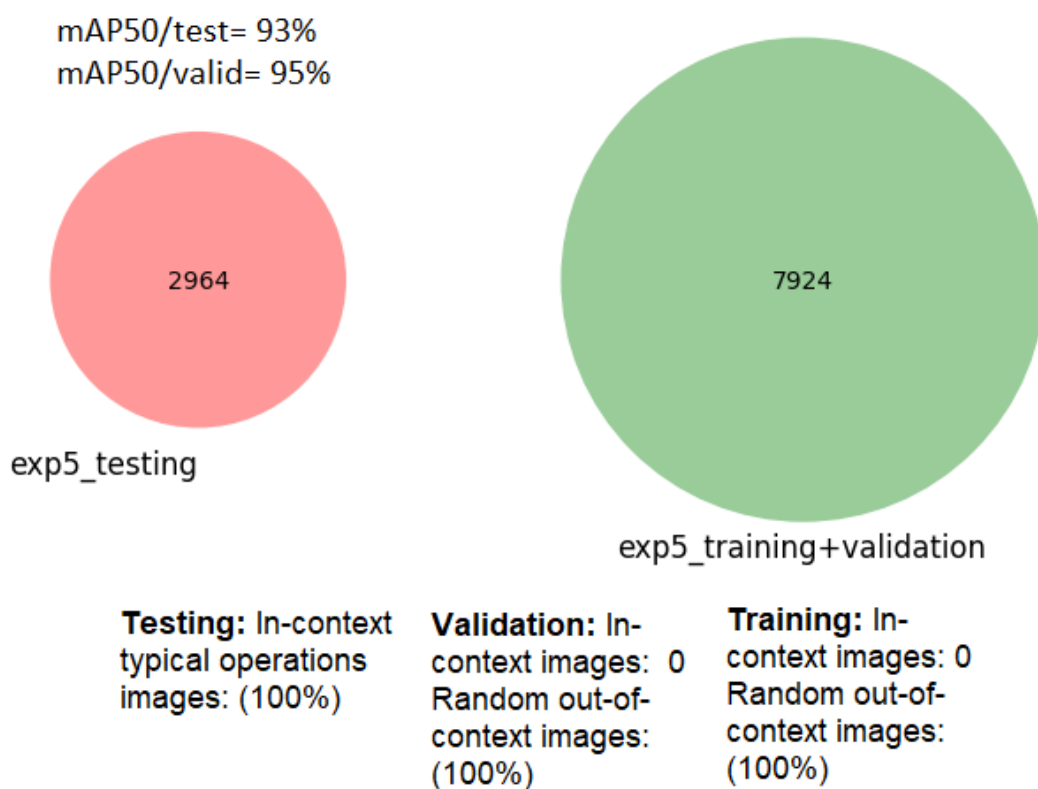


Figure 5.10 Exp.5 reveals that it is possible some OOD scenarios can be mitigated by the diversity of the training and validation set

The behaviour over White Ghosts patterns, cannot be guaranteed to stretch over OOD Black Swans, because the same dataset for training and validation in Exp.5 were reused and was

augmented further with added 85000+ strong training set in Exp.10, even with the additional images, the model of the same architecture did not perform reliably.

Although data augmentation can offer incremental improvements, it remains secondary to achieving accurate epistemic coverage of rare events. Prioritising the explicit representation of anticipated Black Swan scenarios ensures that augmentation complements, rather than substitutes for, robust, context-aware training. Finally, our experience is that incorporating visual evidence, similarity diagrams and performance plots that illustrate unmitigated failures and subsequent mitigations, could provide a powerful tool for safety assurance. Such pictorial demonstrations make it clear to regulators and stakeholders that Black Swan vulnerabilities have not only been identified but also systematically addressed. In summary, the following is a list of findings which inspired the approach for Stage 6.

Our experiments confirm that a context-aware, Black Swan-driven development process is essential for building reliable object detection models in safety-critical domains. Systematic inclusion and validation of out-of-distribution (OOD) scenarios not only improve empirical robustness but also underpin rigorous assurance arguments, thereby reducing the legal and operational risks associated with rare but catastrophic model failures.

5.8.2 Black Swan-driven training pipeline approach

Note: For a fully elaborated justification behind this process in light of the experiments and requirements of standards, see Section E.9.

Based on the findings presented in the previous Section, we have developed an approach to ensuring that a trained model consistently preserves safety properties and performance reliability, thereby providing a coherent argument for a safety case. This is contingent upon the architect adhering to the specified training protocols and regimes:

1. **Validating datasets' conformity to CuneiForms:** Before model development, the architect needs to ensure that datasets conform to CuneiForms and that CuneiForms satisfy requirements. One way to do this is to take a statistically sound sample of the dataset that is claimed to be conformant to a Cuneiform and then use the CuneiForm validation process (see Section).
2. **Typical operations model development and testing:** The architect ensures that datasets comprehensively represent the operational domain (apart from Black Swan scenarios). The architect decides on the architecture and performs the training and testing of ML models.

3. **Performing Model Testing Under Black Swan Conditions:** ML models are tested against unexpected conditions (separate from the rest), such as drones camouflaged against urban infrastructure or spoofed GPS signals.
4. **Validating System Performance in Real-World Deployments:** The final step is deploying the system in controlled live environments and evaluating its predictive performance against emergent complexity.

Stage 6 ensures that autonomous decision-making remains safe, adaptable, and aligned with mission objectives (characterised by the CuneiForms) during both typical and Black Swan operations by verifying that the system undergoes thorough ML validation. The process strategy is based on our experiments in Appendix H, Section H.10 and H.11. It is one thing to claim the discovery of a Black Swan, and it's another to demonstrate that we truly caught one in a safety case. Figure 5.11 describes our approach to how we can do that. Note that we did not include Black Swan in the initial training phase. The reason is that generating in-context datasets can be costly and time-consuming. We hope that the diversity of the gathered out-of-context and in-context typical operations data would be enough in general to enable a model that can handle data shift (out-of-distribution Black Swans).

A further reason for this process is to demonstrate that the design process has deliberately accounted for Black Swan scenarios. If we don't do this way, how do we demonstrate that the Black Swan scenarios we predicted are out-of-distribution and that we discovered that a trained model did or did not perform well over them? With this approach, we can demonstrate that the risk has been reduced in the safety case. Black Swan enables us to assess the effectiveness of our training syllabus and determine whether our prediction of Black Swan in Stage 4 has helped identify unmitigated potential data shifts resulting from changes in the problem domain's complexity.

The ultimate goal behind our chosen ML training pipeline is to systematically ensure safety properties are provisionally accounted for in ML model's generalisation development datasets, serving as evidence to achieve the following assurance objective:

Target Assurance Objective: The ML model's generalisation shall preserve the traced safety properties during data shift and unforeseeable conditions²⁵.

25 See Section E.9 for definitions of data shift and unforeseeability and their relationship to UL4600 Standard for Safety for the Evaluation of Autonomous Pr

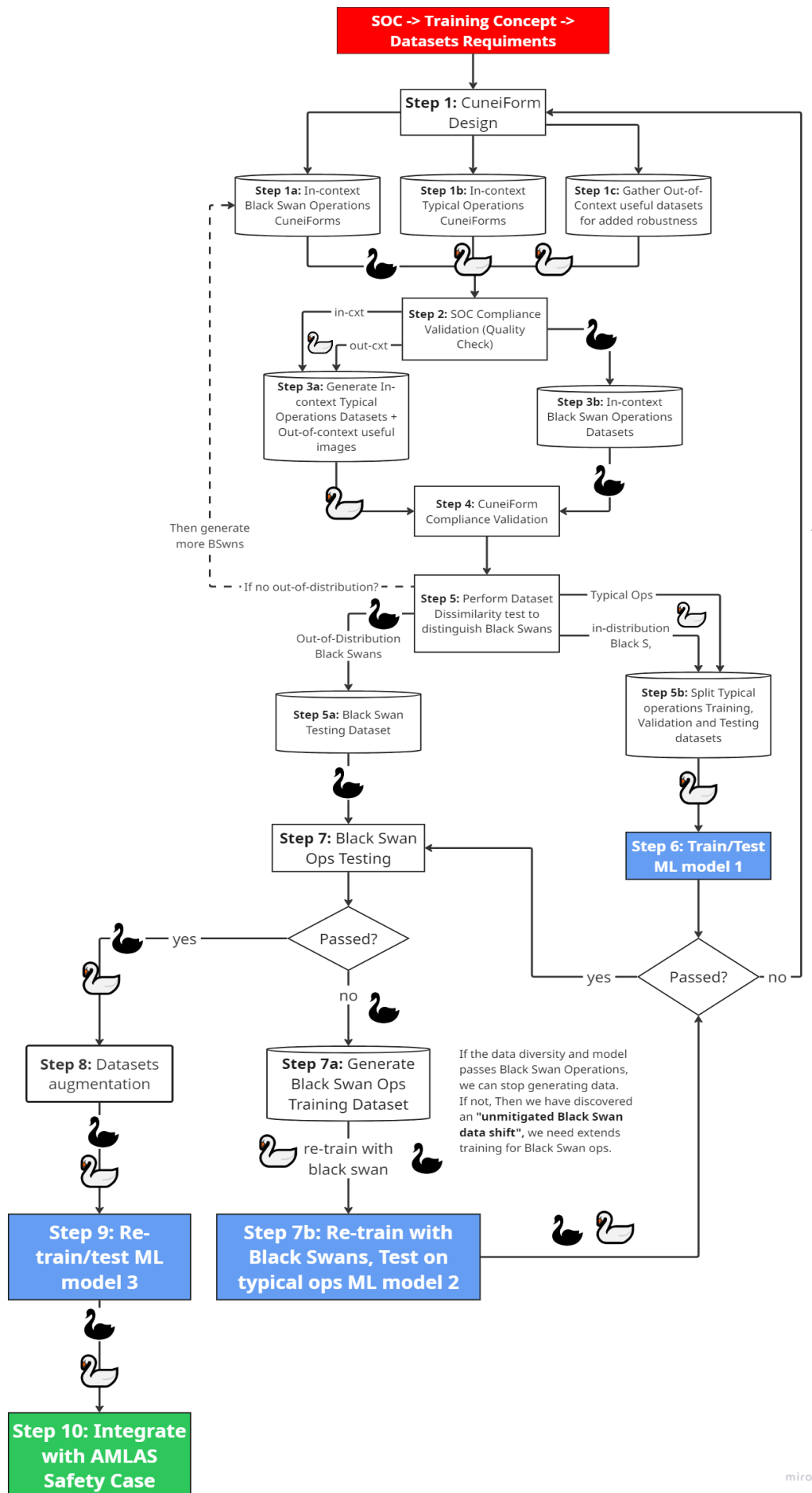


Figure 5.11 Black Swan-driven safety-critical ML development pipeline

To achieve the assurance objectives, we developed a Black Swan-driven training and testing pipeline which can demonstrate compliance:

1. **Design CuneiForms:**
 - a. In-context Black Swans.
 - b. In-context Typical operations.
 - c. Out-of-context useful datasets for added robustness.
2. **Validate CuneiForm compliance with SOC:** Validate that CuneiForm correctly captures safety-training requirements.
3. **Instantiate CuneiForms:** generates datasets as per type. This activity will produce two types of datasets:
 - a. In-context typical operations and out-of-context useful datasets. Let's call it the White Swan dataset.
 - b. Black Swan dataset.
4. **Validate Instantiated Images Compliance with CuneiForms:** Validate that the datasets are compliant with their designated CuneiForms. Use CuneiForm validation process played out in Section F.11.
5. **Dataset Dissimilarity testing:** In this Stage, we try to distinguish which Black Swans are in or out of distribution with White Swans. Those that appear to be in distribution can be added to the White Swan dataset. At the end of this step, we have two distinct datasets:
 - a. Black Swan testing dataset.
 - b. White Swan development dataset.
6. **Develop the White Swan model:** The first model will be trained using the White Swan dataset.
7. **Black Swan operations training and testing:** Test the initial model on out-of-distribution Black Swans. The expectation is that it should pass if the dataset's diversity and size are sufficient. Decide on acceptance criteria for how much would be a successful pass rate. We opted to expect a 100% pass rate over Black Swans.
 - a. If the Black Swan testing fails the acceptance criteria. Then we have discovered an unmitigated Black Swan data shift.
 - b. Generate Black Swan training dataset and re-train the model, then test as usual. Producing ML model version 2 (Black Swan trained).
8. **Dataset augmentation for adversarial attacks robustness:** If Black Swan testing passes, we have achieved sufficient epistemic diversity and dataset size by producing a model that can handle some plausible data shifts. Only then do we apply data augmentations for added robustness.
9. **Develop final model:** We re-train and test the model again.
10. **AMLAS Safety Case:** We then integrate the patterns into the AMLAS safety case.

5.9 Stage 7: SACE and AMLAS Safety Case Production

Output design artefacts of the AIC approach are integrated into each Stage of the SACE and AMLAS Safety Case frameworks to ensure that uncertainties are addressed, operational domain assumptions are clearly articulated, and autonomous system behaviour is predicted and verified. Below, we outline how each Stage of SACE and AMLAS is developed using outputs from the AIC Systems Approach. As we recalled earlier, SACE and AMLAS do not directly provide clear guidance on how to tackle Black Swans and assure safety in such conditions, which is recommended in Stage 4. For a comprehensive and detailed description of the process, refer to Chapter 8 and Appendix L.

In **SACE Stage 1: Operating Context Assurance**, the primary goal is to establish a clear Operational Domain Model (ODM) that delineates the boundaries and assumptions under which the Autonomous System must operate safely. Our AIC Systems Approach addresses this by first articulating the problem domain in depth (capturing environmental factors, stakeholder needs, and high-level capabilities) before introducing any solution elements. We generate a comprehensive problem-domain model and environmental assumptions, then employ our Predictive Thinking Pipeline to forecast complicated interactions among the parts, which in turn inform the set of operating scenarios. These scenarios are validated through stakeholder engagement and expert peer review, ensuring that each assumed condition and operational case is both relevant and sufficiently detailed.

SACE Stage 2: Autonomous System Hazardous Scenario Identification builds directly on the context established in Stage 1 by uncovering, analysing, and validating potential hazards. Within our framework, HazTOPS analysis rigorously captures safety hazards, threats, and even opportunities arising from environmental affordances. Using the same predictive pipelines, architects model AS decisions and environment interactions, then systematically identify hazardous scenarios. Validation occurs through structured semantic checks (SECoTs) and expert review, guaranteeing that each documented hazard reflects real-world agent behaviours, environmental elements, and decision points, and that safety requirements can be clearly derived.

In **SACE Stage 3: Safe Operating Concept Assurance**, the focus shifts from hazard discovery to hazard mitigation via a well-defined Safe Operating Concept (SOC). Our approach leverages the first two predictive pipelines to map AS behaviours onto the problem domain and to anticipate environmental responses. This dual modelling allowed us to specify detailed safety requirements, which help define the Reduced Operating Domain (ROD) constraints, ensuring the AS avoids or safely manages identified hazards. Validation again relies on semantic traceability

and peer review of design decisions made through predictive thinking pipelines, confirming that the SOC's safety constraints are unambiguous and sufficient to address all hazardous scenarios.

SACE Stage 4: Safety Requirements Assurance decomposes the system-level safety requirements defined in the SOC into concrete, subsystem-level specifications. We employ SECoTs and the "AIC-based 4-whats-and-hows" (4WnH) method within our predictive thinking pipelines in Stage 3A to trace each requirement back to the architect's original intent and environmental analyses. This rigorous decomposition and subsequent validation process ensures that every lower-tier safety requirement is logically justified, semantically consistent, and capable of being verified through design and testing.

During **SACE Stage 5: AS Design Assurance**, we translate validated safety requirements into concrete design elements. In this PhD work, the focal design tier is the dataset construction process: CuneiForms define how training, validation, and test datasets must satisfy safety-driven SOC requirements. Each CuneiForm generation is accompanied by a SECoT that documents the architect's rationale for including specific data. A formal quality review of these forms, coupled with ongoing peer validation, demonstrates that our dataset design fully supports the AS's safety and operational objectives.

Finally, **SACE Stage 6: Hazardous Failures Management** ensures that any residual hazardous failure modes, deviations from intended behaviour are identified, analysed, and mitigated. Our HazTOPS-driven predictive pipeline revisits potential deviations, while subsequent modelling and SECoTs derive targeted design changes, operational limitations, and additional safety requirements. This structured cycle of failure identification, mitigation specification, and argumentation mapping provides a transparent record of how each hazard is managed.

Note 1: Stages 7 and 8 of SACE lie beyond the current PhD's scope.

Note 2: For elaboration on the AMLAS process, to avoid duplication at this Stage, we opted to not describe it in this chapter. Refer to Chapter 8 for a detailed description.

5.10 SECoT Validation Process

The following process provides a structured approach for validating Systems Engineering Chains of Thought (SECoTs) in a systematic and repeatable manner. It ensures that each SECoT follows a rigorous predictive thinking process, aligning with general systems principles and enabling a thorough exploration of emergent behaviours. The template can be adapted for any SECoT by defining specific elements and applying the methodology to various systems engineering

contexts. For a full description of SECoTs, see section 4.3 for a full definition of SECoT methods. Examples of implementation can be found in sections 6.10 and 7.9.

The main use of SECoT validation is to provide the “justification” arguments for design decisions that is required in some of AMLAS and SACE stages. The approach to validate would be better understood if we express a SECoT in GSN format. See section 4.11 of how a SECoT can be understood and thus how we approach its validation process.

The following are the main activities which can constitute sections in a validation report:

5.10.1 Purpose of validation

- Clearly define the purpose of the validation report.
- Explain why the SECoT needs to be validated.

Example: This report validates the "AIC Perspective Shift SECoT", ensuring that its predictive thinking steps systematically uncover unforeseen scenarios that impact autonomous safety assurance. Also, validate that the architect's predictive thinking had not been blindsided by the focus on a limited number of subjectively desired factors and ignoring other hidden factors.

5.10.2 Define the Scope of validation

- Define the boundaries of what is being validated.
- Specify whether the validation applies to a single SECoT, a set of SECoTs, or a broader system application.
- You may also include “Out of Scope” to summarise what would be outside the scope of validation.

Example: The scope of this validation includes examining predictive thinking pipelines, general systems rules, and structured outputs to ensure alignment with the SECoT_1 principle.

5.10.3 Define SECoT output validation criteria

5.10.3.1 Validation Criteria

- Define the goal of validation (the output prediction is true) meaning the correctness of SECoT activities leads to a correct inference goal (output prediction).
- Define measurable criteria (pass/fail) to determine whether SECoT predictions are valid and valuable.
- Include qualitative and quantitative measures.
- Examine if the step completion criteria in the SECoT are preserved.

Example criteria:

- The output prediction is true.
- An initial shallow scan of the problem space does not blindside the architect's prediction of deeper factors.
- The predictions align with general systems axioms.
- The SECoT process reveals previously unrecognised emergent behaviours.
- The thinking pipelines yield multiple alternative scenarios.
- Step completion criteria are preserved.

5.10.3.2 Validation Results Table

Define the appropriate textual description of the validation test that needs to be checked for a given SECoT.

Table 5.4 SECoT validation results table

Validation Step	Evaluation Criteria	Pass/Fail	Notes
Step 1	Alignment with general systems rules.	Pass	Predictions adhere to known system behaviours.
Step 2	Generation of unforeseen scenarios.	Pass	Identified new emergent behaviours.
Step 3	Applicability to real-world cases.	Pass	Scenario aligns with real-world system failures.

Step 4	Step completion criteria are preserved.	Pass	The output predictions preserve step completion criteria.
--------	---	------	---

5.10.4 Define Summary and Next Steps

5.10.4.1 Summary of Findings

- Summarize the key takeaways from the SECoT validation.
- Highlight whether the SECoT successfully identified emergent scenarios and preserved general systems rules.

Example: The SECoT validation confirmed that predictive thinking pipelines effectively identify unforeseen scenarios in autonomous drone operations, revealing three unaccounted failure cases.

5.10.4.2 Recommended Actions

Outline the next steps for refining the SECoT or integrating it into system design.

Example:

- Incorporate SECoT predictions into risk mitigation planning.
- Expand SECoT pipelines to cover additional interaction types.
- Validate SECoT predictions through simulation or real-world testing.

5.10.5 Appendices (Optional)

Include supporting materials such as:

- Diagrams of the SECoT predictive steps.
- Simulation results validating SECoT predictions.
- Additional examples of general systems rules in use.

5.10.6 SECoT Validation Review Process

- Validation reports must follow a peer review process.
- The process starts with the naming of the architect who is responsible for generating the report.

- It must also include signatures of at least one more reviewer to affirm the validity of the process.

Chapter 6 Unsafe Train Tracks Part A

Chapter 6 presents a case study on the Unsafe Train Tracks Problem, utilising the AIC Systems Approach to systematically address uncertainty, risk, and autonomous safety challenges in railway. The study applies a predictive thinking framework to model and mitigate operational hazards, particularly in autonomous perception systems. The analysis is structured into six Stages, starting with the articulation of uncertainty problems and operational environment modelling, defining complexity, resolving complications, and forecasting emergent behaviours.

It progresses to defining architectural intent, aligning autonomous system needs with safety objectives, followed by a HazTOPS analysis to derive structured safety requirements. The study then examines disordered AIC-driven black swan scenarios, applying AIC-matrix analysis, perspective shifts, and emergent scenario predictions to model high-risk events. Machine learning-based perception training and validation (CuneiForm Strategy Development) further refine AI models for track safety monitoring. Finally, dataset generation and validation experiments compare Black Swan-enhanced and Black Swan-lacked models, assessing AI resilience under unpredictable conditions. Key findings emphasise the necessity of robust AI safety strategies and propose recommendations for enhancing railway perception systems.

6.1 Introduction

It is important that the reader consider this chapter as a “front-end” description of what we did in Appendix H and J (detailed application of methods). Appendix H captures the detailed SECoTs and full tables, which we briefly describe in the Sections of this chapter with limited examples.

In previous chapters, we highlighted the motivation and described the system's scientific grounding behind our approach. In this chapter, we focus on applying the ideas and concepts of AIC systems approach. The core challenge we are trying to address in our approach can be described in two dimensions:

- Addressing the problem of predicting harder-to-predict unforeseen emergent scenarios (Black Swans) related to unexpected emergent behaviour of the Eagle Robot system (as described in Section 1.8.1), autonomous systems in harder-to-predict Complicated operational environments (unsafe train tracks zone in residential area, Holytown, Scotland).
- Ascertaining those expected Eagle Robot's emergent behaviours fulfil requirements (especially safety ones) when autonomous systems interact with their environment in any possible emergent scenario that may or may not have been thought of during design and training time.

Chapter 6

Both dimensions are attempted to be addressed in our approach. Our solution for handling complicated problems is to enhance the architect's predictive cognition through systemic (utilising general systems rules) and systematic thinking approaches. The second concern is that we propose justification-based traceability of how engineering judgement is developed by demonstrating the thinking and intuition employed when artefacts are created by applying SECoT processes. The ultimate purpose is to systematically enhance the perception system development dataset coverage for potential scenarios and demonstrate that we have traced the multitude of images to requirements using a set of CuneiForms. In Stage 6 we examine multiple options of the type of training and testing of models, from a set of out-of-context (meaning no association to any CuneiForm), in-context typical operations (meaning there is at least a partial specification using CuneiForms). Finally, we examined the impact of Black Swans on the performance of the output model.

6.1.1 A Thought Experiment

Before we start the process, we would like to prompt your predictive thinking about the scenario. We used this test in Chapter 3 to solve this challenge by examining HAZOP, STAMP and FRAM. We discovered that their fundamental focus on the notion of “a system boundary is what I deem to be my concern, and I am concerned about control interactions within that boundary” had limited our foresight to arrive naturally at an objective conclusion.

We have a particular reason for doing so, which will be revealed in later Sections. Let us take you back to the case study we described in Chapter 1. Imagine a situation like in the image below:



Figure 6.1 A snapshot taken from DuckDuckGo web browser maps of a strip of train tracks in Holy Town, Scotland, UK

The train tracks have been infiltrated by drones on multiple occasions, especially when trains were passing by.

Now evaluate the following questionnaire, and write up an answer (by simply looking at the image and speculating) to the following questions:

4. Is the train tracks problem domain completely defenceless against adversarial drones, with no effective natural defence mechanism already in place in the existing structure of the train tracks zone?
5. Is there a concerning risk that train tracks (as they are) pose to the surrounding vegetation, which impacts the safety of the train track zone?
 - a. Why should we discuss the interaction between train tracks and vegetation instead of addressing the adversarial drone? If the primary problem is the adversarial drone, why are we concerned about train tracks and vegetation?

As you explore the systematic process for analysing these issues, consider your initial thoughts on the vulnerabilities and interplay within the domain, and your prior beliefs, and examine how the process updates your prior beliefs about the problem and the system of interest. Comparing these to structured findings will illuminate gaps in perception and reinforce the value of comprehensive systems thinking in Complicated operational environments.

Our initial impression of what the problem is about can be summarised in three main factors:

- Adversarial drone.
- Train.
- Train tracks infrastructure.

The train tracks are open and exposed to adversarial attacks, necessitating an effective prevention mechanism. Usually, to develop an autonomous drone like the Eagle Drone, we would gather as many pictures as possible that capture flying drones. Then we would train a model to recognise them, triggering a chain of behaviours to capture the adversarial drone. However, in Stage 6 we discovered that if we solely relied on out-of-context images of drones to train our model, they do not perform well during Black Swan operations, as demonstrated by Black Swan testing.

We will follow AIC systems approach process and assess whether our current perception of the problem is sufficient to begin deriving system-level requirements based on this level of understanding. You can check our findings in Chapter 9, Section 9.2.1.

6.2 Stage 1: Uncertainty Problem Articulation and Operational Environment Modelling²⁶

In this process, we will apply the SECoT_1 process. The input to this Stage is an initial description of the problem. The problem is (or should be) considered a Confusing Complex at this Stage. This

²⁶ See Section H.4 in appendix H.

means whatever certainty stakeholders may have in their initial perception of the problem is deemed incomplete or yet to be validated. An accurate, acceptable view of the problem has yet to be synthesised. At this Stage, we need to validate whether the perceived problem is the only right problem to solve or if there are any hidden, more concerning problems to consider. To do so, we need to re-write the initial problem brief in a way that shows the qualities of a Confusing Complex. This is an essential intellectual step to stimulate ArcPT's (Architect Predictive Thinking) solving potential:

- The complex of involved systems behaving or allowing behaviour that contradicts expected (by ArcPM) purposes adds to the confusion of the whole problem.
- The organisational dynamics in the train track zone is not behaving as a cooperative system. There are undesirable conflicts among the parts.
- The causal complicatedness of the problem complexity is not clearly and explicitly specified.

In Chapter 1, the initial problem description from the customer was summarised to focus on the high-level confusing aspects of the problem without delving into details that may not be directly related. The goal is to abstract the problem brief to include only what is observed on the ground and to avoid adding speculative potential factors at this Stage. This approach aims to identify any hidden factors through a systematic reasoning process. For instance, if the problem involves an intruder drone, human trespassers, a passing train, and a train track zone, these are the main visible factors of concern that should not be doubted or speculated upon. Introducing other invisible potential factors at this Stage, such as animals or the train driver, would be based on assumption rather than clear reasoning. Our approach aims to help the architect arrive at these assumptions through comprehensive analysis and reasoning. Given the above, we identify the following description of our problematic, Confusing Complex as the following:

“An anonymous Drone is intruding into a bounded train track zone while the train is passing by.”

The predictive thinking pipelines in Appendix H enable the architect to achieve the following activities:

6.2.1 Predictive Thinking Pipeline 1: Appreciate the Complex of the Problem Complexity Field

The architect begins by listing behaviours that contradict the expected order of the environment, leading to either confusion or direct risks. Each of the following is a predictive thought step in the activity SECoT. The output of each thought step is a prediction based on a set of general systems rules.

In the first step, we identified unsafe behaviours by analysing the initial problem definition to distinguish those unsafe problematic behaviours. Using the thought step, we can assert that the following are the primary unsafe problematic behaviours:

1. **Drone Hovering without Clear Purpose.**
2. **Drone Moving at Low Altitude near Active Tracks.**
3. **Fence Ineffectiveness and Perceived Openness to Drones.**
4. **Drone Disruption of Surveillance Systems without Direct Contact.**

Once we understood, we generated a descriptive image that visualises the unsafe behaviour. The idea here is to create a depiction that aligns with our perspective of what the problem stereotypically looks like. In our case, we used DALL-E to generate an image of what we think a typical problem domain looks like. Utilising the thought step, we can assert that the following depiction faithfully captures the problem.



Figure 6.2 Problem Environment Depiction

Then we defined the complex of the complexity field: with the above articulation and the help of depiction, now we can start to predict the most obvious components of the problem. Using the thought step, we can assert that the following is the most immediate composition of the complexity field: {train, intelligent adversarial drone, train tracks zone fence, vegetation around the fence, electric power lines}.

In the next step (1.4), we attempted to group the apparent components into a potential system of systems in order to predict what PrimeP they may be serving. To be more objective about the behaviours of the components we have yet to expect, we need to determine the primary purpose of the central system of systems involved (supra-complexes). We use the term “complex” because we do not know precisely what they are. Utilising the thought step, we can assert that the following are systems of systems’ PrimePs:

- **Train Network:** {train, train tracks zone fence, vegetation around the fence, electric power lines}
- **Adversarial Scheme:** {adversarial drone}
- The adversarial drone is assumed to be part of a larger Adversarial Scheme targeting the Train Network.
- **Train Network PrimeP:** transports passengers and goods along the designated train tracks.
- **Adversarial Scheme PrimeP:** Disrupt Train Network operations.

6.2.2 Predictive Thinking Pipeline 2: Resolve the complicatedness pattern of the observed complexity

Predictive Thinking Pipeline 1 helped us to clearly, objectively and justifiably define the initial impression of the problem holistically. We exposed the overall structure of the problem domain complex. In this Thinking Pipeline, we will attempt to uncover the intricate interactions among the parts with each other. The first step in this process we systematically mapped out the interactions between components, defining how they influence or interact with one another. Using the Actions Matrix method, we identified and documented binary relationships across the elements previously identified as key parts of the problem domain, such as the train, adversarial drone, train tracks, fences, vegetation, and electric power lines (Table 6.1):

Table 6.1 Actions Matrix example

	Train	adversarial drone	train tracks
Train		approaches	drives over
adversarial drone	approaches		recognises and follow
train tracks	guide and stabilise	guides roaming	

using which we resolved the complicatedness into 42 interactions, for example:

- n1: adversarial drone approaches the train
- n7: train tracks guide roaming adversarial drone

Note: We utilised lateral thinking to solve the intricate description of the interactions, which we included a reference third-party observer [Lateral perspective with respect to ...]. To give an example of how we applied the Objective Lateral Predictive Thinking Process, we include only one interaction n26:

We could not predict the relevant interactions between the source “train” and the sink “electric powerline” so it may pose a threat, hazard or opportunity for an adversarial

drone or our potential solution drone. Thus, we needed to evoke a lateral thinking process to help us imagine what could happen. We eventually arrived at the following prediction:

n26: train sparks and wobble electric powerline wires [Lateral perspective concerning train pantograph connection]

To help us resolve the interaction between train and powerline and reach n26 conclusion, we needed to consult the lateral thinking process in Section 5.5 (the full application of this process can be found in Section H.4.2.1). By systematically applying the three-step thought process, we uncovered hidden interactions between the train's pantograph and the electric powerline, leading to the new prediction (n26).

1. Broadening Perspective:

- Challenged the assumption that powerlines are purely static energy providers by considering dynamic mechanical and electrical interactions.

2. Generating Analogies:

- Compared train-powerline interactions to guitar string vibrations and tire-road contact loss, refining our understanding of oscillatory effects.

3. Formulating Hypotheses and Scenarios:

- Identified that pantograph-induced oscillations propagate along the power line, potentially interfering with perception systems and affecting other infrastructure elements.
- Developed prediction (n26), establishing an unexpected but safety-relevant interaction.

This systems-thinking approach helps make sense of harder-to-imagine interactions are accounting for them.

With an output of 42 interactions, we then classified them into two problematic interaction categories:

- Unsafe problematic activities
- Beneficial or non-problematic activities

For example:

Table 6.2 Problematic interactions classification

Unsafe problematic activities	Beneficial or non-problematic activities
1. Train:	2. Adversarial Drone:
	2.5. Adversarial drone collides with vegetation around the fence [lateral

Chapter 6

1.1. A train's unsafe approach to an perspective, for the point of view of inherent adversarial drone may lead to striking the train defence]

[derived from n6].

1.4. The train induces random vegetation motion around the fence, which may impede any trained computer vision agent's perception capability [derived from n21].

2. Adversarial Drone:

2.1. Unsafe intended adversarial drone approach train, which may lead to striking the train [derived from n1].

2.2. Adversarial drones follow train tracks, which may lead to striking the train [derived from n12].

The process in Table 6.2 helped us scrutinise our list of priority problems and examine whether there are beneficial aspects we could capitalise on. With such contextualisation of the problem, we can now decide what to solve and drop.

Now we classify interactions based on those we chose to solve and those we chose to drop. Problem selection process encourages the architect to consult with stakeholders on what problems to solve and what to drop. For example:

Table 6.3 Design problem selection process

Interaction	Potential Concern	Decision	Elaboration or Justification
n1 (Drone 2.1)	An adversarial drone intentionally approaches the train, risking collision.	To be solved	A primary safety hazard. The design must detect/prevent drones from dangerously encroaching upon the train's path.
n22 (C) (Drone 2.7)	An adversarial drone mildly contact electric powerlines, leading to train delays (instead of a crash).	To be dropped	Though undesirable, it is a lesser impact scenario (delay vs. collision). The design scope primarily focuses on preventing catastrophic collisions that endanger train safety.

This Stage lays the foundation for risk mitigation by articulating the uncertainty landscape and creating structured models that will inform system design and autonomous response planning.

6.2.3 Predictive Thinking Pipeline 3: Predict the Emergence of AIC Complexity Field for Detailed Operational Scenario Articulation

For every unsafe, problematic, or beneficial activity, and considering the source complex's Prime, visualise the emergence of AIC Complicated Behaviour using an AIC modelling schema. In this case, any factor modelled must be written with a specific situation in the format of {adjective_name}. For example, if you define train as a factor, a situation needs to be recalled with it, for instance {moving_train} or {roaming_adversarial_drone}. To do so, we need to define a conventional way to refer to any complex of interest. To model the AIC complexity field for Unsafe problematic activities, we used only the Forward-feed partial AIC-SECoT (Figure 6.3).

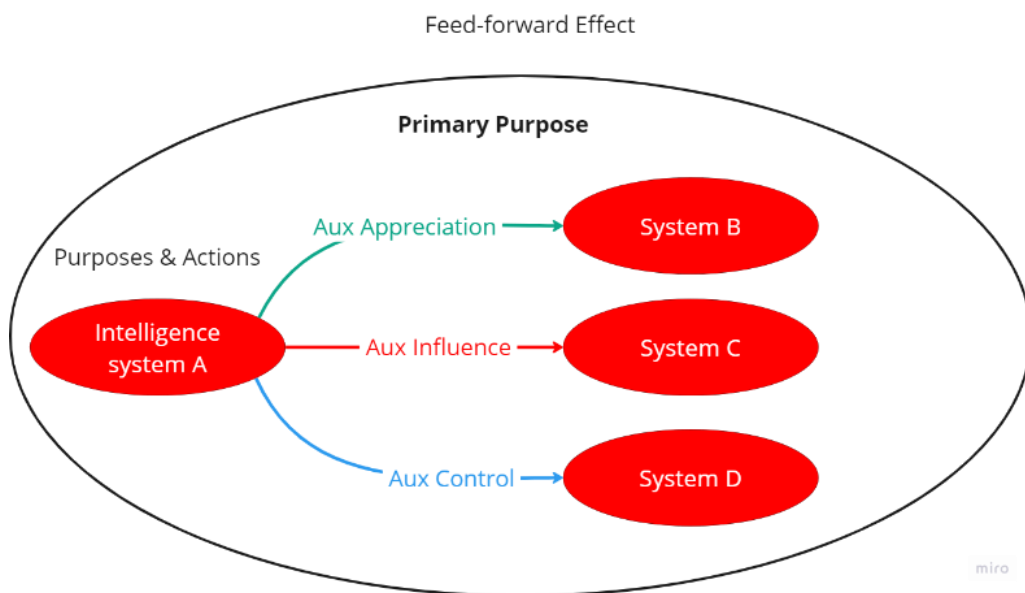


Figure 6.3 Forward-feed partial AIC-SECoT

For example, consider adversarial drone PrimeP: disrupt trains operations. Our case study will consider only the Forward-Feed partial AIC CoT. We will choose the unsafe approach of a Train to an adversarial drone, which may lead to striking the train [derived from n6].

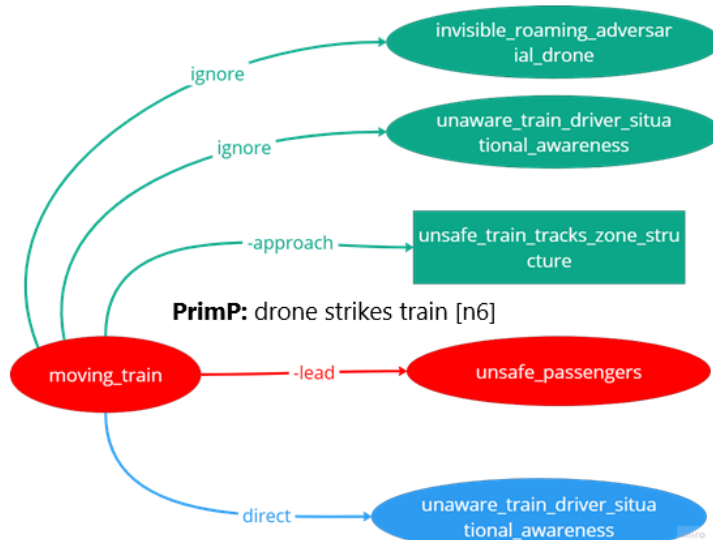


Figure 6.4 Modelling AIC scenario from interaction n1

Figure 6.4 considers a problematic PrimeP where the adversarial drone is tasked to strike a passing train²⁷.

As we proceeded to Step 3.2, we aimed to predict an extended list of emergent AIC interaction scenarios. This step involved utilising a structured interaction format, $\{ \{ \text{situation} \}_ { [\text{action}]_ { \{ \text{situation} \} } } \}$ ²⁸, which enabled a systematic capture of the dynamics involved in each interaction as visualised in our earlier modelling. More detailed capture of this process can be found in Table J.7 in Appendix J, examining multiple inputs and identifying emergent behaviours through collective interactions. The following is an example of artefacts that capture knowledge produced in image 6.4:

Table 6.4 AIC extended scenario for n1 Interaction SECoT definition

Step 1: Unsafe problematic activities	Unsafe approach of Train to an adversarial drone. Which may lead to striking the train [derived from n6].	
Step 2: Observed System (obs)	Step 3: Observed Action	Step 4: supra source Primary Purpose
roaming_adversarial_drone, train_derailment	Roaming adversarial drone approach train	Adversarial Scheme PrimeP: Disrupt Train Network operations.
Step 5: Auxiliary Influence Interaction	Step 6: Auxiliary Control Interaction	Step 7: Auxiliary Appreciation Interaction
$\{ \{ \text{moving_train} \}_ { [- \text{lead}]_ { \{ \text{unsafe_unsecured_passengers_and_goods} \} } } \}$	$\{ \{ \text{moving_train} \}_ { [+ \text{direct}]_ { \{ \text{unaware_train_driver_situational_awareness} \} } } \}$	$\{ \{ \text{moving_train} \}_ { [\text{approach}]_ { \{ \text{unsafe_train_tracks_zone_structure} \} } } \}$ $\{ \{ \text{moving_train} \}_ { [\text{ignore}]_ { \{ \text{unaware_train_driver_situational_awareness} \} } } \}$ $\{ \{ \text{moving_train} \}_ { [\text{ignore}]_ { \{ \text{roaming_adversarial_drone} \} } } \}$
Step 8: Predicted Problem Domain Factors or Features (with repetition)		
Appreciation = [Train_Network, moving_train, unsafe_train_tracks_zone_structure, train_transit_through_track_zone] Influence = [moving_train, unsafe_unsecured_passengers_and_goods] Control = [moving_train, unaware_train_driver_situational_awareness]		

²⁷ For more detailed examples, see Section J.6 in Appendix J

²⁸ See Section

Table 6.4 outlines the interaction between a train and an adversarial drone on a constrained track. This SECoT process identifies unsafe situations (Step 1), particularly the train's risky approach to the drone, which threatens derailment. The n6 interaction highlights the drone's potential disruption during transit.

The observed system (obs) (Step 2) includes key elements like the drone and derailment risks. Step 3 maps actions, detailing the drone's approach and operational disruption. The primary purpose (Step 4), known as the Adversarial Scheme Prime Purpose (PrimeP), aims to interfere with train operations, classifying the drone as a deliberate actor.

Steps 5- 7 analyse interactions across three dimensions: auxiliary influence, control, and appreciation. Influence interactions reveal how the train creates unsafe conditions. Control interactions show the driver's lack of situational awareness, heightening vulnerability. Appreciation interactions evaluate the train's approach to danger zones, complicated by the drone's concealed presence. Overlapping ignorance of threats and lack of awareness signifies critical system failure.

Finally, predicted problem domain factors (Step 8) consolidate elements into three dimensions of systemic engagement: appreciation touches on structural and operational factors like unsafe zones; influence addresses unsecured passengers; and control examines the driver's deficiencies. These insights establish a framework for understanding risks and designing interventions in train-track dynamics.

6.2.4 Predictive Thinking Pipeline 4: Predict and Evaluate Problem Domain Factors and Assumptions.

Evaluate AIC factors and list them without repetition from the most frequent factor to the least frequent. We have predicted 94 factors involved in the problem domain. For example,

1. roaming_adversarial_drone
2. moving_train
3. train_tracks_structure

Those factors are the most influential factors on our perception of the problem. However, they are predictable factors; we want to discover those heavy-tail Black Swan factors. Not all low-architect's attentive mentions factors are Black Swans, but a proportion of them at least are.

Evaluation architect's attentive mentions of factors

The fascinating finding in this analysis is that the architect predictive attention distribution of the factors followed a power-law distribution, which is an expected trend for Black Swan events and

heavy tail distributions. This trend line created an easily fitted curve, a power-law distribution curve with 88% fitness. This also indicates that the analysis produced a reasonable result, as the output predicted a reasonable set of factors corresponding to an anticipated observed statistical distribution.

Table 6.5A Sample of Predicted Factors Output²⁹

Factor label	Predicted Factor	Architect's attentive mentions
1	roaming_adversarial_drone	65
2	moving_train	58
3	train_tracks_structure	10
4	any_trained_computer_vision_agent_perception_capability	8
5	train_tracks_zone_fence_structure	8

The table shows the top 5 factors with the architect paid attention to more than 8 times. These factors are rather obvious for the architect. However, 89 factors have been cited 7 times or fewer. All of which were the hidden black swan events of the problem domain. In this step, evaluate each factor's repetition architect's attentive mentions and calculate the influence level. The term "roaming_adversarial_drone" has been cited 65 times out of 328 factors (including repetitions). This means the influence level of "moving_train" is roughly 18% of the total influence, calculated as 65 divided by 328. Similar to moving_train, which has a 17% influence on the architect's perception. This indicates that our overall perception of the problem pays much more bias towards roaming_adversarial_drone, and the bias has blinded our perspective to notice other hidden factors which the analysis has helped to reveal. Thus, it appears to be the most important factor in the problem domain is not whether an adversarial drone is accessing the train track zone, but whether the train is in motion or not.

In Figure 6.5, the architect predictive attention distribution curve shows an interesting fit to an exponential observed statistical distribution curve. Upon applying the skewness statistical test (using Excel function SKEW()), the results returned a value of 6.28, which indicates right-skewed data, confirming that our predicted complexity is indeed a long-tailed complexity field. Such a realisation does not tell us whether our prediction is right or wrong. Still, it consolidates our assumption that the complexity of the real world is a long-tailed distribution of event probabilities. And that our perception of it is skewed.

The long tail represents some of the black swan events that were hiding under the simplicity of the problem domain description, demonstrating that our approach has assisted us in predicting black swan events and thus reduced our residual ignorance.

²⁹ For the full table see Table H.13A, Section H.4.4.1

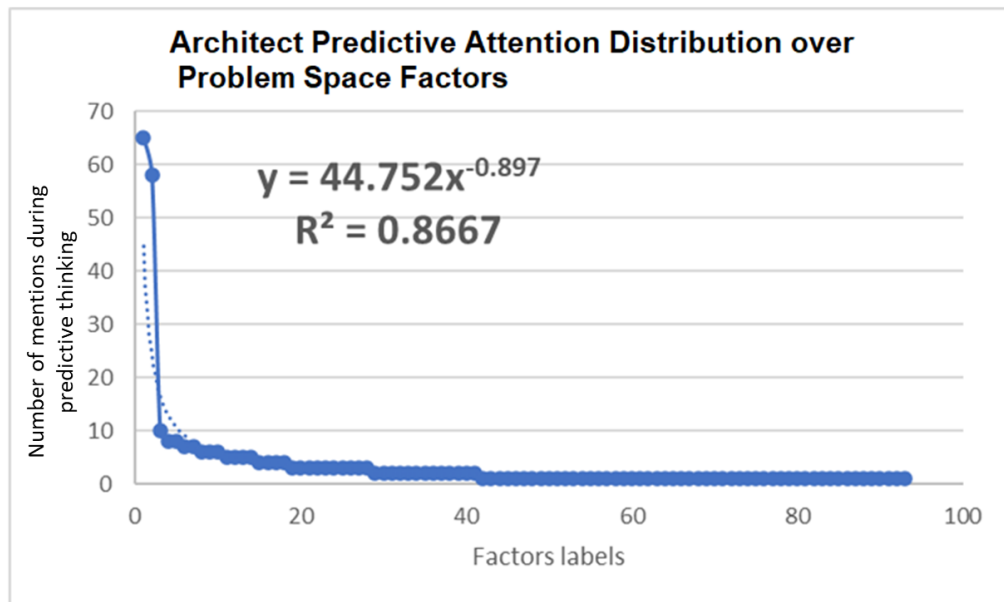


Figure 6.5 Architect predictive attention distribution curve of the derived factors for unsafe train tracks problem domain

After acquiring the factors, we defined each to clarify our meaning. In this process, the architect presents the findings to stakeholders to confirm accurate definitions. The following is an example of factors that only occurred once, which indicates factors that we are not paying too much attention to³⁰:

- **leaves_&_pollen:** falling leaves and pollen carried in the air, which impacts the reliability of autonomous systems.
- **open_fence_to_train_space:** the space between the fence and the moving train.
- **routes_around_maintenance_activities:** if maintenance engineers are present, the route between their activities can be open for adversarial drones to get access to.

We then identify the assumptions about all factors. List all predicted emergent interactions without repetition to define the assumptions related to predicted factors. Then, describe the associated assumed situations, specifying which aspects of the factors are being assumed and which other aspects are not. Lateral Predictive Thinking Processes can be used to help with imagining those assumptions³¹. For example,

Interaction 1: {roaming_adversarial_drone}_[-recognise]_{powered_powerlines_cables_structure_visual_appearance}:

³⁰ See Table H.13B, Appendix H.

³¹ For the entire table, see Appendix J, Table J.8.

Chapter 6

It is assumed that roaming adversarial drones are trained to recognise powered powerline cables based on their visual appearance. The trainer may not have considered random sparks or the architect's attentive mentions of powerline vibrations (wabbling).

These factors were not part of the initial problem brief, but the process helped us identify them objectively. We define those factors as harder-to-predict relative to our perception of the problem at the beginning of the analysis.

The final step in this pipeline, we identify the Black Swans based on the following criteria:

- Given our knowledge and experience with traditional systems, is the factor rarely a concern for us such that if is, then it would be a surprise?
- Is the factor impactful on the ML model?

in Table H.13B, Appendix H, we colour coded the table. If cells are coloured green then the factor is a Black Swan with respect to the architect and the customer knowledge. The green factors are factors which are believed to be sources of surprise and potential Black Swan event may emerge by them or related to some aspect about them.

Table 6.5B Sample Black Swan analysis taken from table H.13B

Predicted Factor	A	B	C
roaming_adversarial_drone	20.00%	no	yes
moving_train	17.85%	no	yes
train_tracks_structure	3.08%	no	yes
any_trained_computer_vision_agent_perception_capability	2.46%	no	yes
train_tracks_zone_fence_structure	2.46%	no	yes
train_derailment	2.15%	no	yes
fallen_vegetation_over_train_tracks_structure	2.15%	yes	yes
powered_powerlines_cables_structure_visual_appearance	1.85%	yes	yes
vegetation_complexity_growth	1.85%	yes	yes

The columns symbols mean the following:

- A. Current Concern Level
- B. Rarely concerning
- C. Impactful on solution ML model detection reliability

Fallen leaves on the sides of the train tracks, is not a common concern for safety of a train system or even infrastructure, hence it is rarely concerning. Also, it is impactful on the reliability of the Eagle Drone because if it create some random visual effect it may impact the detection capability of the perception system. Therefore, we need to include images with leaves on the sides and over

the structure of train track zone. We counted 41 potential sources of Black Swan events within the problem space.

6.3 Stage 2: Architect Intent and Autonomous Solution Needs

Definition³²

Following the problem articulation, this Stage defines the architect's intent and the necessary autonomous capabilities to address the identified risks. The goal is to determine the autonomous system's objectives by aligning its actions with the operational problem domain in Stage 1. The architect prescribes solutions to selected problems, capturing their high-level intents and assessing customer needs. Such as deploying an Eagle Drone System to patrol train tracks and neutralise unauthorised drone threats. The architect then ensures that:

- The system can appreciate the train track security conditions.
- The system can influence unauthorised actors through deterrent actions.
- The system can control its operational area by mitigating unsafe behaviours.

This process begins with identifying primary input knowledge, which includes interaction definitions, AIC model schema, and definitions of key factors and assumptions. The following steps involve a series of systems thinking activities.

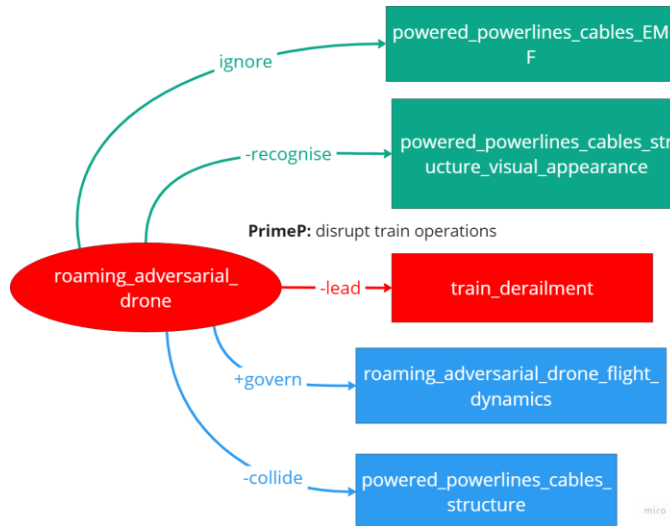
First, we perform a plausibility test to assess whether the assumption is reasonable. Then, we formulate a justification for this evaluation, answering the "Why?" behind our plausibility assessment. Next, we define the Architect's Intent, which clarifies the architect's strategy for addressing the situation, potentially engaging additional stakeholders for support. We also analyse the needs of autonomous solutions to determine their objectives in managing the situation effectively. Finally, we articulate the primary purpose of the overall solution, integrating the needs of both the autonomous systems and support systems, culminating in the formulation of the "Architect High-Level Solution Prescription". For example³³;

Table 6.6 Architect High-Level Solution Prescription

Situation	{roaming_adversarial_drone}_[-lead]_{train_derailment}.
	An assumption that roaming adversarial drones lead to train derailment.

³² See also Section H.5 for more detail.

³³ See Section H.5 for more examples and detailed description.



Plausibility
(Plausible/
Not
plausible)
Why?

Plausible

Given the potential for adversarial drones to deliberately interfere with train operations, such as obstructing the train's path, damaging critical infrastructure, or distracting operational systems, the scenario of train derailment is plausible under these conditions.

Architect
Intent
(mission)

The Whole Solution PrimeP is to ensure train network operations remain uninterrupted by adversarial drones by detecting and neutralising threats effectively. The architect aims to use a combination of autonomous systems and human interventions to prevent derailment and maintain safety.

Autonomous
solution
needs

- The autonomous system should monitor and detect drones approaching train tracks and assess their threat levels.
- Implement intelligent interception mechanisms to neutralise adversarial drones.
- Provide live surveillance data and alerts to ground control for real-time decision-making.

Autonomous
solution
constraints

- Autonomous drones must avoid physical collisions with trains or track infrastructure.
- Operations must be restricted to defined zones to avoid affecting neighbouring areas.

Support
Systems
Needs

- Ground control officers must receive real-time alerts and video feeds to make informed decisions.
- Transport police officers need accurate situational reports to coordinate appropriate on-ground responses.

Support Systems Constraints

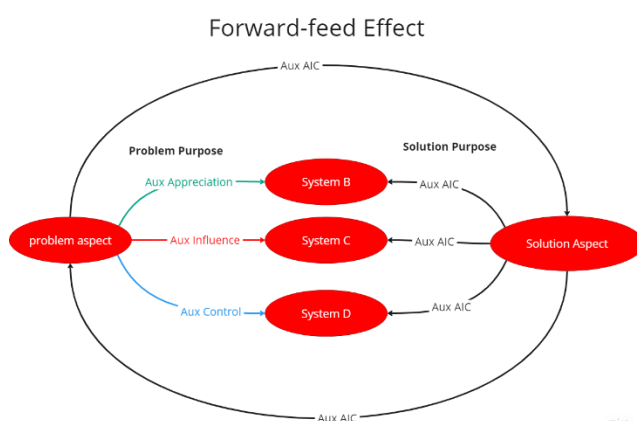
- Integration with existing railway monitoring systems for enhanced situational awareness.
- Ground control officers must have override capabilities to manage critical situations.
- Coordination protocols must be in place to prevent delays or errors in emergency response.
- Supporting systems must maintain compatibility with autonomous drones and other railway infrastructure systems.

6.4 Stage 3A: HazTOPs and Ordered AIC-Driven Autonomous System Requirements Development³⁴

In Stages 1 and 2, we rigorously articulated the problem, prioritising the complexity of the situation over the solution, and considering any hidden Black Swan factors. This Stage will analyse hazards, threats, and opportunities when integrating autonomous systems into the problem. It will also examine the autonomous decisions that need to be made. For every defined Architect Prescription identified in Stage 1, including the architect's intent, we model the chosen solution within the mix of interacting systems and re-evaluate the observed complexity. We need to model how autonomous systems handle every factor in the model.

6.4.1 Predictive Thinking Pipeline 1: Introducing Autonomous systems into Forward-Feed complexity³⁵

The first step is to take the AIC models provided by Architect Solution Prescription and the associated AIC model of the problem. We then introduce the solution into the model, mapping



the capabilities required to manage the problem domain, including any autonomous decisions to be made. We followed the following general systems rule (Figure 6.6) as an assumption of how systems in general would interact in a complexity:

³⁴ For more details see Section H.6

³⁵ More detailed description can be found in Appendix H, Section H.6.1

Figure 6.6 AIC general systems rule to model the engineered solution's complete reaction to the problem complexity.

In our case, we identified an appreciative interaction between the roaming adversarial and Eagle Drones. This means we intend to re-engineer the complexity such that the adversarial drone shall never have any influence or control over the Eagle Drone and will always be influenceable by the Eagle Drone. This also means that autonomous systems' interactions should facilitate such outcomes. The diagram stipulates that if the Eagle Drone:

- Recognises the visibility of vegetation appearance.
- Avoids crashing into vegetation structures.
- Physically inhibit the by-passing drone.
- It cannot effectively be influenced or controlled by the adversarial drone.

Then we can sufficiently trust that the Eagle Drone can inhibit the adversarial drone and prevent derailment of the train incident. See Figure 6.8, which describes the reaction between datasets and the problem:

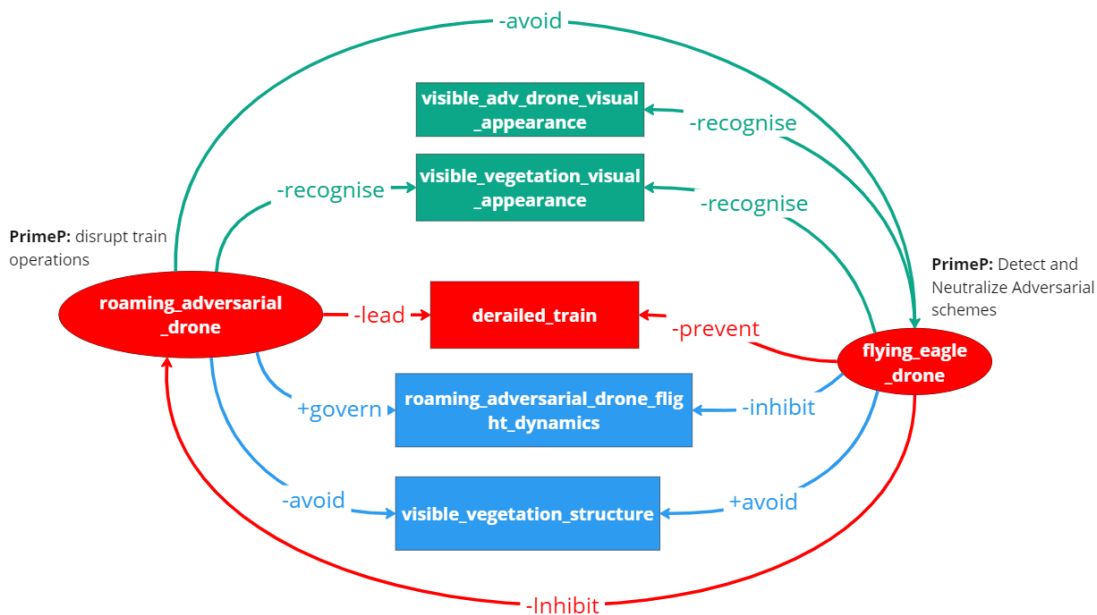


Figure 6.7 Introducing part of the solution into the mix of AIC problem complexity.

Once we have the complete problem-solution AIC model, we then defined the set of interactions between the sink and the source using AIC structured interaction format of: `{source situation}_{+,- or no sign, AIC-action}_{sink situation}`, Written in the following syntax: `{adjective+noun}_{verbal phrase}_{adjective+noun}` For example, `{flying_police_robot}_{learns humans' visual profiles}_{distressed_people}` Capture the output in the following table:

Table 6.7 Mapping AIC interactions of the Eagle Drone and adversarial drone behaviours in mitigating train derailment risks

Source: {eagle_drone}	
Output Behaviour	Input Behaviour that impacts the emergence of Output Behaviour
I1: {flying_eagle_drone} [+prevent]_ {derailed_train}	A1: {flying_eagle_drone} [-recognise]_ {visible_vegetation_visual_appearance}
	C1: {eagle_drone} [-inhibit]_ {roaming_adversarial_drone_flight_dynamics}
	C2: {flying_eagle_drone} [-avoid]_ {visible_vegetation_structure}

Table 6.7 delineates the AIC interactions between the Eagle Drone and the roaming adversarial drone within the complexity field. It highlights that for the Eagle Drone to achieve its influence in preventing train derailment, it must inhibit the adversarial drone and avoid collisions with vegetation while doing so. To ensure the control interactions, the Eagle Drone must recognise the visual appearance of vegetation around the track zone.

6.4.2 Predictive Thinking Pipeline 2: Designing the affecting Backward-Feed complexity field³⁶

With AIC Forward-feed defined, we have ourselves a complete complexity field. However, this complexity does not exist in isolation from the environment. The interactions between the components in an open, complicated environment have a ripple effect on other components' purposes and vice versa. Here, we extend the Forward-feed AIC model to include a Backwards-feed complexity field. However, to do so, we need some visualisations help to see the type of environment in which the AIC problem-solution exists. We place the model over the generated image we created in Stage 1 to give some appreciation of what the real world looks like.

³⁶ More detailed description can be found in Appendix H, Section H.6.2

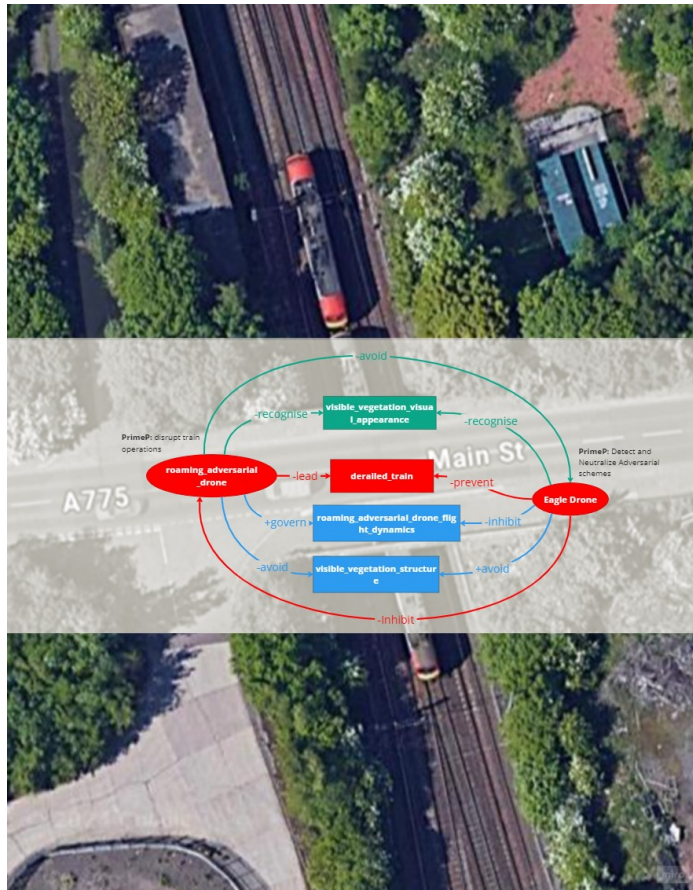


Figure 6.8 Eagle Robot operational environment

With the operational environment in sight, we extend the Forward-feed model with the factors within the operational domain complexity by identifying which complexes are identified in the real-world problem complexity. From Figure 6.8, we then produce the context diagram of the operational domain in Figure 6.9³⁷.

The model in Figure 6.9 is designed to assist the architect in analysing its immediate operational environment and identifying potential risks and areas of interest in real-time. The model breaks down the environment into nine generic regions surrounding the Eagle Drone, categorising factors and elements that impact the drone's decision-making and operational capabilities. These regions provide context for object detection, tracking, avoidance, and interaction, enabling safe navigation and effective response to dynamic environmental conditions.

³⁷ For the complete description of the model see Section H.6.2.

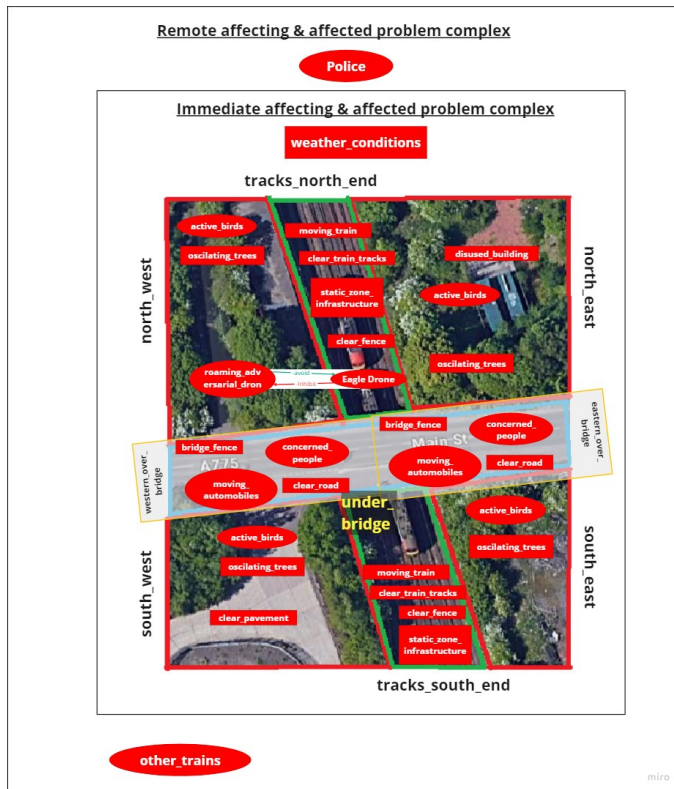


Figure 6.9 Unsafe Train Tracks Operational Domain context diagram

In step 2.2, we extended the Forward-Feed AIC model to include the Backwards-Feed Complicated Behaviours. In this step, we extend the forward-feed model with the factors within the operational domain by identifying which complexes are of concern in the real-world problem complexity. To do so, we need to abstract the Forward-Feed model for ease of modelling. We abstract the interaction model into the following definition³⁸:

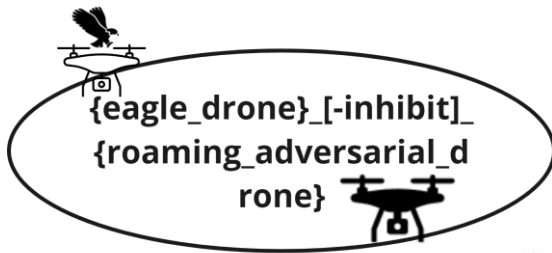


Figure 6.10 Abstract interaction for Eagle Drone denying adversarial drone access

Then, we model the Backwards-Feed AIC interactions. We do so by re-interpreting and tailoring the AIC-Core thought process described in Section 4.4.11:

³⁸ Note we encourage the use of icons and keeping the icons consistent throughout the practice.

Then, we model the Backward-Feed AIC interactions by asking the following questions³⁹:

1. What complex appreciates the interaction? **We answer:**

- Cars driving over the bridge can not influence or control the interaction.

2. What complex influences the interaction? **We answer:**

- The visual dynamics of objects around the train tracks impact both Eagle Drone and Adversarial Drone. Perhaps, oscillating movements of trees could present a visual challenge for perception systems.

3. What complex has control over the interaction? **We answer:**

- Police are in control of the interaction.

We will use icons to represent the complexes visually. Adding pictorial representations to the verbal description provides the designer with another dimension in visualising the scenario. Imagination is a key enabler for predictive thinking.

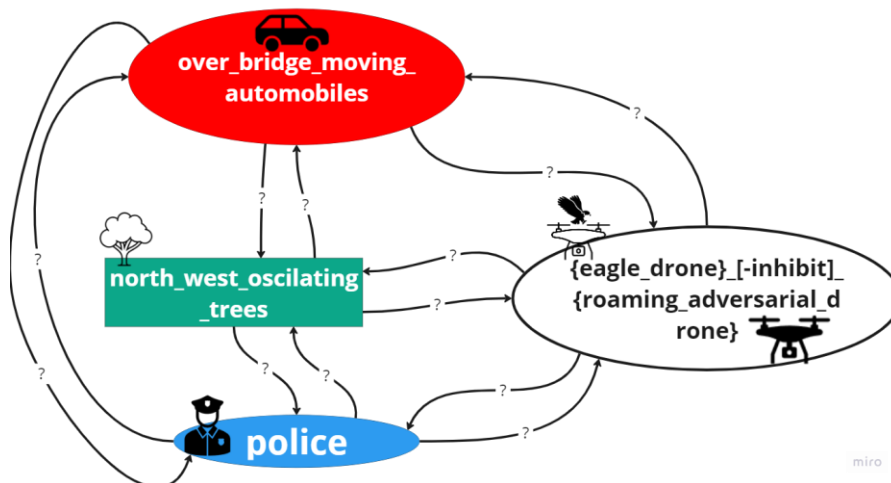


Figure 6.11 Backward-feed complexity field

Looking at the operational domain context, we can see that the trees will impact both drones. Additionally, we recognise that police involvement is necessary, as cars driving over the bridge could also be impacted by a potential dogfight between two drones. Figure 6.11 summarises the

³⁹ Note, in this scenario we followed a disordered form of AIC-Core thought process. Instead of staring with influence, we started with appreciation. The reason being is that we are more concerned about what could prevent the eagle drone from inhibiting the adversarial drone. Such attitude can be recognised as a safety-driven thinking because appreciation is the most vulnerable situation a complex or a system can be in.

Table 6.9 AIC interactions table for Eagle Drone in 6.12 the complexity field

Source or Sink: {eagle_drone}	
Output Behaviour	Input Behaviour that impacts the emergence of Output Behaviour
I1: {flying_eagle_drone} _ [+prevent]_ {derailed_train}	A1: {flying_eagle_drone} _ [-recognise]_ {visible_adv_drone_visual_appearance}
I2: {flying_eagle_drone} _ [-prevent]_ {roaming_adversarial_drone}	C1: {eagle_drone} _ [-inhibit]_ {roaming_adversarial_drone_flight_dynamics}
	A1: {flying_eagle_drone} _ [-recognise]_ {visible_adv_drone_visual_appearance}
	C1: {eagle_drone} _ [-inhibit]_ {roaming_adversarial_drone_flight_dynamics}

Let's take a slice of the complexity field and evaluate the complicatedness surrounding the Eagle Drone. The architect predicts that if an Eagle Drone is released into the train tracks, it will experience the complicated interactions with the environment as defined in Figure 6.13:

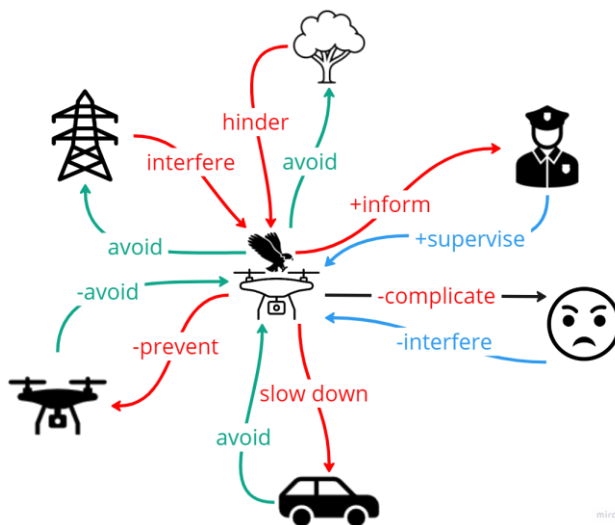


Figure 6.13 Complexity surrounding Eagle Drone

6.4.3 Predictive Thinking Pipeline 3: Hazards, Threats and Opportunities Scenarios (HazTOPS) Analysis⁴¹

We apply the HazTOPS SECoT outlined in Section [reference]. The following is the application of the process:

⁴¹ More detailed descriptions and examples can be found in Appendix H, Section H.6.3

6.4.3.1 Step 1) Scope the HazTOPS context domain

Scope the potential safety and security challenges on the AIC schema of the problem domain.

Add the following icon  for safety hazards,  for opportunities and  for potential security threats (cyber-attacks) to every interaction on the mode.

Note that the n interaction between *agitated_people* and *flying_eagle_drone* has changed from complicated to “threaten” this is because when we produced the HazTOPS for this interaction, we discovered that the complicated interaction is “threatening”.

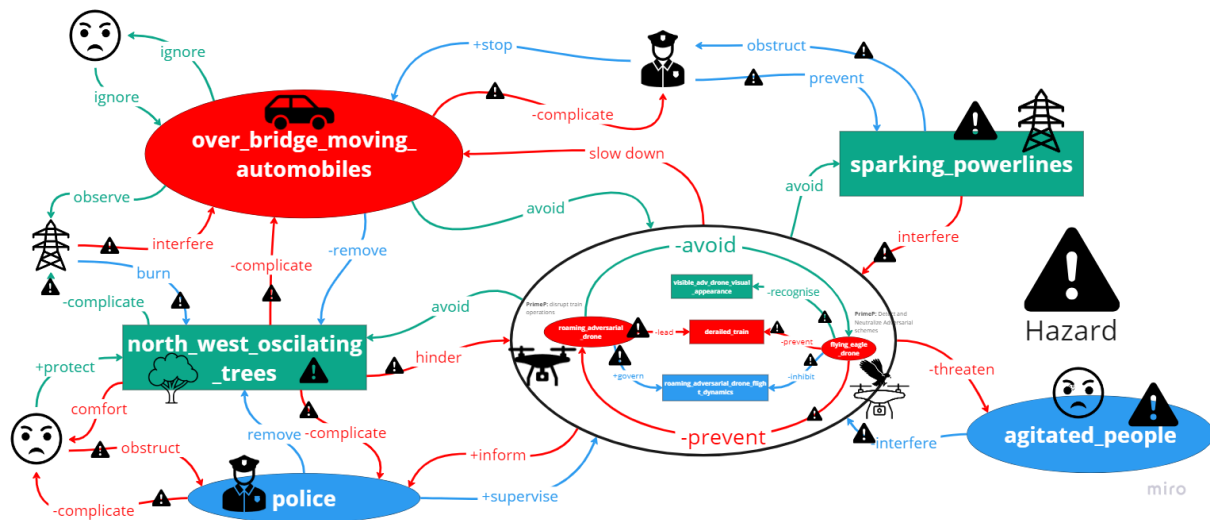


Figure 6.14 Hazards Complexity Field Scope: graphically scoping the hazards within the complexity field by placing hazard icons on target interaction.

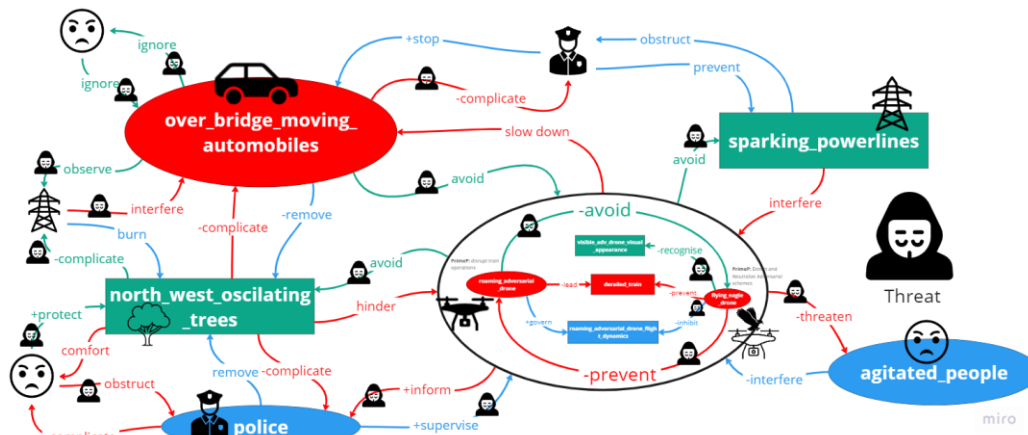


Figure 6.15 Threats, Complexity Field Scope

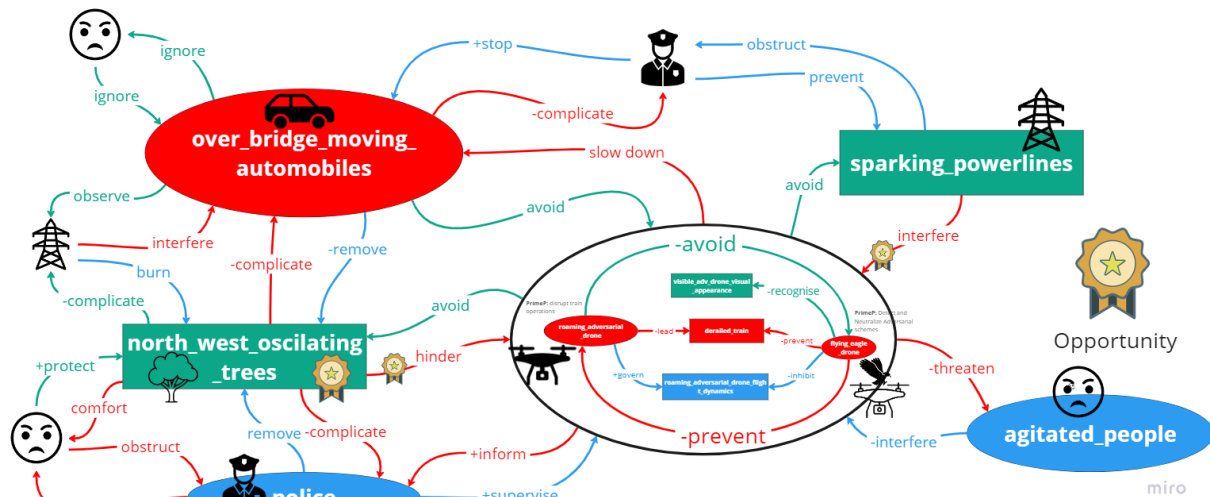


Figure 6.16 Opportunities Complexity Feels Scope

The opportunities identified in the Complexity field are the following:

- Trees can also hinder adversarial drone operations, thus providing natural deterrence. Knowing this, the police may decide to keep the trees rather than remove them or even increase their density, as it is a cheaper deterrent.
- Sparking powerlines can also interfere with adversarial drone operations.

Therefore, we can conclude that the train track zone is not totally defenceless. There are natural defences that can be leveraged and included in the overall strategy, thus reducing the overall solution's costs.

6.4.3.2 Step 2) Characterise the scoped interactions.

Roaming adversarial drone problem solution space complexity field: Clearly define the nature of the scoped interactions. We will use only the intelligent system's Forward-Feed model in this step. However, the process requires considering all interactions in the complexity field. Let's take the Eagle Drone sphere in Figure 6.17 and model the interactions:

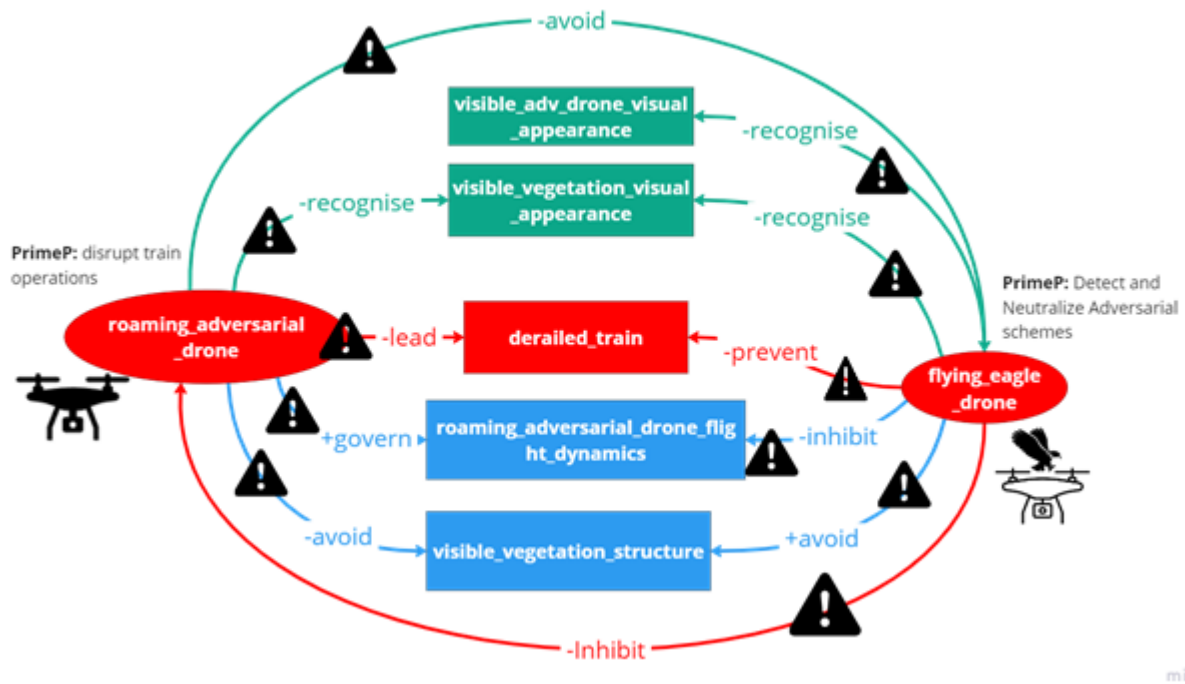


Figure 6.17 Hazards associated with Eagle Drone preventing derailed train complexity field

We capture the AIC interactions in Table 6.11. The table illustrates the behaviours and interactions within the AIC model in Figure 6.18, focusing on Eagle Drones as the source. It categorises specific behaviours (C Behaviours) of the Eagle Drone, such as inhibiting adversarial drones and their associated contributing (AC) behaviours. These include recognition of visual appearances, avoidance of environmental structures like vegetation, and flight dynamics inhibition. The table emphasises the intricate dependencies and hazard mitigation strategies within surveillance operations.

Table 6.10 The table describes the AIC interaction dynamics between Eagle Drones and adversarial drones

Source: {eagle_drone}	
Output Behaviour	Input Behaviour that impacts the emergence of Output Behaviour
I2: {{flying_eagle_drone}}[-inhibit]_{roaming_adversarial_drone}	A1: {{flying_eagle_drone}}[-recognise]_{visible_vegetation_visual_appearance}
	A3: {{flying_eagle_drone}}[-recognise]_{visible_adv_drone_visual_appearance}
	C1: {{eagle_drone}}[-inhibit]_{roaming_adversarial_drone_flight_dynamics}
	C2: {{flying_eagle_drone}}[-avoid]_{visible_vegetation_structure}

6.4.3.3 Step 3) Apply potential complications predictive guide words.

Then, we identify potential complexity by utilising the following modified keywords: More, Part of, Less, Early, and Late. We follow SECoT_2 to derive a variety of potential deviations.

Table 6.11 HazTOPS Analysis of "3 Drones Attack" Scenario with Risk and Surprise Assessment

Title	3 drones attack
AIC interaction	I2: {{flying_eagle_drone}_[-inhibit]_{roaming_adversarial_drone}}
HazTOPS Aspect	Definition
Hazards, Threats or Opportunities Scenario: Guide word	More: 3 adversarial drones involved.
Operating Scenario Context	Clear day, and the Eagle Drone is conducting a patrol.
Hazard, Threat or Opportunity definition (consider used systems)	More than 1 adversarial drone approach.
Foreseeable Sequence of Events	• The Eagle Drone is hovering over a designated 360-degree panoramic survey. • While facing the fence direction, three adversarial drones appear within 10 meters of the Eagle Drone. • The Eagle Drone correctly identify three adversarial drones. • It decides to contact HQ. • It launches pursuit and inhibits on one of the drones. • It crosses a boundary fence. • The aerial dog fight occurs within the track zone.
Potential harm or benefit	• Potential crash on track zone infrastructure. • Potential ingress over local houses' back gardens. • The other drones are freely roaming around train track zone.

Table 6.12 provides a deviation analysis of the "3 Drones Attack" scenario, focusing on risks, threats, and opportunities in a situation where an Eagle Drone must manage multiple adversarial drones. The AIC interaction highlights the Eagle Drone's objective to inhibit roaming adversarial drones, while the HazTOPS Aspect introduces the guideword "More", signifying an increased threat with three adversarial drones involved.

Soft Safety Hazard example:

While capturing the list of interactions, we re-considered some of the actions in the AIC model; for example, the model shows the police performing a “remove” control action with a neutral effect on the trees. That is because we thought the police may not have a direct intention to do so. This interaction may lead to social uproar and thus indirect impact on the safe deployment of the system. This is an example of a Soft Hazard, and we could identify it immediately. Ethical considerations can be demonstrated objectively as a design aspect in a safety case. The figure below shows the complexity field related to the emergent soft hazard. While they don’t directly impact the safety of the Eagle Drone, they affect the harmony of achieving the overall ideal whole.

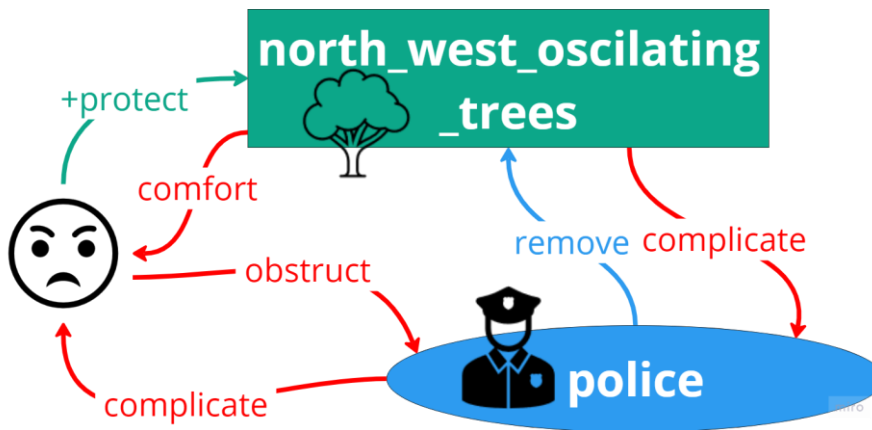


Figure 6.18 Soft Hazard Complexity Field Model for local people's reaction to police decisions, which could lead to an unsafe environment for people and the system

6.4.4 Predictive Thinking Pipeline 4: Elicitate ordered AIC System-Level Requirements and training requirements⁴²

In this step, we will model every derived HazTOPS and define mitigating system-level requirements. We performed the process on two examples. However, we are presenting one in this chapter:

Adversarial drone hiding behind the fence example

Let's take the following HazTOPS scenario:

Table 6.12 Adversarial Drone hiding behind fence HazTOPS

Title	Adversarial drone behind fence
AIC interaction	A3: {flying_eagle_drone} [-

⁴² More detailed description can be found in Appendix H, Section H.6

	recognise]_{ visible_adv_drone_visual_appearance}]
HazTOPS Aspect	Definition
Hazards, Threats or Opportunities Scenario: Guide word	Less: Adversarial drones are less visible
Operating Scenario Context	Operational environment difficulty grade A. Eagle Drone conducting a usual patrol. The area adjacent to the train track zone fence is not part of Eagle Drone training data.
Hazard, Threat or Opportunity definition (consider used-systems)	Adversarial drones hide behind train track fence
Foreseeable Sequence of Events	The adversarial drone cunningly lands on the floor behind the fence; the Eagle Drone does not notice its presence. A train passes, and the adversarial drone launches to strike the train; the Eagle Drone fails to act swiftly enough.
Potential harm or benefit	Train derailment

6.4.4.1 Step 1) Model the Complex of Interest Operating Scenario Context

We will start with an abstract relationship between adversarial drones and the Eagle Drone patrolling capability. Initially we considered the “detect” action as an appreciative interaction.

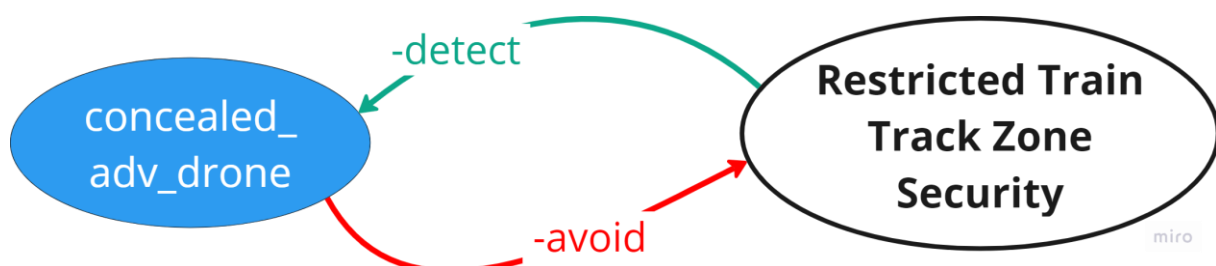


Figure 6.19 Problem context definition for concealed adversarial drone

Note that we chose the interaction between the Eagle Drone and the adv drone to be appreciative. This is because we assume the system can not influence or control a well-concealed adv drone.

6.4.4.2 Step 2) Model hazards mitigation ordered-AIC complexity field

Refine the model by introducing the hazard and mitigating solutions, then update the interactions table⁴³We resolve influence interactions into AC components, thus replacing the influence interaction “-avoid.” The refinement includes the possible AC interactions required for the adversarial drone to successfully avoid the Eagle Drone detection.

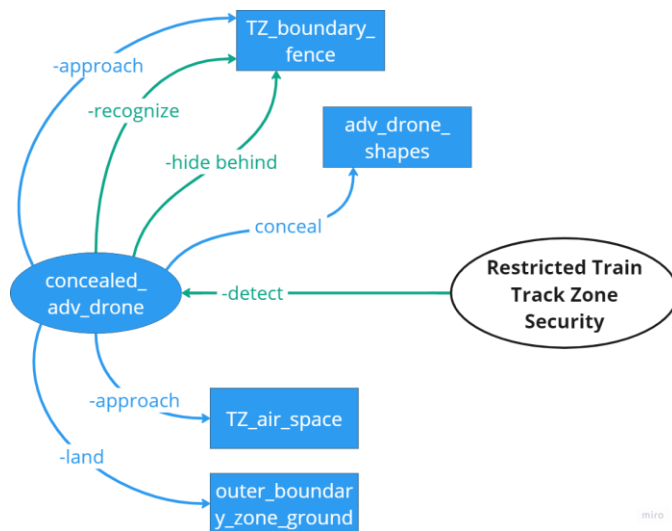


Figure 6.20 Refining the concealed adversarial drone problem context

Then, we refine it further by defining Eagle Drone patrolling capability to mitigate reactions.

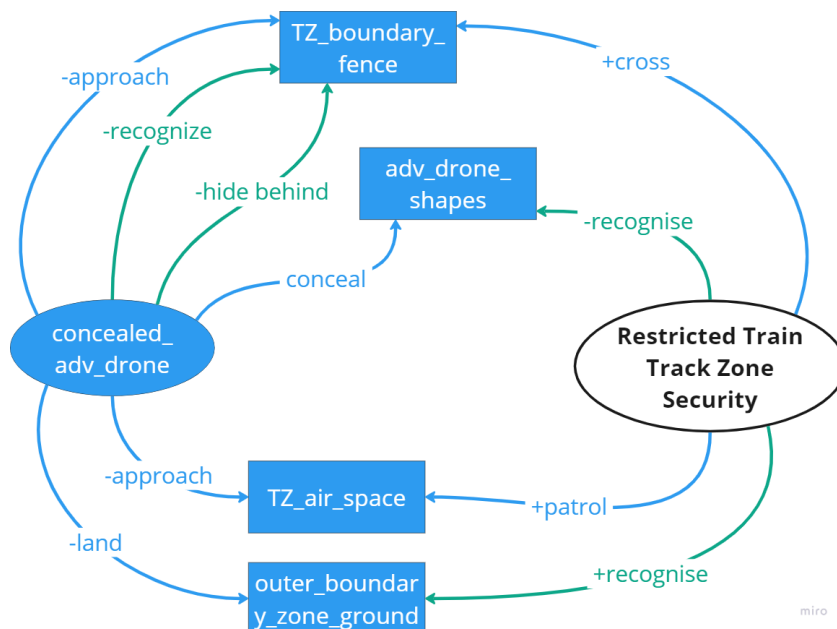


Figure 6.21 Mitigating the Concealed Drone problem scenario

⁴³ We use the AIC interactions table we developed in Section H.4.6.1

However, as we progressed to define the AIC interactions, we realised that the “-detect” interaction between the Eagle Drone and adversarial drone should not be an appreciative interaction but rather an influence interaction of “-prevent”. This is particularly true since the detection of adv_drone influences its behaviour and decision-making process. So we updated the initial interaction into an influence-influence design pattern:

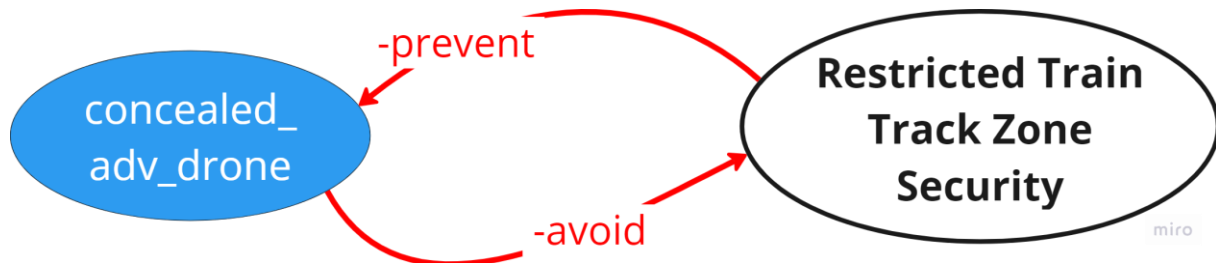


Figure 6.22 Updated abstract relationship

With the information discovered, we update the AIC interaction table. We will use one example in this table; the full table can be viewed in Section H.6.4.2:

Table 6.13 AIC interactions table

Agent Output Behaviour	Agent Input Behaviour that delivers Output Behaviour	Supportive systems AIC Behaviour
I4: {concealed_adv_drone}_[- avoid]_{flying_eagle_drone}	A13: {concealed_adv_drone}}_[- recognises]_{track_zone_ boundary_fence}	
	A14: {concealed_adv_drone}}_[-hide behind]_{track_zone_ boundary_fence}	
	C6: {concealed_adv_drone}}_ [+conceal]_{ adv_drone_shapes}	
	C7: {concealed_adv_drone}}_[- approach]_{track_zone_ boundary_fence}	
	C8: {concealed_adv_drone}}_[- approach]_{track_zone_air_space}	
	C9: {concealed_adv_drone}}_[- land]_{outer_boundary_zone_ground}	

6.4.4.3 Step 3) Ordered-AIC-based System or Safety Requirements Derivation

In this step, we convert the identified interactions into system requirements. We have comprehensively articulated the problem domain through extensive analysis and discussion by this point in the process. Below is an example of the requirements:

Table 6.14 Safety requirements derivations to mitigate the concealed drone problem.

AC interaction	Mitigating Safety or Systems Requirements (Safe Operating Concept)
A11: {flying_eagle_drone}_[+recognise]_ {adv_drone_shapes}	Safety Requirement 2: Eagle Drone recognises approaching adversarial drone shapes. Given: adversarial drone recognises track zone boundary fence, adversarial drone approaches track zone airspace, In order to: avoid Eagle Drone detection and intrusion prevention, Then: the Eagle Drone shall recognise approaching adversarial drone shapes, In order to: prevent adversarial drone incursion.
A12: {flying_eagle_drone}_[+recognise]_ {outer_boundary_zone_ground}	Safety Requirement 3: Eagle Drone recognises adversarial drones during approach and landing Given: adversarial drone landed on outer boundary zone ground, In order to: avoid Eagle Drone detection and intrusion prevention, Then: 3.1: The Eagle Drone shall recognise adversarial drone approach to the fence during landing. 3.2: The Eagle Drone shall recognise an adversarial drone landed on the outer ground zone. In order to: prevent adversarial drone incursion.

C5: {flying_eagle_drone}_[+cross]_
{ track_zone_boundary_fence}

A13: {monitoring_police}_[+
dictates]_{predefined_eagle_drone_
incursion_outside_boundary_fence}

Safety Requirement 4: Police offer sets predefined patrolling coordinates

Given: adversarial drone recognises track zone boundary fence, adversarial drone approaches track zone airspace, adversarial drone landing outside track zone fence, adversarial drone landed outside track zone fence, adversarial drone attempts to conceal shape,

In order to: avoid Eagle Drone detection and intrusion prevention,

Then:

4.1: The monitoring police officer shall set predefined distances across the fence for extended patrolling.

4.2: The Eagle Drone shall follow a predefined cross-boundary distance during patrol to monitor for any landing or about to land adversarial drones,

In order to: prevent adversarial drone incursion.

Table 6.14 outlines the safety requirements to mitigate the concealed adversarial drone problem through specific AIC interactions. The first requirement (Safety Req 2) mandates that the Eagle Drone recognise the shapes of adversarial drones approaching the track zone airspace to prevent intrusion. Safety Req 3 further specifies that the Eagle Drone must detect adversarial drones while landing in the outer boundary zone, ensuring comprehensive coverage of potential entry points. Lastly, Safety Req 4 focuses on extending the Eagle Drone's patrol capabilities beyond the track zone boundary fence, as dictated by the monitoring police controller, to detect adversarial drones attempting to conceal themselves or land just outside the boundary. These requirements collectively emphasise proactive recognition and extended surveillance to prevent adversarial drone incursions effectively.

6.4.4.4 Step 4) Extended Concrete Safe Operating Concept and ML Safety-Training Concept

In this example, we will take the following safety requirements:

Safety Requirements 3: Eagle Drone recognises adversarial drones during approach and landing.

Given: adversarial drone landed on outer boundary zone ground,

In order to: avoid Eagle Drone detection and intrusion prevention,

Then:

3.1: The Eagle Drone shall recognise adversarial drone approach to the fence during landing.

3.2: The Eagle Drone shall recognise an adversarial drone landed on the outer ground zone.

In order to: prevent adversarial drone incursion.

Note that requirements 3.1, 3.2 are related to the recognition of an adversarial drone. This is an ML training issue rather than a mechatronic issue. The architect is free to use 4WnH process to derive the ML training safety requirement. However, we will refrain from doing so in this example for variety.

Step 4.1) ML Safety-Training Requirement derivation (Training Concept):

Part of satisfying Safety Requirement 3 is training Eagle Drone to recognise adversarial drones approaching and landing on the ground. Thus, we need to specify a general training requirement for the ML model, which we refer to as the Training Concept. In this case, this is a Safety-Training requirement since it mitigates a safety hazard. A training concept refers to the training scenarios where we want to increase the machine's epistemic experience about the real-world we want it to operate in. Eventually, this concept turns into training, validation and testing datasets. Which then will turn into CuneiForm training syllabus, which includes training classes. Imagine you are a teacher, and the machine is your apprentice, and you are tasked to introduce this apprentice, who has 0 knowledge about the whole world, into the real-world, how would you train it? where do you even start? How would you prove that your knowledge is complete? And that you trained the machine to be safe and act safely?.

To define the training concept, we used the following structure:

ML Safety-Training Requirement n: [system of interest] ML component shall be trained to [training scenario].

In this case:

Table 6.15 ML Safety Requirements Derivation

Mitigating Safety or Systems Requirements (Safe Operating Concept)	ML Safety-Training Requirements (Training Concept)
3.1: The Eagle Drone shall recognise the adversarial drone approach to the zone boundary and the fence.	ML Safety-Training Requirement 2: The Eagle Drone's ML component shall be trained to recognise adversarial drones approaching the train tracks zone boundary, fence, and train and attempting to land.
3.2: The Eagle Drone shall recognise an adversarial drone landed on the outer ground zone.	ML Safety-Training Requirement 3: The Eagle Drone's ML component shall be trained to recognise an adversarial drone landed on the outer ground zone.

Step 4.2) ML dataset requirements derivation

Thus, a dataset training requirement can be derived from the training concept. We use the following structure to define requirements over actual datasets:

Dataset requirement structure:

The [system of interest] ML component [Training/Testing/Black Swan Validation] Dataset shall provide the trainee model with a valuable minimum variety⁴⁴ of..

In this case:

ML development datasets requirement n: The Eagle Drone ML component [Training/Testing/Black Swan Validation] Dataset shall provide the trainee model with a valuable minimum variety of..

Table 6.16 ML dataset requirements derivation

ML Safety-Training Requirements (Training Concept)	ML Safety-Training Dataset Requirement
---	---

⁴⁴ See Section 4.5.4.1 for the definition of Valuable Minimum Variety

ML Safety-Training Requirement 2: The Eagle Drone’s ML component shall be trained to recognise adversarial drones approaching the train tracks zone boundary, fence, and train and attempting to land.

ML Safety-Training Requirement 3: The Eagle Drone’s ML component shall be trained to recognise an adversarial drone landed on the outer ground zone.

ML development datasets requirement 2: The Eagle Drone’s ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drones (typical off-the-shelf drones) approaching the train, train tracks fence and attempting to land next to the train tracks boundary. **In order to:** train the drone’s machine learning algorithms to dynamically identify, track, and trace adversarial drones during typical operations.

ML development datasets requirement 3: The Eagle Drone’s ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drones (typical off-the-shelf drones) landed next to the train tracks boundary. **In order to:** train the drone’s machine learning algorithms to dynamically identify, track, and trace adversarial drones during typical operations.

6.5 Stage 3B: Comprehensive Operational Environment Definition⁴⁵

We reduced the comprehensive ODD classification into the following categories for this application and considered Grade A⁴⁶ Uncertainty. For a detailed description of how to define ODD, refer to Sections 4.7 and 4.8.

Appendix B defines the A-E types of uncertainty an ODD complexity can be in. For example, **Grade A uncertainty**. It specifies ideal natural conditions for perception, including sunny weather, minimal wind (<5 km/h), high visibility (>8 km), and clear skies. Additional parameters include a

⁴⁵ See also Section H.7

⁴⁶ See Appendix A and B for more details on ODD characteristics

temperature range of 15–25°C, 12 hours of sunshine, and no pollen, sand, or dust interference. The operation is limited to a single type of season, midday timing, and one specific landscape type with minimal infrastructure variation. The table ensures a clearly defined, controlled environment, essential for reliable system performance and accurate perception during deployment. In this case study we choose Grade D:

Table 6.17 Operational Design Definition for Eagle Robot Deployment in Train Track Zone

System of interest		Eagle Robot
Solution Operational Space		Train Track Zone in the UK
ODD Uncertainty Grade		Grade D
Development purpose		Training and Testing
Environment Characteristics		Less favourable natural environment system
Natural Lighting Conditions		Cloudy / Overcast
Weather	Precipitation mm/h	20-50
Conditions	Wind km/h	20 - 30
	Humidity %	0 – 20 & 70 - 90
	Visibility km	0.4 - 1
	Cloud Cover	Mostly Cloudy or Cloudy (5/8 to 7/8 oktas)
	Snow mm/12 hrs	150 – 250
	Pollen	Very high
	Sand	Sandstorm
	Temperature	To be defined by the architect
	Sunshine Duration	4 hours
Time of the Year: Seasons-specific environmental characteristics		3 types
Landscapes type variety definition		7 types
Geographical region-specific natural phenomena		7 types
Time of the Day		6 types
Perceived Horizon Attitude		5 types
Sun sphere positioning		3
Moon sphere positioning		3
Specialised zones features		3+ features

Table 6.17 defines the operational design domain (ODD) and environmental characteristics for deploying the Eagle Robot within the train track zone in the UK under **Grade D uncertainty**. We may choose different sets for ODD when developing testing datasets. However, we will keep the same ODD for generating the datasets in this application.

6.6 Stage 4: Disordered AIC-Driven Black Swan Scenarios

Prediction⁴⁷

In this Stage, you may consider disordered AIC timing in Section E4. Some of the output of this Stage is to generate Black Swan training and validation datasets for validating ML components to handle Black Swan scenarios.

In this Stage, you may consider disordered AIC timing (before or after). Consider Section 6.4 (AIC Timing). Some of the output of this Stage is to generate Black Swan validation datasets for validating ML components to handle Black Swan scenarios. However, nothing may stop the architect from dedicating some of the black swan scenarios to be part of the training and testing process of the ML component.

At first, we choose the complexity field we intend to perform the AIC perspective shift. For this we will select the complexity field in Figure H.31:

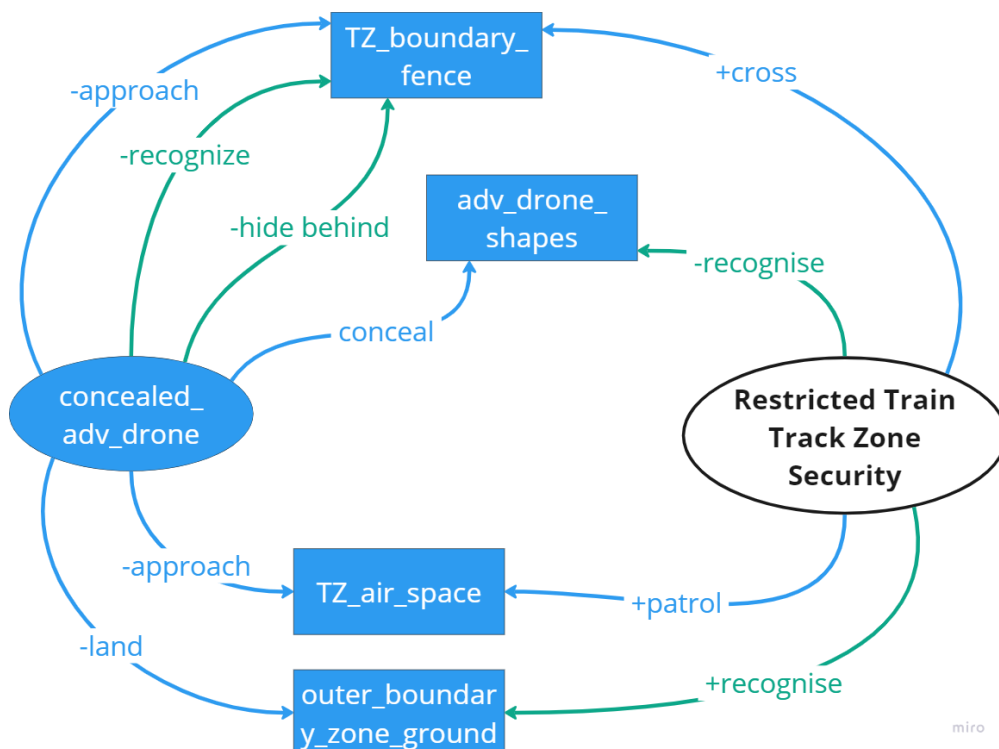


Figure 6.23 Example complexity field. TZ stands for track zone.

To implement the AIC perspective shift, we need to perform the following process:

6.6.1 Step 1) Define the interactions

Define the interactions needed to predict a potential emergence. We chose the following relationship:

⁴⁷ For more detail see Section H.8

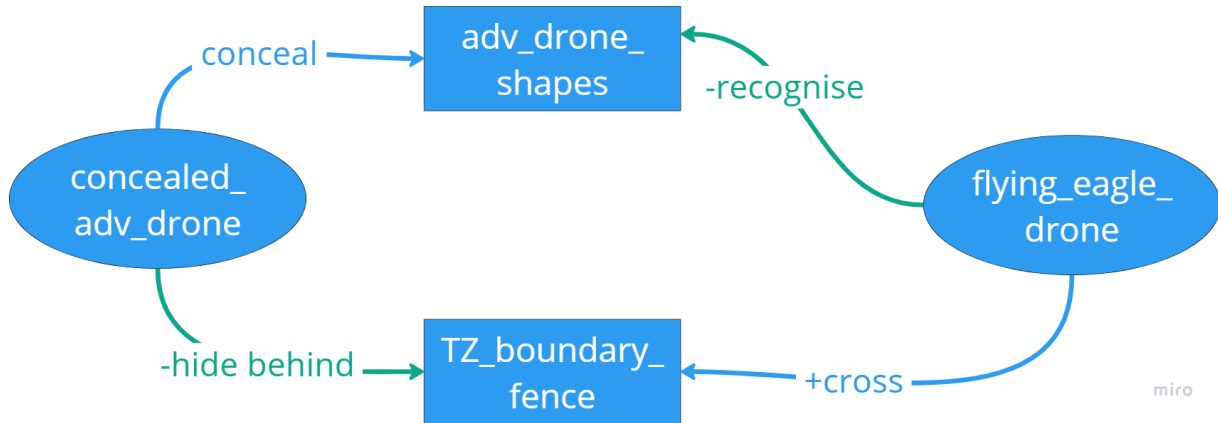


Figure 6.24 Example relationship of interest to perform the perspective shift

To increase the detailed context of the relationship, we need to extend interactions with the reactions between each node.

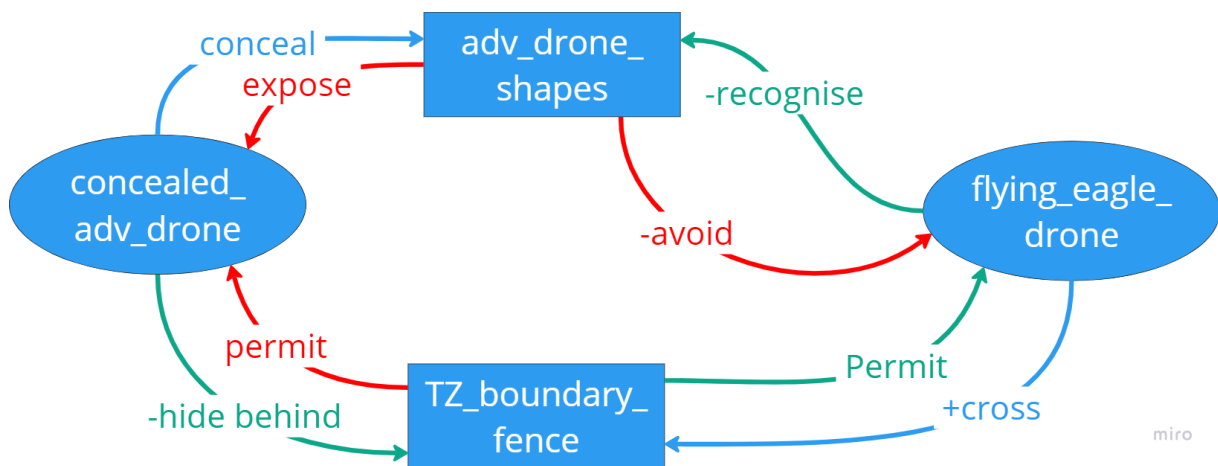


Figure 6.25 Extending the interactions with reactions among systems (binary relationships)

6.6.2 Step 2) Define the AIC-Matrix

Define the interaction's current AIC factors using the AIC-Matrix (Section 5.10). We choose the binary relationship between the Eagle Drone and the adversarial drone shapes.

Table 6.18 Deep AIC factorisation for Eagle Drone and adversarial drone shapes

	Flying_eagle_drone	Adv_drone_shapes
Flying_eagle_drone		Supra Source: Train Network. PrimeP: Safely transport people and goods. Goal: detect adversarial drone presence. Goal type: Appreciation. Action: recognise adversarial drone shape.

		Action type: Appreciation. Effect: Obstructive.
Adv_drone_shapes	Supra Source: Adversarial Scheme. PrimeP: Disrupt Train Network operations. Goal: avoid detection by Eagle Drone. Goal type: influence. Action: obscure Eagle Drone perception. Action type: Influence. Effect: Obstructive.	

6.6.3 Step 3) Perform the Perspective Shift

Perform the perspective shift using AIC perspective shift SECoT. In this step, we choose an appropriate perspective shift that we believe will be a potential black-swan event not foreseen during modelling the interaction between `adv_drone_shapes` and `flying_eagle_drone`. We will select the following shift:

1. AIC type shift.

For this, we will apply the following Thought Step from SECoT_3:

General Systems Rule: Given a source or sink, it is possible that over time and with the change of complexity, the AIC goals are altered, leading to a new situation of complicatedness.

Predictive question: What would happen if the interaction flow and effect type remained the same, but the goal's AIC type and action type were altered in the future?

Guiding Prompt: Review the AIC dynamics of the observed complexes and alter the nature of the goal and action. Then, define an appropriate action to bear alternative AIC types and describe a scenario in the shifted context.

Completion criteria: The step is considered complete when a scenario demonstrating an alternative AIC goal is detailed.

As per the following schema:

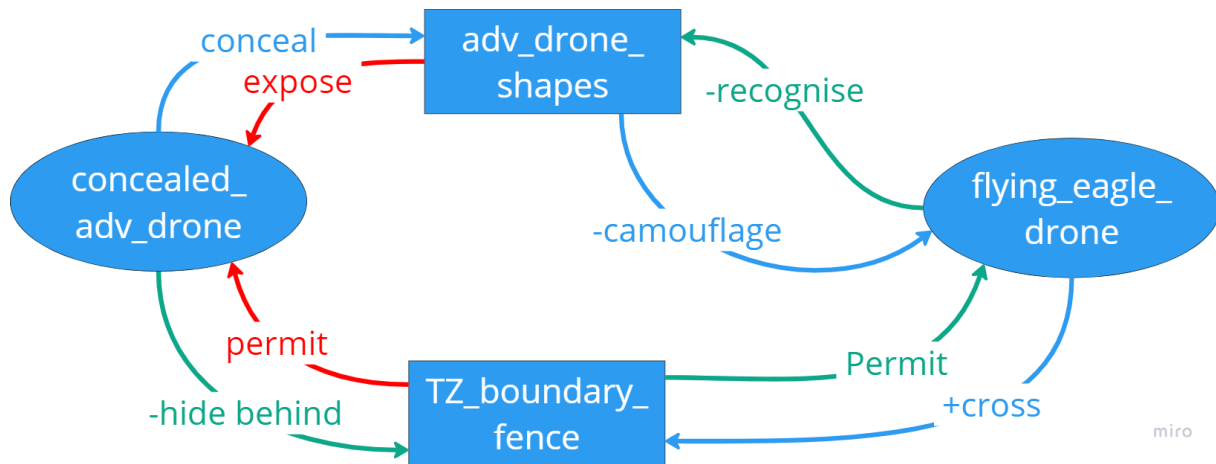


Figure 6.26 Shifted perspective from influence to control

The above steps can be captured in the following table:

Table 6.19 AIC-type perspective shift

Interaction	A11: {flying_eagle_drone}_[+recognise]_{adv_drone_shapes}	
AIC factors	Initial perspective	Shifted Perspective
Source	Adv_drone_shapes	Adv_drone_shapes
Sink	Flying_eagle_drone	Flying_eagle_drone
Supra Source	Adversarial Scheme.	Adversarial Scheme.
PrimeP	Disrupt Train Network operations.	Disrupt Train Network operations.
Source's Goal	obscure Eagle Drone perception.	Obscure Eagle Drone perception.
Source's Goal type	<i>Influence</i>	<i>Control</i>
Source's Action	<i>Avoid</i>	<i>Camouflage</i>
Source's Action type	<i>Influence</i>	<i>Control</i>
Source action effect on sink	Obstructive	Obstructive

The shifted scenario demonstrates an alternative AIC goal where the adversarial drone fully controls the interaction, obscuring its presence and rendering the Eagle Drone ineffective. The mitigation strategy involves enhancing the Eagle Drone's situational awareness and adaptability to counteract the adversarial drone's control-type actions effectively. This ensures the system can respond dynamically to evolving threats and maintain train network security.

6.6.4 Step 4) Predict Harder-to-foresee emergent scenarios (Black Swan scenario)

In this step, we elaborate on the scenario further to define the sequence of potential actions. We will use Table 6.19 to predict a Black Swan scenario. The architect can potentially use 4WnH process (see Section...) to break down the scenario further or elaborate a scenario guided by the perspective shift.

Table 6.20 Harder-to-foresee Emergent Scenario (black swan scenario)

Black Swan Scenario	Rationale for prediction: it is possible that ...
Black Swan 1: The adversarial drone may shift tactics, using advanced camouflage technology to blend in with environmental elements such as the sky, trees, or even the train infrastructure.	Adversarial drones may evolve their tactics beyond simple evasive manoeuvres and incorporate advanced camouflage technology to avoid detection. With modern developments in adaptive camouflage, such as optical cloaking and AI-driven pattern adaptation, adversarial drones could blend seamlessly into environmental elements like the sky, trees, or train infrastructure.
Black Swan 2: The Eagle Drone, still tasked with monitoring, struggles to adapt to the adversarial drone's increasingly sophisticated tactics, leading to degraded performance in identifying and tracking the adversarial drone.	As machine intelligence evolves, adversarial drones will likely incorporate increasingly complicated evasive tactics to degrade the eagle drone's monitoring performance. These tactics could include AI-driven flight pattern modifications, multi-agent coordinated evasive manoeuvres, or self-modifying behaviour based on the eagle drone's response patterns.

The AIC analysis reveals a shift in the adversarial drone's goal and action type, resulting in a significant challenge for the Eagle Drone. Initially, the adversarial drone aimed to influence the Eagle Drone's perception by avoiding detection. However, the shifted perspective introduces a control-type goal, where the adversarial drone actively seeks to neutralise the Eagle Drone's monitoring capabilities through advanced camouflage tactics.

6.6.5 Step 5) Define mitigating ML Development and Safety Requirements.

Each safety requirement outlined for Black Swan scenarios necessitates corresponding ML component safety requirements to ensure the Eagle Drone's systems can handle unforeseen

threats. Below, we define the ML component safety requirements in alignment with each system safety requirement.

Table 6.21 Eagle Drone Safety-Training Requirements for Black Swan Scenarios

Black Swan Scenario	Safety or systems requirements (Safe Operating Concept)	ML Safety-Training Requirement (Training Concept)
Black Swan 1: The adversarial drone may shift tactics, using advanced camouflage technology to blend in with environmental elements such as the sky, trees, or even the train infrastructure.	Safety Requirement 5: Eagle Drone track and trace camouflaged adv drones on the ground Given: Adversarial drone uses camouflage tactics on the ground, In order to: avoid detection by the Eagle Drone, Then: The Eagle Drone shall implement machine learning algorithms that can dynamically identify, track, and trace adversarial camouflage tactics on the ground, In order to: prevent adversarial drone incursion.	ML Safety-Training Requirement 4: Recognise Camouflaged drones on the ground The Eagle Drone's ML component shall be trained to detect, classify, and track adversarial drones using camouflage tactics on the ground by recognising subtle movement patterns, environmental inconsistencies, and spectral differences using AI-driven computer vision models.

Then for every ML training concept requirement, we derive an appropriate set of ML-component development datasets requirements:

Dataset requirement structure:

The [system of interest] ML component [Training/Testing/Black Swan Validation] shall provide the trainee model with a valuable minimum variety of ..

Table 6.22 Black Swans driven dataset requirements

ML Safety-Training Requirements (Training Concept)	ML development datasets requirements
ML Safety-Training Requirement 4: Recognise Camouflaged drones on the ground The Eagle Drone's ML component shall be trained to detect, classify, and track adversarial drones using camouflage tactics on the ground by recognising subtle	ML development datasets requirement 3: The Eagle Drone's ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drones using various ground camouflage tactics, such as blending in with, bushes,

movement patterns, environmental gravel, soil, trash or pavement next to track inconsistencies, and spectral differences zone fence. using AI-driven computer vision models.

In order to: train the drone’s machine learning algorithms to dynamically identify, track, and trace adversarial drones obscured by ground-level camouflage.

6.7 Reflections on applying SECoT_1 in Stage 1 process

6.7.1 Reflection 6.1: AIC exposed the nature of problems

AIC exposed and visualised the long-tailed nature of open complicated problems, exposing our epistemic ignorance.

We experienced the long-tailedness of open, complicated problems during the application of Stage 1 in both of our case studies (see Sections H.4.4.1, I.4.4.1). We identified 90 out of 93 potential interrelated events that were outside our initial peripheral perception of the problem in the unsafe train tracks scenario. For example, if we hadn't applied Sage 1 in the case study, we would have thought the only real impactful problem would have been:

- An adversarial drone.
- A train approaching an adversarial drone, and vice versa.
- Train track structures.

This was demonstrated in Step H.4.4.1 and Step 4.1, where we performed the most and least frequent factor evaluations. The output showed the distribution of factors, with the top three events accounting for approximately 40% of occurrences in our analysis. The following is a snippet of the full table, which can be reviewed in Section H.4.4.1 of Appendix H.

Table 6.23 Unsafe Train Tracks problem space complexity (long-tailed distribution)

Predicted Factor	Architect’s attentive mentions	Concern Level
roaming_adversarial_drone	65	20.00%
moving_train	58	17.85%
train_tracks_structure	10	3.08%
Hidden Black Swans Analysis		
Predicted Factor	Rarely Concerning?	Impactful?
train_tracks_zone_fence_structure	no	yes
train_derailment	no	yes

fallen_vegetation_over_train_tracks_structure	yes	yes
powered_powerlines_cables_structure_visual_appearance	yes	yes
vegetation_complexity_growth	yes	yes

Table 6.23 (Table H.13A and 13B in Appendix H) shows the top 5 factors with the architect paid attention to equal or more than 8 times. These factors are rather obvious for the architect. However, 89 factors have been recalled 7 times or less. All of which were the hidden black swan events of the problem domain. In this step, evaluate each factor's repetition architect's attentive mentions and calculate the influence level. The term "roaming_adversarial_drone" has been cited 65 times out of 328 factors (including repetitions). This means the influence level of "moving_train" is roughly 18% of the total influence, calculated as 65 divided by 328. Similar to moving_train, which has a 17% influence on the architect's perception. This indicates that our overall architect's perception of the problem pays much more bias towards roaming_adversarial_drone, and the bias has blinded their perspective to notice other hidden factors which the analysis has helped to reveal. Thus, it appears to be the most important factor in the problem domain is not whether an adversarial drone is accessing the train track zone, but whether the train is in motion or not.

Furthermore, we were able to identify 41 Black Swan events within the complexity field of the articulated problem for unsafe train track zone. Which further demonstrate the ability if AIC approach to and our predictive thinking process to identify such harder to predict events. See Section 6.2.4, Table 6.5B.

In summary, while we were implementing the SECoT of Stage 1, we recalled the top three factors in our analysis 40% of the time. This is indicative of how we are perceiving the problem. The influence level refers to the extent to which a factor affects our perception of the problem. For example, roaming_adversarial_drone has 20% influence over our concern. The percentage comes from the number of mentioning we recalled the factor, which reflects the bias of our concern and thus our epistemic ignorance.

6.7.2 Reflection 6.2: Reduction of epistemic uncertainty could be validated

We are indeed dealing with epistemic uncertainty, and AIC architect's attentive mentions curve helps to demonstrate its capture and thus consequent reduction.

In Figure 6.5, the architect's predictive statistical attention distribution curve exhibits an interesting fit to the observed statistical distribution curve. The long tail represents some of the black swan events that were previously hidden under the simplicity of the problem domain description, demonstrating that our approach has helped us predict black swan events and thus

reduce our residual ignorance. The skewness test (using the Excel function SKEW()) revealed that the distribution is right-skewed in both case studies.

In this analysis, we can look at all the factors: those to which we are paying more attention and those (the least frequent ones at the bottom of the list) to which we are not paying attention. In this case study, we counted nearly 23% of the discovered factors with the minor architect's attentive mentions range (3 to 1 occurrence), demonstrating our epistemic uncertainty we have about the problem domain. The AVOIDDS case study also found the same trends (see Section I.4.4).

Regarding the AVOIDDS case study, we identified 102 interrelated factors involved in the problem domain that necessitated defining the architect's assumptions about each of these factors. The top three factors which define our peripheral perception of the problem are captured in the Table below, which is part of Table I.12 in Appendix I. The table shows the top 16 factors with the architect paid attention to more than 3 times. Some of these factors are rather obvious for the architect. However, there are 88 factors that have been recalled 3 times or less. Some of which may be considered the hidden Black Swan events of the problem domain (relative to our initial knowledge about the problem). See Figure 6.27, which shows how the long tail of Black Swan events has exposed the problem domain.

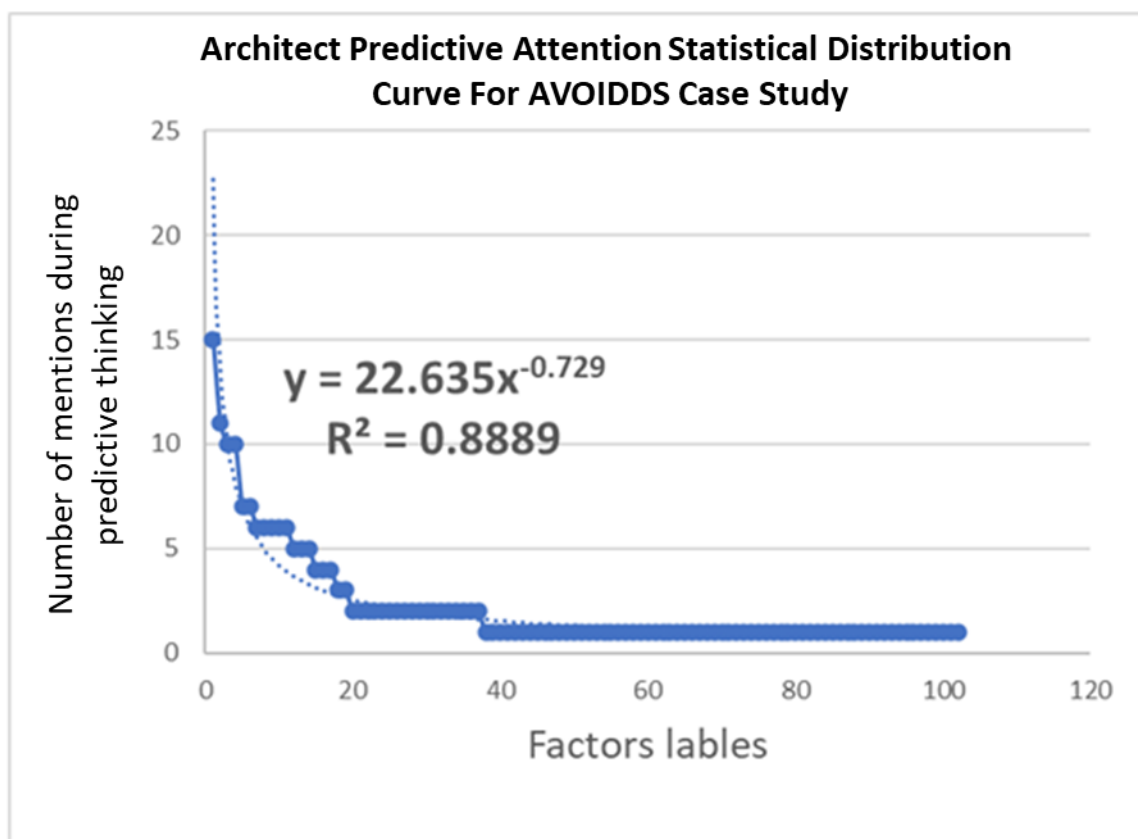


Figure 6.27 Architect predictive attention distribution curve of the derived factors in AVOIDDS problem domain complexity field

Our concern about the problem seems predominantly biased towards what we can perceive concurrently. It reflects that our perception of the problem may always be incomplete, which may indicate a fundamental flaw in any ArcPM (predictive architect), which we call **Architect Perception Deficiency**. The AIC Systems Engineering process aims to minimise this deficiency by applying lateral and vertical thinking approaches. The definition is minimised in the process of pursuing ideal completeness (Ideal System).

6.7.3 Reflection 6.3: AIC reveals a Heavy-Tail distribution of complexity

The AIC approach exposed the Power Law or potential Heavy-Tail Distribution of the open, complicated environment, indicating a good alignment with our definition of Complexity and Complicatedness. Making the problem classification to be abnormally distributed (non-normally distributed problem).

In Section 4.1.1 we provided a measurable definition of what a system is. We believe that complexity is akin to the observed statistical distribution of events of an observed complex; complicatedness is how predictable this observation is by a predictive observer. Perhaps if we can fit an exponential curve well over the observed statistical distribution of predicted factors, it may reveal something measurable about the complexity of the problem domain. However, this is a speculation, and more investigation may be required. One has been confirmed that the observed statistical distribution of complexity may be a good measurable estimation of complexity. If we accept this, systems engineering may become a mathematical based discipline, just like other engineering disciplines. This is one of our founding visions for our research.

In Figure 6.27, the fitted power-law curve ($R^2 = 0.88$) indicates a potential heavy-tail pattern in the distribution of factors as we perceived them. This means a small number of critical factors (e.g., "Own_aircraft_pilot_decision_making_process", which appears 10 times) will have a significant impact on an architect's expectation of how the problem will unfold, while most factors have low frequencies. In Figure 6.5, we observed a curve-fit of ($R^2 = 0.86$) for an exponential distribution, and a skewness value of 6.2.

Upon applying the skewness statistical test (using the Excel function SKEW()), the results returned a value of 2.85 (for AVOIDDS case study), indicating right-skewed data, which confirms that our predicted complexity is indeed perceived to be long-tailed complexity field. The same is also observed for the unsafe train track problem domain, with a skewness value of 6.2.

We were expecting such long-tail distributions, a characteristic of confusingly complicated complexity fields, where some elements exert outsized influence (over the observer's perception)

while most remain relatively hidden from the architect's foresight. This distribution aligns with the nature of many real-world complexes, where only a few key variables or conditions drive most outcomes, while others contribute less predictably.

Such a realisation does not tell us whether our prediction is right or wrong. Still, it consolidates our assumption that the complexity of the real world is a long-tailed distribution of event probabilities. And that our perception of it is skewed. also consolidating the suspicion that there may be a link between the curve-fit quality and how complicated we think the problem is. Thus, we may be able to quantify complicatedness based on a reference curve-fitting. **So what?**

6.7.3.1 Connection to Black Swan Theory:

Nassim Nicholas Taleb's Black Swan Theory explains the occurrence of rare, high-impact events that are challenging to foresee within standard predictive models, especially those based on normal distributions. These "Black Swan" events lie beyond typical expectations and have a disproportionately large effect. We re-adapted the definition of Black Swan in Section 4.1.16 in our approach to Predictive Thinking and how this would be assured at pictorial datasets.

The observed power-law distribution in the AIC analysis consolidates our belief, suggesting that most factors have minimal impact on our concern, and a few rarely concerning may become sources of Black Swan. We would perceive them as harder-to-predict or unpredictable factors that could significantly disrupt our assumptions about CUP. These low-architect's attentive mentions factors are difficult to anticipate with conventional engineering approaches (Safety I, Safety II), which primarily address White swan failure events. We believe this is partly due to the following reasons:

1. **The lack of lateral thinking in current approaches:** Current systems engineering approaches are heavily focused on vertical thinking and a control-driven perspective, only. However, in CUPs, control alone will not serve as an efficient means of predictive thinking for forecasting rare, concerning, and impactful events. It may take more factors to go wrong and more surprises for the control-based predictive thinker to reach their resilient ideal configuration of their engineered system. Hence, we proposed the embrace of both Vertical and lateral thinking to allow for more cognitive flexibility, enabling the recognition of rarely seen interactions when perceiving a problem domain.
2. **Guidewords are a limiting factor:** Basing predictive thinking and triggering imagination on a limited number of guideword-based heuristics is useful for traditional ideal and normal systems (see Section 4.1.1 for definitions of ideal and normal systems). However, when we face CUPs, any deviation that falls outside the guideword guidance is classified by default as the least concerning. The architect does not intend to ignore these

deviations; however, this is a side effect of limiting predictive capability to trigger only a small set of possible ways things may go wrong. AIC helps support these approaches by offering a dynamic method to discover deviations outside the guide words. In Stage 1 of the case study and in HazTIPS analysis, we were able to think outside the constraints of our generalised guidewords. We are not dismissing their effectiveness; after all, they did a good job in the past. We just need extra work done to bag those valuable Black Swans.

6.7.3.2 Assessing the risk of rarely concerning factors

In the context of the AVOID Dataset, the most frequently recalled factors, like decision-making processes and environmental elements, represent predictable concerns (known unknowns) central to system operation. However, the infrequently recalled factors at the tail of the distribution might represent the set of those factors that are “Black Swans” such as unforeseen risks like "hacks into the AVP detection confidence_score" or environmental challenges like "cloud turbulence."

These low-architect’s attentive mentions factors could have an outsized impact in extreme or unexpected scenarios. The heavy-tail distribution underscores that even rarely encountered factors can be crucial in particular circumstances, particularly when typical control mechanisms may not account for these rare events.

One clear advantage of step 4.1 (Section F.3.4): Perform Most and Least Frequent Factor Evaluation, is that it can provide risk assessment methods (for holistic systems engineering practice) under CUP conditions as a reference for estimating the likelihood of events occurring. Regarding impact assessment, I find it rather pointless because if Black Swans can cause a significant break or impact on our engineered solutions, we should consider all possible events as equally impactful.

In fact, we recommend not to bother with applying the traditional red, amber, and green classification of risk metrics, as they are essentially meaningless when a Black Swan affects our engineered system's reliability and safety. While they may be used to demonstrate that some form of rigour was involved in risk assessment failures, beyond being a fancy legacy tool, they are likely of no practical use in CUP conditions. An effective approach (although time-consuming) from a safety perspective is to consider all events as equally impactful. By doing so, we will begin to recognise those least concerning events that we normally (if we are engineering ideal and normal systems) consider.

6.7.4 Reflection 6.4: power-law curve fitting as a measure

There may be a connection between predicting factors using AIC Approach and how well we can fit an exponential curve, indicating some measurable quality of epistemic uncertainty reduction performance.

The analysis in both case studies suggests that the fitness of the power-law curve (as indicated by the R^2 value) may reflect the quality of AIC application. Maybe the stronger the alignment with a power-law distribution, the more effective the AIC approach has been in capturing the complexity and potential Black Swan events within the problem domain. For instance, the Eagle-Nest project exhibited higher curve fitness due to a more granular application of AIC, analysing every interaction comprehensively. In contrast, the AVOID Dataset project showed lower curve fitness, possibly due to a more targeted approach that focused on primary system interactions rather than exhaustive analysis.

We may want to ask whether we have approached the problem with a reasonable bias? What if there is an even more fundamental problem that we are not paying enough attention to? In this analysis, we can look at all the factors: those to which we are paying more attention (higher influence over our perception) and those (the least frequent ones at the bottom of the list) to which we are not paying attention.

6.7.5 Unsafe Train Tracks Thought Experiment

At the beginning of implementing our approach for the first case study, we initiated the evaluation with a thought experiment (see section 6.1.1). Initially, we assumed that the real problem could be characterised by three main factors: the adversarial drone, the train, and the infrastructure. However, at the end of the Stage 1 analysis alone, we discovered that 93 problematic factors are involved in the complexity. If we had started the derivation of systems-level requirements by assuming only three main factors and assumptions, we would have missed 89 potential factors that should be considered. In the thought experiment, we were asked to answer two main questions. Below is a table that describes our initial thought (perception of the problem) and how our perception changed after applying the process:

Table 6.24 Answering thought experiment tests

Thought experiment concern	Initial perspective	Shifted perspective
----------------------------	---------------------	---------------------

Q1) Is the train tracks problem domain completely defenceless against adversarial drones?	Initially, we would have started the system-level requirement derivation based on the assumption that train tracks are obviously defenceless.	The track zone is not entirely defenceless, it is surrounded by vegetation and possibly fauna, which likely introduce visual uncertainties to the reliability of adversarial drones in executing their missions.
---	--	---

Q2) Is there a concerning risk that train tracks, as they are, pose to the surrounding vegetation, which impacts the safety of the train track zone?	Usually, such interaction is not a concern.	The answer, we discovered, is yes. It primarily affects the reliability of the Eagle Drone's perception system and also provides cover for adversaries.
--	--	--

For **Q1**, after conducting the analysis, we realised that assuming adversarial drones deploy ML-based perception, there is a good chance that their reliability will be affected by visual uncertainties in the train tracks. For example, leaves_&_pollen or the motion of plants around train tracks. Therefore, it is not entirely defenceless. Consequently, it is plausible that we can enhance the effectiveness of the solution by simply increasing the visual uncertainty of the train tracks in a way that the adversarial architect would not have guessed. Such a strategy would have been accounted for in our design for Eagle Robot, so the operational environment would be entirely familiar for our Eagle Drone (because we would have included in our training CuneiForms), yet completely visually alien to any intruding drone.

For **Q2**, the answer is yes, it is a genuine concern as any visual uncertainty related to how components interact with the train track environment can impact the reliability of our engineered solution's effectiveness. For example, adversarial drones can hide behind trees, and any vortices generated by passing trains could agitate plants, fallen leaves, or debris that may impact the Eagle Drone's performance. Therefore, it is reasonable to believe that SECoT processes had improved the architect's perception of the problem domain, thus increasing requirements coverage.

6.7.6 Reflection 6.5: Application of Action Matrix and Lateral Thinking Process

The Lateral thinking process is necessary to provide convincing justification for risk assessment about rarely concerning and emerging interactions among things in a problem

To unpack the complexity field hidden in long-tailed interactions (those which we are not observing at the moment of consideration), we utilised an Action Matrix (see Section??). The matrix challenged our current peripheral perception of the problem with interactions we had never considered before. For example, an interaction between {police}_[_?_]_{oscillating_trees} and {oscillating_trees}_[_?_]_{police} (see Section H.4.2.1). It was rather a strange interaction; why might there be a potentially unsafe interaction between the police and the Oscillating Trees movement? We realised that those hard-to-describe interactions are actually the unknown interactions that exist outside our peripheral perception problem. Therefore, these are sources for “unknown unsafe” scenarios. Hence, the Action Matrix (the same as AIC Matrix) helped us to discover them and potentially reduce the incompleteness of requirements.

The Action Matrix only exposed our ignorance but did not tell us how to make sense of those rather strange interactions. Here, we needed a systematic process to help us distill such laterally challenging interactions. To resolve this thought dilemma, we employed the General Lateral Predictive Thinking Process (Section F.9) to guide our thinking and generate potential scenarios. The output of the process was three potential hypotheses of what scenario might manifest into.

6.8 Applying SECoT_3 in Stage 3A

SECoT_3 (HazTOPS-driven Ordered AIC Requirements Elicitation Process) also involves SECoT_2 (Hazards, Threats and Opportunities/Affordances Scenarios (HazTOPS)). We applied SECoT_3 during Stage 3A for both case studies to further distil the problem domain and identify more potential Black Swan Scenarios. Predictive thinking pipeline 1 helped us to have a holistic overview of the actual operation environment.

6.8.1 Reflection 6.6: AIC brings lateral thinking to guidewords-approach

The AIC approach can help mitigate psychological biases and improve preparedness for the least concerning events outside the limit of guidewords.

In HAZTOPS, we use a generalised form of HAZOP guidewords; however, we include AIC complexity field to help with reaching out to those least concerning events further. In Stage

1, the analysis of H.4.4 Predictive Thinking Pipeline 4: table H.13, also attracted our attention to the following rare considerations for such a problem:

Table 6.25 lists of rare factors associated with the problem

Rarely predicted factor	Why it's unlikely to be considered	Source thought step that led to the prediction
roaming_adversarial _drone_noise_ suppression _systems camouflage_ mechanisms	The idea that adversarial drones may intentionally suppress their noise to avoid auditory detection is often reserved for military-grade systems or spy technology, and may not occur to those designing civilian applications in rural areas(non-hostile). Makes such encounters relatively rare, yet impactful if they do occur. Hence, they are by definition a Black Swan scenario.	Appendix J, Section J.1.3, Step 3.2, based on the AIC-analysis of Unsafe Scenario 2.1. Unsafe intended adversarial drone approach train, which may lead to a train crash.
leaves_&_pollen	Leaves and pollen are seen as natural and minor environmental phenomena, not typically classified as engineering risks. It may be unlikely that such a factor would have been considered in any traditional problems related to train tracks, perhaps when solving pantograph problems, as access debris and foliage between the contact area of the mast and the power cables could cause a hazard.	Appendix J, Section J.1.2, step 2.1. The prediction stemmed from resolving the complexity matrix (Table J.4), where we applied lateral thinking to determine how electric powerline wires, combined with vegetation around the fence, may pose a concern for the architect. In this case we thought that: Fallen vegetation such as leaves and pollen, adds

visual complexity to
 electrical powerlines
 recognisability with respect
 to intelligent perception.

In Table 6.25, we provide an example of rare factors and explain why we consider them rare scenarios. In the output table of factors, they were recalled only once throughout our implementation of SECoT_1 during Stage 1. This has informed us to think that those factors are, in fact, outside our peripheral perception of the problem. Additionally, Table 6.25 illustrates how predictive thinking processes facilitate the development of arguments supporting the prediction that the factors are impactful and should be considered as part of the design. It also provides a systematic demonstration to back any justification of design choice or engineered judgement for safety cases. For example, artefact [Y]: AS Design Justification in SACE framework.

6.8.2 Reflection 6.7: Safety and Security Concepts in the Context of AIC

The application of HazTOPS enabled us to understand safety and security in an AIC complexity field more objectively. We have come to learn that:

Safety⁴⁸ refers to a system's ability to appreciate and prevent harm or danger to the primary purpose of the engineered system and cause harm or danger to other complexes' primary purposes within the complexity field. It is a relative concept depending on which point of view in the AIC complexity field, and there are

- **The architect's point of view:** The overseeing architect will have a different view of what is safe and what is not in an AIC complexity field.
 - **So What?** If we assure the architect's objective view using the application of general systems rules, we can then help minimise personal bias about how a complexity may evolve in the future, and allow us to validate their decision-making process. Ethical regulator theory by Mick Ashby (we developed a process for its integration, but we did not include it in this research) could offer an effective solution to assuring the objectiveness of the architect's view on what is safe and what is not.

⁴⁸ More detailed definition of safety and security concepts (especially properties of soft hazards) in the context of AIC can be found in Appendix E, Section E.3.

- **From a complex's point of view:** Each node has its view on what is safe or not. The AIC field provides the architect with the opportunity to stand in the shoes of a node and view the entire complexity from a bird's-eye view, using the definitions of interaction and emergent properties (see Section 4.1.5).

We took safety from an architect's point of view, so it often boils down to harming humans, property, or the environment, whether functioning or malfunctioning. It encompasses the system's capability to operate safely under normal conditions and to mitigate risks arising from unforeseen circumstances. Safety is evaluated by identifying **hard** and **soft safety hazards** and implementing mitigating interventions or capabilities.

In CUPs, Safety Hazards can be classified into:

6.8.2.1 Hard Safety Hazard

A direct, immediate threat within the system's complexity field or external environment that poses physical harm to the system or its surroundings.

- **Example:** An autonomous vehicle fails to detect a pedestrian due to a malfunction in its object detection algorithm, creating an immediate risk to the pedestrian, vehicle occupants, and nearby property. The malfunction originates within the system's complexity field.
- **AIC Relation:** This represents a hazardous control interaction where the system's control over its environment and itself fails, resulting in an unsafe scenario.

6.8.2.2 Soft Safety Hazard

A potential future threat that could evolve into a hard safety hazard due to changes in the complexity field. Although this may not pose an immediate danger, anticipating and mitigating it requires foresight.

- **Examples:**
 - **Wildlife Interaction:** Animals near a train track zone monitored by a security drone may not present an immediate hazard but could become one if deployment conditions change.
 - **Weather Conditions:** Adverse weather outside the immediate operational zone of a drone might impact future deployment.
 - **Human Factors:** Local population temperament could evolve into a safety threat in certain contexts.

- **Human or free-will agents' intent** is also a soft hazard since it can only be clear over time and change of complexity.
- **AIC Relation:** interactions within the complexity field that have to appreciate the system of interest behaviour and the architect misses their impact in the future. This can be predicted using AIC perspective shift.

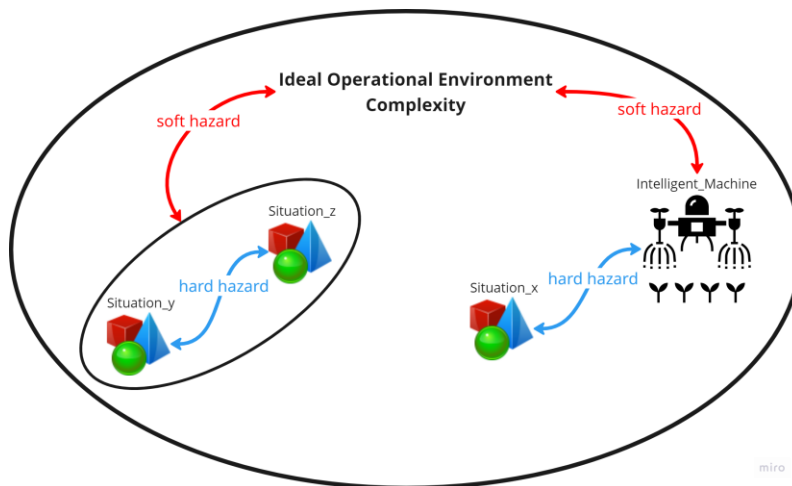


Figure 6.28 Hard and Soft Hazards definition

In general, a Soft Hazard impacts the overall holistic relative harmony (relative, in the sense of alignment to the architect's intent of how they wish the dynamics flow for the whole) of the complexity. They may not pose direct hazards or threats in the immediate sense, but when potential circumstances and conditions arise, they become impactful and troublesome. After all, in lateral thinking, everything is possible; a small flap of a butterfly wing could lead to disaster in CUPs.

6.8.2.3 The inherent security threat of appreciation

In the AIC complexity field, every appreciative interaction carries with it an inherent security threat, because if A appreciates B, this means A has no influence or control over B, and B has full influence over A. Therefore, whoever controls B, most definitely will control A (see Section 4.4.16). We applied this rule in Section H.6.3.1. We found that the discovery of rarely concerning threats was intuitive; for example, we discovered a potential security threat that exploits local people's appreciation of preserving local trees around the train track zone, which hinders the success of deploying the eagle drone. An adversarial drone may work out a very efficient and less expensive social engineering scheme that forces the local people to pressure operators to abandon the deployment of Eagle Robot. SHARCS, HAZOP or STAMP would not have predicted such threats or hazards because the interaction between local people and nature is outside the immediate concern of the architect, who would be more focused on ways local people may directly damage the Eagle Drone.

6.9 Applying AIC Perspective Shift (Stage 4)

6.9.1 Reflection 6.8: AIC perspective shift helped to predict Black Swan pictorial scenarios

One of this study's pivotal findings was the realisation that as adversarial schemes evolved in complexity, their operational goals transitioned from an appreciation-influenced dynamic to a control-oriented strategy (see Section H.8). This shift fundamentally altered our prior perspective about the interaction between the adversarial drone and the eagle drone in Stage 3A.

Table 6.26 AIC-type perspective shift for unsafe training tracks case study

Interaction	A11: {flying_eagle_drone}_[+recognise]_{adv_drone_shapes}	
AIC factors	Initial perspective (prior)	Shifted Perspective (posterior)
Source	Adv_drone_shapes	Adv_drone_shapes
Sink	Flying_eagle_drone	Flying_eagle_drone
Supra Source	Adversarial Scheme.	Adversarial Scheme.
PrimeP	Disrupt Train Network operations.	Disrupt Train Network operations.
Source's Goal	obscure Eagle Drone perception.	Obscure Eagle Drone perception.
Source's Goal type	<i>Influence</i>	<i>Control</i>
Source's Action	<i>Avoid</i>	<i>Camouflage</i>
Source's Action type	<i>Influence</i>	<i>Control</i>
Source action effect on sink	Obstructive	Obstructive

The interaction landscape between the adversarial drone and the Eagle Drone fundamentally changed at interaction emergent property levels. Our definition of the interaction (see Section 4.1.5) and Deep AIC Matrix (see Section F.1.1), using AIC factorisation SECoT (see Section F.2) helped us to objectively describe and explain how a Black Swan scenario may emerge (demonstrating no personal bias had skewed our perception of complexity). By employing the AIC framework and executing a strategic perspective shift, we recognised how these changes in goals and actions could generate potential Black Swan events (situations in which the adversarial drone could perform undetected incursions into train network operations, thereby compromising security).

Moreover, we generated a Black Swan testing dataset (designated as the Black Swan A dataset, see Section 6.8) constructed from scenarios characterised by these unforeseen dynamics in Table 9.5. From it, we derived Black Swan CuneiForms (see Table 6.35), example below:

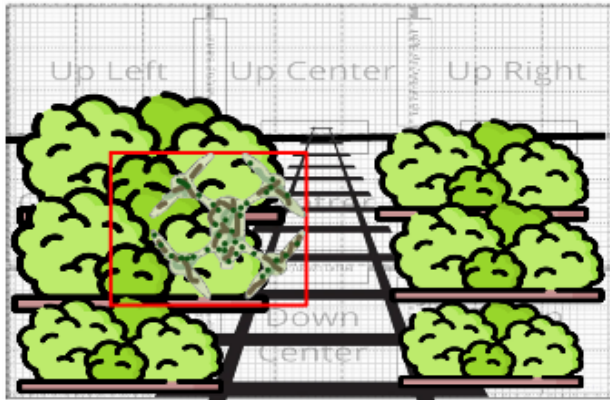


Figure 6.29 Example Black Swan CuneiForm H.37 used to assure the final model performance during the Black Swan operational scenario

This dataset was a benchmark against which we tested our final machine learning model (see Section 6.8). The model's performance on the Black Swan A dataset diverged dramatically from expectations, revealing its inadequacy in handling out-of-distribution instances not represented during training. This discrepancy underscored the importance of the AIC perspective shift, as it reinforced our understanding of how subtle changes in interaction dynamics can lead to significant model performance degradation. In AVOIDDS case study, Stage 4 provided good help with predicting a potential Black Swan scenario that was missing from AVOID dataset (see Section 1.8).

In summary, the investigation in our experiments highlighted the necessity of predicting the complicated adaptability of operational domain behaviours through the AIC perspective shift lens when developing predictive models for complicated problems. The insights gained from this Stage validated the importance of incorporating Black Swan scenarios into our training processes and emphasised the need for continuous revaluation of models in light of emerging complexities.

6.9.1.1 Evidence of AIC-perspective shift led to predicting missing Black Swans

This is one of the main interesting findings in our research. Initially, we were reluctant to make a strong claim that by shifting the AIC perspective of AIC models, we would end up predicting Black Swans. However, when we conducted similarity measures between the Black Swan dataset and the training dataset in experiment 10 (a typical, popular, economic way to train perception models), as well as in experiments 1 and 2, we were able to verify that the AIC-perspective shift indeed helped us to discover an out-of-distribution subset. However, when we subsequently tested the trained models on datasets that were missing those Black Swans, specifically on the

Black Swan images, the model performed significantly poorly. This provided us with significant evidence that the AIC perspective shift indeed helped us to predict an unmitigated Black Swan scenario such that if we did not shift our AIC perspective about the problem, we would not have thought about such scenario.

6.9.1.2 Predicting Unknown-Unknowns through Perspective Shifts

Why is the concept of Unknown-Unknown relevant to our approach? Because Black Swan Scenarios is a subset of the Unknown-Unknowns of any complexity. By understanding the concept of unknown unknowns from a systems approach perspective, we can appreciate the nature of black swan scenarios and problems from another worldview.

In general systems wisdom, we often encounter the concepts of known and Unknown-Unknown factors. While known unknowns are often easier to handle as they stem from past experiences and experts are usually good at spotting them, Unknown-Unknowns are those events or factors the designers don't even realise they don't know. The latter is the spawn of hidden, confusing, complicatedness, and often, thinking techniques (SHARCS, HAZOP ..etc) are methods to enhance the designer's chances of spotting them, using predominately vertical thinking. In the SHARCS method, there is an attempt to use lateral thinking by using the STAMP failure discovery table. However, it is limited to the assumption that interactions are a sequence of domino effects. The approach doesn't directly claim as such. However, we view it as effectively an attempt to employ some systemic consideration, though it is limited.

Regardless of the solution methodology, "unknowingness" can be associated with the level of the designer's ignorance about a given observed complexity field. The more the designer unpacks the complexity, the more chances they have to discover new emergent information that they didn't know before. The complexity of the mental model method and its ability to generalise well enough are crucial factors in helping designers discover the edge of their ignorance, "Unknown-Unknowns".

AIC modelling techniques, attempt to complicate the thought process in a general sense to tease out those hidden factors when repeated iteratively. The more AIC fractals are unravelled from a complexity field, the more chances a designer will discover Unknown-Unknowns. When the designer applies an AIC perspective shift, some of the shifts become harder to imagine. Those hard-to-imagine shifted interactions are the "Unknown-Unknowns" relative to the designer. Therefore, when an interaction becomes hard to imagine at the moment of analysis, it should be labelled as "Unknown-Unknown".

For example, suppose you are designing an autonomous vehicle (AV) system and are conducting an AIC (Appreciation, Influence, Control) analysis to model interactions between the vehicle and

its environment. You are examining an interaction between the AV and pedestrians at a busy city intersection.

Known Unknowns:

You already know that the AV must detect pedestrians and predict their movements. However, factors like unexpected pedestrian behaviours (e.g., someone suddenly appear from between cars into the street) or external conditions (like bad weather) are **unknowns**. Based on past experience, you can anticipate these challenges, and you can design safety features like improved sensors or predictive algorithms to handle these situations.

AIC Modelling to Uncover Unknown-Unknowns:

During the AIC analysis, you break down the behaviour of the AV interacting with a pedestrian with supportive systems like sensor arrays. As you iterate through different possible scenarios, you consider shifts in perspective to tease out deeper complexities:

- **Influence Interaction (I):** The pedestrian's movement influences the AV's behaviour by triggering its sensor system to apply the brakes.
- **Control Interaction (C):** The AV controls its response by adjusting braking force based on real-time environmental data.
- **Appreciative Interaction (A):** The AV appreciates the pedestrian's presence and adjusts speed to avoid collision.

You continue to model variations in behaviour, such as different types of pedestrians (children, distracted individuals) and varying levels of sensor visibility (day, night, fog).

Uncovering an Unknown-Unknown

As you continue to iterate, you would consider the AIC complexity field for pedestrians. Given an urban city environment, you may consider a pedestrian appreciating the path on the pavement, for example, walking past shops. But what are the other aspects about a pedestrian that the AV has no influence or control over?

- The way a person looks.
- What the person is wearing.

At the appreciation thought step, you would know that appreciation means considering the least concerning aspect. One way we advise about this step is to consider “humour.”

What would be a hilarious thing that I would have to appreciate for its randomness?

It would be hilarious if a fancy dress like a cupcake or a banana distorted the pedestrian form. You might consider this a ridiculous thought, and you can, but at least in the UK, fancy dressing is quite common and popular. But you get the idea here, I presume. Thinking in humour helps to consider trivial scenarios, and those are the Black Swans, which normally do not occur to you when you use logic thinking. Fancy Dress is an unknown-unknown; you did not know that you did know about the impact of Fancy Dress on your AV perception. It would be rather awkward for a professional company if the AV makes the headline on the news for ruining some stag-do party because the design team did not consider British people sense of humour as failure mode.

Let's assume you apply the AIC perspective shift, you may change the appreciative interaction to control and try to imagine a shift in the scenario that becomes increasingly hard to conceptualise;

How could a pedestrian control the walking path?

You struggle to envision this at first but then realise a possibility: **what if a pedestrian is interacting with a secondary object in the environment in an unexpected way?** A pedestrian might push a shopping cart or carry a large object that blocks sensor detection, causing the AV to misinterpret their behaviour.

This type of interaction wasn't initially in your mental model, but after repeated AIC modelling, you realise it poses a significant risk. The **shopping cart** is effectively an **unknown unknown** that emerges from the interaction between the pedestrian and the environment (objects carried or pushed). You hadn't previously considered this case because it fell outside the bounds of typical pedestrian behaviour.

Labelling the Unknown-Unknown

Initially, controlling a path is labelled as unknown unknown because it was not something you even realised you didn't know when first analysing the system. This realisation prompts a need for additional requirements to handle unusual object interactions in pedestrian detection systems, such as more sophisticated object recognition algorithms or sensors with enhanced field-of-view.

Chapter 7 Unsafe Train Tracks Case Study B

Chapter 6 presents a case study on the Unsafe Train Tracks Problem. We covered the articulation of the problem domain up to the Stage of predicting Black Swan scenarios in Stage 3A and Stage 4. We derived the Safe Operating Concept and Training Concept. We then derived the ML Dataset requirements. In the following Stages, we will delve into closing the semantic gap between Safety Requirements and the required images through the development of CuneiForms.

7.1 Stage 5: CuneiForm-based Syllabus for Safety-Driven ML Epistemic Intelligence Development

Outcomes from Stages 3 and 4 reveal the epistemic uncertainties faced by the architect and their trainee machine regarding the problem domain. The architect must define the real world and outline potential unexpected scenarios for the trainee machine to meet objectives. Minimising these uncertainties to a reasonable level (ALARP) is crucial, which can be achieved by creating CuneiForms that capture the required training regimes. Thus, this process can be termed ML Epistemic Uncertainty Reduction Training, focusing on enhancing the machine's robustness by increasing the variety of real-world scenarios, rather than increasing the variety of image augmentations. Dataset augmentation techniques (e.g., colour modifications) will follow, seen as “Aleatoric Uncertainty Reduction” since they alter pixel randomness rather than actual scenarios.

7.1.1 Step A) Articulate the pictorial problem context:

We start the process by choosing the ML development datasets requirement. In this case study we choose the following:

ML development datasets requirement 3: The Eagle Drone’s ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drones using various ground camouflage tactics, such as blending in with bushes, gravel, soil, trash or pavement next to the track zone fence. In order to: train the drone’s machine learning algorithms to dynamically identify, track, and trace adversarial drones obscured by ground-level camouflage.

The above requirement mitigates **ML Safety-Training Requirement 4** from the Safe Operating Concept. To articulate the pictorial situation, we followed the process in Table 7.1, which uses the AIC-Core thought process.

Table 7.1 CuneiForm Pictorial situation articulation

Pictorial Situation CoT step	Definition
------------------------------	------------

Step 1) Define a minimum variety of TOIs and their pictorial appearances

Architect prediction: The architect asserts that the perception system may face a pictorial situation with a drone landing on the ground. The drone has camouflaged skin.

Step 2) Consider a minimum variety of other objects that TOIs aim to influence

Architect prediction: The architect asserts that the adversarial drone aims to influence the Eagle Drone detection capability.

Step 3) Consider a minimum variety of objects that TOIs may appreciate

Architect prediction: The architect asserts that the adversarial drones may have to appreciate the following environmental **scenery aspects:** distance from the track fence, trees, local birds, bushes, gravel, soil, train tracks, trash and pavement.

Step 4) Consider a minimum variety of what other objects TOIs may control the correctness of predicting their shapes.

Architect prediction: The architect asserts that adversarial drones autonomously look to exploit the following aspects:
The density of trash on the ground, the density of vegetation on the ground.

Step 5) Produce pictorial problem situation context.

Pictorial problem situation context: The architect asserts that the perception system may face a pictorial situation with:
A drone landing on the ground {1}. The drone has a camouflaged skin {2} landing at various distances from the track fence or train tracks or both {3}, trees {4}, local birds surrounding the landed drone{5} bushes {6}, gravel{7}, soil{8}, pavement{9} trash {10}.

7.1.2 Step B) Characterise the Training Classes for CuneiForms

In this step, we will define a CuneiForm that satisfies the pictorial situation context. We will consider a maximum of 5 objects in this Cuneiform. Table 7.2 captures the main activities:

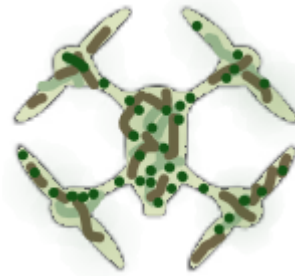
Table 7.2 Characteristic Training Classes definitions for a CuneiForm abstract image

CuneiForm	Characteristic	Definition
Training Class		
Step 1) Define visible horizon attitude.	At least one horizon attitude type:	Bird's Eye View: no horizon is visible

Frame roll = any°, Frame pitch >-45°{11}

Step 2) Define all the TOIs and their aesthetic complexity. Then, generate abstract representative icons for the CuneiForm abstract image.

At least one adversarial drone{1}, quad-copter type {1.1}, camouflaged in military green pattern {2}.



Step 3) Define TOI's motion trajectory and dynamic optical situations. Then, update the generated abstract representative icons for the CuneiForm abstract image.

Motion trajectory: static, no motion {1.2}.

Dynamic optical state: captured without optical blur {1.3}.

Step 4) Define the background objects associated with TOIs and environmental scenery in the background of the CuneiForm. Then, generate abstract representative icons for the CuneiForm abstract image.

Train tracks zone fence or train tracks or both {3}, bushes {6}, gravel{7} trash {10}Train{12}.

Step 5) Define the background Objects' Motion situations and dynamic optical situations . Then, update the generated abstract representative icons for the CuneiForm abstract image.

Background objects' motion trajectory: static or in relative motion {3.1,6.1,7.1,10.1,12.1}

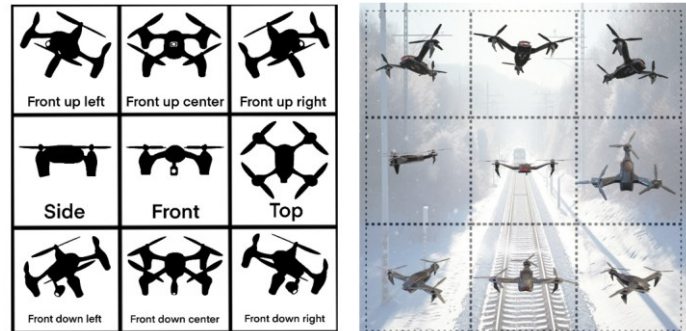
Dynamic optical state: no motion blur or with blur {3.2,6.2,7.2,10.2,12.2}. Below is an example blurred background object due to relative motion.



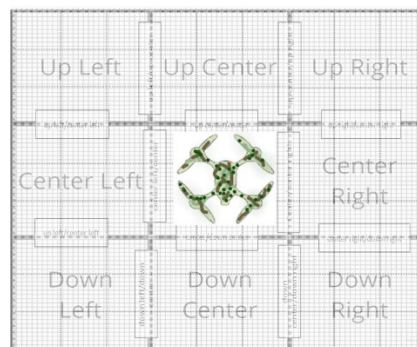
Step 6) Define TOI's positioning in the CuneiForm. Then, generate abstract representative icons for the CuneiForm abstract image.

Center also various {1.4}

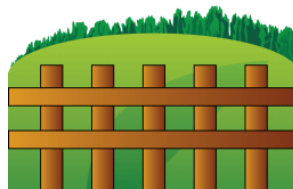
Step 7) Define TOI's 3D orientation. Top {1.5} the following is the definition of an acceptable Then, update the generated definition of a minimum variety of possible TOI's 3D abstract representative icons for orientations the CuneiForm abstract image.



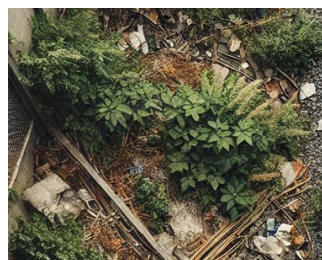
Step 8) Define the optical distance for each TOI in nindans. Then, update the generated abstract representative icons for the CuneiForm abstract image. The drone is represented in a pictorial distance of 9 nindans {1.6} bellow is a relative scale of TOI at 9 nindans pictorial distance.



Step 9) Design the relevant icons to produce the CuneiForm and give an example of an instantiating image Train tracks zone fence or train tracks or both {3}



bushes{4}



gravel{7}



trash {10}



7.1.3 Final CuneiForm

The output of the process above is then combined. An example of an instantiated image is shown in Table 7.3. Can you spot the drone?

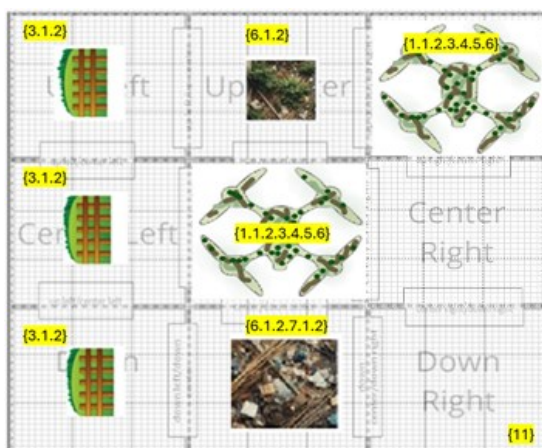


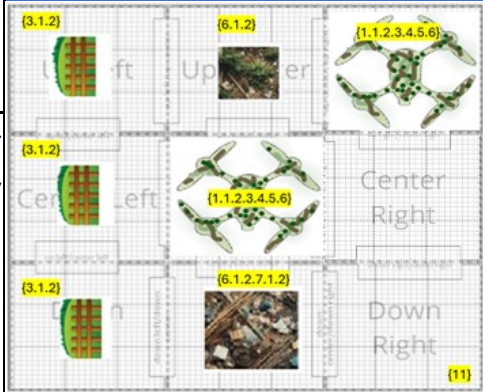
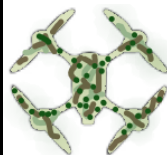
Figure 7.1 represents the CuneiForm (H.36) characterisation for the Black Swan Scenario Validation.

The CuneiForm (Figure 7.1) can be used in the safety case to demonstrate how safety requirements, covering Black Swan Scenarios, have been incorporated into the epistemic training of ML components.

The following is the final CuneiForm artefact:

Table 7.3 CuneiForm-H.36 artefact

ML development datasets requirement	Mitigating ML Safety-Training Requirement 4. ML development datasets requirement 3: The Eagle Drone's ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drones using various ground camouflage tactics, such as blending in with, bushes, gravel, soil, trash or pavement next to track zone fence. In order to: train
--	--

	the drone's machine learning algorithms to dynamically identify, track, and trace adversarial drones obscured by ground-level camouflage.		
Pictorial problem situation context	A drone landing on the ground {1}. The drone has a camouflaged skin {2} landing at various distances from the track fence or train tracks or both {3}, trees {4}, local birds surrounding the landed drone{5} bushes {6}, gravel{7}, soil{8}, pavement{9} trash {10} Train{12}.		
CuneiForm Training Process Design			
Abstract CuneiForm Characteristics (Training Classes)	Abstract CuneiForm Training Classes Characteristics definitions: Minimum Valuable Variety	Output CuneiForm and an example image	
Visible horizon attitude	At least one horizon attitude type: Bird's Eye View: no horizon is visible Frame roll = any°, Frame pitch >-45°{11}	CuneiForm Abstract Image: 	
TOIs definition and their aesthetic complexity	At least one adversarial drone{1}, quad-copter type {1.1}, camouflaged in military green pattern {2}. 		
TOI Motion and Dynamic optical situations	Motion trajectory: static, no motion or appears to be in relative motion{1.2}. Dynamic optical state: captured without optical blur or with blur {1.3}.		
Background Objects associated with TOIs	Train tracks zone fence or train tracks or both {3}, bushes {6}, gravel{7} trash {10}Train{12}.		
Background Objects Motion and Dynamic optical situations	Background objects' motion trajectory: static {3.1,6.1,7.1,10.1,12.1} Dynamic optical state: no motion blur {3.2,6.2,7.2,10.2,12.2}		Instantiated Concrete Image example:

TOI's Pictorial Positioning		Center also various {1.4}	
TOI's 3D Orientation			
			Top(1.5)
TOI's Pictorial Distance	Both drones are represented in a pictorial distance of 1 nindan (equivalent to 1/9 total area of pictorial frame) {2.5}		



7.1.4 Develop the Training, Testing and Black Swan Validation Datasets

With the CuneiForms produced in the previous step, the ML developer can be tasked with gathering an appropriate number of instantiations constrained by each CuneiForm characterisation. Conventional wisdom can be used to collect and split the training and test. In our case study, we will focus on generating a small-size Black Swan instantiation of the CuneiForm H.36. we will also develop a Black Swan training dataset that would conform to the CuneiForm. The full Black Swan Scenario Dataset can be found in [120].



Figure 7.2 Example of a compliant image to CuneiForm in H.36 in Black Swan Validation set

Black Swan Validation dataset for CuneiForm H.36:

We generate a set of 1158 images that capture the CuneiForms: H.36, 37, 38, 39, 40, 41. We created the images by manually designing a drone's shell or fuselage and covering it with camouflaged tape. Then, we sought a training track zone where we took pictures near a fence (outside the train track zone) resembling the CuneiForms and what it might look like on the other side of the wall. Most of the images we generated were synthetic by placing the drone's pictures on a background that satisfied the CuneiForms. Figure 7.3 is an example image from the Black Swan dataset.

As you can see, the camouflaged drone landing on foliage is rather hard to detect. We tested a pre-trained model on drones on the Black Swan validation dataset, where the training set had no Black Swan training subset. The result is captured in Experiment 4 in the next Section. A random drone image model performed poorly in a customised operational domain under such Black Swan scenarios. This may indicate that training models on public data is not necessarily reliable for performing well during Black Swan event scenarios. Below is a snapshot of what the training Black Swan looked like:

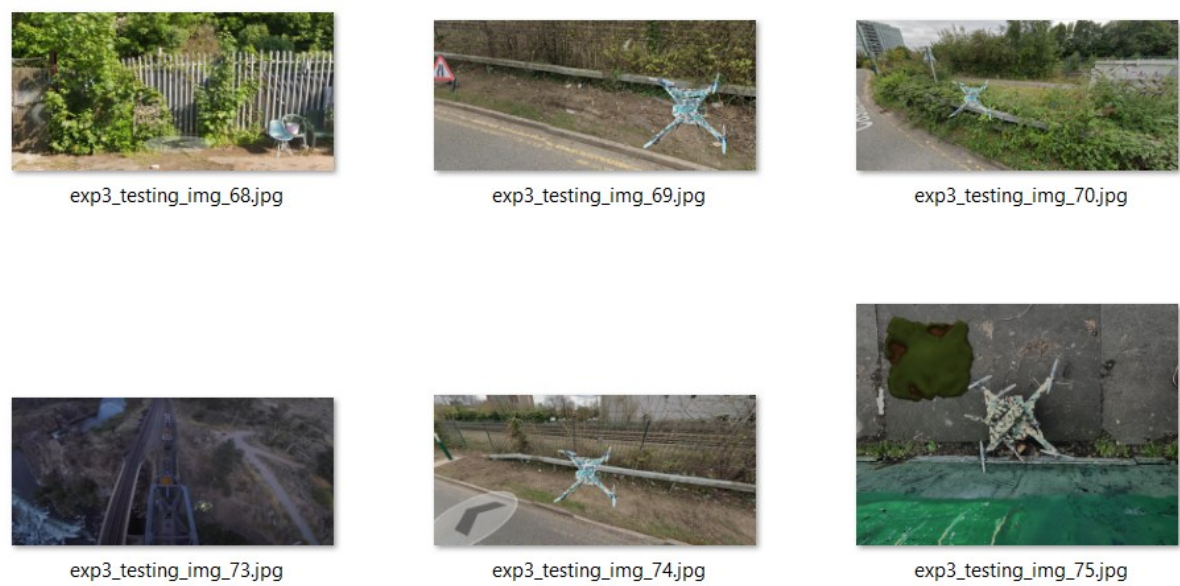


Figure 7.3 Example of compliant images to CuneiForm in H.36-41

7.1.5 More CuneiForms

In this Section and for the sale of demonstration only we produce several CuneiForms we will directly generate CuneiForms to facilitate the development of the dataset. We will refrain from explicitly defining the traceability definitions (the CuneiForm artefact), as the example of cuneiForm H.36 is adequate. This Section serves solely to illustrate how a set of CuneiForms can fulfil one requirement for a machine learning development dataset. We used those CuneiForms to create further instantiations for a dataset in Stage 6.

7.1.5.1 A CuneiForm-based non-typical Black Swan scenario

We will use those examples to diversify our Black Swan Scenario further. We develop ad hoc cuneiForms for the following criteria:

Table 7.4 Set of Black Swan CuneiForms examples

Safety Requirements
ML Safety-Training Requirement 4: Recognise Camouflaged drones on the ground
The Eagle Drone’s ML component shall be trained to detect, classify, and track adversarial drones using camouflage tactics on the ground by recognising subtle movement patterns, environmental inconsistencies, and spectral differences using AI-driven computer vision models.
ML development datasets requirement 3:

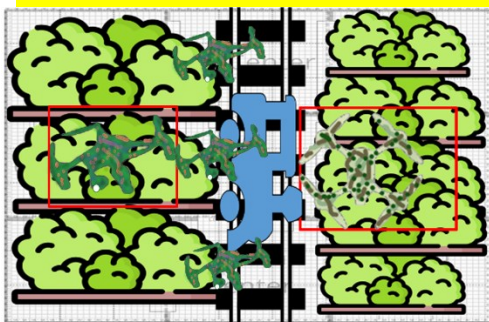
The Eagle Drone's ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drones using various ground camouflage tactics, such as blending in with, bushes, gravel, soil, trash or pavement next to track zone fence.

In order to: train the drone's machine learning algorithms to dynamically identify, track, and trace adversarial drones obscured by ground-level camouflage.

Pictorial problem situation context: A drone landing on the ground {1}. The drone has a camouflaged skin {2} landing at various distances from the track fence or train tracks or both {3}, trees {4}, local birds surrounding the landed drone {5} bushes {6}, gravel {7}, soil {8}, pavement {9} trash {10} Train {12}.

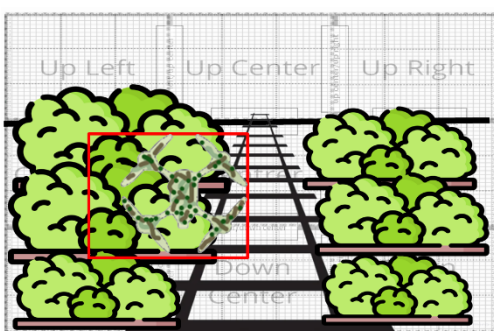
Black Swan CuneiForm Scenarios (Batch Instantiating Image

A)



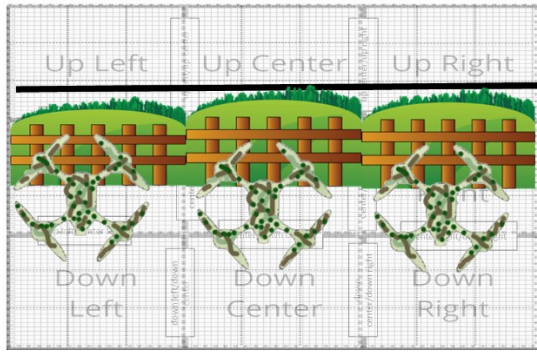
Black Swan CuneiForm H-37

Note, there are several TOIs in the CuneiForm. These are linked by an OR relationship. Meaning we do not expect to see all of them in one image, but rather any of those specific locations.



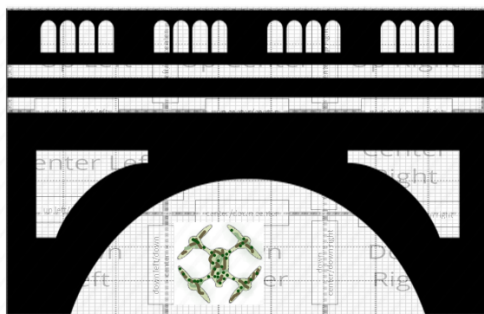
The straight black line represents the Pictorial Horizon

Black Swan CuneiForm H-38

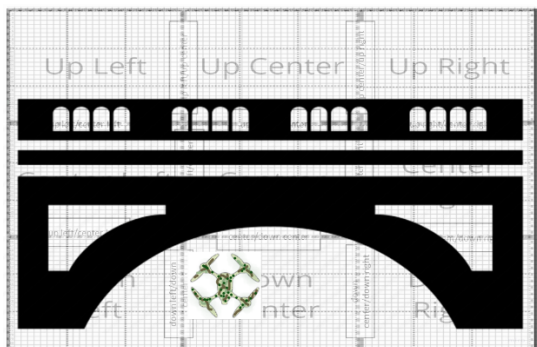


The straight black line represents the Pictorial Horizon. The TOI can be at any position as depicted.

Black Swan CuneiForm H-39

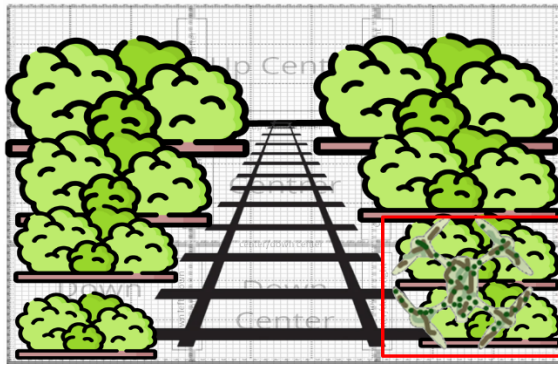


Black Swan CuneiForm H-40

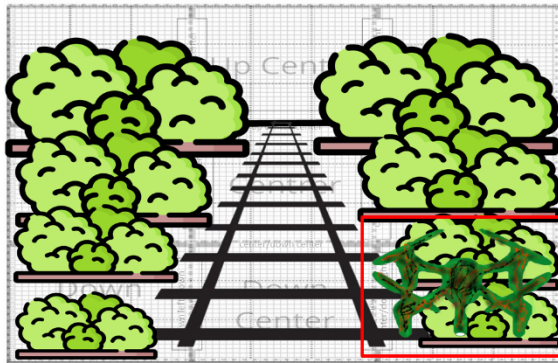


Black Swan CuneiForm H-41

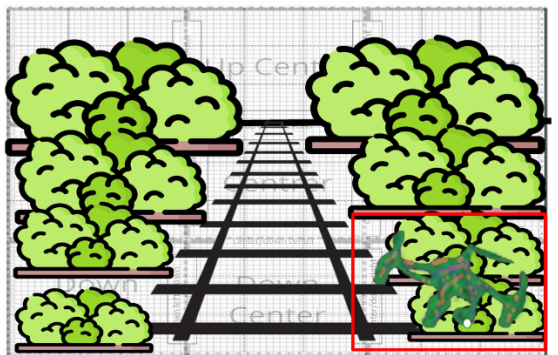
Black Swan CuneiForm Scenarios (Batch Instantiating Image B)



Black Swan CuneiForm H-51



Black Swan CuneiForm H-52



Black Swan CuneiForm H-53

7.1.5.2 A CuneiForm-based typical operations scenarios

In this Section, we will present a possible training syllabus based on carefully selecting which aspects of CuneiForms to focus on for a specific ML development Stage. For example, we will choose the TOI 3D orientations and define the split of training, validation and testing strategies.

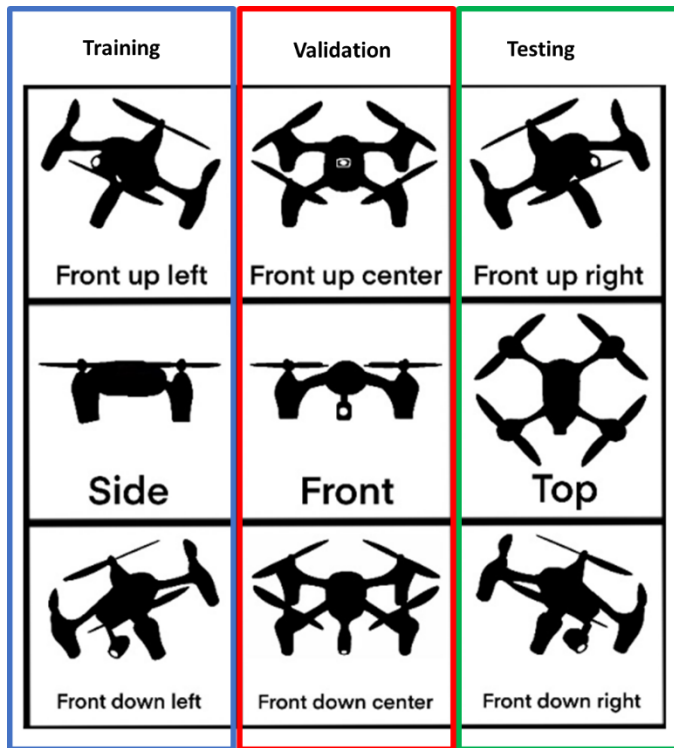


Figure 7.4 CuneiForm ML development strategy based on TOI 3D orientation.

Figure H.39 highlights our intended development strategy, to based on training the ML model to recognise specific orientations for the TOI (Front up left, Side, Front down left), then validating it against the following orientation variety (Front up center, Front, Front down center), and finally testing its performance against the following orientations (Front up right, Top, Front down right). For the next batch of datasets, we will fix the following general characteristics of the Cuneiform and only alter the orientations of the 3D orientation.

The CuneiForm strategy will be based on mitigating the following safety and dataset development requirements:

Table 7.5 Safety Training dataset development requirements

ML Safety-Training Requirements (Training Concept)	ML Safety-Training Dataset Requirement
ML Safety-Training Requirement 2: The Eagle Drone's ML component shall be trained to recognise adversarial drones approaching the train tracks and during their landing by the tracks side.	ML development datasets requirement 2: The Eagle Drone's ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drones (typical off-the-shelf drones) approaching the train, train tracks and attempting to land next to the train tracks boundary. In order to: train the drone's machine learning algorithms to dynamically identify, track, and trace adversarial drones during typical operations.

Since Black Swan scenarios and Typical Operations are somewhat related, albeit one has a camouflaged drone and the other has a non-camouflaged drone, we will reuse the pictorial situation articulation in Table 7.6, as a satisfaction articulation to ML development datasets requirement 2. Then we will reuse the same H.36 cuneiform characteristics, only changing the camouflaged icon into a non-camouflaged drone icon. The following Table 7.6 is a CuneiForm characterisation for the training syllabus over the typical operations domain on a train track:

Table 7.6 Typical operations CuneiForm characterisation

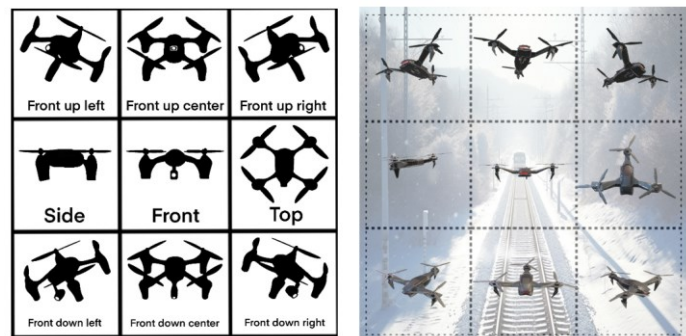
CuneiForm	Characteristic	Definition
Training Class		
Step 1) Define visible horizon attitude.	visible horizon	Elevated Level Horizon: Frame roll = $-1 \leq \text{ROLL} \leq 1$ Frame pitch = $1 < \text{PITCH} \leq 45$ Also , various elevated horizon attitudes are acceptable {1}
Step 2) Define all the TOIs and their aesthetic complexity. Then, generate abstract representative icons for the CuneiForm abstract image.		At least one adversarial drone {2} 
Step 3) Define TOI's motion trajectory and dynamic optical situations . Then, update the generated abstract representative icons for the CuneiForm abstract image.		Motion trajectory: static, no motion {2.1}. Dynamic optical state: captured without optical blur {2.2}.
Step 4) Define the background objects associated with TOIs and environmental scenery in the background of the CuneiForm. Then, generate abstract representative icons for the CuneiForm abstract image.		Train tracks zone or train tracks or both {3}, bushes and trees {4}, Side of the Train {5}.
Step 5) Define the background Objects' Motion situations and dynamic optical situations . Then,		Background objects' motion trajectory: static or in relative motion {3.1,4.1,5.1}

Chapter 7

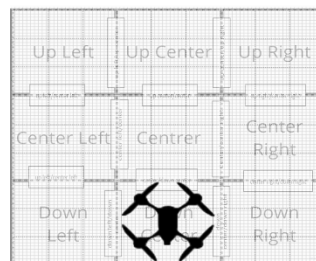
update the generated abstract representative icons for the CuneiForm abstract image. **Dynamic optical state:** no motion blur or with motion blur {3.2,4.2,5.2}

Step 6) Define TOI's positioning in the CuneiForm. Then, generate abstract representative icons for the CuneiForm abstract image. Down Center {1.4}

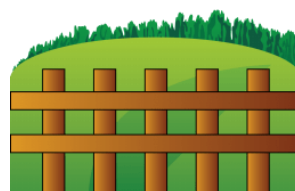
Step 7) Define TOI's 3D orientation. Then, update the generated abstract representative icons for the CuneiForm abstract image. Training TOI at (Front up left, Side, Front down left), validation TOI at (Front up center, Front, Front down center), testing TOI at (Front up right, Top, Front down right){1.5} the following is the definition of an acceptable definition of a minimum variety of possible TOI's 3D orientations



Step 8) Define the optical distance for each TOI in nindans. Then, update the generated abstract representative icons for the CuneiForm abstract image. The drone is represented in a pictorial distance of at least 9 nindans {1.6} below is a relative scale of TOI at 9 nindans pictorial distance.



Step 9) Design the relevant icons to produce the CuneiForm and give an example of an instantiating image Train tracks or train tracks or both {2}



bushes{3}



Side of train{4}



The following is a set of CuneiForms in Table 7.7 that enact the above requirements: typical drone and typical operational behaviour i.e. no black swan events:

Table 7.7 Set of typical operations CuneiForms satisfying Table 7.28

Safety Requirements

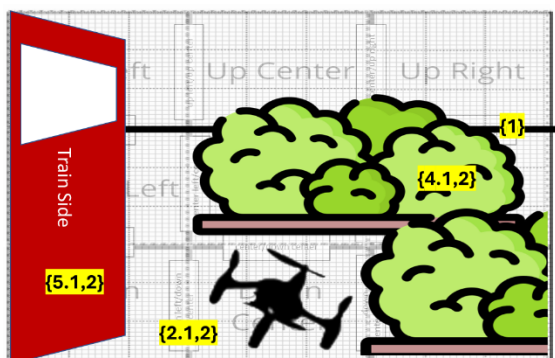
ML Safety-Training Requirement 2: The Eagle Drone's ML component shall be trained to recognise adversarial drones approaching the train tracks and during their landing by the tracks side.

ML development datasets requirement 1: The Eagle Drone's ML component Training Dataset shall provide the trainee model with a valuable minimum variety of adversarial drone shapes as they approach the track zone boundary fence and airspace from different angles and altitudes, including both direct and evasive approaches.

CuneiForm-based Training Scenarios

Instantiating Image (Total 1000 similar, not exactly the same, images generated)

Typical Operations CuneiForm H-42

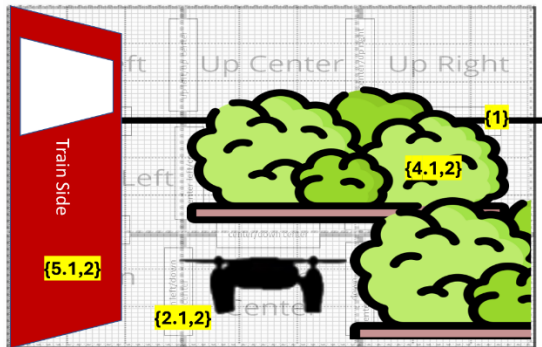


TOI at 3D orientation of: Front up left



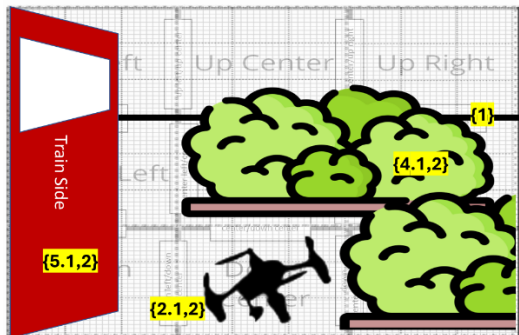
Chapter 7

Typical Operations CuneiForm H-43



TOI at 3D orientation of: side

Typical Operations CuneiForm H-44

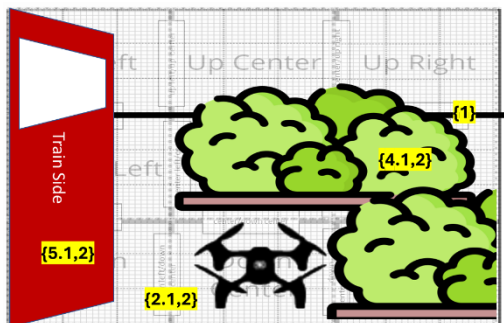


TOI at 3D orientation of: Front down left

CuneiForm-based Validation Scenarios

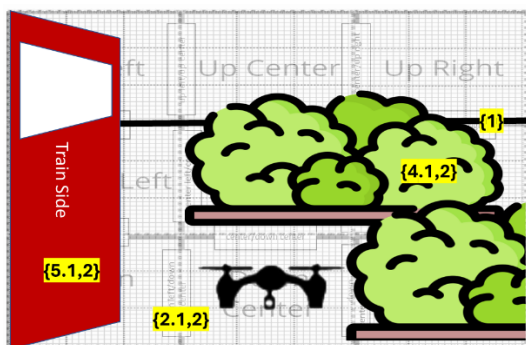
Instantiating Image

Typical Operations CuneiForm H-45



TOI at 3D orientation of: Front up center

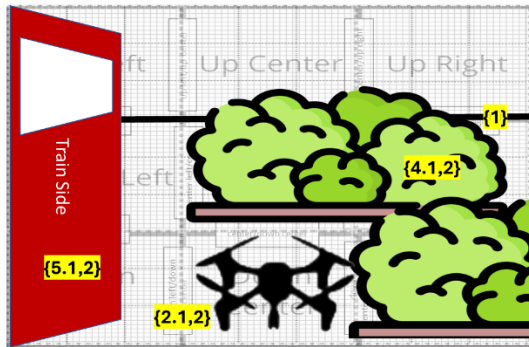
Typical Operations CuneiForm H-46



TOI at 3D orientation of: Front

Chapter 7

Typical Operations CuneiForm H-47

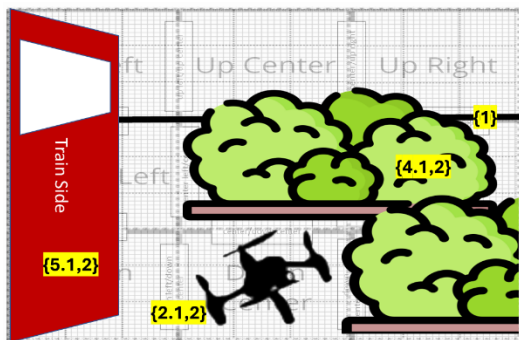


TOI at 3D orientation of: Front down center

CuneiForm-based Testing Scenarios

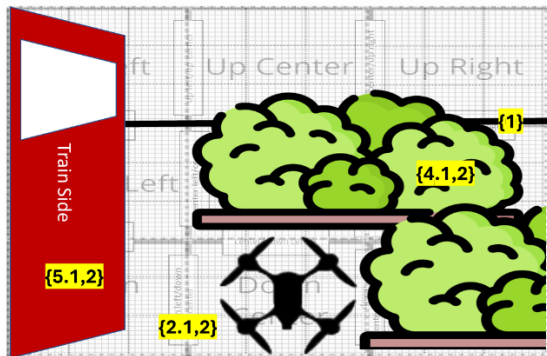
Instantiating Image

Typical Operations CuneiForm H-48



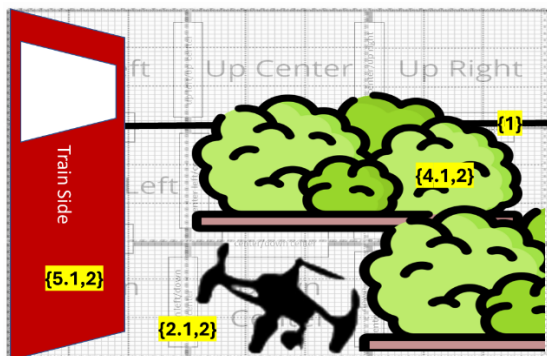
TOI at 3D orientation of: Front up right

Typical Operations CuneiForm H-49



TOI at 3D orientation of: Top

Typical Operations CuneiForm H-50



TOI at 3D orientation of: Front down right

7.1.5.3 A CuneiForm-based out-of-context example

To further increase robustness, the architect may use freely available images that carry useful labelled TOIs. However, if they use an existing online dataset with mixed context-unaware data, great care needs to be taken to ensure biases are picked up and examined. CuneiForm can provide a general abstraction to signify an out-of-context (out/context) dataset. It can be as simple as a white background and an image of a needed TOI. For example, in our case, we will specify the following CuneiForm in Figure 7.5:



Figure 7.5 H.54 Out-of-context CuneiForm of drones

7.2 Stage 6: Black Swan-driven ML Development and Testing⁴⁹

In this Section, we tailor our approach to demonstrate the reduction of risk ALARP during Black Swan scenarios, which we delineated in Section 4.9. We tailor the approach in line with guidance in AMLAS.

AMLAS mentions a distinct type of requirement:

- Performance
- Robustness

The primary difference in performance relates to prediction accuracy and precision. At the same time, robustness is associated with various real-world scenarios the model can generalise well over. The general expectation is that training a model on training data and testing it on out-of-statistical distribution (relative to the training dataset) yields lower performance than testing the same model within the training context. Thales had published a work confirming our expectation, referring to such datasets as either “InD” or “OOD.” [121]. The authors asserted that:

"Mathematically, when an ML-based model is successfully trained on a given dataset, then the model is expected to produce accurate predictions for unseen, InD test data. Conversely, the accuracy of the model is expected to fall when processing data instances drawn from OOD test data."

AMLAS guidelines suggest that models test datasets that are required to be different but “similar” for the sake of robustness assurance.

Why and how do we validate that a test dataset is different but similar?

Hence, we need to incorporate a similarity test for a meaningful safety-related purpose.

Our answer to this is: OOD, Black Swan scenario testing process to demonstrate that safety properties (characterised in CuneiForm) are also preserved during Black Swan data shifts.

The selected ML training syllabus (ML Config 7/Exp. 9.6) achieves **robustness** across typical operations and rare Black Swan scenarios. The dataset comprises 23,533 (non-augmented) images, split into training (71%, 16,788 images), validation (19%, 4,426 images), and testing (10%, 2,319 images). The training set balances 65% out-of-context images with structured scenarios: 11% Black Swan Batch A (CuneiForms H.36–41), 4% Black Swan Batch B (H.51–53), and 19% in-context typical operations (H.42–44). The validation set reinforces detection accuracy with 14% Black Swan A, 5% Black Swan B, and 36% typical operations (H.45–47), while the test set evaluates generalisation on 26% Black Swan A, 10% Black Swan B, and 51% typical operations (H.48–50).

⁴⁹ For more details see Section H.10

No pre-processing or augmentations were applied to isolate dataset effects. The model achieves exceptional performance, with 99% mAP@50 on testing and 98% on validation, demonstrating that integrating both Black Swan batches and phased typical scenarios into training ensures robustness across rare and routine conditions. This holistic approach validates the efficacy of context-aware dataset design for safety-critical systems, where high-stakes rare events demand proactive mitigation. Table 7.8 captures the Training Syllabus and the output model that is a Black Swan-trained model 9.6.

Table 7.8 Chosen training and testing syllabus with required performance

Experiment 9.6: All	Total Training:	Total Valid: 4426	Total Test: 2319
in dataset	16788 (71%) (no augm.)	(19%) (no augm.)	(10%) (no augm.)
See link	Out-of-context	Out-of-context	Out-of-context
Total: 23533 images	images: 10954 (65%)	images: 1969 (44%)	images: 268 (11%)
(without augmentation)	Black Swans A	Black Swans A	Black Swans A
	CuneiForm	CuneiForm	CuneiForm
	Scenarios: H.36, 37,	Scenarios: H.36, 37,	Scenarios: H.36, 37,
With augmentation:	38, 39, 40, 41: 1866	38, 39, 40, 41: 622	38, 39, 40, 41: 622
40321	(11%)	(14%)	(26%)
	Black Swans B	Black Swans B	Black Swans B
	CuneiForm	CuneiForm	CuneiForm
	Scenarios:	Scenarios:	Scenarios:
	H.51,52,53: 695 (4%)	H.51,52,53: 231 (5%)	H.51,52,53: 232
			(10%)
	Typical Operations	Typical Operations	Typical Operations
	CuneiForm	CuneiForm	CuneiForm
	Scenarios: H.42,	Scenarios:	Scenarios: 48,49,50:
	43,44: 3273 (19%)	H.45,46,47: 1604	1197 (51%)
		(36%)	
	Applied Pre-processing: Grayscale: Applied		
	Applied Pre-processing:		
	Outputs per training example: 2		
	Noise: Up to 2.39% of pixels		

Output Performance: mAP50/test 99%, mAP50/validation 98%.

Performance Requirements: mAP50/test 90%+, mAP50/validation 90%+.

The approach we took in this case study involved the following activities:

7.2.1 Training syllabus configurations

In the earlier Section, we experimented with multiple configurations of training syllabus (Table 7.9). We will use the experiments as examples of different development training configurations.

Table 7.9 Renaming experiments to training syllabus configuration

Experiment	Configuration
Exp.6	Config1 (White Swan)
Exp.7	Config2
Exp. 9.1	Config3
Exp. 9.2	Config4
Exp. 9.5	Config5
Exp. 9.4 (chosen)	Config6
Exp. 9.6 (augmented)	Config7

It will make sense how this configuration management process works in step 7 to provide a safety assurance audit trail demonstrating how safety requirements (using Black Swan CuneiForm) are systematically introduced and contribute to the assurance of the training syllabus to handle Black Swan scenarios. The following process instantiate the general process defined in Section 5.8.2.

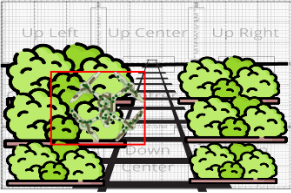
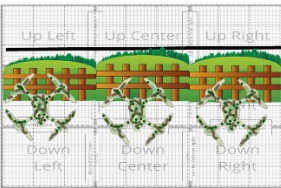
For a detailed justification and description of how the following process indeed is sufficient to validate regulator requirement regarding Black Swan operations, see Section E.9.

7.2.2 Step 1: Design CuneiForms

To achieve the assurance objectives, we need to develop three types of CuneiForms:

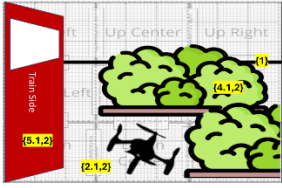
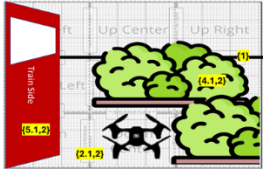
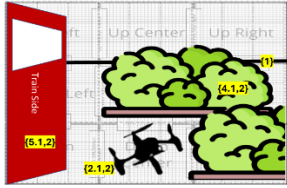
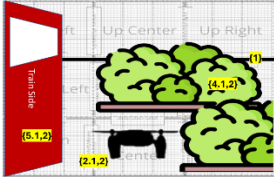
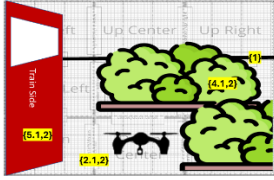
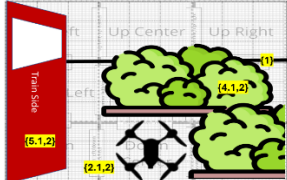
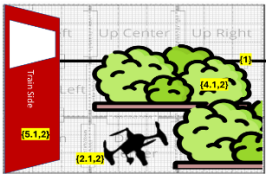
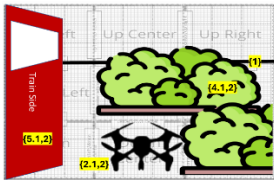
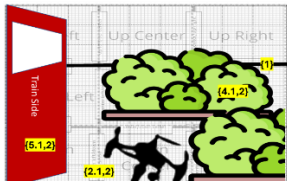
7.2.2.1 In-context Black Swans.

Table 7.10 Black Swan Scenarios Batch A and B CuneiForms

In-context, Black Swans CuneiForm (Batch B)	In-context, Black Swans (Batch A)
 H.51	 H.37
 H.52	 H.38
 H.53	 H.36

7.2.2.2 In-context Typical Operations.

Table 7.11 Typical operations CuneiForms

Training Scenarios	Validation Scenarios	Testing Scenarios
 H.42	 H.45	 H.48
 H.43	 H.46	 H.49
 H.44	 H.47	 H.50

7.2.2.3 Out-of-context CuneiForms (OOC dataset)

Out-of-context images are valuable datasets to demonstrate the model's ability to handle operational context shifts.



Figure 7.6 CuneiForm H.54 Out-of-context depiction of drones

The instantiated images of this generic depiction must not conform to any White or Black Swan CuneiForms. This is not a CuneiForm, only a pictorial depiction to be used as demonstration in a safety case or a dataset validation report. We sourced an out-of-context dataset from the open-source online datasets of drones.

Validation can be done by taking a sample from an out-of-context dataset and comparing it to all CuneiForms. It should be determined that they are not compliant.

7.2.3 Step 2: Validate CuneiForm compliance to SOC

Validate that CuneiForm correctly captures safety-training requirements. See the following Sections in this chapter, which capture the way we would validate CuneiForms tractability to safety requirements:

see Sections:

- 7.1.3
- 7.1.5

7.2.4 Step 3: Instantiate CuneiForms

generates datasets as per type. This activity will produce three types of datasets:

7.2.4.1 White Swan datasets generation

In-context typical operations and out-of-context useful datasets. Let's call it the White Swan dataset. We generated synthetic images according to the CuneiForm characterisation.

Final Dataset: Typical Ops (output of stage 3)

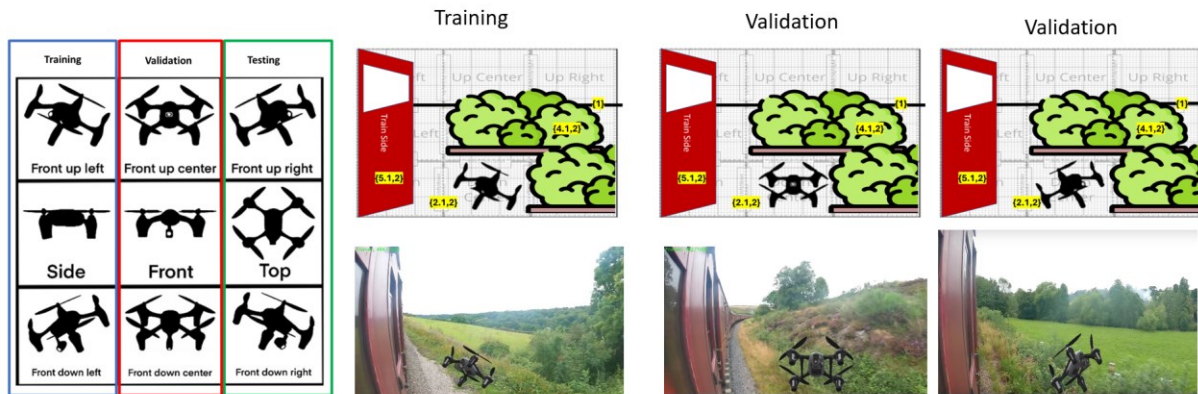


Figure 7.7 Typical operations of CuneiForms and instantiated images

7.2.4.2 Black Swan datasets generation.

Some of the Black Swans were generated by visiting local places where the scenery matches the CuneiForm, for example, H. 36. For other Black Swans, we synthetically generated images according to the scenery that matches the CuneiForm, and we included drones in them.

Final Dataset: Black Swans (output of stage 4)

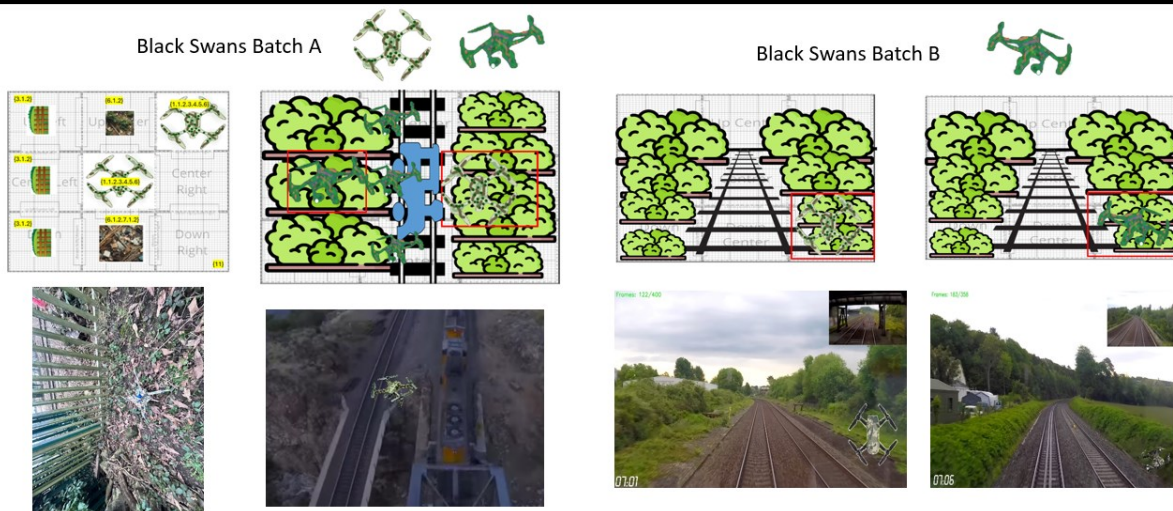


Figure 7.8 Black Swan images generation

We split the data generated in this step into two subsets:

- **The Black Swans set**, which demonstrates performance during unforeseeable data shifts relative to some initial development training set.

7.2.4.3 Out-of-context dataset generation

We gathered online repositories of datasets sourced from Roboflow Universe. See Section H.11.3.1. Below is an example of an out-of-context dataset we used:

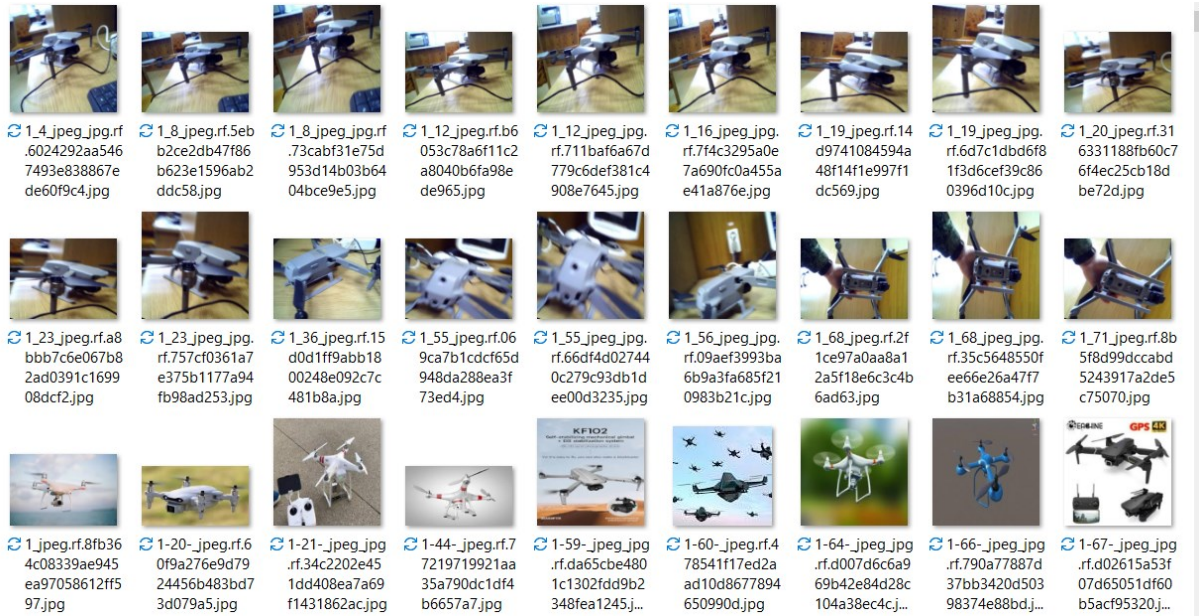


Figure 7.9 Examples of out-of-context dataset

Figure 7.9 demonstrates a sample of OOC data that does not conform to any CuneiForm.

7.2.5 Step 4: Validate Instantiated images compliance to CuneiForms

In this example, we will show an example validation process of how an image is compliant to a CuneiForm:

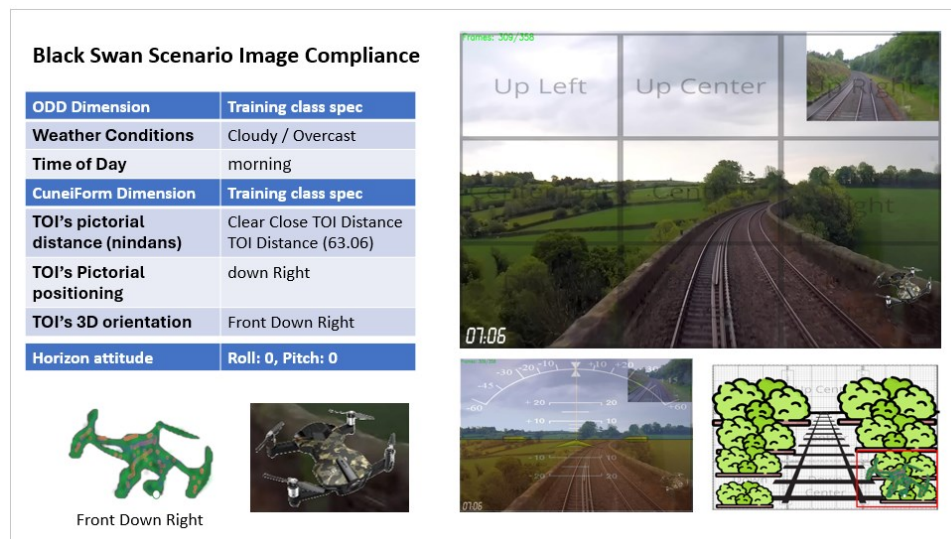


Figure 7.10 Validating a Black Swan image to a Black Swan (batch B) CuneiForm

The AVOIDDS case study provides a more detailed descriptive example (see Sections F.11, M.8.1 and Appendix D demonstrate the CuneiForm validation process and report). Figure 7.10 captures the CuneiForm dimensions we chose to validate as an example. The architect can choose which dimensions to validate. Starting with weather conditions, we are confirming that the weather front is as Staged in Stage 3B; cloudy conditions can be validated. We used a Python code to validate the pictorial distance of the TOI (see Section E.7.7 for a detailed definition of this quality). And for TOI's position (see Section E.7.6), the CuneiForm canvas shows clearly that the TOI is located in

down down-right quadrant as per the CuneiForm. 3D orientation (see Section E.7.5) can be clearly validated per the visual similarity between the TOI and the icon, including the texture. The PHI tool (see Section E.7.8.3) shows that the horizon is maintained as expected from the CuneiForm image.

7.2.6 Step 5: Dataset Similarity Testing

In this Stage, we try to distinguish which Black Swans are in or out of distribution with White Swans. Those that appear to be in distribution can be added to the White Swan dataset. At the end of this step, we have three distinct datasets:

- a. **White Swan development dataset:** represents the reference level of knowledge about the operational domain. The premise here is that if the ML model architecture is good enough, then this dataset should be sufficient. To empirically test this hypothesis, we need an unforeseen Black Swan to prove that both the architect and the level of knowledge about the operational environment stored in the datasets.
- b. **Black Swan testing dataset:** represents a possible unforeseen data shift from some initial reference knowledge about the operational environment. If the gathered information is sufficient, then surely the trained model should have no problem handling such scenarios.

We did not include all generated datasets in one go, because we still need to demonstrate that a Black Swan dataset represents a data shift that is challenging for a model configuration to handle. Unmitigated pictorial Black Swans are the real hazards we must systematically predict and train for.

We performed the similarity test in Experiment 7 and discovered that Black Swan batch A images are OOD. But we don't know whether they are mitigated Black Swans. Therefore, we need to develop the White Swan model and test it on the Black Swan to see if that is indeed the case.

In this step, we need to validate two key aspects:

- Dissimilar datasets are truly dissimilar relative to each other. We do so using similarity measures.
- Unforeseeable images are truly unforeseen relative to a set of CuneiForms.

What are the dataset-level Similarity Validation objectives that must be demonstrated and logged?

1. **Similarity Validation Objective 1:** The Black Swan dataset is reasonably OOD from the White Swan dataset.

What are the OOC Validation objectives that must be demonstrated and logged?

1. **Unforeseeability Validation Objective 1:** OOC datasets are unforeseen to be relevant to the train track problem, and that they all do not comply with any white or black swan CuneiForm.

7.2.7 Step 6: Develop the White Swan model

The first model will be trained using the White Swan dataset. In our case, the Config1 training syllabus will be our initial reference model, which we will use to incrementally and systematically introduce Black Swan datasets, those that are OOD with the White Swans.

7.2.8 Step 7: Black Swan operations training and testing

We then initiated the systematic training process by testing config1 on Black Swan batch A scenarios (see Table 7.10). Table 7.12 captures the process we followed⁵⁰, which clearly demonstrates a deliberate introduction and enhancement of performance during a potential Black Swan data shift, and thus provides an audit trail that assures claims of sufficient training, as demonstrated in the AMLAS safety case artefact [U]: Model Development Log. We set a performance acceptance criterion to be at 90% across all Black Swan images, and 98% across Black Swans, Typical operations and out-of-context test dataset.

Table 7.12 Black Swan-driven incremental ML development process to assure model performance under Black Swan operations

Step	Config	Training Size	Train + Validation strategy			Test / mAP@50		
			out/ context	Black Swan	Typical ops	Black Swan	Typical Ops	Out/ context
1	Config1 (White Swan)	10015	yes	0	yes	-	99%	-
2	Config2	10015	yes	0	yes	A: 13%	-	-
3	Config3	14961	yes	A	yes	-	99%	
4	Config4	14961+	yes	A	yes	B: 40%	-	-
5	Config5	16788	yes	A+B	yes	B:98%	-	-
6	Config6 (chosen)	16788	yes	A+B	yes	99%		
7	Config7 (augmented)	33576	yes	A+B	yes	99%		

⁵⁰ For a complete description

7.2.8.1 Config 1: White Swan Model

We developed the Config1 model and tested the model on Typical ops, which yielded 99% success. The visual characteristics of the training and testing datasets can be demonstrated in the following similarity test Venn diagram in Figure 7.11. The figure shows that the test set of typical operations is OOD compared to the training and validation sets. The fact that the model scored well (99%) should, in theory, indicate that the diversity of the out-of-context data is sufficient to handle data shifts like the one we have. But let's reevaluate what we are seeing:

- The OOD quality of the test is relative to out-of-context data, not our problem domain. This means the OOD, which is in-context with our operational domain, is not the same as the OOD scenario with respect to our specific operational domain. While it is great that the online dataset helped us to train a model that passed some typical operations, from an assurance during OOD, in-context scenario perspective, this does not provide any objective, trustworthy result.

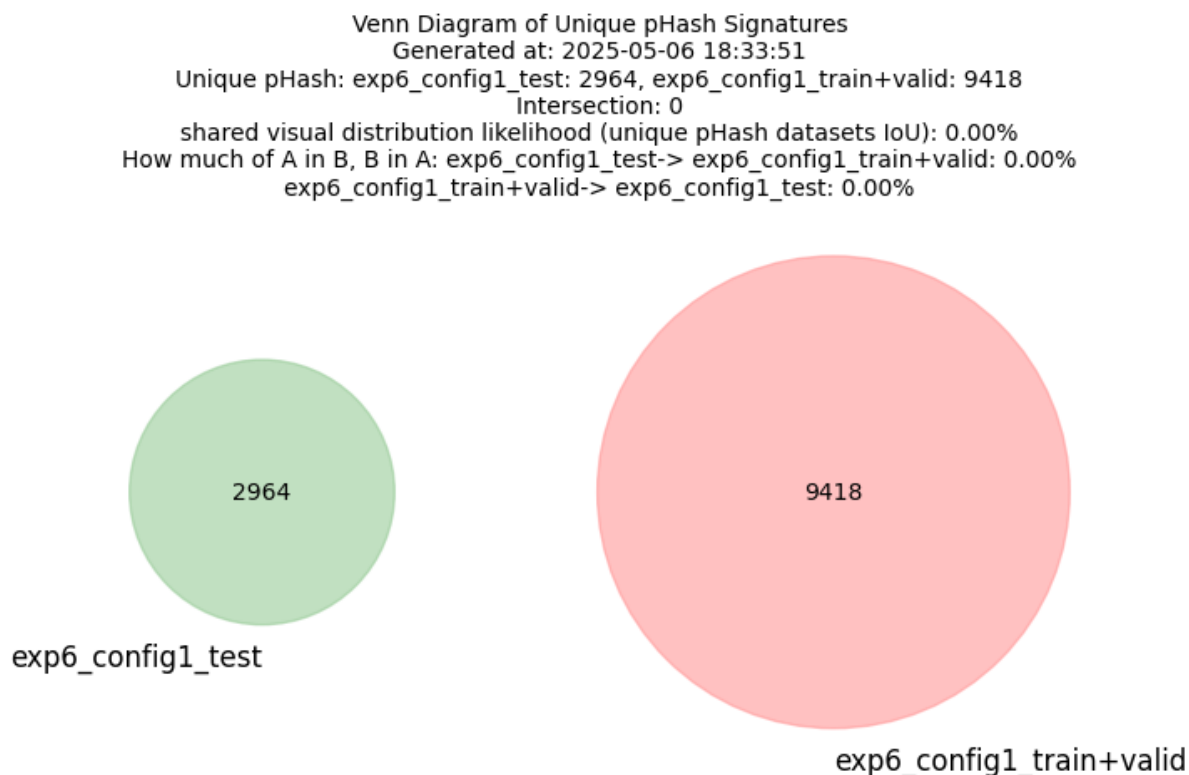


Figure 7.11 Config1 training and testing strategy

Figure 7.41 interpretation:

exp6_config1_test dataset: 2964 unique perceptual hashes found in this dataset, with a visual similarity (within the dataset) rate of approximately 5.06%.

exp6_config1_train+valid dataset: 9418 unique perceptual hashes found in this dataset, with a visual similarity (within the dataset) rate of approximately 27.80%.

Note: Visual similarity does not mean exact copies.

Common unique signatures: 0 unique signatures are common to both datasets.

shared visual distribution likelihood (unique pHash datasets IoU) between exp6_config1_test and exp6_config1_train+valid: 0.0%

comparing dataset without in-dataset similarities (considering only one hash number if repeated multiple times):

exp6_config1_test -> exp6_config1_train+valid: (0 / 2964) $\approx$ 0.00%.

exp6_config1_train+valid -> exp6_config1_test: (0 / 9418) $\approx$ 0.00%.

Ok, if this is good enough, let's give it out of distribution, Black Swan and see whether the results hold.

7.2.8.2 Config 2: Black Swan testing 1

Now we take the training of the model and test it on the Batch A Black Swan scenario (see Section 7.2.2). However, first we need to confirm that the test dataset (Black Swan A) is OOD with training and validation of config 1, which is now config 2. The following is the similarity test results within config 2 (exp 7):

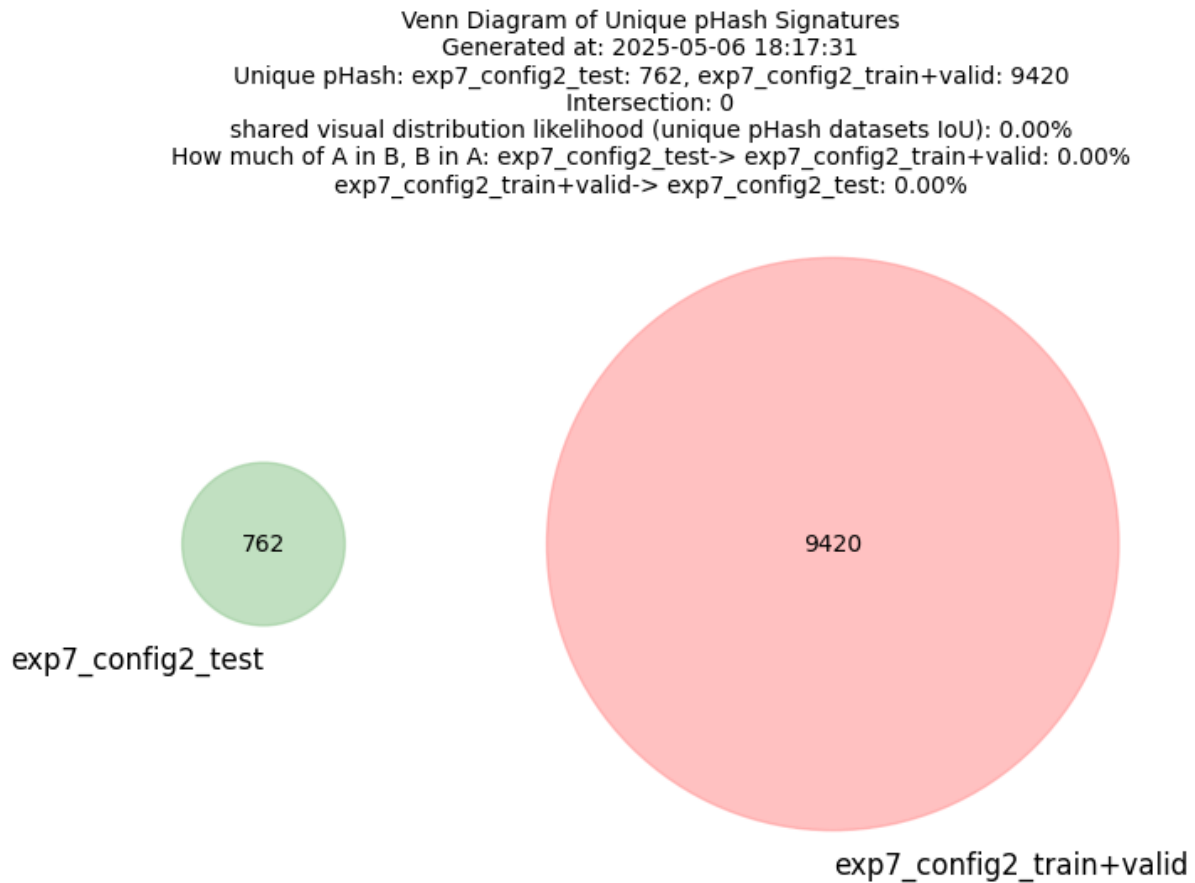


Figure 7.12 Config 2 similarity test result

With such validation, we can trust that Black Swan A's test set is OOD. Now, we need to check whether this type of OOD scenario (potential data shift in the operational domain) is also mitigated by config2's out-of-context diversity. In the config1 development, the test results over typical operations told us it should be trusted. Now, we have another example of a data shift scenario. Will the results be the same?

The config2 test result over Black Swan A testing revealed that the model's performance dropped to only 13%. This means that if we had believed the config1 test result, we would have been fooled into thinking it was good enough.

7.2.8.3 Config 3: Black Swan training 1

Now that we have identified an unmitigated Black Swan data shift, we can be confident that including it in our training for the config3 Stage will result in a model trained on a potential Black Swan scenario A. We retrain the model and test it on both typical operations and out-of-context data. We did so, and the result was a model with performance similar to config1, with the main difference being that we reassured the training with an actual unmitigated Black Swan scenario (as evidenced in the process).

7.2.8.4 Config 4: Black Swan testing 2

Now, we want to check the quality of the training syllabus we did in config 3. So we choose Black Swan batch B scenarios. But first, we need to validate that Black Swan batch B set is OOD with config3 training.

Figure 7.43 Interpretation:

exp9_2_test dataset: 1058 unique perceptual hashes found in this dataset, with a visual similarity (within the dataset) rate of approximately 14.42%.

exp9_2_train+valid dataset: 18264 unique perceptual hashes found in this dataset, with a visual similarity (within the dataset) rate of approximately 27.56%.

Note: Visual similarity does not mean exact copies.

Common unique signatures: 0 unique signatures are common to both datasets.

shared visual distribution likelihood (unique pHash datasets IoU) between exp9_2_test and

exp9_2_train+valid: 0.0%

comparing dataset without in-dataset similarities (considering only one hash number if repeated multiple times):

exp9_2_test -> exp9_2_train+valid: $(0 / 1058) \approx 0.00\%$.

exp9_2_train+valid -> exp9_2_test: $(0 / 18264) \approx 0.00\%$.

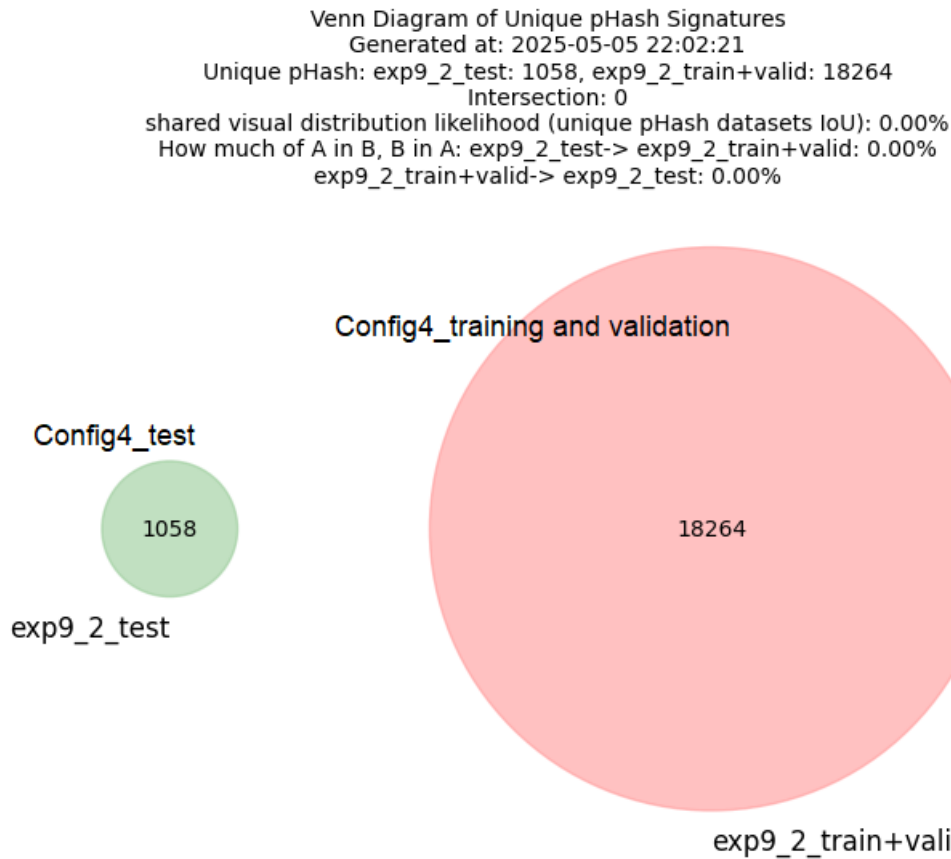


Figure 7.13 Config4 training and testing similarity test result

We tested the config3 strategy and got a 40% success rate over Black Swan B. This means the following:

- Black Swan B scenarios are unmitigated data shift, and though the diversity of config 3 improved in comparison to config 1 (from 13% to 40%) over data shift scenarios (in-context), it remains below the expected 90% performance rate.
- Now, we can see tangible improvement over Black Swan data shifts. This audit trail is essential to demonstrate a deliberate and systematic effort to ensure the model's safety during unmitigated, in-context data shifts.

7.2.8.5 Config 5: Black Swan training 2

Now we have successfully caught a second unmitigated Black Swan. We can add the training images to the training process, retrain, and test over the Black Swan B scenario. We did so and we got ourselves a model with two validated and demonstrated Black Swan data shifts and a trained model that performed 98% well on the Black Swan B data shift scenario.

7.2.8.6 Config 6: Testing on all types of data

Now that we have an assured model for Black Swan scenarios, we test the chosen training syllabus on a test set that includes Black Swan, typical operations, and out-of-context datasets. This gives us a more trustworthy general performance. In our case, we achieved a 99% success

rate. If we compare the model of config6 to the model of config1, we realise that the config1 model performed well on typical operations but failed on the Black Swan data shift. The config6 model has gained experience through challenging Black Swan scenarios and has achieved the same performance on a more meaningful test that included typical operations and Black Swan datasets.

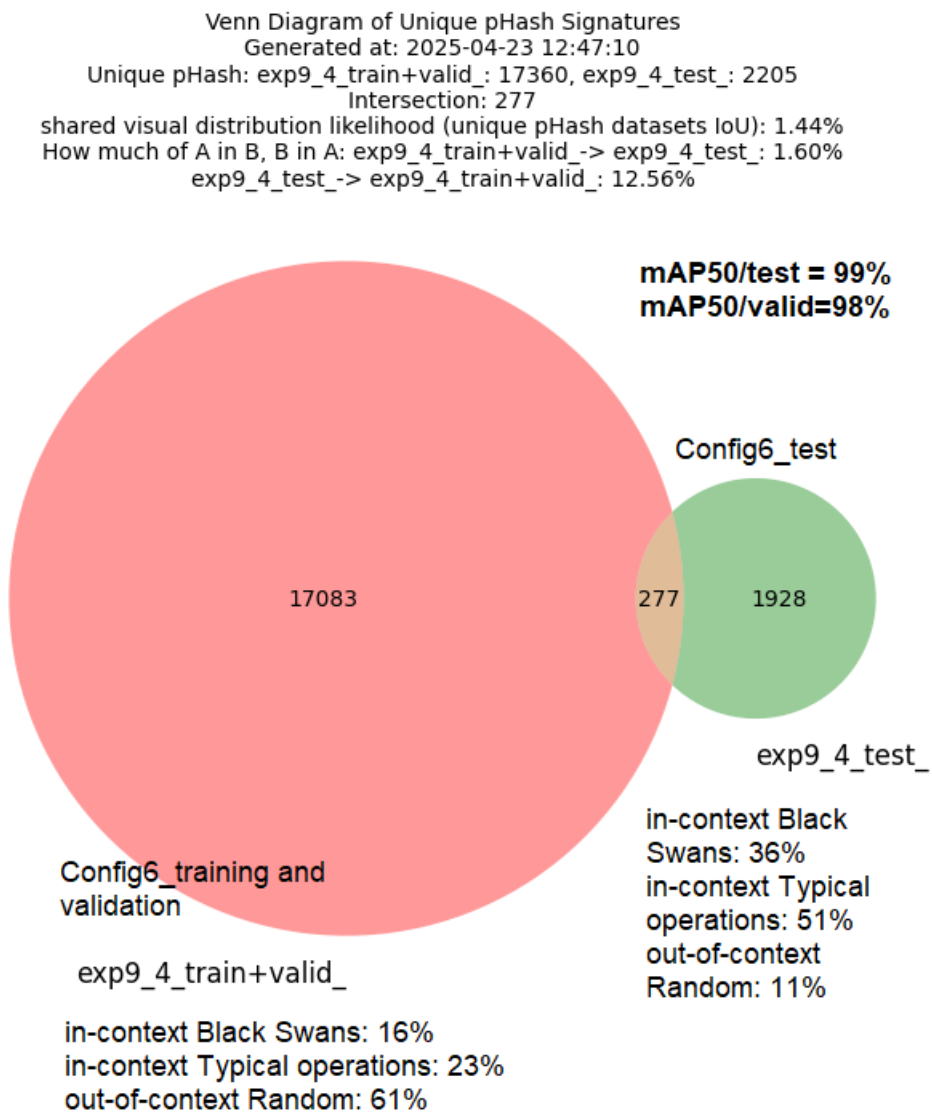


Figure 7.14 Chosen final training and test strategy configuration 6

Figure 7.44 Interpretation: The Similarity method used is imagehash phash.

exp9_4_train+valid_ dataset: 17360 unique perceptual hashes found in this dataset, with a visual similarity (within the dataset) rate of approximately 27.38%.

exp9_4_test_ dataset: 2205 unique perceptual hashes found in this dataset, with a visual similarity (within the dataset) rate of approximately 8.06%.

Note: Visual similarity does not mean exact copies.

Common unique signatures: 277 unique signatures are common to both datasets.

shared visual distribution likelihood (unique pHash datasets IoU) between

exp9_4_train+valid_ and exp9_4_test_: 1.4361260887598508%

comparing dataset without in-dataset similarities (considering only one hash number if repeated multiple times):

exp9_4_train+valid_ -> exp9_4_test_: $(277 / 17360) \approx 1.60\%$.

exp9_4_test_ -> exp9_4_train+valid_: $(277 / 2205) \approx 12.56\%$.

The final performance shows that the model is tested in scenarios where:

- 12.56% roughly 13% of them had similar experience in the training (thus foreseeable by the training and validation).
- Roughly 87% of the testing scenarios were OOD (unforeseeable) by the training and validation.

If we isolate the foreseeable 277 images from the testing set and re-test the model, we do not expect significant reduction in performance.

Henceforth, the model is trained sufficiently to handle Black Swan operations and unexpected data shifts (which are visually similar to 87% of the testing dataset).

7.2.9 Step 8: Dataset augmentation for adversarial attacks robustness (Config 7)

We now have a training syllabus with a proven track record of enhanced epistemic coverage, enabling participants to experience various scenarios, including rare and challenging ones. To add the icing on the cake, we augment the training set. This would further increase resilience against adversarial scenarios.

At this Stage, we have systematically demonstrated deliberate training and testing during Black Swan scenarios. However, there still residual risk that would impact our trust in the ability of the model to perform as intended during scenarios which are:

1. in-context of the operational domain.
2. Unaccounted for in any dataset up to config 7. However, we know that the diversity of the training examples extends to those invisible scenarios yet considered part of the training dataset.
3. The model training is expected to be safe in those unseen scenarios.

Hence, we need to show empirically that the model can indeed perform safely to some InD datasets to Black Swans. Thus providing systematic evidence of compliance to expected risk reduction. Hence, step 9 is our final destination.

7.2.10 Step 9: Develop final model

Now we can retrain and test the augmented datasets and have a comprehensive audit trail with objective evidence for constructing a justifiable assurance case that the risk of model failure during typical operations and some rare data shift scenarios has been lowered ALARP.

7.2.11 Step 10: Develop AMLAS Safety Case

We then integrate the following artefacts into the AMLAS artefacts. Stage 4 of Model learning in AMLAS would be the target safety case Section, which documents the assurance activities of Stage 6 in our approach. The following is our interpretation of how the process above captures AMLAS artefacts:

Table 7.13 Integrating model training activities in AMLAS safety case

AMLAS artefact	Example evidence from the process
[H] : ML Safety Requirements	7.2.3 Step 2: SOC Compliance
	7.2.5 Step 4: CuneiForm Compliance Validation
[N]: Development Data	Table 7.10 Black Swan Scenarios Batch A and B CuneiForms.
[O]: Internal Test Data	Table 7.11 Typical operations CuneiForms.
	7.2.4 Step 3: Instantiates CuneiForms
	7.2.6 Step 5: Dataset Similarity Testing
	Figure 7.15 CuneiForm Consolidation report
[V]: ML Model	7.9.10 Step 9: Develop final model [see the following model 9.6]
[X]: Internal Test Results	
[Y]: ML Learning Argument	See chapter 8 and Appendix L

[U]: Model Development Log

Table 7.12 Black Swan-driven incremental ML development process to assure model performance under Black Swan operations

7.2.12 Dataset Holistic Consolidation

The total number of images used (before augmentation) is 23,533, represented by 15 CuneiForms. We successfully traced the entire dataset to safety requirements and clearly explained and validated that safety is indeed considered in the training syllabus of the model.

By holistic, we mean a non-mathematically rigorous approach; however, if the project team deems it practical to employ mathematical means, then the more the merrier. In our approach, we opt for holistic consolidation, based on sampling the data and comparing it to the claimed CuneiForm through a manual inspection method. Because the CuneiForms are tightly traced to safety requirements, regulators can trust that by comparing a CuneiForm to data, they are in fact checking compliance against safety requirements themselves. If the models are indeed used for the public and transparency is a requirement, CuneiForms could be presented along with sample data that instantiates them.

To perform consolidation, we propose a CuneiForm Consolidation Report (Figure 7.15) that clearly shows how datasets are traced to safety requirements. We utilised the Roboflow platform to demonstrate the capture of safety requirements and CuneiForms, which are ready for manual inspection by the quality assurance team or a regulator. Roboflow can be a valuable platform to provide holistic summarisation to ensure the capture of requirements. For example, we used “tags” to name each image that corresponds to a CuneiForm. See Figure 7.15:

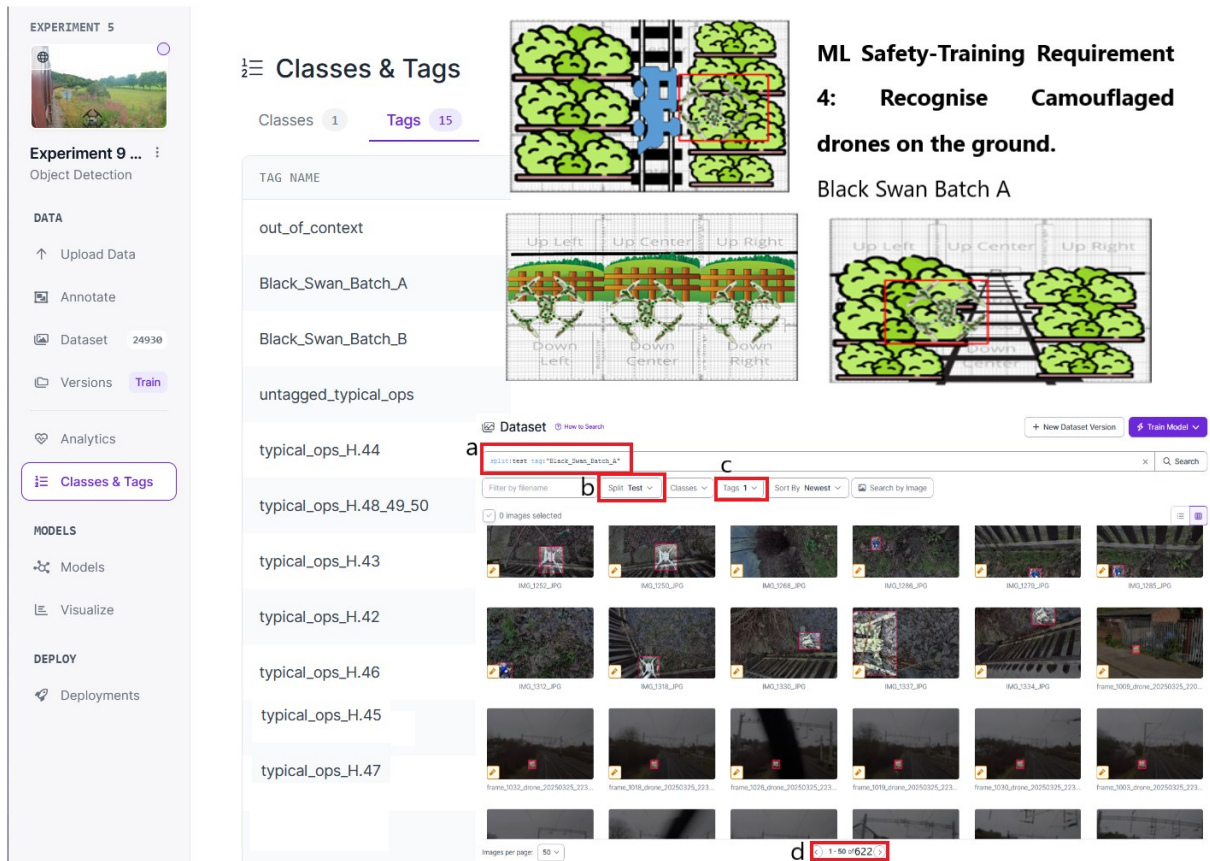


Figure 7.15 CuneiForm consolidation report

A regulator can utilise filters by tags and view a corresponding CuneiForm manually. “a” defines the CuneiForm given ID, in this case: split:test tag:“Black_Swan_Batch_A”. “b” shows the dataset split, in this case “test”. “c” shows the selected tag which only displays the instantiated images that meant to instantiate the CuneiForms. And “d” indicates the number of instantiated images, in this case is 622 images are used to test the model over Black Swan A.

7.3 SECoT Validation Report for Unsafe Train Tracks case study

This report implements the SECoT validation process in section 5.10.

7.3.1 Purpose of the Report

This report aims to validate the SECoT step “H.4.1.4 Step 1.4) Define the Supra-Complexes and Their PrimePs” ensuring that its predictive thinking processes and outputs adhere to the established general systems rules and follow the method’s intended objective.

The SECoT step under validation identifies two supra-complexes within the train tracks problem domain, Train Network and Adversarial Scheme, each associated with a distinct Prime Purpose (PrimeP). Validating this SECoT output is crucial to confirm that the methodology correctly

organises the system elements into supra-complexes with consistent, clearly stated PrimePs, thereby guiding subsequent design and solution development.

7.3.2 Scope of Validation

- **Focus:** The validation is limited to the single SECoT step “Define the supra-complexes and their PrimePs” as documented in Table H.4 and summarised in Architect Assertion 1.4.6.
- **Boundaries:** The assessment excludes detailed downstream solutions, mitigations, or expansions of these supra-complexes beyond verifying that the SECoT step (1) correctly applies the relevant general system rules (D, E, F) and (2) satisfies the completion criteria (1.4.5).
- **Related References:** This validation specifically examines integrating with the “General Systems Rules” (especially D, E, F) and aligning with the AIC-based method for clarifying purpose and system scope.

7.3.3 SECoT Output Validation

7.3.3.1 Validation Criteria

The following criteria (Pass/Fail) are applied to judge whether the SECoT step output is acceptable:

1. Alignment with General System Rules

- **General Rule D:** Has the architect defined supra-complexes as “larger collections of complexes”?
- **General Rule E:** Are the purpose statements consistent, stable, and clear across scenarios?
- **General Rule F:** Does each supra-complex have a well-defined Prime Purpose in the sense of a “Primary Purpose” that influences design?

2. Clarity of Supra-Complex Scope

- Each supra-complex must logically group relevant elements to achieve its chosen PrimeP.

3. Correctness and Completeness

- The final list of supra-complexes must encompass all visible elements.
- The step must not omit significant elements or add extraneous ones that do not fit the scenario.

4. Step Completion Criteria (1.4.5) Preservation

- The step's deliverable is considered complete when at least one supra-complex is identified to represent each group of coexisting elements, and each is assigned a PrimeP.

5. Absence of Unwarranted Solutions

- This phase must remain descriptive about system structure and purpose rather than specifying a design or solution.

Table 7.14 SECoT Validation acceptance criteria

Validation Step	Evaluation Criteria	Pass/Fail	Notes
Step 1: Application of Gen. Rules D, E, F	Are supra-complexes formed properly (Rule D)? Are the defined PrimePs consistent (Rules E, F)?	Pass	Two supra-complexes, Train Network and Adversarial Scheme, are logically separated, each with a stable, domain-consistent PrimeP.
Step 2: Clarity of Supra-Complex Scope	Do these supra-complexes collectively include all identified elements?	Pass	The Train Network includes the train, fence, vegetation, and power lines. The Adversarial Scheme consists of the adversarial drone. Comprehensive coverage.
Step 3: Correctness and Completeness	Are the key elements (train, fence, drone, etc.) correctly allocated?	Pass	All visible elements are accounted for; each belongs to a supra-complex that logically groups them by purpose.
Step 4: Step Completion Criteria 1.4.5 Preserved	Is each supra-complex assigned a PrimeP?	Pass	Train Network → “transport people/goods safely” Adversarial Scheme → “disrupt train operations” Aligned with the method's instructions.
Step 5: No Solutions Included	Does it avoid design solutions?	Pass	The final output remains at the level of problem domain structuring, no solutions or mitigations overshadow the definition process.

7.3.4 Summary and Next Steps

7.3.4.1 Summary of Findings

1. General Systems Rule Application

- The step properly operationalises Rule D by defining supra-complexes that unite sets of systems with shared overarching objectives.
- Rules E and F are upheld: each supra-complex's PrimeP is well defined and consistent across potential scenarios.

2. Thoroughness in Coverage

- All major observable elements (train, tracks zone, vegetation, fence, power lines, drone) are included and correctly assigned to either the Train Network or Adversarial Scheme supra-complex.

3. Clarity for Future Stages

- The logic behind choosing these two supra-complexes clarifies how future design or risk analysis can proceed (e.g., focusing on how adversarial elements threaten the train network's primary purpose).

Therefore, the SECoT step "Define the supra-complexes and their PrimePs" meets the validation criteria, with no unmet requirements or contradictions discovered.

7.3.4.2 Recommended Actions

1. Incorporate into Next Steps

- Use these supra-complex definitions to guide scenario-based hazard analyses or system design choices, especially in subsequent steps focusing on emergent interactions.

2. Validate Worst-Case Assumptions

- Validate or refine the assumption that the "Adversarial Scheme" might be more widespread than a single drone (i.e., a multi-drone orchestrated threat).

7.3.5 Appendices (optional)

- (If desired) Diagrams showing the two supra-complexes and their sub-elements.
- Additional references or notes on re-scope definitions, if the "Adversarial Scheme" requires broadening.

7.3.6 SECoT Validation Review Process

- **Report Author/Architect:** Haider Al-Shareefy [University of Southampton], responsible for generating the SECoT validation content.
- **Reviewer 1:** Prof Michael Butler [University of Southampton], validating alignment with General Rules D, E, F.
- **Reviewer 2:** Dr Son Hoang [University of Southampton], verifying coverage of elements and clarity of PrimePs.
- **Reviewer 3:** Prof Hamid Asgari [Thales], performs customer acceptance of validation report output.
- **Sign-off:** The above signatories affirm that the **SECoT validation** process was carried out thoroughly, and the results are valid as of this report date.

By signing below, the reviewers confirm that **H.4.1.4 Step 1.4) Define the supra-complexes and their PrimePs** of the SECoT is validated as meeting the defined criteria, ready to support.

Signatures:

- Architect: _____ (Date: _____)
- Reviewer 1: _____ (Date: _____)
- Reviewer 2: _____ (Date: _____)
- Reviewer 3: _____ (Date: _____)

End of SECoT Validation Report

7.4 Reflection from Applying CuneiForm-based assurance framework (Stage 5)

7.4.1 Reflection 7.1: We managed to bridge the semantic gap using CuneiForms

The main findings from the Unsafe Train Tracks Case Study's Stage 5 focus on developing a safety-driven training and validation strategy for our final ML model, for example, in Eagle Drone detecting adversarial drones (see Sections 7.1 for the Eagle Robot, M.6 for AVOIDDS case study). Key points to make about our experience in implementing Stage 5 of the approach:

Demonstrating the Reduction of Epistemic Uncertainty: The study identified various uncertainties faced by both the architect and the trainee ML model concerning real-world scenarios. Reducing these uncertainties to an acceptable level (As Low As Reasonably Practicable - ALARP) is critical. The CuneiForm we developed allowed us to directly validate datasets capturing the main pictorial features in a traceable manner. Table H.42B captures the traceability between

CuneiForms Development: Generating CuneiForms has helped us capture diverse training scenarios that mimic real-world situations, thereby demonstrating how we accounted for safety requirements in the robustness of the ML model in the AMLAS safety case.

Real-world Training Focus: The approach prioritises modelling real-world scenarios into training of models, over superficial dataset augmentations. It distinguishes between “epistemic uncertainty” (related to the model’s comprehension of the world) and “aleatoric uncertainty” (random variations in data). We distinguished three types of operational pictorial scenarios:

- In-context typical operations.
- In-context Black Swan operations.
- out-of-context practical additional scenario for robustness.

Training Classes Definition: A structured process is established to define visible attitudes, target objects of interest (TOIs), and their interactions within the pictorial context. Various steps include understanding the environment, potential influences, and the adversarial drones’ appearance complexity. We then linked those pictorial dynamics to safety requirements, thus tracing that the architect’s intent is preserved in the dataset.

Comprehensive Scenario Development: The construction of the problem context considers environmental variables such as proximity to track fences, types of camouflage, and surrounding objects, ensuring the training data reflects a range of practical scenarios that the drone may encounter.

7.4.2 Reflection 7.2: CuneiForm Pictorial Complexity Field limit

While applying our method, we realised that it is possible to end up with a very complicated CuneiForm characterisation. Such a situation can negatively impact the purpose of this approach, which is to provide a means to reduce the architect’s cognitive burden in understanding the semantic link between textual descriptions (requirements) and pictorial representations and ensuring that a dataset faithfully captures these requirements. Therefore, we found ourselves in a situation where we had to make an engineering judgment based on the criterion that determines the information limit for designing a CuneiForm. Our method employs heuristic numbers, such as “no more than 3 TOI’s aesthetic aspects” or “9 orientational views.” Those numbers are chosen based on cognitive psychology theories [122]. In this Section, we will describe the rationale behind these numbers.

The selection of three distinct aesthetic design aspects is a deliberate constraint that leverages human cognitive processing limits while optimising the dataset’s clarity and utility. This decision is informed by several mental and psychological theories [123], including Miller’s “magic number” [9], which posits that humans can comfortably process five to nine chunks of information.

Furthermore, this approach aligns with research in human-computer interaction and machine learning, which suggests that a manageable chunk of information of a maximum of 4 categories can significantly improve the quality of information comprehension [124]. Therefore, A subset of three informational aspects is comfortably within the range that promotes cognitive ease and clarity, especially when instantiating images are validated against their corresponding parent abstract CuneiForm. Similarly, we selected 9 possible varieties of orientations and zones within the range of the 7 ± 2 rule.

In this case, one may ask why we chose an upper limit of 3 aesthetic aspects for a single CuneiForm, as opposed to choosing 9, as we do in the TOI's orientation and positioning zone. We chose a lower limit of 3 in the case of aesthetic aspects because recognising and validating aesthetic characteristics of a TOI in an image is more descriptive and conceptually subtle for the human perception capacity to detect than visually recognising the general location of a TOI as it may put more cognitive burden on the human validator. More cognitive overload means more mistakes, which can lead to potential failures.

7.4.3 Reflection 7.3: Addressing the problem of sufficiency

One of the primary challenges encountered during the design of CuneiForms involved addressing the issue of "proving sufficiency of training syllabus. " How can we demonstrate to ourselves and to the regulator that the set of CuneiForms is adequate for training a trustworthy model for a safety-critical application? Although we did not endeavour to tackle this challenge in our research, we can assert a limited contribution towards its resolution. While a strict, formal, absolute mathematical proof of completeness (ensuring that not a single, unforeseen, microscopic interaction or property has been overlooked in the vast universe applicable to our model's training) is practically unattainable and leads to a philosophical conundrum, we successfully implemented a ***rigorous and auditable process*** that approaches this ideal with a reasonable degree of engineering confidence. In other words, the architect may not make a strong claim of the sufficiency of training and testing. Still, they can indeed showcase and explain what was included to allow for a negotiated sense of trust in the value of the chosen variety with the regulator.

It is possible to compute the size of the available configurations of CuneiForms that can potentially be designed by determining the cardinality of all possible combinations of CuneiForm characteristics. The size of such a set is exceedingly large, as it involves factorial calculations. Suppose we assert that a sufficient training set encompasses all possible combinations of CuneiForm characteristics. In that case, we will ultimately arrive at an impractically extensive set of datasets, as each CuneiForm constitutes a definition of a dataset comprising multiple images.

Therefore, perhaps instead of pursuing an infeasible (indeed, factorially explosive) exhaustion of every conceivable CuneiForm configuration, we adopt a risk-based, coverage-guided approach: we identify high-impact dimensions of variability, enumerate representative instantiations along each axis, and employ stratified sampling to construct a tractable yet diverse syllabus. By coupling this design with continuous monitoring (where emerging gaps are detected through systematic Black Swan-driven testing, error-mode analysis, and stakeholder feedback) we create an auditable, evolving corpus whose documented provenance and demonstrable valuable (traced to requirements) coverage support defensible claims of sufficiency to regulators. In this way, the architect trades absolute mathematical completeness for a pragmatic, evidence-driven process that both minimises unforeseen failure modes and fosters a negotiated trust in the model's readiness for safety-critical deployment.

7.4.4 Mid-air Collision problem Thought Experiment

In Section M.1.1, Appendix M, we started with a thought experiment examining whether the authors' approach to data gathering may have helped them to consider Black Swan scenarios such as hot-air balloons. The validation findings presented in Appendix D address the efficacy of the AVOIDDS dataset when applied to a perception system designed to manage the complicatedness of aircraft landing scenarios, particularly during the launch of hot-air balloons. While AVOIDDS serves as a benchmark, the analysis indicates that significant improvements are essential for the dataset to support real-world, safety-critical applications in aviation AI.

7.4.4.1 Reflection 7.4: CuneiForm validation process helped us to discover gaps in existing dataset

CuneiForm validation process helped to discover gaps in the AVOID training sample dataset.

7.4.4.1.1 Evaluating Cloud Types Coverage

The evaluation of cloud coverage indicates a balanced representation across various categories, asserting that the perception system encounters a balanced distribution of cloud experiences. Notably, the dataset meets the ALARP (As Low As Reasonably Practicable) criteria concerning balanced cloud type exposure. However, gaps remain wherein certain cloud types, specifically high cirrus clouds, are underrepresented. This inadequacy, coupled with the presence of broken or scattered cloud images, underscores the need for a more robust dataset to ensure comprehensive training. By “robust” we mean a dataset whose epistemic coverage does not suffer from misalignment with real-world complexity too fast

after deployment due to unexpected data shift. Such quality is directly impacted by the architect epistemic uncertainty and effectiveness of their predictive thinking processes.

7.4.4.1.2 Analysis of Black Swan Event Handling

Despite the balanced cloud coverage, the dataset demonstrates an alarming disparity in temporal exposure, with 47% of instances occurring at night or midday. This imbalance indicates a strong bias and poses a significant risk of hazardous unpredictability during emergent black swan events. It reveals that the dataset fails to meet ALARP standards in terms of time-of-day representation. Such deficiencies potentially compromise the reliability of the perception system, especially in scenarios characterised by unexpected and high-risk situations.

7.4.4.1.3 Pictorial Distance Coverage

In the Mid-air collision Case Study, the findings concerning pictorial distance reveal an essential imbalance, with 50% of training instances concentrated in only 20% of the required coverage. Moreover, instances categorised as dangerously close distances are entirely absent. The limited exposure risks unforeseen black swan behaviour, escalating uncertainty in the perception system's operational integrity. Thus, the dataset does not satisfactorily fulfil the ALARP requirements for pictorial distance coverage.

7.4.4.1.4 TOI Positioning and Horizon Attitude Analysis

In the Mid-air collision Case Study, the analysis of the dataset further uncovers significant deficiencies in covering TOI (Traffic of Interest) positioning, where 60% of the perception experiences are confined to only 20% of possible quadrants. Importantly, specific critical positional experiences are entirely missed, exemplifying the dataset's lack of comprehensiveness in this domain. Similarly, the horizon attitude analysis reveals that 60% of instances are limited to just 23% of the potential horizon attitudes. The absence of diverse horizon experiences limits the perception system's ability to respond to varied and complicated scenarios effectively.

In conclusion, while AVOIDDS provides a foundational framework for perception system training, the identified gaps in cloud type representation, time-of-day exposure, pictorial distance coverage, and TOI positional and horizon attitude analysis indicate an urgent need for dataset enhancement. Without these improvements, we cannot trust that the dataset will enable the preservation of safety properties during emerging Black Swan scenarios.

7.5 Reflections on Black Swan-driven training pipeline

We conducted a series of experiments in Section 7.2 and H.10. We found that explicitly including in-context Black Swan scenarios in training is critical for robustness. However, they are very good as an initial validation for the training syllabus. They can give us an indication of the quality the diversity of the dataset. Furthermore, we need to provide deliberate evidence in the safety case about how we truly caught Black Swans that were in-context and out-of-distribution with respect to some initial dataset. So we must be more strategic about how and when to start training for Black Swan operations. We recommend devising a process (Figure 5.11, Section 5.8) that describes how we validate the capture of Black Swans, and then intentionally trains and prepares the model for them. The following are the results of our tests:

7.5.1 Reflection 7.5: Impact of Black Swan Inclusion in Training

Hypothesis 1 Validated: Models trained without Black Swan examples (e.g., Experiment 4, Group 1) showed poor performance on Black Swan test sets (24% mAP50). Conversely, models explicitly trained on Black Swan scenarios (e.g., Experiment 3, Group 3) achieved significantly higher performance (67% mAP50), confirming that Black Swan inclusion in training is critical for robustness.

Dataset Similarity Tests revealed that Black Swan test sets (e.g., Exp. 4/10) had **0% overlap** with training/validation data, and poor performance of the model confirmed their out-of-distribution nature.

7.5.2 Reflection 7.6: Limitations of typical operations and out-of-context training

The experiment showed us that models trained solely on out/context images (e.g., Exp. 1, 4, 5) performed well on in-distribution tests (e.g., 97% map50 in Exp. 1) but failed catastrophically on Black Swans (24–35% map50 on Black Swan A dataset).

Furthermore, increasing dataset size (Exp. 10: 85,677 training images) with mixed typical and random data marginally improved Black Swan performance (35% mAP50) but remained insufficient, highlighting that **diversity of out/context alone cannot mitigate Black Swan risks**.

7.5.3 Reflection 7.7: Testing Strategy Implications

Testing Must Simulate Rare Events: Traditional validation metrics (e.g., mAP50 on typical data) are insufficient for safety-critical assurance; dedicated Black Swan testing is required.

- **Typical Operational Testing is Misleading:** Models validated on typical scenarios (e.g., Exp. 5) achieved high mAP50 (93%) but provided no insight into Black Swan readiness.
- **Black Swan Testing is Essential:** Dedicated Black Swan test sets (e.g., Batch A/B CuneiForms) exposed vulnerabilities undetectable through traditional validation, enabling targeted improvements (e.g., Exp. 3's 30% Black Swan training boosted performance to 67%).

7.5.4 Reflection 7.8: Role of Stage 4 in Assurance

The Stage 4 process identified **unmitigated data shifts** (e.g., Exp. 4/10's 0% similarity between training and Black Swan tests). This validated the utility of CuneiForms for predicting operational risks and guiding dataset design.

Stage 4 Enhances Trustworthiness: By generating CuneiForms to simulate Black Swan operations, Stage 4 enabled proactive risk identification and dataset refinement, aligning with ALARP (As Low As Reasonably Practicable) risk management principles.

7.5.5 Reflection 7.9: Demonstrating lowering of risks during Black Swans ALARP

Developing the model is one thing; providing objective, evidence-based safety assurance that the desired properties will be preserved during unforeseeable scenarios is a completely different mindset. To demonstrate that the training and validation strategies are effective to enable the development of a model that will perform as expected during unforeseen scenarios (assuring safety requirements are preserved during Black Swan operations), we need:

- A practical definition of “unforeseeability” (which we did in Section E.9.2).
- A practical method that translates such a definition into an assurance approach (see Section 5.8, and its implementation in Section 7.2, including Black Swan assurance audit trail in Table 7.12).

We needed to show the following:

1. Pictorial scenarios are visually (pictorially) unforeseen by the model. We interpret this as a dataset being out of distribution with respect to the training and validation.
2. To validate that an out-of-distribution dataset contains scenarios that represent data shift with respect to training and validation, a trained model needs to fail in such datasets. In other words, these are scenarios unmitigated by the diversity of the training and validation of images. Meaning the epistemic diversity of the training and validation is not sufficient to enable a model that can generalise as expected.

3. If we demonstrate that we can catch those unmitigated data shift scenarios and include them in the training, we demonstrate that we have reduced the epistemic risks of the model (due to ignorance of the operational domain complexity) would have been lowered ALARP.

The three principles are the basis of our Black Swan-driven assurance process of ML models during Black Swan operations, which we used in Stage 6.

These findings underscore the need for context-aware ML pipelines that prioritize rare but high-impact scenarios during both training and testing. The results provide a framework for building assurance arguments in safety-critical applications, demonstrating that robustness to Black Swan events is achievable through deliberate dataset design and targeted validation.

7.6 Reflections with regard SECoT Validation Process

The 5.10 SECoT Validation Report process (in Chapter 5) provides a structured framework for systematically evaluating Systems Engineering Chains of Thought (SECoTs), ensuring a rigorous and repeatable validation process. This template helps assess whether an SECoT output prediction effectively identifies emergent behaviours and contributes to the safety assurance of autonomous systems by exposing the architect's predictive thinking processes that led to engineering judgements. The report is divided into key Sections, including defining the purpose and scope of validation, establishing quantifiable validation criteria, and documenting the results in a structured evaluation table. It also includes a summary of findings, outlining whether the SECoT successfully applies the general systems rules (or any abstract principle involved in the decision-making process) to uncover previously unrecognised hazards or scenarios. It also suggests recommended actions for refining SECoT outputs or integrating into system safety design.

7.6.1 Reflection 7.10: The SECoT validation report was a useful tool

To maintain credibility and consistency, the validation process follows a peer review protocol, requiring at least one additional reviewer to affirm the integrity of the validation. This template ensures that SECoTs remain transparent, methodologically sound, and adaptable across various systems engineering applications, making them a tool for autonomous system safety validation. Especially for artefacts in SACE and AMLAS that require demonstrating a thorough justification of decisions made, for example, [Y]: AS Design Justification in SACE framework.

Chapter 8 Safety Case Development

This chapter integrates the AIC systems approach with AMLAS and SACE assurance approaches to develop safety cases for our AIC systems. While we have successfully applied this methodology to two case studies, our work remains incomplete in fully addressing the safety assurance requirements, without the standard patterns to establish and document the trustworthiness of our design. Our comparisons to SACE and AMLAS revolve around two primary considerations:

- AMLAS and SACE as design processes (as we evaluated in Section 5.9, See Figure 5.2).
- AMLAS and SACE as a safety case construction pattern.

This chapter mainly focuses on using them as a Safety Case construction pattern. Thus, capturing our processes in response to the artefacts. Some Stages of the AMLAS and SACE frameworks extend beyond the core scope of this PhD research and have not been fully developed. For

example, we are not considering Stages 7 and 8 in SACE or Stages 5 and 6 in AMLAS. Consequently, we are constructing a partial safety case to demonstrate how the output artefacts of our approach can be mapped into SACE and AMLAS assurance patterns. Surprisingly, the AIC systems approach aligns well with the AMLAS and SACE methodology. Initially, we had not planned to align our strategy with either of them. The alignment between the AIC systems approach and the safety assurance methods was an emergent output. In Appendix L we define the full detailed mapping between AMLAS and SACE artefacts and AIC artefacts in case studies. This chapter attempts to summaries and explain the work we have done. In this research, we only focused on the functional requirements for the ML component rather than the non-functional requirements (for example, robustness constraints or specific ML performance metrics).

8.1 Developing Partial SACE Safety Case

A comprehensive table that captures the SACE artefacts and their definitions can be found in Section L.4.9. See Appendix L for the complete set of Safety Case patterns.

The SACE process assumes that the solution is already decided. This is where the AIC Systems Approach differs from SACE; the AIC approach is problem domain-driven rather than solution-driven. If a client requests a specific system to be developed, the AIC architect assumes that there is a problem, and that no viable solution exists. They would then execute Stage 1 to validate their understanding of the problem and the efficiency of the solution.

In this Section, we will demonstrate the natural synergies between our approach and SACE, with our approach being the “know-how” for each SACE Stage, while the SACE Stage provides the “what” to consider. Another point to make here, SACE process follows the convention of pursuing safety and security concerns with lesser deliberation to pursuit Black Swans. Our approach stands on the concept that it is wiser (for long-tailed problem domains where Black Swans dominate (see sections 4.1.1, 4.1.2, 4.1.3, E.1) for the architect to intentionally pursuit the prediction of all possible Black Swans and along the way, the architect would pick up safety and security concerns.

8.1.1 SACE Stage 1: Operating Context Assurance

The first Stage of SACE aims to define the operating context of the Autonomous System (AS) by establishing the Operational Domain Model (ODM). To compare how our approach achieves SACE Stage 1 objectives, we formulate the following comparative table:

Table 8.1 AIC systems processes to SACE Stage 1 design process

SACE Stage 1 activities	AIC systems approach to activities
-------------------------	------------------------------------

Activity 1: Define Autonomous Capabilities of AS

- Define the scope of AS capabilities.
- High-level user requirements and objectives of the AS operation.

Activity 3: Define and Validate the Operating Scenarios

- The operating scenario of an AS involves a specific collection of actions and events that the AS can undertake within a defined environmental context.
- Resilience is a critical factor in designing ASs, as these unexpected operating scenarios demand that the system can safely adjust to unforeseen events.

Activity 2: Define and Validate the Operational Domain Model (ODM)

- defines the scope of the operation of an AS within which the AS can be demonstrated to be acceptably safe.
- Explicitly define assumptions related to ODM where AS is expected to encounter.
- After creation, the ODM will be validated to ensure that both the model's scope and detail level are appropriate.

AS system-level capabilities are described in Stage 2 via the Architect High-Level Solution Prescriptions. However, to do so, we need to trace capabilities to actual problematic activities in the problem domain. Hence, we opted for problem articulation without considering any solutions at Stage 1.

Although Stage 3A directly defines AS operating scenarios, Stage 2 provides the autonomous solution-level capabilities that could satisfy activity 3.

SACE mentions rare impactful scenarios without providing detailed know-how on how to derive those scenarios. However, they emphasise the importance of predicting them. We have a dedicated process (Stage 4) that accomplishes this.

Stage 1 comprehensively models the operational domain and outputs AIC problem domain models, assumptions and factors. All of them describe SACE ODM.

Stage 3B further details the conditions of the operational environment.

The set of AIC interactions outputted from “Predictive Thinking Pipeline 3: Predict the Emergence of AIC Complexity Field for Detailed Operational Scenario Articulation” defines the set of operating scenarios in which the AS will participate.

Those scenarios are validated during Stage 2, when the architect discusses selected problematic scenarios with stakeholders.

Activity 4: Instantiate AS Operating Context Assurance Argument Pattern. Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.4.1 maps AIC artefacts to Activity 4 Assurance argument patterns (see table L.4.1.1 for the Eagle Robot, and L.4.1.2 for AVOIDDS case study). Table L.2 maps SACE artefacts to AIC Sections.

8.1.2 SACE Stage 2: Autonomous System Hazardous Scenario Identification

The second Stage of SACE focuses on identifying, validating, and mitigating hazardous scenarios that may arise in the AS's operational environment. The AIC Systems Approach contributes through HazTOPs analysis, which provides a rigorous framework for discovering and managing hazards. To compare how our approach achieves SACE Stage 2 objectives, we formulate the following comparative table:

Table 8.2 Mapping AIC systems processes to SACE Stage 1 design process

SACE Stage 2 activities	AIC systems approach to activities
Activity 5: Define interactions between the AS and the environment: Accurately understanding the AS's interactions with operating environment elements is vital for recognising hazardous scenarios.	Stage 1 comprehensively models the operational domain and outputs AIC problem domain models, assumptions and factors. All of them describe SACE ODM.
Activity 6: Analyse AS Decisions.	Stage 3A then prompt the architect to model the AS's AIC behaviour.
Activity 7: Identify AS Hazardous Scenarios.	From both activities, the architect defines decisions and interactions with respect to the problem's complexity.
	As for hazards associated with AS, Stage 3A HazTOPS captures safety hazards and threats. It also goes beyond hazardous scenarios and includes opportunities (exploitation of environmental affordances).

Activity 8: Validate the AS Hazardous Scenarios:

The documented AS Hazardous Scenarios will undergo validation to ensure that:

All AS Hazardous Scenarios are recognised.

The assessment of hazardous scenarios comprehensively considers all relevant factors:

- Agents
- ODM Elements
- Decision points and their associated outcomes.

Each AS Hazardous Scenario is clearly defined with sufficient detail to identify mitigating safety requirements.

Activity 9: Instantiate AS Hazardous Scenarios Assurance Case Pattern

Section 5.10 defines a validation process wherein SECoTs uphold semantic relevance to prior decompositions of the design. This process encompasses a comprehensive review of the Chain-of-Thought steps involved and any articulation. We depend on an expert peer review to substantiate the validation processes.

Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.4.2 maps AIC artefacts to Activity 9 Assurance argument patterns (see table L.4.2.1 for the Eagle Robot, and L.4.2.2 for AVOIDDS case study). Table L.3 maps SACE artefacts to AIC Sections.

8.1.3 SACE Stage 3: Safe Operating Concept Assurance

This third Stage of SACE focuses on developing a Safe Operating Concept (SOC) for the AS. This involves defining what constitutes a sufficiently safe operation and validating that all safety constraints and requirements have been met. In the SACE Stage, the focus is on system-level safety as part of the operational environment. This is exactly what predictive thinking pipelines 1,2, and 3 in AIC Stage 3A focus on. The AIC Systems Approach provides structure to ensure this process is rigorous and well-justified. If SACE Stage 2 was the discovery of hazards, then SACE Stage 3 is about mitigating those hazards into structured safety requirements.

To compare how our approach achieves SACE Stage 3 objectives, we formulate the following comparative table:

Table 8.3 Mapping AIC systems processes to SACE Stage 1 design process

SACE Stage 3 activities	AIC systems approach to activities
Activity 10: Define Safe Operating Concept of AS: - Define SOC: The SOC shall include a definition of system-level safety requirements that specify how the AS must behave to mitigate hazardous events properly. - Define Reduced Operating Domain (ROD): additional constraints on the ODM that reduce the operational scope of an autonomous capability under certain conditions. - By operating in a ROD, the intention is that the AS is not exposed to scenarios that may lead to hazardous events under those defined conditions. Activity 11: Validate Safe Operating Concept for AS. The AS safety requirements established in the SOC effectively mitigate all identified hazardous scenarios. The AS safety requirements are unambiguous in their guidelines. Strategies outlined in the RODS and reduced AC's adequately address the identified hazardous scenarios under specified conditions.	Stage 3A then prompts the architect to model the AS's AIC behaviour, predictive thinking pipeline 1 directly map AS AIC behaviours to the operational problem domain AIC behaviours. In predictive thinking pipeline 2, the architect models how the rest of the environment would react to the designed AS behaviour. In this Stage, the architect can define ROD constraints in response to environmental behaviours. From both activities, the architect defines decisions and interactions with respect to the problem's complexity. Section 5.10 defines a validation process wherein SECoTs uphold semantic relevance to prior decompositions of the design. This process encompasses a comprehensive review of the Chain-of-Thought steps involved and any articulation. We depend on an expert peer review to substantiate the validation processes.

Activity 12: Instantiate SOC Assurance Argument Pattern.

Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.4.3 maps AIC artefacts to Activity 12 Assurance argument patterns (see table L.4.3.1 for the Eagle Robot, and L.4.3.2 for AVOIDDS case study). Table L.4 maps SACE artefacts to AIC Sections.

8.1.4 SACE Stage 4: Safety Requirements Assurance

This fourth Stage of SACE ensures that safety requirements are correctly captured, validated, and justified. The AIC Systems Approach ensures that the process remains structured and traceable. In this SACE Stage, the Safe Operating Concept SOC level safety requirements are decomposed to more subsystem-level requirements.

To compare how our approach achieves SACE Stage 4 objectives, we formulate the following comparative table:

Table 8.4 Mapping AIC systems processes to SACE Stage 4 design process

SACE Stage 4 activities	AIC systems approach to activities
Activity 13: Define Safety Requirements • Characterise system-level Safety Requirements. • Decompose system-level Safety Requirements into subsystem-level requirements. This activity examines how sub-system safety requirements are outlined at each level of decomposition within the system development process, ensuring they accurately represent the <i>intentions</i> of the safety requirements set at the prior level of decomposition (SOC).	SECoTs, in general, clearly demonstrate how the architect's intent is maintained across all design Stages. The General Systems Rules and predictive thinking process show the background thoughts involved in the architect's assertion (prediction), which lead to articulating particular requirements. Stage 3A then prompts the architect to model the AS's AIC behaviour. Predictive thinking pipeline 1 directly maps AS AIC behaviours to the operational problem domain AIC behaviours.

In predictive thinking pipeline 2, the architect models how the rest of the environment would react to the designed AS behaviour. In this Stage, the architect can define ROD constraints in response to environmental behaviours.

Also, predictive thinking pipeline 4, step 4 extends the SOC requirements in the earlier step by decomposing them into sub-system-level concrete requirements. We use the 4wnh (4-whats-and-hows) method to decompose the SOC requirements.

Activity 14: Validate Safety Requirements.

- Check that each of the higher-level requirements can be satisfied if the Safety requirements for this tier are correctly implemented.

Section 5.10 defines a validation process wherein SECoTs uphold semantic relevance to prior decompositions of the design. This process encompasses a comprehensive review of the Chain-of-Thought steps involved and any articulation. We depend on an expert peer review to substantiate the validation processes.

Activity 15: Instantiate Safety Requirements Argument Pattern.

Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.4.4 maps AIC artefacts to Activity 15 Assurance argument patterns (see table L.4.4.1 for the Eagle Robot, and L.4.4.2 for AVOIDDS case study). Table L.5 maps SACE artefacts to AIC Sections.

8.1.5 Stage 5: AS Design Assurance

The fifth Stage of SACE focuses on ensuring that the AS concrete design meets safety and operational requirements. The AIC Systems Approach ensures that design choices are fully justified and validated through structured predictive analysis.

To compare how our approach achieves SACE Stage 4 objectives, we formulate the following comparative table:

Table 8.5 Mapping AIC systems processes to SACE Stage 5 design process

SACE Stage 5 activities	AIC systems approach to activities
Activity 16: Create Design at tier n Given a set of safety requirements defined at a tier of decomposition of the AS design, a design for that tier shall be produced that ensures those safety requirements can be met.	In this PhD we focused as the Dataset Design tier. Stage 5 defines the CuneiForms that specify the training, validation and testing datasets driven by SOC and Safety-Training concept requirements.
Activity 17: Review and Justify Design at tier n Justification shall be provided for how each of the key design decisions made at tier n helps ensure that the AS can meet the safety requirements. In particular, this should consider how the design decisions relate to the AS's robustness and fault tolerance and how this supports the system's safety.	As part of the CuneiForm generation process, the architect follows a SECoT that helps them to articulate SCO into a practical problem context. This process shows how the architect decided what to include in the dataset to satisfy safety requirements. Also, in Section 7.8 (I.10), the architect validates the CuneiForms, which is a form of quality review process of design decisions conducted in Stage 5. All SECoTs used in the approach provide evidence of the thought steps involved in making predictions and engineering decisions.
Activity 18: Instantiate Design Assurance Argument Pattern.	Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies. Section L.4.5 maps AIC artefacts to Activity 18 Assurance argument patterns (see table L.4.51 for the Eagle Robot, and L.4.5.2 for AVOIDDS case study). Table L.6 maps SACE artefacts to AIC Sections.

8.1.6 Stage 6: Hazardous Failures Management

The final Stage of SACE ensures that all hazardous failures are identified due to deviation from intended behaviours, analysed, and mitigated. The AIC Systems Approach provides structured processes to manage failure scenarios and validate mitigation strategies systematically.

To compare how our approach achieves SACE Stage 4 objectives, we formulate the following comparative table:

Table 8.6 Mapping AIC systems processes to SACE Stage 5 design process

SACE Stage 5 activities	AIC systems approach to activities
Activity 19: Identify Potential AS Hazardous Failures at Tier n. Potentially hazardous failures are identified by considering possible deviations from intended behaviour that may arise for the AS.	Stage 3A then prompts the architect to model the AS's AIC behaviour, predictive thinking pipeline 1 directly map AS AIC behaviours to the operational problem domain AIC behaviours. In predictive thinking pipeline 2, the architect models how the rest of the environment would react to the designed AS behaviour. In this Stage, the architect can define ROD constraints in response to environmental behaviours. With that in mind, HazTOPS (predictive thinking pipeline 3) prompts the architect to identify sources of deviations based on a generalised version of HAZOP guidewires.
Activity 20: Define Mitigations for Identified Hazardous Failures • defining required design changes • limitations to the operating concept • deriving additional safety requirements	Also, predictive thinking pipeline 4, step 4 extends the SOC requirements in the earlier step by decomposing them into sub-system-level concrete requirements. We use the 4wnh (4-whats-and-hows) method to decompose the SOC requirements.
Activity 21: Instantiate Hazardous Failures Argument Pattern	Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.4.6 maps AIC artefacts to Activity 21 Assurance argument patterns (see table L.4.6.1 for the Eagle Robot, and L.4.6.2 for AVOIDDS case study). Table L.7 maps SACE artefacts to AIC Sections.

SACE Stages 7 and 8 are out of the scope of this PhD.

8.2 Developing Partial AMLAS Safety Case

See Section 2.1.7.4 for a detailed comparison between AMLAS and SACE. Additionally, for a comprehensive capture of AMLAS patterns, refer to Appendix L, Section L.5. We have only covered 4 Stages out of 6, as Stages 5 and 6, which involve model deployment and verification, are outside the scope of this research. The process involved matching the output of the AIC systems approach in our case studies to AMLAS GSN patterns. Like the above, we will summarise the list of artefacts in AMLAS and define how AIC systems' artefacts substantiate the assurance goals. We will reuse the literature review summary on AMLAS (Section 2.1.7.2) and attempt to explain how the AIC systems' approach to artefacts matched the requirements for demonstration.

The Assurance of Machine Learning in Autonomous Systems (AMLAS) framework is developed systematically to ensure safety, reliability, and robustness in ML-based autonomous systems. By integrating predictive scenario analysis, uncertainty articulation, hazard identification, and safety-driven ML training syllabus, the AIC framework substantiates the AMLAS assurance process at each Stage. Below, we describe how the outputs of the AIC Systems Approach support each AMLAS Stage.

8.2.1 AMLAS Stage 1: ML Safety Assurance Scoping

This Stage defines the safety argument for the ML component by setting system boundaries, identifying safety objectives, and articulating operational conditions. The AIC Systems Approach provides a structured process to ensure that this process remains comprehensive and traceable. To compare how our approach achieves AMLAS Stage 1 objectives, we formulate the following comparative table:

Table 8.7 Mapping AIC systems processes to AMLAS Stage 1 design process

AMLAS Stage 1 activities	AIC systems approach to activities
--------------------------	------------------------------------

Activity 1: Define the Safety Assurance Scope for the ML Component:

- This activity requires as input the system safety requirements.
- Descriptions of the system and the operating environment.
- A description of the ML component that is being considered.

Activity 2: Instantiate ML Safety Assurance Scoping Argument Pattern

Stage 3A and 4 outputs involve the following types of requirements:

- SOC

As for system description, this is captured during Stage 2 and the 3a pipelines 1 and 2, which also includes description of the capability the ML component will need to be able to deliver.

Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.5 maps AIC artefacts to Activity 2 Assurance argument patterns (see table L.5.1 for the Eagle Robot. Table L.8 maps AMLAS artefacts to AIC Sections.

8.2.2 AMLAS Stage 2: ML Requirements Assurance

This Stage ensures that ML safety requirements are explicitly defined, justified, and validated. The AIC Systems Approach ensures traceability from system-level safety requirements to ML-specific constraints. To compare how our approach achieves AMLAS Stage 2 objectives, we formulate the following comparative table:

Table 8.8 Mapping AIC systems processes to AMLAS Stage 2 design process

AMLAS Stage 2 activities	AIC systems approach to activities
Activity 3: Develop ML Safety Requirements Translating complex real-world concepts and cognitive decisions into a format and a level of detail that is amenable to ML implementation and verification	When we say “ML requirements”, we mean ML component-level capability. We focus on the functional aspects of the ML components rather than on performance aspects (like the size of bounding boxes, mAP performance rates, false positives, and negatives)⁵¹.

⁵¹ In this research we are concerned more about robustness coverage and ability to handle diverse operational scenarios than performance.

When we model HazTOP scenarios, we derive ML safety requirements and introduce the ML components to mitigate those scenarios. This process happens in Stages 3A, pipeline 3, and 4.

Activity 4: Validate ML Safety Requirements

Section 5.10 defines a validation process wherein SECoTs uphold semantic relevance to prior decompositions of the design. This process encompasses a comprehensive review of the Chain-of-Thought steps involved and any articulation. We depend on an expert peer review to substantiate the validation processes.

Activity 5: Instantiate ML Safety Requirements Argument Pattern

Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.5 maps AIC artefacts to Activity 5 Assurance argument patterns (see table L.5.2 for the Eagle Robot. Table L.9 maps AMLAS artefacts to AIC Sections.

8.2.3 AMLAS Stage 3: Data Management Assurance

This Stage ensures that the data used for ML training and validation is of high quality, representative, and aligned with safety requirements. The AIC Systems Approach provides a structured process for defining, validating, and managing ML training datasets.

To compare how our approach achieves AMLAS Stage 2 objectives, we formulate the following comparative table:

Table 8.9 Mapping AIC systems processes to AMLAS Stage 3 design process

AMLAS Stage 3 activities	AIC systems approach to activities
Activity 6: Define Data Requirements	Stage 3A (step 4.1) defines SOC requirements from which the architect derives general Safety-Training concept requirements. In Step 4.2, Stage 3a, the architect then defines ML development dataset requirements (with no specification of what to

Activity 7: Generate ML Data**Activity 8: Validate ML Data**

Data validation shall consider:

- The relevance,
- completeness, and
- balance of the data sets.

include in each dataset). The split and strategies of the dataset are determined in Stage 5.

Stage 4 also involves deriving SOC and Training concepts, driven by Black Swan scenarios. Those dataset requirements are then allocated to Black Swan datasets, which are later used in the testing and Training syllabus.

Here, Stage 5 (CuneiForm) provides a direct value in bridging the semantic gap between ML dataset requirements and actual pictures.

Stage 5: Articulate the pictorial problem for a given safety-training concept and SOC. In this Stage, we generate abstract cuneiforms which are then translated into datasets. There are three types of images we include :

- In-context, typical operations
- In-context, Black Swans
- Out-of-context, relevant robustness augmentation images.

The systematic generation of CueniForms from SOC and the Training concept allows for a validation process to check whether a CueniForm truly captures a SOC requirement, thus demonstrating relevance. Additionally, completeness is shown through the systematic mapping of CueniForms to all SOC requirements.

For balance, we count the number of images for a given CueniForm characterisation dimension. Any bias would be discovered at this Stage. See Sections 7.8 and I.10 for

examples of how we used CuneiForm to validate AVOID datasets.

Activity 9: Instantiate ML Data Argument Pattern Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.5 maps AIC artefacts to Activity 9 Assurance argument patterns (see table L.5.3 for the Eagle Robot. Table L.10 maps AMLAS artefacts to AIC Sections.

8.2.4 AMLAS Stage 4: Model Learning Assurance

This Stage ensures the ML model has been correctly trained and evaluated, satisfying safety objectives. The AIC Systems Approach systematically integrates learning assurance into the ML development lifecycle. To compare how our approach achieves AMLAS Stage 2 objectives, we formulate the following comparative table:

Table 8.10 Mapping AIC systems processes to AMLAS Stage 4 design process

AMLAS Stage 4 activities	AIC systems approach to activities
Activity 10: Create ML Model	The creation of the ML model is done at Stage 6. The activities involved are: • CuneiForm-based datasets validation. • developing in-context typical operations model. • Incorporating in-context Black Swan datasets. Only those data scenarios where the typical ops model fail. Pre-processing comes after the model achieves the required performance over CuneiForms for added robustness.
Activity 11: Test ML Model	In our testing Stagey, we distinguish two types of testing: • Test over in-context, typical operations examples, as well as out-of-context examples.

Once the model achieves the required performance, we then perform the second type of tests:

- Black Swan testing: We then test the model over challenging Black Swan examples.

We then assess which Black Swan CuneiForms the diversity of the typical operation dataset field enables the model to generalise well, and those Black Swan CuneiForms where the model fails to generalise well.

The failed Black Swan CuneiForms require the architect to generate more data and retrain the model.

A Black Swan dataset has two characteristics:

- Out-of-distribution with typical operations and out-con-context data.
- Derived explicitly from Black Swan CuneiForms.

We used a data dissimilarity measure to evaluate whether a dataset is out of or in distribution.

Activity 12: Instantiate ML Data Argument Pattern Chapter 8 and Appendix L demonstrate how we mapped the arguments with evidence gathered from our case studies.

Section L.5 maps AIC artefacts to Activity 12 Assurance argument patterns (see table L.5.4 for the Eagle Robot. Table L.11 maps AMLAS artefacts to AIC Sections.

8.3 AMLAS and SACE application reflection

In this chapter and Appendix L, we have systematically mapped our AIC Systems Approach onto the AMLAS and SACE frameworks to evaluate how well our methodology aligns with established safety assurance processes. Our primary objective was to construct a partial safety case that demonstrates the traceability and sufficiency of our approach in covering the essential safety assurance Stages required by these frameworks.

8.3.1 AIC can enhance AMLAS and SACE

By structuring our validation process through SACE's Staged assurance framework and AMLAS's ML-specific safety validation process, we identified key areas where our approach naturally aligns with existing methodologies. Our predictive epistemic reasoning, structured scenario analysis, and CuneiForm-driven assurance techniques provided a sufficient foundation for addressing most SACE and AMLAS assurance objectives.

Our key synergies findings from this integration process include:

Table 8.11 Mapping AIC systems approach to SACE and AMLAS

Key finding	Elaboration
Operational Domain Definition and Uncertainty Modelling	Our structured predictive thinking methodology (Stage 1-3 of the AIC Systems Approach) comprehensively articulated operational contexts, uncertainty factors, and scenario-driven safety requirements. This aligns well with SACE's Stage 1 (Operating Context Assurance) and AMLAS's Scoping Stage, ensuring our system's operating conditions were thoroughly assessed.
Hazard Identification and Safety Requirement Definition	The HazTOPs methodology (Stage 3A) was successfully mapped to SACE's Stage 2 (Hazardous Scenario Identification) by systematically capturing both conventional and emergent hazard cases. Our approach extended beyond conventional SACE hazard identification through Black

	Swan Scenario Predictions (Stage 4), addressing unpredictable failure conditions not typically captured in standard safety assessments.
Safety Concept Justification and Design Assurance	Our ordered AIC-based mitigating Safety Requirements (Stages 3A & 4) provided structured justification for safe operational concepts, fulfilling SACE Stage 3 (Safe Operating Concept Assurance). Similarly, our CuneiForm-driven ML safety justification methodology helped validate safety-critical learning models in compliance with AMLAS's safety requirements assurance Stage.
Validation of ML Components and Data Management Assurance	Our systematic ML safety training process (Stage 5) provided a structured framework for ensuring dataset integrity and ML training sufficiency, meeting AMLAS Stage 3 (Data Management Assurance). Our experiments (Unsafe Train Tracks case study) confirmed that our methodology may effectively identify failure in Black Swan scenarios and data biases, strengthening our alignment with AMLAS Stage 4 (Model Learning Assurance).
Identifying Areas Beyond Our Scope	While our approach effectively supports most AMLAS and SACE Stages, some areas (such as system deployment assurance and final verification (AMLAS Stage 6, SACE Stage 8)) were not fully developed within the scope of this research. Future extensions of our approach should incorporate deployment-Stage verification

techniques to ensure safety validation across the entire system lifecycle.

8.3.2 Potential Enhancement of AMLAS and SACE

Our experience with SACE and AMLAS suggests that they are based on a traditional safety-driven paradigm, where the primary concern is identifying safety and security issues and providing assurances against them. With that in mind, SACE placed less emphasis on critical general problems associated with complicated open environments and deterministic safety-critical autonomous systems operations, which are **Black Swan** scenarios. By “less”, we mean the approach did not deliberately align the Stages to be primarily purposed to predict Black Swan scenarios (sources of operational data shifts after deployments). SACE refer to Black Swans in a traditional way of labelling them as “Edge Cases”. We argued in Section F.7 that the term Edge Case is a misrepresentation of a well-defined concept in deterministic software testing, which does not make sufficient sense in the context of non-deterministic AI and a complicated open environment.

The AMLAS process does not directly guide the architect in predicting Edge Cases. While the SACE process includes a brief general requirement to incorporate sufficient edge cases as part of the testing, our investigation into the challenge of assuring trustworthy safety-critical autonomous systems in an open, complicated environment made us realise that Black Swans are the Achilles Heel of any safety case. A safety case for such a system is only as good or as valid as its assumptions about how the environment may shift, leading to data shifts. This reliance on the correctness of the dataset distribution as representative of the present and future complexity of the operational environment is problematic. Suppose the architect overlooks the slightest aspect of the environment (for example, the possibility of the moon resembling a traffic light). Such a minor oversight can invalidate the assumptions upon which the safety case is built.

SACE and AMLAS view this issue as a “safety” or “security” issue. We take a slightly different perspective; we believe the entirety of systems and safety engineering should be directed toward the sole purpose of “predicting Black Swans” as much as possible. From this perspective, safety and security issues become part of the endeavour, but the primary focus of designing safer autonomous systems is the team's dedication to discovering Black Swans. We noted the long-tailed open, complicated problem domains during Stage 1 of our case studies, which confirmed our approach. In Stage 6 for the Unsafe Train Track Zone, we realised that the diversity of unique examples plays a role in performance over out-of-distribution (shifted data) relative to training and validation. Experiment 5 showed that training with random images may suffice to cover such situations for out-of-distribution typical operations.

However, as guided by Stage 4 of our process, Black Swan scenarios batch A (a camouflaged drone), which relates to the operational problem, demonstrated that random images are insufficient to ensure performance in a non-typical black swan scenario. This suspicion was reaffirmed in experiment 10, where we trained the same architecture, included most random images from experiment 5, increased the dataset size to over 100,000 images, and included over 4,000 typical in-context scenarios. In experiment 10, the Black Swan scenario batch A appears to challenge the expectation of satisfactory performance with increased diversity of datasets. With that in mind, we concluded that the search for Black Swans should be the primary purpose of a comprehensive and practical approach. We bring that to SACE and AMLAS to address this issue that is not sufficiently emphasised in the processes.

The findings of this chapter indicate that the AIC Systems Approach demonstrates alignment with the AMLAS and SACE frameworks, effectively addressing a majority of safety assurance objectives. Through structured hazard identification and safety requirement justification (leveraging methodologies such as HazTOPs and Black Swan Scenarios prediction) a practical approach is established that enhances resilience by incorporating high-impact, low-probability risk analysis. Furthermore, the CuneiForm methodology employed for machine learning training and data validation integrates seamlessly with the structured safety case process of AMLAS, underscoring the strength of our techniques in ensuring the reliability of AI-driven safety-critical systems. It is noteworthy that while the research identifies areas for further enhancement, particularly in system deployment and post-deployment safety verification, these components, though currently beyond the project's immediate scope, offer promising opportunities for future development.

8.4 Summary: AIC complementarity with SACE and AMLAS

The AIC Systems Approach and the SACE/AMLAS frameworks share a common goal of delivering safe and reliable autonomous systems. Still, they approach this goal from different vantage points and bring distinct strengths. Below is a concise overview of how AIC both complements and extends SACE and AMLAS.

8.4.1 Problem-Domain Versus Solution-Domain Starting Points

SACE/AMLAS typically assume a chosen solution (the autonomous system or ML component) and builds safety assurance “around” it. AIC deliberately begins in the problem domain, articulating the underlying operational challenges, stakeholder intents, and environmental complexity before prescribing any solution. This ensures that all derived requirements, scenarios, and constraints remain firmly tied to real-world problems rather than to preconceived system

designs. By front-loading problem validation (AIC Stage 1), we avoid “solution fixation” and maintain traceability back to the client’s true needs.

8.4.2 Gap in SACE/AMLAS: Detailed Scenario & Black-Swan Prediction

While both frameworks acknowledge the importance of rare or unanticipated events, they provide limited guidance on how to derive and analyse such “Black Swan” scenarios systematically. AIC utilise Predictive Thinking Pipelines (especially Stage 1 and 4) offer a structured methodology for uncovering long-tailed, high-impact events by exploring emergent interactions in the problem space. These pipelines yield rich, validated scenario sets that extend and deepen SACE’s Operational Domain Model (ODM) and HazTOPS analyses.

8.4.3 Rich Artefacts & Traceable Evidence

SACE/AMLAS Assurance Patterns prescribe what must be argued and evidenced at each Stage (e.g. ODM validation, Hazardous Scenario Assurance, ML Requirements Argument). AIC’s “Know-How” supplies how to generate those artefacts:

- Operational Domain Models, Assumptions & Factors (supporting SACE Stage 1’s ODM)
- HazTOPS Reports mapping each interaction and decision point to hazards, threats, and opportunities (feeding SACE Stage 2)
- Predictive Scenario Reactions (Pipeline 2) that validate Safe Operating Concepts and Reduced Operating Domains (feeding SACE Stage 3)
- 4-What-and-How Decompositions that transform system-level safety requirements into concrete, subsystem-level requirements (SACE Stage 4)
- CuneiForms & Dataset Prescriptions that justify ML training and testing data (AMLAS Stages 3–5).

Each of these artefacts comes with embedded semantic trace links, from stakeholder needs, through problem-domain complexities, to final design decisions, directly supporting the GSN/CA patterns laid out in your appendices.

8.4.4 Holistic Coverage Across SACE & AMLAS

While SACE focuses on system-level safety in an operational context, and AMLAS zeroes in on the ML component lifecycle, AIC bridges both by:

1. Starting with the full problem landscape,

2. Layering solution-agnostic predictive analytics,
3. Then drilling down into ML-specific data and performance requirements.

The result is a seamless assurance continuum from problem discovery through to ML dataset generation, closing the methodological gaps that can emerge when SACE and AMLAS are applied in isolation.

In summary, the AIC Systems Approach:

1. **Anchors** every safety activity in the real problem domain, avoiding premature solution lock-in and any bias towards unsustainable solutions.
2. **Augments** SACE/AMLAS with systematic Black-Swan prediction and detailed scenario derivation.
3. **Generates** richly linked artefacts that directly instantiate the assurance argument patterns outlined. Then link them to datasets via CuneiForms.
4. **Validates** each step via transparent, peer-reviewed rationales, ensuring nothing is a “black box.”

Together, these strengths mean that AIC doesn’t merely parallel SACE and AMLAS, it complements, enriches, and closes their most critical gaps.

Chapter 9 Conclusion

Some of the concepts and proposals we mention here, for example, (Residual Ignorance) require further investigation, which is outside the scope of this PhD at this Stage. Rather will be part of future work. Other concepts, such as Compounded Uncertainty Problems, Architect Predictive Thinking, AIC complexity field mental model, and ArcPM, are based on our understanding of the methods we used to solve our case studies.

Fundamentally, our approach aims to propose a scientifically based predictive system thinking method to solve Compounded Epistemic Uncertainty problems involving harder-to-predict intelligent systems operating in equally complex operational environments. This is a black-box approach, meaning we view the AI component as a black box and attempt to address AI trustworthiness holistically. We aim to enhance or complement any white-box methods to ensure the trustworthiness of AI systems in safety-critical applications.

By understanding the concepts and ideas presented in this chapter, the reader will develop a strong appreciation for the worldview from which we approach engineering safety-critical autonomous systems. This chapter explains what predictive architecting is as a Systems Science-based Systems Engineering practice and why we chose the methods we proposed. For example, the Complexity Field hypothesis highlights our interpretation of general complexity and the role of Architect Predictive Thinking Methods (ArcPM) in resolving it.

As a whole, our chosen dataset strategy is the one we conducted in Exp9.4, then we augmented it further in Exp9.6. If we compare it with what we usually would do in Exp.10, producing in-context typical operations and adding more out of context. In Exp10, we gathered an 85,000+ image dataset. Yet it couldn't handle a dataset of Black Swan scenarios A. The approach in Exp.9.4, even though we had a smaller training set (circa 16000 images), we produced a more trustworthy model. Because it was able to handle two types of out-of-distribution tests. Additionally, the process can systematically demonstrate our deliberate training and enhancement of Black Swan (Edge cases), which can be documented in a safety case for further scrutiny. The process can help to discover gaps in knowledge further.

The following are the main findings from our research as a whole:

9.1 Predictive Lateral, Vertical and T-Thinking

Our research has led to an understand comprehensively the difference between Lateral, Vertical thinking, and how AIC approach possibly balances both thinking modes. We used T-thinking to describe a balanced approach to lateral and vertical thinking. A thought experiment inspired the findings in this Section, we articulated about the nature of Vertical and lateral thinking in Section E.4. The main observations we made in Section E.4 involved:

- **Vertical thinking:** Systematically refining resilient and trustworthy rules and addressing contradictions, aiming for a stable and predictable system. It is effective if we have precise and strong trust in the resilience of priors and initial assumptions. Therefore, it effectively resolves known issues (White Swans) but may struggle with unexpected scenarios or patterns. The approach is somewhat fundamentally deductive reasoning-based.
- **Lateral Thinking:** embraces irregular concerns in assumptions and rules, and explores new possibilities through analogies and associations. It generates diverse predictions and is better suited for counterintuitive or novel scenarios. However, compared to Vertical methods, it may lack precision and stability. The approach is somewhat

fundamentally inductive reasoning-based. Effective for the discovery of least concerning Black Swan scenarios.

In this Section, we will describe what we have learned about these concepts throughout our experience in this research:

Lateral thinking: commonly known as a problem-solving approach that involves tackling challenges through indirect and creative methods, often by viewing the problem from new and unconventional perspectives [3]. We view such a definition to be rather too broad. Through our research into systems thinking and the application of AIC in case studies as a predictive mental model, we understood lateral thinking as a critical approach to assure the resilience of safety and desired engineered emergent properties such as intelligence, in the context of CUPs.

During the research and the application of the case study in solving the issues of predicting unconventional interactions in the real world (for example, how vegetation around the train track zone fence can interact with the wall that leads to impacting autonomous drone operations), we realised we needed to think outside the box in order to arrive to such usually trivial events, thus being more creative.

In our effort to try to systemise Lateral Thinking, we have realised that a predictive thinker needs to apply the indifference principle effectively [115] whereby the predictive thinker assumes that all events have the same probability of occurring regardless of whether the vertical thinking assumes that some are more or less likely to happen, including contradictions (regardless of whether they are logical or not). For example, a Vertical thinker may conclude that a moon is not a traffic light (in human common sense, it is rightly so); however, concerning a machine, this might not be the case [109]. Therefore, a Vertical thinker who intensely assumes the world can be reduced to an algorithm dependent on some common-sense assumption: a moon will never make an autonomous car slow down in the middle of the road. A lateral thinker would consider such a scenario a possible Black Swan event.

Fundamentally, the lateral thinker approaches solving problem domains as a uniformly distributed set of events or scenarios, whereby all possible events are equally impactful and should not be ignored. Lateral thinking can be made a systematic or asystematic approach to prediction inference where no initial assumption is a hard truth, but rather a soft possibility that may turn out to be true or false. More into this topic can be reviewed in Section 9. An example for lateral thinking would be De Bono's 6 thinking hats process

Vertical thinking refers to the thought processes involved in formulating problems so their solutions can be represented as Vertical steps and algorithms [4].

Fundamentally, Vertical thinking perceives the problem domain as a complex interaction of elements that can be reduced to an algorithmic set of predictable interactions. It relies on strong common-sense assumptions (hard-to-change assumptions) or axioms established or believed to be true. Once the problem solver sets the assumptions at the beginning of algorithmic problem solving, the entire correctness of the approach will hinge on one or two assumed facts. This could be troublesome, as a slight alteration in those assumptions could lead to the collapse of the adequacy of the solution. Therefore, it may be acceptable to think that Vertical thinking produces inherently chaotic solution systems, as it is susceptible to the butterfly effect, where a small perpetuation of uncertainty in the founding assumptions can lead to the loss of a solution's relevance.

In our experience with the AIC approach, we realised that a Vertical thinker addresses the uncertainty problem in complex fields as being reducible or assumed to follow a normal distribution of possible events, such that certain unlikely occurrences can be disregarded or need not even be considered. This approach is efficient for definable or controlled problems. An open environment, by definition, is not controlled; thus, relying solely on this approach may lead to an incompatibility that results in ignoring or failing to account for Black Swan events scenarios. An example of vertical thinking approaches would be: HAZOP and Fault Tree Analysis.

T-Thinking is a balanced, strategised thought process that intelligently applies Lateral and Computer-based thinking to solve problems, thus benefiting from each approach's advantages. We call it “T” because the letter T has two parts: the top horizontal line, which signifies horizontal thinking, a uniformly distributed assumption that equally accepts all possibilities. At the same time, the vertical line represents the step-wise, vertical, algorithmic process to maintain traceable inference. AIC thought process is a T-thinking approach where the predictive thinker is invoked to use lateral and vertical thinking across different processes.

9.1.1 Fundamental Framework for AIC Balanced Predictive T-Thinking

The claim of “Balance” refers to equally balancing all possibilities of potential events and outcomes, including those we deem as relevant or logical, and those we decide to exclude as irrelevant and illogical. A biased distributed problem definition results from an imbalanced thought process compared to a uniformly distributed view of the problem, where all possible events are equally concerning. Guidewords-based approaches prompt the architect to consider limited types of deviations, and by training the predictive capability of the architect to those guide words only, they inadvertently prompt the architect to exclude any other type of deviations as irrelevant or outside their sphere of concern. AIC attempts to make the architect think and not forget about those that are outside of their concern through AIC modelling and AIC Matrix. Hence, we represented AIC approach in the following diagram:

The fundamental thought process for balanced-perspective-based thinking can be understood to involve three principal steps:

1. **Identifying the Complexes Involved in the Interaction (constructing the context of the complexity field):** The first step is to define the primary complexes engaged in the interaction of concern. Complexes interact dynamically with entities within its self-control, such as components, subsystems, or external environmental factors. This step establishes the context by identifying the roles and relationships of these complexes.
 - **Predictive Question:** What key complexes drive or participate in this interaction?
2. **Define Influence and Control Dynamics (Vertical thinking):** The second step focuses on Vertical reasoning to map how other complexes influence or control the interaction and vice versa. This analysis includes identifying the external forces, rules, or parameters that govern the behaviour of the primary complexes. Architects can predict how control dynamics may affect the outcome by simulating or evaluating these influences.
 - **Predictive Question:** Given the context, what other complexes influence or control the interaction?
3. **Define Appreciation Dynamics (Lateral Thinking):** The final step introduces lateral thinking to explore the interaction's appreciation dynamics. This involves shifting perspective to understand how other complexes interpret, react to, or value the interaction. By appreciating how these dynamics unfold, architects can identify indirect influences or emergent behaviours that may not be apparent from the primary perspective.
 - **Predictive Question:** What other complexes need to be appreciated or will be appreciating the interaction, and how might that appreciation shape the outcome?

9.1.2 Architect Predictive Systems Thinking

Uncertainty is the most challenging aspect of Compounded Epistemic Uncertainty Problems. Thus, we cannot avoid dwelling on understanding the problem from an architect's predictive thinking perspective. And know how we approach enhancing architect predictive thinking in general. This notion also aligns with the core aspect of intelligent systems: predictive inference. Therefore, in Chapter 3, Section 3.3.2, we reviewed some literature sources about predictive thinking. In this Section, we elaborate on how that knowledge helps us understand the nature of challenges associated with human predictive thinking, which will help us understand the intellectual challenges associated with machine intelligence training processes.

To solve Compounded Epistemic Uncertainty problems, the common wisdom is to adopt systematic predictive thinking that expands the scope of the architect's attention to enhance

their chances of discovering emergent behaviours. This means encouraging the architect not to confine their perspective to a single predefined framework (like control only, for example) but to incorporate diverse worldviews. Such a comprehensive approach helps in forming a more holistic understanding of the problem space, akin to "seeing the forest for the trees". Furthermore, achieving a balance between **lateral thinking** (exploring novel and unconventional interactions, including illogical ones) and **vertical thinking** (logical-consistency-based thinking) is essential. This dual approach fosters both creative exploration and rigorous analysis, enabling architects to predict and address complicated systems more effectively and innovatively.

In our approach, we aim to integrate systematic and systemic thinking approaches into a unified thought process. The systemic part of our thought processes is based on general axioms about any complexity, and the systematic part is composed of a series of prediction-provoking questions. We call those thought processes "Systems Engineering Chain-of-Thought (SECoT)"; see the figure below for a depiction of what a SECoT is.

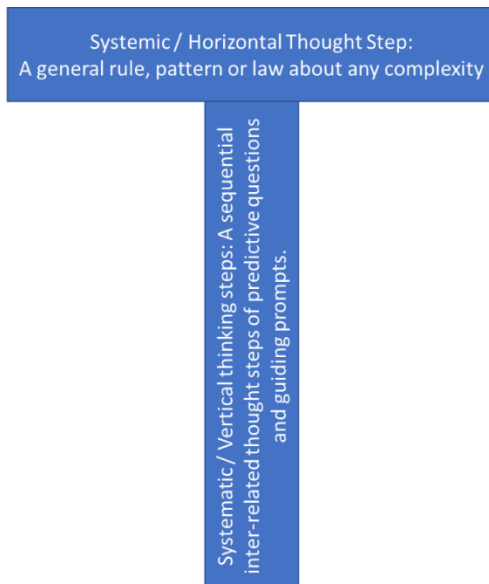


Figure 9.1 illustrates the structure of the SECoT method.

Compounded Epistemic Uncertainty problems should require more time and energy to think about, as they involve discovering long-tail, hard-to-predict scenarios based on a deep comprehension of the factors involved in the problem domain. Such a cognitive tendency is hazardous in terms of making correct engineering judgments. Hence, guided systematic approaches to problem-solving served as a way to control the pace of the decision-making process, forcing the human mind to look wider. More thinking time means more potential factors can be considered, and they are more likely to converge on an accurate prediction and judgement.

9.2 SECoT helped us trust our predictions under CUP

One of the major problems with solving CUP situation is truly trusting that we have made the right assumptions about how the complexity of the problem will play out in the real world. The issue here is the resiliency of assumptions, meaning how long will we realise that whatever we assumed during our design time will actually last in the real world complexity. In CUP conditions, such issue is of extreme importance; the most resilient general rule about any CUP scenario can be reasonably accepted as an axiom:

*99% of assumptions made at design time about CUP will be inaccurate at some point; hence
never assume without thinking deeper and checking with someone else*

Designing for safety under highly volatile uncertainty can significantly undermine the architect's confidence in their assumptions about the problem. Our findings revealed that SECoT provided insight into our invisible thought processes, which we utilised to make predictions. When multiple individuals are involved in a single SECoT during implementation validation, we develop a more robust thought process. This means that the consideration of the problem is shared among multiple systems thinkers, thereby fostering trust within the cohort about their collective engineering judgment. Systems rules help objectivity the language between experts, where no expert bias may dominate the prediction.

The primary purpose of SECoT is to:

- Enhance the architects' predictive thinking by influencing their vertical thinking-based decision-making processes. It does so by provoking the architect's lateral thinking, prompting the designer to apply a general systems pattern learned from different systems.
- By balancing lateral and vertical thinking, the architect is assisted in being less biased towards hard-systems thinking. The architect can also be prompted to consider soft-systems thinking during their thought processes. Thus increasing the chances of discovering Black Swan scenarios.
- Validators or design assurance entities can ascertain what principles have been implemented throughout the design decision-making process, thus providing evidence that can back up justifications for an informative safety case (like AMLAS and SACE). For example, it is hard to ascertain soft aspects such as ethical principles. SECoT includes the implementation of philosophical requirements related to ethics.
- The general axioms used in SECoT define the quality criteria of satisfying output assertions and predictions. It defines what is an acceptable output and what is not.

Objective lateral thinking is key to our approach; it helps with the following:

1. It exposes the Epistemic uncertainty of architect intelligence that may accumulate through constant vertical thinking.
2. Guides the architect's predictive thinking process to consider counterintuitive scenarios.
3. It allows us to influence the architect's thinking bias objectively. We can conceptually influence the bases (in General Systems Rules).
4. It can handle the implementation and the demonstration of important soft aspects such as ethical considerations.
5. It allows design authorities, quality assurance teams, or the public to scrutinise the thinking processes that went into the design of the autonomous systems, adding another layer of trustworthiness.
6. Exposes the architect's invisible thinking processes for scrutiny and enhancement.

9.3 PhD research formulation reflections

This Section provides a brief explanation of our responses to the research questions highlighted in Chapter 1. We will use the list of research requirements to map the concepts and Sections, demonstrating how we answered our research questions.

9.3.1 Research Question 1

Research Question 1: How do we comprehensively appreciate problem and operational complexity domains to reduce the architect's epistemic uncertainty as low as reasonably practical and predict problem-level Black Swan events?

Sub question: How can we demonstrate that Epistemic uncertainty is effectively reduced at the engineered dataset level?

We propose the following solutions:

9.3.1.1 Solution Methods for RQ1

Our primary methodology, **SECoT_1** (An AIC-based Compounded Epistemic Uncertainty Problem Articulation), comprises four Predictive Thinking Pipelines. Pipeline 1, "Appreciate the Complexes of the Problem Complexity Field", systematically identifies ambiguous and unsafe behaviours within the operational domain and visually represents them to facilitate a more profound analysis and understanding of complexity dynamics. For instance, in the Unsafe Train Tracks scenario, we illustrated problematic behaviours such as adversarial drones hovering near restricted zones, explicitly articulating confusing and potentially unsafe behaviours.

Pipeline 2, "Resolve the Complicatedness Pattern", employs structured analyses, including the Actions Matrix method, to systematically map and categorise interactions, distinguishing between problematic (unsafe) and advantageous scenarios. This was crucial in understanding dynamic relationships and their systemic impacts on operations, particularly regarding the interactions among adversarial drones, train tracks, surveillance systems, and other environmental elements.

Pipeline 3, "Predict the Emergence of AIC Complexity Field for Detailed Operational Scenario Articulation", involved comprehensive predictive modelling utilising the AIC (Appreciation, Influence, Control) thought process. We developed emergent problem domain scenario models that detailed complex interactions, such as how adversarial drones might affect or control operational safety. This pipeline ensured thorough scenario predictions, thus enabling pre-emptive risk identification and mitigation.

Pipeline 4, "Predicting and Evaluating Problem Domain Factors and Assumptions", was fundamental in systematically assessing and defining assumptions and factors that influence operational complexity. We effectively illustrated how epistemic uncertainty is identified, evaluated, and mitigated by ranking these factors according to architect's attentive mentions and relevance and by critically defining assumptions. We then analysed which of those factors are Black Swans. For instance, assumptions regarding adversarial drone detection capabilities and limitations were articulated, ensuring transparency and traceability within our uncertainty reduction strategy.

- **For foundational theoretical concepts:** read Section 4.1, 4.2, F.1, E.1, E.5, E.6.
- **High-level theoretical description:** read Section 5.2.
- **For detailed SECoT description:** read Section F.2.
- **Example detailed implementation:** read Section H.4 and I.4.

Furthermore, we needed a method to define a comprehensive **Operational Design Domain (ODD)** comprehensively to cover air and ground variety of conditions and objects regardless of the location of the autonomous system may operate within. Hence, we further developed the Multi-Purpose Operational Design Domain Definition Section. The ODD is defined as a structured table that outlines the conditions in which the autonomous system must operate. This structured definition explicitly outlines the system of interest, for example, Eagle Robot, operating within an operational domain, for example, the Train Track Zone in the UK. An ODD uncertainty of Grades further characterises the ODD uncertainty to indicate the level of challenging conditions.

Chapter 9

The primary purpose of defining the ODD is specifying the conditions required for rigorous training, testing and validation activities. The ODD table meticulously specifies environmental characteristics, including consistent natural lighting (sunny, clear skies), stable weather conditions (no precipitation, low wind), high visibility, controlled temperature ranges, and detailed landscape, geographical, and temporal parameters. This structured approach significantly reduces the architect's epistemic uncertainty by precisely delineating the operational environment's boundaries, characteristics, and assumptions, providing clear guidelines for stakeholders during system development and validation.

- **For foundational theoretical concepts:** read Sections E.8.
- **High-level theoretical description:** read Sections 5.1.4.
- **For detailed description:** read Section F.6, Appendices A and B.
- **Example implementation:** read Section H.7 and I.7.

9.3.2 Research Question 2

Research question 2: How can we ensure that safety requirements and architect intent will be preserved during Black Swan Pictorial situations (unforeseeable high-impact scenarios)?

Sub-question: How can we validate designed intelligent behaviour safety properties during Black Swan scenarios?

9.3.2.1 Solution Methods for RQ2

SECoT_2, our **Hazards, Threats, and Opportunities Scenarios (HazTOPs)** Analysis method, systematically identified potential complications using modified HAZOP guidewords. This approach enabled the explicit capture of hazardous and opportunistic scenarios, thereby enhancing the robustness of our predictive capabilities. Our comprehensive approach in employing HazTOPs SECoT analysis ultimately provided objective and structured evidence of effective epistemic uncertainty management, particularly concerning the engineered dataset level.

- **For foundational theoretical concepts:** read Section 4.1, 4.2, F.1, E.1, E.3, E.5, E.6.
- **High-level theoretical description:** read Section 5.4.
- **For detailed description:** read Section F.4, F.5.
- **Example implementation:** read Section H.6.3 and I.6.3.

Predictive Lateral Thinking Technique: Predictive Disordered AIC Perspective Shift for Predicting Black Swan Scenarios: The Predictive Disordered AIC Perspective Shift technique introduced a structured lateral-thinking method to uncover hard-to-foresee, rare, and impactful scenarios proactively, termed "Black Swan Scenarios" (inspired by the concept of Black Swan events). Architects could foresee unconventional or counterintuitive operational situations by systematically altering the perspectives of pre-defined AIC interactions. For example, previously unseen risks were uncovered by shifting perspectives on a drone initially intended to appreciate (by monitoring) public access to control public access, inadvertently attracting public intrusion into train tracks. This explicit, structured lateral-thinking approach directly addressed epistemic uncertainty by methodically uncovering and preparing for unpredictable, emergent scenarios, ensuring comprehensive coverage of the operational domain's complexity and effectively answering Research Question 2.

- **For foundational theoretical concepts:** read Section 4.1, 4.2,4.4, F.1, E.1,E.2,E.3,E.4,E.5, E.6 .
- **High-level theoretical description:** read Sections 5.6.
- **For detailed description:** read Sections F.8, F.7.
- **Example implementation:** read Section H.8 and I.8.

The output of the above methods will ultimately result in more training scenarios for the ML component, hence reducing the epistemic uncertainty of the ML development dataset by reducing the architect's epistemic uncertainty about the problem domain.

9.3.3 Research Question 3

Appendix A **Research question 3:** How can we trace and validate developed pictorial datasets to written requirements (closing the semantic gap between requirements and pictorial datasets)?

9.3.3.1 Solution Methods for RQ3

CuneiForm: Similar to how formal mathematical specifications create clear links between human language and software coding, CuneiForm acts as a requirement that connects conventional written requirements with computer vision development datasets. This method allows for the conversion of textual descriptions of operational environment behaviours into visual representations that meet the training and validation needs of perception-based systems, especially in safety-critical contexts. By functioning as a bridge between abstract written

specifications and tangible examples, Cuneiform enhances the thorough demonstration of expected system behaviours across various operational situations and datasets.

- **For foundational theoretical concepts:** read Section 4.3,E.7,E.8.
- **High-level theoretical description:** read Sections 5.7.
- **For detailed description:** read Sections F.10, F.11 and Appendix D.
- **Example implementation:** read Sections H.9 and I.9.

9.4 Future Research Direction

AIC Systems Approach only opens up the door for further research into the topic, Applied Systems Science in Systems Engineering for Safety Critical applications. The following are some examples of future research and speculative analysis which could become a starting point to something more.

9.4.1 Fine-Tuning on Black Swan-driven ML development pipeline

Building on our AIC Systems Approach, a promising avenue is to refine the Black Swan-driven ML development pipeline through hybrid “freeze-and-tune” strategies that balance efficiency with robustness. Rather than full retraining, we will initialise from a large-scale pretrained backbone (e.g., ResNeXt or EfficientNet), freeze early convolutional layers during White Swan training, and progressively unfreeze deeper layers and the detection head as new Black Swan batches arrive. By staging layer unfreezing, starting with just the head and only unlocking mid-level feature extractors when accuracy plateaus, we aim to significantly reduce training time and cost (particularly for third-party platforms) while still capturing critical anomalous patterns.

We will rigorously quantify the trade-offs between lightweight fine-tuning and full retraining in terms of convergence speed on Black Swan test sets and the quality of final feature representations. Furthermore, integrating continual-learning techniques (such as replay buffers and Elastic Weight Consolidation) promises a scalable framework that can continually adapt to an open stream of novel scenarios without full retraining. Ultimately, this research aims to produce a unified regime that “inoculates” perception systems, such as drone detectors against extreme weather, sensor artefacts, or adversarial perturbations, thereby preserving the systematic anomaly-discovery advantages of our current approach while unlocking significant vertical savings and faster adaptation.

9.4.2 AIC-enhanced FRAM process

By embedding AIC’s lateral-thinking ontology and SECoT rationale into the Functional Resonance Analysis Method, an AIC-enhanced FRAM would become a proper Safety III approach, one that starts with “Influence” (resonance) and iteratively defines controls and appreciation before refinement, rather than treating resonance as a post-hoc step. In this integration, foreground functions capture the problematic situations architects seek to control, while background functions represent ungovernable yet influential contexts. Each function is described via a CRIPTO-Prime schema, purpose, inputs and outputs as AIC influence/control actions, preconditions, resources, controls, and timing, ensuring that both direct control loops and subtler indirect influences are explicitly modelled.

Once the enriched FRAM graph is complete, our Stage 4 predictive pipeline can reverse AIC interactions to surface potential Black Swan events. The result is a unified complexity graph that reduces residual epistemic ignorance, uncovers hidden dependencies (for example, how surveillance indirectly shapes an adversarial drone’s behaviour), and supports more comprehensive risk assessments. Future work will define semi-quantitative metrics for appreciation-based variability and influence strength, develop software to distinguish AIC link types in FRAM diagrams, and validate this methodology across various domains, including healthcare robotics and maritime navigation.

9.4.3 White Ghosts, Black Ghosts

Alongside identifying Black Swan anomalies, a critical future challenge is ensuring that safety properties also hold in truly unforeseeable “invisible” pictorial scenarios, patterns the model has never seen and that aren’t even implied by known data. We term in-distribution but untrained-on images that the model nonetheless handles correctly “White Ghost Patterns”, since preliminary experiments (Section 7.2) suggest safety requirements may reliably extend to them even in the absence of explicit examples; rigorously validating an ALARP claim for White Ghosts is therefore a prime research target. In contrast, “Black Ghost Patterns” lie outside both training distributions and the emergent White Ghost space, and any preservation of safety properties here would be astonishing, and thus demands new methods for both detection and formal assurance. Future work must develop frameworks for characterising, testing, and ultimately bounding risk in these two classes of invisible scenarios, so that ALARP arguments can comprehensively account for not only known and discovered anomalies but also the fully unknown “ghosts” that lurk beyond.

9.5 In Conclusion

This thesis outlines and examines a Systems Thinking and Science-based approach for designing and ensuring the safety of autonomous systems operating in an open, complex environment. In general, it turned out that our hypothesis can be summarised as the following principle:

Ensuring the trustworthiness of intelligent systems involves establishing a comprehensive safety case that the underlying Epistemic Uncertainty has been reduced to as low as reasonably practical at both the architect's predictive thinking and intelligent machine predictive model levels. To demonstrate such quality, the approach must explicitly address both White Swans events and Black Swan scenarios.

In other words, we have arrived at the overall conclusion that:

Appendix B Autonomous Systems Engineering practice for safety-critical applications should be a process that is fundamentally purposed to pursue the predictions of Black Swan Scenarios and equips the system with relevant safeguards.

Only through the active pursuit of the Black Swan scenario, regulators and customers can be assured that the product is worth the trust. In this process, architects would have considered known unsafe scenarios and made a deliberate effort to identify unknown unsafe scenarios. Our approach aims to provide such assurance. We argue that any systems approach that does not deliberately pursue the discovery of black swan scenarios may be less trustworthy. The reason for our conclusions is the following:

1. Our case studies have demonstrated to us that a set of long-tailed distributed events inherently characterises an open, complicated problem environment. In other words, unlike the assumption of a normally distributed set of events, rare scenarios cannot be dismissed as negligible in such an environment of uncertainty.
2. Black Swan scenarios pose a significant risk to system safety design. This means that, regardless of how effectively safety measures are implemented for an autonomous system, there will always be a Black Swan scenario that challenges the adequacy of the initial assumptions about the system's behaviour in a given operational domain (as seen with Tesla mistaking the moon for traffic lights).
3. Validating ML components in Black Swan scenarios demonstrates readiness for a wide variety of situations within the operational domain, thus enhancing the value of safety cases produced. We argue that safety cases that do not validate performance against predicted Black Swan scenarios may be considered less valuable than safety cases that thoroughly demonstrate such performance.

Chapter 9

This chapter delineates the key components of the hypothesis by aligning them with the research questions posed in earlier chapters. These questions address critical issues in autonomous systems engineering (ASE), such as managing Residual Epistemic Ignorance, bridging the semantic gap between datasets and requirements, and preparing for Black Swan scenarios. This chapter lays the groundwork for answering these questions and advancing the research vision by introducing theoretical insights and practical methodologies.

Glossary of Terms

Bibliography

- [1] New York Post, “Food delivery robot casually rolls through Los Angeles crime scene,” CNN, 16 Sep 2022. [Online]. [Accessed 03 Feb 2024].
- [2] H. Bwayo, “Food Delivery Robot Forces Its Way Across a Cordoned-Off Police Crime Scene in LA,” *autoevolution*, 14 09 2022. [Online]. Available: <https://www.autoevolution.com/news/food-delivery-robot-forces-its-way-across-a-cordoned-off-police-crime-scene-in-la-198617.html#>. [Accessed 01 02 2024].
- [3] E. e. a. Jenn, “Identifying Challenges to the Certification of Machine Learning for Safety Critical Systems,” in *European Congress on Embedded Real Time Systems* , 2020.
- [4] H. Kuwajima, H. Yasuoka and T. Nakae, “Engineering problems in machine learning systems,” *Machine Learning*, no. 109, p. 1103–1126, 2020.
- [5] I. Zhang, “A Training Method for Enhancing Neural Network Model Generalisation,” IEEE, Newcastle upon Tyne , 2002.
- [6] A. Banks and R. Ashmore, “Requirements Assurance in Machine Learning,” Defence Science and Technology Laboratory, Salisbury, 2019.
- [7] D. Karunakaran, S. Worrall and E. Nebot, “Efficient statistical validation with edge cases to evaluate Highly Automated Vehicles,” *2020 IEEE 23rd International Conference on Intelligent Transportation Systems*, no. doi: 10.1109/ITSC45102.2020.9294590, pp. 1-8, 2020.
- [8] H. Al-Shareefy, M. Butler and S. Hoang, “An AIC-based approach for articulating unpredictable problems in open complex environments,” in *19th European Dependable Computing Conference*, Leuven, Belgium, 2024.
- [9] H. Al-Shareefy, M. Butler and S. Hoang, “AIC Approach for Intelligent Systems Requirements Elicitation,” in *7th International Conference on System Reliability and Safety (ICSRS)*, Bologna, 2023.
- [10] H. Al-Shareefy, M. Butler and T. Hoang, “AIC-based unforeseen emergent scenarios (Black Swan events) discovery method for autonomous systems safety design,” in *68th International Society for Systems Sciences Annual Conference*, Washington, USA, 2024.

Bibliography

- [11] H. Al-Shareefy, M. Butler and T. Hoang, “Comprehensive Multi-environment operational design domain (ODD) taxonomy for intelligent systems safety design,” in *68th International Society for Systems Sciences Annual Conference*, Washington, D.C, 2024.
- [12] H. Al-Shareefy, M. Butler and T. S. Hoang, “CuneiForm Method for Assuring the Safety of ML-Based Computer Vision Development Datasets,” in *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*, Reykjavik, Iceland , 2024.
- [13] F. Crawley, *HAZOP: Guide to Best Practice*, Glasgow, Scotland: Elsevier, 2015.
- [14] N. G. Leveson, *Engineering a Safer World*, London, UK: MIT Press, 2011.
- [15] F. A. Salehi, S. C., D. Dghaym, T. Hoang, F. Alotaibi and M. Butler, “Designing Critical Systems Using Hierarchical STPA and Event-B,” in *ABZ 2023*, Nancy, France, 2023.
- [16] E. Hollnagel, *FRAM: The Functional Resonance Analysis Method*, London: CRC Press, 2012.
- [17] G. J. Klir, *Facets of Systems Science*, New York: Springer Science+Business Media, LLC, 2001.
- [18] R. Hawkins, C. Paterson, C. Picardi, Y. Jia, R. Calinescu and I. Habli, “Guidance on the Assurance of Machine Learning in Autonomous Systems,” University of York, York, UK, 2021.
- [19] R. Hawkins, M. Osborne, M. Parsons, M. Nicholson, J. McDermid and I. Habli, “Guidance on the Safety Assurance of Autonomous Systems in Complex,” *Assuring Autonomy International Programme (AAIP)*, University of York, York, 2022.
- [20] INCOSE, *Systems Engineering Handbook*, San diego: Willey, 2015.
- [21] Carnegie Mellon University, “Phil Koopman,” Carnegie Mellon University, [Online]. Available: <https://users.ece.cmu.edu/~koopman/>. [Accessed 24 06 2022].
- [22] S. Burton and B. Herd, “Addressing uncertainty in the safety assurance of machine-learning,” *Front. Comput. Sci.*, vol. 5, 2023.
- [23] P. W. M. Koopman, “Challenges in Autonomous Vehicle Testing and Validation,” in *2016 SAE World Congress, 2016-01-0128 / 16AE-0265*, Detroit, 2016.

Bibliography

- [24] C. K. R. A. H. G. A. A. S. A. Crowe S, "The case study approach," BMC Med Res Methodol, doi:10.1186/1471-2288-11-100, 2011.
- [25] Department for transport, "Rail Factsheet," December 2019. [Online]. Available: https://assets.publishing.service.gov.uk/government/uploads/system/uploads/attachment_data/file/851082/rail-factsheet-2019.pdf. [Accessed 07 October 2021].
- [26] Network Rail, " BTP launch enhanced drone capability," Network Rail, 01 May 2025. [Online]. Available: <https://www.youtube.com/watch?v=ecFyokpu4UQ&t=84s>. [Accessed 14 May 2025].
- [27] Airbus, "Airbus concludes ATTOL with fully autonomous flight tests," Airbus , 29 06 2020. [Online]. Available: <https://www.airbus.com/en/newsroom/press-releases/2020-06-airbus-concludes-attol-with-fully-autonomous-flight-tests>. [Accessed 22 01 2025].
- [28] E. Q. Smyers, S. M. Katz, A. L. Corso and M. J. Kochenderfer, "AVOIDDS: Aircraft Vision-based Intruder Detection Dataset and Simulator," arXiv:2306.11203 [cs.CV], 2023.
- [29] G. K. M. Mobus, Principles of systems science, London: Springer, 2015.
- [30] A. Bogdanov, Essays in Tektology, California, USA: Intersystems Publications, 1980.
- [31] R. Nunes, "From the organizational point of view: Bogdanov and the Augustinian Left, Part 1," *E-flux*, no. 152, 2025.
- [32] A. A. Aleshin, "System Theory in International Relations: Alexander Bogdanov's Tektology Potential," *Obshchestvennye Nauki I Sovremennost*, vol. 4, p. 40–53, 2024.
- [33] W. Ulrich, Critical Heuristics of Social Planning: A New Approach to Practical Philosophy, Bern: Wiley, 1983.
- [34] P. Checkland, Soft systems methodology: a 30 year retrospective and systems thinking,, Chichester: Wiley, 1999.
- [35] W. Smith, Director, *LESI SIG Workshop Part 1 - Advances in Systems Sciences*. [Film]. ISSS, 2022.
- [36] N. N. Taleb, The Black Swan, Harlow: Penguin Books, 2008.

Bibliography

- [37] P. Kalia, M. Menzel, K. Grello and A. Walker, “Integrated Systems Engineering, Safety, Reliability and Risk Management – Minimizing Black Swan Events,” in *2024 Annual Reliability and Maintainability Symposium (RAMS)*, Albuquerque, NM, USA, 2019.
- [38] N. Moller and S. O. Hansson, “Principles of engineering safety: Risk and uncertainty reduction,” *Reliability Engineering & System Safety*, pp. 798-805, 6 June 2008.
- [39] D. Dubois, H. Prade and P. Smets, “Representing partial ignorance,” *IEEE Transactions on Systems*, vol. 26, no. 3, p. 361–377, 1996.
- [40] S. Hora, “Aleatory and epistemic uncertainty in probability elicitation with an example from hazardous waste management,” *Reliability Engineering and System Safety*, vol. 54, no. (2-3), p. 217–223, 1996.
- [41] F. Yang, H. Z. Wanga, H. Mi, C. de Lin and W. W. Cai, “Using random forest for reliable classification and cost-sensitive learning for medical diagnosis,” *BMC Bioinformatics*, vol. 10, p. 118–131, 2009.
- [42] A. Lambrou, H. Papadopoulos and A. Gammerman, “Reliable confidence measures for medical diagnosis with evolutionary algorithms,” *IEEE Transactions on Information Technology in Biomedicine*, vol. 15, no. 1, p. 93–99, 2011.
- [43] E. Hüllermeier and W. Waegeman, “Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods,” *Machine Learning*, p. 457–506 , 2021.
- [44] K. Varshney and H. Alemzadeh, “On the safety of machine learning: Cyber-physical systems decision sciences, and data products.,” *Big data*, pp. 246-255, 2017.
- [45] D. Sales and L. B. Becker, “Systematic Literature Review of System Engineering Design Methods,” in *VIII Brazilian Symposium on Computing Systems Engineering (SBESC)*, 2018.
- [46] INCOSE, IEEE Systems Council, “Brief History of Systems Engineering,” SEbok , 20 May 2022. [Online]. Available: https://www.sebokwiki.org/wiki/Brief_History_of_Systems_Engineering. [Accessed 15 Sep 2022].
- [47] W. Royce, “Managing the development of large software,” in *9th international conference on Software Engineering*, 1987.

Bibliography

- [48] B. Boehm, "A spiral model of software development," *ACM SIGSOFT Software Engineering*, vol. 11, no. 4, p. 14–24, 1986.
- [49] A. Rausch, C. Bartelt, T. Ternité and M. Kuhrmann, "The v-modell xt applied–model-driven and documentcentric development," *3rd World Congress for Software*, vol. 3, p. 131–138, 2005.
- [50] INCOSE, *Systems Engineering Handbook*, INCOSE, ISBN: 978-1-119-81431-3, 2023.
- [51] M. Kaufmann, *Agile Systems Engineering*, Elsevier Inc., 2016.
- [52] M. Kaufmann, *A Practical Guide to SysML*, Elsevier Inc., 2015.
- [53] Open University, "Systems diagramming," Open University, [Online]. Available: <https://www.open.edu/openlearn/science-maths-technology/computing-ict/systems-diagramming/content-section-3.2>. [Accessed 20 06 2022].
- [54] J. S. Hsu, N. Parolia and J. J. K. G. Jiang, "The Impact of Team Mental Models on IS Project Teams' Information Processing and Project Performance," in *eProceedings of the 2nd International Research Workshop on Information Technology Project Management*, Québec, Canada, 2007.
- [55] J. R. Hudson, "Using diagrams in systems thinking," johnrhudson.me.uk, 1998.
- [56] D. Long and Z. Scott, *A primer for Model Based Systems Engineering*, Vitech, 2011.
- [57] J. M. Wing, "A specifier's introduction to formal methods," *Computer*, vol. 23, no. 9, pp. 8-22, 1990.
- [58] L. E. G. Martins and T. Gorschek, *Requirements Engineering for Safety-Critical Systems*, River Publishers, 2021.
- [59] S. Tariq, A. Ibrahim, A. Usama and M. S. Shahbaz, "An Overview of Requirements Elicitation Techniques in Software Engineering with a focus on Agile Development," in *4th International Conference on Computing & Information Sciences (ICCIIS)*, Karachi, Pakistan, 2021.
- [60] D. Kritzinger, *Aircraft System Safety*, Woodhead Publishing, 2006.
- [61] D. P. Nolan, *Safety and Security Review for the Process Industries*, Gulf Professional Publishing, 2015.

Bibliography

- [62] D.-H. Ham, "Safety-II and Resilience Engineering in a Nutshell: An Introductory Guide to Their Concepts and Methods," *Safety and Health at Work*, vol. 12, no. 1, pp. 20-19, 2021.
- [63] F. Crawley and B. Tyler, "The Detailed HAZOP Study Procedure," in *HAZOP: Guide to Best Practice Guidelines to Best Practice for the*, Oxford, UK, Elsevier, 2015, pp. 13-28.
- [64] S. Hoang, "An introduction to the Event-B Modeling Method," in *Industrial Deployment of System Engineering Methods*, Springer.
- [65] J.-R. Abrial, *Modeling in Event-B*, Cambridge: Cambridge University Press , 2010.
- [66] A. S. Fathabadi and e. al., "Systematic Hierarchical Analysis of Requirements for Critical Systems," ECS, University of Southampton, Southampton.
- [67] H. Giles, "A rigorous tool-supported methodology for assuring the security and safety of cyber-physical systems," UNIVERSITY OF SOUTHAMPTON, SOUTHAMPTON, 2019.
- [68] M. Cossentino, *From Requirements to Code with the PASSI Methodology*, Idea Group Inc, 2005.
- [69] L. Fischer, L. Ehrlinger, V. Geist, R. Ramler, F. Sobieczky, W. Zellinger, D. Brunner, M. Kumar and B. Moser, " AI System Engineering,Key Challenges and Lessons Learned," *Mach. Learn. Knowl. Extr*, vol. 3, no. <https://doi.org/10.3390/make>, p. 56–83, 2021.
- [70] R. a. M. M. Matthaei, "Functional System Architecture for an Autonomous on-Road Motor Vehicle," in *Automotive Systems Engineering II*, Cham, Switzerland, Springer International Publishing, 2018, pp. 93-120.
- [71] R. Ashmore, R. Calinescu and C. Paterson, "Assuring the Machine Learning Lifecycle: Desiderata, Methods, and Challenges," arXivLabs, 2019.
- [72] A. Géron, *Hands-on machine learning with Scikit-Learn and TensorFlow: concepts, tools, and techniques to*, Sebastopol, USA: O'Reilly Media, Inc., 2017.
- [73] J. Heaton, "An empirical analysis of feature engineering for predictive modeling," in *Southeast Con. IEEE pp. 1-6*, doi: 10.1109/SECON.2016.7506650, 2016.
- [74] "How to use data analysis for machine learning," R-Bloggers Data Analysis 2019, Sharp Sight Labs , 31 May 2016. [Online]. Available: <https://www.r-bloggers.com/how-to-use-data-analysis-for-machine-learning-example-part-1>. [Accessed 3 April 2021].

Bibliography

- [75] I. J. Goodfellow, “Explaining and Harnessing Adversarial Examples,” arXiv:1412.6572v2 , 2015.
- [76] S. B. Kotsiantis, D. Kanellopoulos and P. E. Pintelas, “Data Preprocessing for Supervised Learning,” *Int. Journal of Computer, Electrical, Automation, Control and Information Engineering*, vol. 1, no. 12, p. 4104–4109, 2007.
- [77] G. Ros, L. Sellart and J. e. a. Materzynska, “The synthia dataset: A large collection of synthetic images for semantic segmentation of urban scenes,” in *IEEE Conf. on computer vision and pattern recognition.* , 2016.
- [78] J. Bergstra and Y. Bengio, “ Random search for hyper-parameter optimization,” *Journal of Machine*, vol. 13, p. 281–305, feb (2012).
- [79] J. D. Miller, Automotive System Safety, MI, USA: John Wiley & Sons Ltd, 2020.
- [80] R. Bellairs, “Why SOTIF (ISO/PAS 21448) Is Key For Safety in Autonomous Driving,” Perforce, 24 Apr 2019. [Online]. Available: <https://www.perforce.com/blog/qac/sotif-iso-pas-21448-autonomous-driving>. [Accessed 2 May 2021].
- [81] Underwriters Laboratories Inc, “UL 4600 Fact Sheet Developing the Standard for Safety of Autonomous Products,” Underwriters Laboratories Inc, 2019.
- [82] NHTSA, “Automated Driving Systems 2.0,” Create space Independent Publishing Platform, 2018.
- [83] The Safety of Autonomous Systems Working Group, Safety Assurance Objectives for Autonomous Systems, UK: Safety Critical Systems Club, 2020.
- [84] Z. Chu, J. Chen, Q. Chen, W. Yu, T. He, H. Wang and e. al., “A Survey of Chain of Thought Reasoning: Advances, Frontiers and Future,” Cornell University arXiv.org, Cornell, Available: <https://arxiv.org/abs/2309.15402>, 2023.
- [85] J. Wei and e. al, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information*, New Orleans, Louisiana, USA, SBN: 978-1-7138-7108-8, 2022.
- [86] M. Mehlhorn, A. Richter and Y. Shardt, “Ruling the Operational Boundaries: A Survey on Operational Design Domains of Autonomous Driving Systems,” in *22nd IFAC World Congress*, Japan, 2023.

Bibliography

- [87] BSI, PAS 1883:2020 Operational Design Domain (ODD) taxonomy for an automated driving system (ADS), British Standards Institution., 2020.
- [88] H. Al-Shareefy, M. Butler and T. Hoang, “An AIC-based approach for articulating unpredictable problems in open complex environments,” in *19th European Dependable Computing Conference*, Leuven, Belgium, 2024.
- [89] AVSC , Best practice for describing an operational design domain: Conceptual framework and lexicon, SAE Industry Technologies Consortia, 2020.
- [90] ASAM e.V. , ASAM OpenODD: Concept Paper (version 1.0), The Association for Standardization of Automation and Measuring Systems, 2021.
- [91] NHTSA, A framework for automated driving system testable cases and scenarios, U.S. Department of Transportation, National Highway Traffic Safety Administration., 2018.
- [92] C. Ugé and S. Hochgeschurz, “Learning to Swim - How Operational Design Parameters Determine the Grade of Autonomy of Ships,” *TransNav, the International Journal on Marine Navigation and Safety of Sea Transportation*, pp. 501-509, 2021.
- [93] Castellanos-Ardila and e. al., “A Context-Specific Operational Design Domain for Underground Mining (ODD-UM),” in *EuroSPI 2022. Communications in Computer and Information Science*, Cham, 2022.
- [94] H. Al-Shareefy, “MULTI-ENVIRONMENT-ODD,” 30 05 2024. [Online]. Available: <https://github.com/AIC-Matrix-Architect/MULTI-ENVIRONMENT-ODD>.
- [95] F. Alotaibi, *Metamorphic Testing- Evaluating And Testing The Performance Of Machine Learning Algorithms*, Southampton, UK: University of Southampton, 2018.
- [96] C. Aggarwal, *Outlier Analysis*, 2nd ed., New York, NY, USA: Springer International Publishing, 2017.
- [97] v. e. a. Rueden, “Informed Machine Learning - A Taxonomy and Survey of Integrating Prior Knowledge into Learning Systems,” *IEEE Transactions on Knowledge and Data Engineering*, no. doi: 10.1109/TKDE.2021.3079836, pp. 1-1, 2021.
- [98] J. Mange, “Effect of Training Data Order for Machine Learning,” in *2019 International Conference on Computational Science and Computational Intelligence (CSCI)*, doi: 10.1109/CSCI49370.2019.00078, 2019, pp. 406-407.

Bibliography

- [99] M. Ramesh, "Impact of Training and Testing Data Splits on Accuracy of Time Series Forecasting in Machine Learning," in *2017 International Conference on Computing, Communication, Control and Automation (ICCUBEA)*, doi: 10.1109/ICCUBEA.2017.8463779, 2017, pp. 1-6.
- [10 0] P. Vani, "Face Recognition in Video Streams and its Application in Freedom Fighters Discovery - A Machine Learning Approach," in *2020 IEEE International Conference on Machine Learning and Applied Network Technologies (ICMLANT)*, doi: 10.1109/ICMLANT50963.2020.9355979, 2020, pp. 1-5.
- [10 1] L. Miao, "2017 Chinese Automation Congress (CAC)," in *Virtual reality and artificial intelligence support future training development*, doi: 10.1109/CAC.2017.8242803, 2017, pp. 416-419.
- [10 2] H. Tong-Yu, "On Automatic Generation of Training Images for Machine Learning in Automotive Applications," in *2019 IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, doi: 10.1109/AICAS.2019.8771605, 2019, pp. 225-228.
- [10 3] T. Gibru and et.al., "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, p. 86–92, 2021.
- [10 4] J. Giner-Miguelez and et.al, "A domain-specific language for describing machine learning datasets," *Journal of Computer Languages*, vol. 76, 2023.
- [10 5] wikipedia, "Sales contract Shuruppak Louvre AO3766," Louvre , [Online]. Available: https://en.wikipedia.org/wiki/File:Sales_contract_Shuruppak_Louvre_AO3766.jpg. [Accessed 10 03 2024].
- [10 6] K. Tanaka, T. Aoki, T. Kawai, T. Tomita, D. Kavvakami and N. Chida, "A Formal Specification Language Based on Positional Relationship Between Objects in Automated Driving Systems," in *2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*, Los Alamitos, CA, USA, 2022.
- [10 7] J. Dick, "Rich traceability," in *1st international workshop on traceability in emerging forms of software engineering*, Citeseer, 2002.
- [10 8] W. Smith, "What are the AIC Maps?," 2023. [Online]. Available: https://powermap.odii.com/intro_signup.php. [Accessed 09 10 2023].

Bibliography

- [10] T. Levin, "Tesla's Full Self-Driving tech keeps getting fooled by the moon, billboards, and Burger King signs," Insider, 26 July 2021. [Online]. Available: <https://www.businessinsider.com/tesla-fsd-full-self-driving-traffic-light-fooled-moon-video-2021-7?op=1&r=US&IR=T>. [Accessed 19 July 2022].
- [11] G. E. Mobus and M. C. Kalton, Principles of systems science, New York : Springer, 2015.
- [11] E. De Bono, The use of lateral thinking, London: Penguin Books, 1990.
- [11] J. Bonnie, "Towards a theory of engineered complex adaptive systems of systems," in 2018 Annual IEEE International Systems Conference (SysCon), Vancouver, BC, Canada, 2018.
- [11] S. D. H. a. H. E. Jackson, *What is the Systems Approach?*, INCOSE Insight, vol. 13, no. 1, pp. 41-43. , 2023.
- [11] S. Guiasu and A. Shenitzer, "The principle of maximum entropy," *The Mathematical Intelligencer*, p. 42–48, 1985.
- [11] B. Eva, "Principles of Indifference," *The Journal of Philosophy*, 2019.
- [11] INCOSE, SYSTEMS ENGINEERING HANDBOOK, INCOSE, ISBN: 978-1-119-81431-3, 2023.
- [11] A. D. Hall and R. E. Fagen, Definition of System, Routledge, 1968.
- [11] E. Pat ´e-Cornell, "On "Black Swans" and "Perfect Storms": Risk Analysis and Management When Statistics Are Not Enough," *Risk Analysis*, vol. 32, no. 11, pp. 1823-1833, 2012.
- [11] J. Pearl, "The Logic of Structure-Based Counterfactuals," in *CAUSALITY: Models, Reasoning, and Inference*, New York, CAMBRIDGE UNIVERSITY PRESS, 2000, pp. 209-210.
- [12] H. Al-shareefy, "Final_Dataset_All_in," 2 24 2025. [Online]. Available: <https://app.roboflow.com/experiment-5/experiment-9-all-in/visualize/5>. [Accessed 2 24 2025].

Bibliography

- [12] G. Incorvaia, D. Hond and H. Asgari, "Uncertainty Quantification for Machine Learning
1] Output Assurance using Anomaly-based Dataset Dissimilarity Measures," in *2023 IEEE International Conference On Artificial Intelligence Testing (AITest)*, Athens, Greece, 2023.
- [12] T. Saaty and M. Ozdemir, "Why the magic number seven plus or minus two," *Mathematical
2] and Computer Modelling*, vol. 38, no. 3-4, pp. 233-244, 2003.
- [12] M. K and e. al., "When Enough Is Not Enough: Information Overload and Metacognitive
3] Decisions to Stop Studying Information," *Experimental Psychology: Learning, Memory, and Cognition*, vol. 42, no. 6, p. 914–924, 2016.
- [12] N. Cowan, "The magical number 4 in short-term memory: A reconsideration of mental
4] storage capacity," *Behavioral and Brain Sciences*, vol. 24, no. 1, p. 87–114, 2001.
- [12] D. Apel, W. Behme, R. Eberlein and C. Merighi, Datenqualität erfolgreich steuern:
5] Praxislösungen für Business-Intelligence-Projekte [Successfully Governing Data Quality: Practical Solutions for Business-Intelligence Projects], Heidelberg, Germany: TDWI, 2015.
- [12] H. Al-Shareefy and S. Wright, "Applied systems science to holistic quality assessment
6] metrics for formal methods-based models," in *68th International Society For Systems Sciences Annual Conference*, Washington, D.C., 2024.
- [12] S. R. Kelleher, "No Pilot, No Problem? Here's How Soon Self-Flying Planes Will Take Off,"
7] *Forbes*, 27 02 2023. [Online]. Available:
<https://www.forbes.com/sites/suzannerowankelleher/2023/02/26/pilotless-autonomous-self-flying-planes/>. [Accessed 22 01 2025].
- [12] S. Bushwick, "The NYPD's Robot Dog Was a Really Bad Idea: Here's What Went Wrong,"
8] *SCIAM*, 07 May 2021. [Online]. Available: <https://www.scientificamerican.com/article/the-nypds-robot-dog-was-a-really-bad-idea-heres-what-went-wrong/>. [Accessed 25 12 2024].